Lectures on Quantum Computing and Completely Positive Maps

A formalisation in Isabelle/HOL of Vern Paulsen's Houston 2011 lecture notes

Dustin Bryant

formal statements and proofs: the theories listed in the table of contents

October 27, 2026

Abstract

This document is generated by Isabelle from the project theories. Every definition and theorem shown here has been checked by Isabelle/HOL (proofs are folded). The commentary restates Paulsen's lecture notes lecture by lecture (Day 1 = 22 August 2011, Day 34 = 18 November 2011): definitions and results in the notes' own language, each followed by the formal statement. The informal proofs of the notes are omitted; the formal proofs replace them. Where the formalisation deviates from the notes, the commentary says so. The companion file `NOTES_AUDIT.md` lists, for every lecture, what is formalised and what is not. General matrix facts that the lectures use but do not teach form the parent session *Essential Functional Analysis*, with its own document. Two companion entries build on it: the functional calculus of Hermitian matrices with product formulas, and quantum phase estimation with the HHL algorithm.

Contents

1 Reading guide **2**

2 Conventions **3**

I Basic mathematics (Days 1–7) **3**

3 Day 1 (22 August 2011): the inner product **3**
3.1 The Hilbert–Schmidt inner product on matrices 4
3.2 Properties of the inner product 5

4 Day 2 (24 August 2011): bases, matrix units, trace, projections **6**
4.1 The trace 7
4.2 Matrix-unit algebra and cyclicity of the trace 9
4.3 Orthogonal projections and Parseval's identity 10
4.4 The projection does not depend on the orthonormal basis 14

5 Day 3 (31 August 2011): unitary matrices, Householder unitaries, Schur's theorem **14**
5.1 Unitary matrices 14
5.2 Householder unitaries 15

6 Day 4 (2 September 2011): Hermitian matrices; positive semidefinite and positive definite matrices 20
6.1 Hermitian matrices . . . 20
6.2 The spectral theorem for Hermitian matrices . . . 23
6.3 Positive semidefinite and positive definite matrices . . . 24
6.4 The positive square root . . . 27
6.5 The converse directions for Hermitian matrices (D4-1) . . . 29
6.6 Ranges; positive definiteness and spanning (D4-8) . . . 29
6.7 Projections . . . 30

7 Day 5 (7 September 2011): tests for positivity; direct sums and partitioned matrices 31
7.1 Practical tests for positivity . . . 31
7.2 Cholesky's algorithm on the notes' two examples (D5-3) . . . 31
7.2.1 One step of Cholesky's algorithm . . . 31
7.2.2 Small matrices entry by entry . . . 32
7.2.3 Example 1: a matrix that is not positive . . . 32
7.2.4 Example 2: a positive matrix, in two steps . . . 33
7.3 Direct sums of vector spaces and partitioned matrices . . . 34
7.4 A basis of $V \oplus W$ (D5-5) . . . 35
7.5 Sylvester's criterion (D5-1) . . . 35

8 Day 6 (9 September 2011): tensor products 36
8.1 The universal properties in the Kronecker model . . . 37
8.2 Coordinates in a tensor product . . . 39
8.3 The Schmidt rank (D6-4) . . . 42

9 Day 7 (12 September 2011): tensor products of Hilbert spaces; the postulates 43
9.1 Postulates of quantum mechanics . . . 43

II An introduction to quantum computing (Days 8–17) 45

10 Day 8 (14 September 2011): the quantum game; projective measurements and expected values 45
10.1 Distinguishing states . . . 45
10.2 Projective measurements, expected values and self-adjoint operators . . . 46
10.3 Non-orthogonal states cannot be distinguished . . . 47

11 Day 9 (16 September 2011): positive operator-valued measures; composite systems; entanglement 48
11.1 Positive operator-valued measures . . . 48
11.2 Composite systems . . . 49
11.3 Entanglement . . . 49
11.4 Measuring one half of a Bell pair . . . 50

12 Day 10 (19 September 2011): superdense coding; classical and quantum gates 51
12.1 One-qubit gates . . . 51
12.1.1 The Pauli matrices . . . 51
12.2 Two- and three-qubit gates . . . 53
12.3 Superdense coding . . . 55

13 Day 11 (21 September 2011): gates revisited, circuit diagrams, no-cloning **56**
13.1 Cloning and no-cloning . . . 56
13.2 SWAP from three CNOTs . . . 57
13.3 The CCNOT truth table (D11-2) . . . 57

14 Day 12 (23 September 2011): quantum parallelism **58**
14.1 $H^{\otimes N}$ on $|0\cdots 0\rangle$ and the counting probability (D12-1, D12-4) . . . 59

15 Day 13 (26 September 2011): ensembles and density matrices **60**
15.1 Ensembles . . . 60
15.2 Von Neumann's density matrix . . . 61
15.3 Probabilities via the trace . . . 62

16 Day 14 (28 September 2011): density matrices, pure states, composite ensembles **63**
16.1 Pure states . . . 63
16.2 Composite ensembles . . . 64
16.3 Partial traces: motivation . . . 65

17 Day 15 (30 September 2011): traces, tensor products of operators, partial traces **65**
17.1 The isomorphism $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) \cong L(\mathcal{H}_A \otimes \mathcal{H}_B)$ . . . 66
17.2 Partial traces . . . 69
17.3 Partial traces and measurements . . . 71
17.4 The partial trace over A (D15-6) . . . 72

18 Day 16 (3 October 2011): partial traces in block form; the postulates for density matrices **73**
18.1 $H_A \otimes H_B \cong H_A^{(p)}$ and $L(H_A \otimes H_B) \cong M_p(L(H_A))$ (D16-2) . . . 73

19 Day 17 (5 October 2011): measurement maps, quantum noise, completely positive maps **75**
19.1 Measurement maps . . . 75
19.2 Kraus maps, positive maps, complete positivity . . . 76
19.3 The environment model of noise . . . 81

III Completely positive maps (Days 18–23) **82**

20 Day 18 (7 October 2011): the theory of completely positive maps **82**
20.1 Densities to densities iff positive and trace-preserving (D17-4, D18-2) . . . 83
20.2 The transpose is positive but not 2-positive (D18-4) . . . 83

21 Day 19 (10 October 2011): Choi's theorem **84**
21.1 From a positive Choi matrix to a Kraus representation . . . 85
21.2 Composition of Kraus maps . . . 88
21.3 Vectorisation of Kraus operators . . . 89
21.4 Choi matrix of a Kraus map . . . 89
21.5 Choi matrix is linear in the map . . . 90
21.6 Choi matrix determines a linear CP map on carrier . . . 90
21.7 Choi's theorem with (2) isolated (D19-1) . . . 90
21.8 Trace preservation and unitality in Choi-matrix form (D19-3) . . . 91

22 Day 20 (12 October 2011): positive functionals, adjoints, Choi rank **93**
22.1 The adjoint of a Kraus map . . . 93
22.2 Complete positivity and the UCP/CPTP duality . . . 94
22.3 Positive functionals are completely positive . . . 94
22.4 Choi rank . . . 95

23 Day 21 (14 October 2011): the Kraus rank bound; unitary freedom of representations **95**
23.1 Unitary freedom of rank-one decompositions and Kraus representations (D21-3, D21-5, D21-6) . . . 96
23.1.1 The Gram matrix of an independent list is invertible; dual vectors . . . 96
23.1.2 D21-3: independent vectors give independent rank-one matrices . . . 97
23.1.3 Isometric mixing of columns preserves VV^* . . . 97
23.1.4 D21-5: two rank-one decompositions are related by an isometry . . . 98
23.1.5 D21-6 (Choi 2): unitary freedom of Kraus representations . . . 98
23.2 Ranges of positive matrices (D21-4) . . . 99
23.3 The conjecture on averages of unital maps (D21-2) . . . 99
23.3.1 Positive matrices as sums of d rank-one terms . . . 101
23.3.2 Measurement maps . . . 101
23.3.3 The case $d = 1$: Horn's lemma . . . 101
23.3.4 Maps with diagonal values . . . 102
23.3.5 The notes' form (i) is false as written . . . 102
23.4 The conjecture D21-2 for $n = 2$ . . . 102
23.4.1 Completing an orthonormal family to an orthonormal basis . . . 102
23.4.2 Singular value decomposition . . . 102
23.4.3 A contraction is the average of two unitaries . . . 103
23.4.4 Bessel's inequality for rows that are orthonormal or zero . . . 103
23.4.5 Factoring a wide matrix . . . 103
23.4.6 The case $n = 2$ . . . 104

24 Day 22 (17 October 2011): minimal Kraus representations, convex sets, UCP and CPTP **104**
24.1 Convex sets and extreme points . . . 105
24.2 The convex sets UCP and CPTP . . . 105
24.3 A minimal Kraus representation is linearly independent (D22-1) . . . 106
24.3.1 Mixing a Kraus list by an isometry does not change the map . . . 106
24.3.2 A unitary with a prescribed first row . . . 107
24.3.3 D22-1 . . . 107
24.4 Choi's rank theorem $\mathrm{cr}(\Phi) = \mathrm{rank}\, P_\Phi$ (D20-4) . . . 107
24.4.1 $\mathrm{rank}(\sum_{l<L} w_l w_l^*) \le L$ . . . 108
24.4.2 Independent vectors inside the column space bound the rank from below . . . 108
24.4.3 The theorem . . . 109
24.5 The adjoint map, an affine bijection from UCP onto CPTP (D22-5) . . . 109
24.6 $\mathrm{UCP}(M_n, M_d)$ is compact (D22-4) . . . 111
24.6.1 Typed PSD predicate . . . 111
24.6.2 The positive matrices form a closed set . . . 111
24.6.3 Trace on lmat . . . 112
24.6.4 Compactness of the unit-trace PSD slice . . . 112
24.6.5 UCP maps in Choi coordinates . . . 113

25 Day 23 (19 October 2011): Choi's Theorem 3, extreme points of UCP and CPTP **114**
25.1 Extreme points . . . 114
25.2 Quadratic Kraus maps . . . 115
25.2.1 The Choi matrix . . . 116
25.2.2 The pair sum and the trace . . . 117
25.2.3 The products of a Kraus list . . . 118
25.3 Extreme maps have independent products . . . 119
25.4 A CP Radon–Nikodym theorem . . . 120
25.4.1 Dominated positive matrices . . . 120
25.4.2 Completely positive maps below a Kraus map . . . 121
25.5 Independent products give extreme maps . . . 121
25.6 Choi's Theorem 3 . . . 121
25.6.1 Unital maps (D23-1) . . . 122
25.6.2 An extreme channel has Choi rank at most n (D22-6) . . . 122
25.7 Extreme and non-extreme maps (D23-4) . . . 123
25.7.1 Linear dependence tests . . . 123
25.7.2 Example 1: unitary conjugation is extreme . . . 123
25.7.3 Example 2: an average of two unitary conjugations . . . 123
25.7.4 Example 3: $X \mapsto \mathrm{Tr}(X)\, I_d$ . . . 124
25.7.5 Example 4: the diagonal map . . . 125

IV Operator systems, Arveson's theorems and entanglement (Days 24–28) **125**

26 Day 24 (24 October 2011): operator systems **126**
26.1 $M_p(OS)$ is an operator system . . . 127
26.2 Every element is a combination of four positives . . . 127

27 Day 25 (26 October 2011): the Hahn–Banach theorem **127**
27.1 Bounded complex-linear functionals on lmat . . . 128
27.2 Real-part functional and linearform/continuous instances . . . 128
27.3 Application of HOL/Hahn_Banach (norm-preserving extension) . . . 128
27.4 Bohnenblust-Sobczyk: complex extension from real extension . . . 129
27.5 The matrix-space Hahn-Banach theorem . . . 130
27.6 Arveson's correspondence between maps and functionals . . . 131
27.6.1 Finite sums of matrices . . . 131
27.6.2 $E_{ij} \otimes M$ and the block decomposition . . . 131
27.6.3 The correspondence . . . 132
27.6.4 Theorem (Arveson): completely positive iff d-positive iff S_Φ positive . . . 133
27.7 A unital functional is positive iff it has norm one (D25-4) . . . 135
27.7.1 Vectors . . . 135
27.7.2 The operator norm . . . 135
27.7.3 Unital functionals . . . 136

28 Day 26 (28 October 2011): Arveson's extension theorem; separability **137**
28.1 Hermitian parts and traces of products . . . 138
28.2 Matrices as points of the Euclidean space *lmat* . . . 138
28.3 Real functionals on typed matrices and Hermitian matrices . . . 139
28.4 Extension of positive functionals . . . 139
28.5 Arveson's extension theorem . . . 139

28.6 Factoring a CP map through a unital or a trace-preserving one (D26-1) 140
28.7 Separability 141
28.7.1 Typed tensor product 141
28.7.2 The separable cone 142
28.7.3 The unit-trace product slice 142

29 Day 27 (31 October 2011): positive maps detect entanglement; convex cones 143
29.1 Tensor products with a matrix in the first factor 143
29.2 The quadratic form of Φ_f 143
29.3 $f(\mathrm{Sep}) \geq 0$ iff Φ_f is positive (D27-3) 144
29.4 Positive maps keep separable matrices positive (D27-1) 144
29.5 Entangled matrices detected by the transpose (D27-2) 144

30 Day 28 (2 November 2011): entanglement witnesses; the Horodecki–Peres criterion 145
30.1 Tensor continuity 146
30.2 Closedness of the separable cone 146
30.3 The lmat inner product as a real-trace inner product 148
30.4 Separating hyperplane for non-separable states 148
30.5 Entanglement witness in real-trace form 148
30.6 Hermitian entanglement witness 148
30.7 Hermitian closure of the separable cone 149
30.8 Main separability characterisation 150
30.9 Density-matrix specialisation 150
30.10 The positive-map criterion (D28-3) 152
30.11 The Horodecki–Peres criterion for two qubits (D28-5) 152

V Quantum error correction (Days 29–34) 153

31 Day 29 (7 November 2011): classical error detection and correction 153
31.1 Binary strings, Hamming weight and distance 153
31.2 Code distance 154
31.3 The parity-check and repetition codes 154
31.4 More on the repetition code 155
31.5 Classical codes over $\mathbb{Z}_2$ 156
31.5.1 The notes' "Hamming [7,1,3]" code does not correct one error 156
31.5.2 The Hamming [7,4,3] code corrects one error 157
31.6 Hamming distance and subspace codes (D29-4) 157

32 Day 30 (9 November 2011): error probabilities; the three-qubit bit-flip code 158
32.1 Probabilities of undetected and uncorrected errors (D30-1) 159
32.2 The three-qubit bit-flip code and its recovery map (D30-3, D30-4) 160

33 Day 31 (11 November 2011): the bit-flip code as a channel; fidelity 163
33.1 The bit-flip channel 163
33.2 Fidelity 164
33.3 The three-qubit code against independent bit flips 165
33.3.1 Errors followed by recovery 165
33.3.2 Minimum fidelities (D31-4) 166

34 Day 32 (14 November 2011): the phase-flip code, the Shor code, Pauli magic 166
34.1 The phase-flip code . . . 166
34.2 Algebra of projections; towards Days 33 and 34 . . . 168
34.3 Pauli magic (D32-6) . . . 169
34.4 The Pauli expansion identity for unitaries (D32-7) . . . 169
34.5 The general recovery proposition (D32-4, D33-2) . . . 169
34.5.1 Matrix sums acting on vectors . . . 169
34.6 The code for Y flips . . . 172
34.6.1 Codes with orthogonal error images . . . 173
34.6.2 The Y-flip code . . . 174

35 Day 33 (16 November 2011): the nine-qubit Shor code 175
35.1 Single-qubit operators on nine qubits, and on a block of three . . . 175
35.2 The block vectors $|m\rangle \pm |7-m\rangle$ . . . 176
35.3 Code words and the images of the errors . . . 177
35.4 The 22 correctable errors . . . 178
35.5 The code space and its projection . . . 179
35.6 Proposition (Day 33): orthogonal images, and $Z_1 v = Z_2 v = Z_3 v$ . . . 180
35.7 Shor's theorem . . . 181

36 Day 34 (18 November 2011): the Knill–Laflamme conditions 181
36.1 Vectors in the range of a projection; rank-one decompositions inside V . . . 181
36.2 Linearity facts for the composite map $\mathcal{R} \circ \mathbb{E}$ . . . 182
36.3 Knill–Laflamme, Theorem 1, necessity: $PE_i^* E_j P = \alpha_{ij} P$ . . . 182
36.4 Kraus lists consisting of multiples of one operator . . . 183
36.5 Theorem 2 . . . 183
36.6 Sufficiency of the Knill–Laflamme conditions (D34-2) . . . 183

VI Beyond the notes 185

37 Extensions beyond the notes 185
37.1 Rank-one projections . . . 185
37.2 The bit-flip channel at $p = 0$ and $p = 1$ . . . 186
37.3 The phase-flip channel . . . 186
37.4 Unitary-conjugation channels . . . 186
37.5 The bit-phase-flip channel and the identity channel . . . 187
37.6 Composition of channels . . . 188
37.7 Conjugation channels are completely positive . . . 188
37.8 Unitary-conjugation channels are UCP and CPTP . . . 189
37.9 The Pauli channels are UCP and CPTP . . . 190
37.10 Pauli matrices as Kraus operators . . . 190
37.11 Kraus maps of Kraus-operator lists are CPTP . . . 191
37.12 Fixed points of Pauli channels . . . 191
37.13 The maximally mixed state . . . 192
37.14 Fixed points of unitary-conjugation channels . . . 192
37.15 Channels preserve density matrices . . . 193
37.16 Pauli product identities . . . 194

1 Reading guide

Paulsen's course had five parts: (I) basic mathematics (Days 1–7), (II) an introduction to quantum computing (Days 8–17), (III) completely positive maps (Days 18–23), (IV) operator systems, Arveson's theorems and entanglement (Days 24–28), (V) quantum error correction (Days 29–34). The theories follow the lectures: lecture n is the theory `Day`nn`_...`, which restates the notes of that day and proves what they state; some lectures continue in a further theory, for a long development or for material that later lectures should not import (Sylvester's criterion brings in another AFP entry's names, the Bell measurement short names like e_0). Every theory builds only on earlier lectures. A result that the notes state before the tools of its proof are available is proved on the first day that has them, and its statement says where: Choi's rank theorem, stated on Day 20, is proved on Day 22 from the independence of minimal Kraus representations, and Day 2's characterisation of orthogonal projections waits for the Hermitian matrices of Day 4. The unformalised items are listed in `NOTES_AUDIT.md`.

Day	Theories
1	`Day01_Inner_Product`
2	`Day02_Trace_Projections`
3	`Day03_Unitary_Matrices`
4	`Day04_Hermitian_Matrices`
5	`Day05_Positivity_Tests`, `Day05_Sylvester_Criterion`
6	`Day06_Tensor_Products`
7	`Day07_Postulates`
8	`Day08_Measurements`
9	`Day09_POVMs_Entanglement`, `Day09_Bell_Measurement`
10	`Day10_Quantum_Gates`
11	`Day11_Gates_No_Cloning`
12	`Day12_Quantum_Parallelism`
13	`Day13_Density_Matrices`
14	`Day14_Pure_States`
15	`Day15_Partial_Traces`
16	`Day16_Block_Form`
17	`Day17_Quantum_Noise`
18	`Day18_CP_Maps`
19	`Day19_Choi_Theorem`
20	`Day20_Positive_Functionals`
21	`Day21_Unitary_Freedom`, `Day21_Average_Conjecture`
22	`Day22_Minimal_Kraus`, `Day22_UCP_Compact`
23	`Day23_Choi_Extreme_Points`, `Day23_Extremality_Examples`
24	`Day24_Operator_Systems`
25	`Day25_Hahn_Banach`, `Day25_Arveson_Correspondence`, `Day25_Unital_Functional_Norm`
26	`Day26_Arveson_Extension`, `Day26_Separability`
27	`Day27_Detecting_Entanglement`
28	`Day28_Entanglement_Witnesses`
29	`Day29_Error_Correction`
30	`Day30_Bit_Flip_Code`
31	`Day31_Bit_Flip_Channel`
32	`Day32_Phase_Flip_Code`, `Day32_Recovery_Proposition`
33	`Day33_Shor_Code`
34	`Day34_Knill_Laflamme`

The last part, `Quantum_Channels`, collects standard channels beyond the notes. What the lectures build on but do not teach is the parent session *Essential Functional Analysis*: general matrix lemmas (`Matrix_Lemmas`), unitary and Kronecker-product algebra (`Unitary_Tensor_Algebra`), and matrices of fixed dimension as a type, with the Hilbert–Schmidt norm (`Matrix_Typed`) and the structure of a Euclidean space (`Lmat_Euclidean`). The companion AFP entries *Hermitian Functional Calculus* and *HHL Algorithm* go beyond the notes: the functional calculus and product formulas, quantum phase estimation and the HHL algorithm.

2 Conventions

- Vectors in $\mathbb{C}^n$ are *complex Matrix.vec* with *dim-vec* $v = n$; matrices in $M_{m,n}$ are *complex Matrix.mat* with $A \in$ *carrier-mat m n* (the Jordan-Normal-Form library). Dimensions are hypotheses, not types, except in the typed wrapper *lmat* used for the topological arguments of Part IV.
- The physicists' adjoint $X^\dagger = X^*$ is $A^\dagger$; $\langle y|x\rangle$ is $\langle y|x\rangle$ (conjugate-linear in the first slot, as in the notes); $|v\rangle\langle w|$ is $v \oslash w$; the Hilbert–Schmidt inner product $\langle Y|X\rangle = \mathrm{Tr}(Y^*X)$ is $\langle\langle$ *Y | X* $\rangle\rangle$; the matrix unit $E_{ij} \in M_{m,n}$ is $E[\![i,j\ ;\ m,n]\!]$.
- Indices start at 0, as the physicists in the notes prefer: the canonical basis of $\mathbb{C}^n$ is $|0\rangle, \ldots, |n-1\rangle$, formally *unit-vec n i*.
- $\sum_i |v_i\rangle\langle v_i|$ for a list of vectors is *outer-sum n vs* $= VV^\dagger$ with V the matrix whose columns are the v_i.
- Tensor products are Kronecker products: $v \otimes w$ and $A \bigotimes B$, with the first factor as the major index.
- Kraus sums $\sum_i W_i X W_i^\dagger$ over a list are *kraus-map-dim d Ws X*.

Verification status. All theories build with *quick-and-dirty = false*; there are no *sorry*s and no axioms beyond those of Isabelle/HOL and the imported libraries.

Part I
Basic mathematics (Days 1–7)

3 Day 1 (22 August 2011): the inner product

Paulsen opens with the dictionary between classical and quantum computing: a bit is an element of $\mathbb{Z}_2$, a qubit a unit vector in $\mathbb{C}^2$; N bits form $\mathbb{Z}_2^N$, N qubits a unit vector in $\mathbb{C}^{2^N} \cong \mathbb{C}^2 \otimes \cdots \otimes \mathbb{C}^2$; operations are Boolean gates versus completely positive maps on M_{2^N}; information is a probability vector versus a positive semidefinite matrix of trace one. He then fixes the physicists' notation.

Definition (notes). $\mathbb{C}^n$ is the space of complex n-tuples with the inner product

$$\langle y|x\rangle = \sum_{i=1}^{n} \overline{y_i}\, x_i,$$

which is linear in the right-hand slot, conjugate linear in the left-hand slot ("sesquilinear"), and positive definite: $\langle x|x\rangle \geq 0$ with equality only for $x = 0$. The Euclidean length is $\|x\| = \sqrt{\langle x|x\rangle}$.

A *Hilbert space* is a complex vector space with a sesquilinear positive definite form, complete for this norm.

The library's inner product $\langle y|x\rangle$ is exactly this form, with indices running from 0 to $n-1$.

lemma *inner-prod-sum*:
  assumes *dim-vec u* = *n* **and** *dim-vec v* = *n*
  shows $\langle u|v\rangle = (\sum\ i \in \{0..< n\}.\ cnj(u\ \$\ i) * (v\ \$\ i))$
  ⟨*proof*⟩

lemma *cpx-vec-length-zero-iff*:
  fixes *v* :: *complex Matrix.vec*
  shows $\|v\| = 0 \longleftrightarrow v = 0_v\ (dim\text{-}vec\ v)$
⟨*proof*⟩

$\langle v|v\rangle = 0$ iff $v = 0$.

lemma *inner-prod-self-eq-zero-iff*:
  fixes *v* :: *complex Matrix.vec*
  shows $\langle v \mid v\rangle = 0 \longleftrightarrow v = 0_v\ (dim\text{-}vec\ v)$
⟨*proof*⟩

lemma *cpx-vec-length-smult*:
  fixes *a* :: *complex* **and** *v* :: *complex Matrix.vec*
  shows $\|a \cdot_v v\| = cmod\ a * \|v\|$
⟨*proof*⟩

The vector inner product is linear on the right, $\langle u|av+w\rangle = a\langle u|v\rangle + \langle u|w\rangle$, and conjugate-linear on the left; correspondingly $(\gamma U)^* = \overline{\gamma}\, U^*$.

lemma *inner-prod-right-smult′*:
  fixes *u v* :: *complex Matrix.vec*
  shows $\langle u \mid a \cdot_v v\rangle = a * \langle u \mid v\rangle$
  ⟨*proof*⟩

lemma *inner-prod-right-add′*:
  fixes *u v w* :: *complex Matrix.vec*
  assumes *vw*: *dim-vec v* = *dim-vec w*
  shows $\langle u \mid v + w\rangle = \langle u \mid v\rangle + \langle u \mid w\rangle$
⟨*proof*⟩

lemma *inner-prod-right-minus′*:
  fixes *u v w* :: *complex Matrix.vec*
  assumes *vw*: *dim-vec v* = *dim-vec w*
  shows $\langle u \mid v - w\rangle = \langle u \mid v\rangle - \langle u \mid w\rangle$
⟨*proof*⟩

lemma *inner-prod-left-minus′*:
  fixes *u v w* :: *complex Matrix.vec*
  assumes *uv*: *dim-vec u* = *dim-vec v*
    and *uw*: *dim-vec u* = *dim-vec w*
  shows $\langle u - v \mid w\rangle = \langle u \mid w\rangle - \langle v \mid w\rangle$
⟨*proof*⟩

3.1 The Hilbert–Schmidt inner product on matrices

Definition (notes) (Example)**.** The space $M_{n,k}$ of complex $n \times k$ matrices is a Hilbert space for

$$\langle (y_{ij}) \,|\, (x_{ij})\rangle = \sum_i \sum_j \overline{y_{ij}}\, x_{ij},$$

and $M_{n,k} \cong \mathbb{C}^{nk}$.

We write $\langle\langle y|x\rangle\rangle$ for this Hilbert–Schmidt (Frobenius) inner product; the double brackets distinguish it from the inner product of vectors.

definition *hs-inner-prod* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *complex* **where**
hs-inner-prod = (λ *y x*.
∑ *i* ∈ {*0* ..< *dim-row x*}.
∑ *j* ∈ {*0* ..< *dim-col x*}.
cnj (*y* $$ (*i*,*j*)) ∗ (*x* $$ (*i*,*j*)))

notation *hs-inner-prod* (⟨⟨ - | - ⟩⟩)

lemma *hs-inner-prod-expand*:
assumes *dim-row x* = *m* **and** *dim-col x* = *n*
shows ⟨⟨ *y* | *x* ⟩⟩ =
(∑ *i* ∈ {*0* ..< *m*}. ∑ *j* ∈ {*0* ..< *n*}. *cnj* (*y* $$ (*i*,*j*)) ∗ (*x* $$ (*i*,*j*)))
⟨*proof*⟩

lemma *hs-inner-prod-square*:
assumes *dim-row x* = *n* **and** *dim-col x* = *n*
shows ⟨⟨ *y* | *x* ⟩⟩ =
(∑ *i* ∈ {*0* ..< *n*}. ∑ *j* ∈ {*0* ..< *n*}. *cnj* (*y* $$ (*i*,*j*)) ∗ (*x* $$ (*i*,*j*)))
⟨*proof*⟩

3.2 Properties of the inner product

The properties listed in the notes, verified for the matrix inner product: conjugate symmetry, linearity in the right-hand slot, conjugate linearity in the left-hand slot, and positive definiteness.

lemma *hs-inner-prod-cnj*:
assumes *dim-row x* = *dim-row y* **and** *dim-col x* = *dim-col y*
shows ⟨⟨ *x* | *y* ⟩⟩ = *cnj* (⟨⟨ *y* | *x* ⟩⟩)
⟨*proof*⟩

Linear in the right slot, conjugate-linear in the left:

$$\langle Y|X_1 + X_2\rangle = \langle Y|X_1\rangle + \langle Y|X_2\rangle, \qquad \langle Y|cX\rangle = c\,\langle Y|X\rangle, \qquad \langle cY|X\rangle = \overline{c}\,\langle Y|X\rangle.$$

lemma *hs-inner-prod-right-add*:
assumes *dim-row X1* = *dim-row y* **and** *dim-col X1* = *dim-col y*
and *dim-row X2* = *dim-row y* **and** *dim-col X2* = *dim-col y*
shows ⟨⟨ *y* | *X1* + *X2* ⟩⟩ = ⟨⟨ *y* | *X1* ⟩⟩ + ⟨⟨ *y* | *X2* ⟩⟩
⟨*proof*⟩

lemma *hs-inner-prod-left-add*:
assumes *dim-row A1* = *dim-row B* **and** *dim-col A1* = *dim-col B*
and *dim-row A2* = *dim-row B* **and** *dim-col A2* = *dim-col B*
shows ⟨⟨ *A1* + *A2* | *B* ⟩⟩ = ⟨⟨ *A1* | *B* ⟩⟩ + ⟨⟨ *A2* | *B* ⟩⟩
⟨*proof*⟩

lemma *hs-inner-prod-right-smult*:
assumes *dim-row x* = *dim-row y* **and** *dim-col x* = *dim-col y*
shows ⟨⟨ *y* | *c* ·$_m$ *x* ⟩⟩ = *c* ∗ ⟨⟨ *y* | *x* ⟩⟩
⟨*proof*⟩

lemma *hs-inner-prod-left-smult*:
assumes *dim-row x = dim-row y* **and** *dim-col x = dim-col y*
shows $\langle\langle\ c \cdot_m\ y \mid x\ \rangle\rangle = cnj\ c * \langle\langle\ y \mid x\ \rangle\rangle$
⟨*proof*⟩

Positive definiteness: $\langle X|X\rangle = \sum_{i,j} |x_{ij}|^2$ is real, non-negative, and 0 only if every entry is 0.

lemma *hs-inner-prod-self-real*:
shows $\langle\langle\ x \mid x\ \rangle\rangle \in \mathbb{R}$
⟨*proof*⟩

lemma *hs-inner-prod-self-nonneg*:
shows $Re\ (\langle\langle\ x \mid x\ \rangle\rangle) \geq 0$
⟨*proof*⟩

lemma *hs-inner-prod-eq-zero-imp*:
assumes $\langle\langle\ x \mid x\ \rangle\rangle = 0$
and $i < dim\text{-}row\ x$ **and** $j < dim\text{-}col\ x$
shows $x\ \$\$\ (i,j) = 0$
⟨*proof*⟩

lemma *inner-prod-vec-sum-right*:
assumes *ud*: $dim\text{-}vec\ u = n$
and *wd*: $\bigwedge J.\ J \in Js \Longrightarrow dim\text{-}vec\ (w\ J) = n$
shows $\langle u \mid vec\text{-}sum\ n\ Js\ w\rangle = sum\ (\lambda J.\ \langle u \mid w\ J\rangle)\ Js$
⟨*proof*⟩

lemma *inner-prod-unit-vec-right*:
assumes *ud*: $dim\text{-}vec\ u = n$ **and** *j*: $j < n$
shows $\langle u \mid unit\text{-}vec\ n\ j\rangle = cnj\ (u\ \$\ j)$
⟨*proof*⟩

4 Day 2 (24 August 2011): bases, matrix units, trace, projections

Definition (notes) (Examples)**.** $\mathbb{C}^n$ has the orthonormal basis $e_1, \ldots, e_n$ and every x satisfies $x = \sum_i x_i e_i = \sum_i \langle e_i|x\rangle e_i$. The *matrix units* E_{ij} (entry 1 in position (i,j), 0 elsewhere) form an orthonormal basis of $M_{n,m}$, and

$$X = \sum_{i,j} X_{ij} E_{ij} = \sum_{i,j} \langle E_{ij}|X\rangle E_{ij}.$$

Formally $E[\![i,j\ ;\ m,n]\!]$ is the $m \times n$ matrix unit (indices first, dimensions second). The notes also recall the conjugate $\overline{X}$, the transpose X^t, the adjoint $X^* = \overline{X}^t$ (written $X^\dagger$ by physicists, formally $X^\dagger$) and the row-times-column rule for matrix products; these are library notions.

definition *mat-unit* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
mat-unit i j m n = Matrix.mat m n (λ(*r*,*c*). *if r = i* ∧ *c = j then 1 else 0*)

notation *mat-unit* (*E*[![-,- ; -,-]!] [*1000*, *1000*, *1000*, *1000*] *999*)

Entrywise, $(E_{ij})_{rc} = \delta_{ri}\,\delta_{cj}$.

lemma *mat-unit-carrier* [*simp*]: $E[\![i,j\ ;\ m,n]\!] \in$ *carrier-mat m n*
⟨*proof*⟩

lemma *mat-unit-dim-row* [*simp*]:
fixes *i j m n* :: *nat*
shows *dim-row* $(E[\![i,j\ ;\ m,n]\!]) = m$
⟨*proof*⟩

lemma *mat-unit-dim-col* [*simp*]:
fixes *i j m n* :: *nat*
shows *dim-col* $(E[\![i,j\ ;\ m,n]\!]) = n$
⟨*proof*⟩

lemma *mat-unit-index* [*simp*]:
assumes $r < m$ **and** $c < n$
shows $(E[\![i,j\ ;\ m,n]\!]$ $\$\$$ $(r,c) =$ (*if* $r = i \wedge c = j$ *then 1 else 0*)
⟨*proof*⟩

Pairing with E_{ij} extracts the (i, j) entry: the matrix analogue of $\langle e_i|v\rangle = v_i$.

lemma *hs-inner-prod-mat-unit-left*:
assumes $i <$ *dim-row x* **and** $j <$ *dim-col x*
shows $\langle\langle\ E[\![i,j\ ;\ (\textit{dim-row}\ x),\ (\textit{dim-col}\ x)]\!]\ |\ x\ \rangle\rangle = x$ $\$\$$ (i,j)
⟨*proof*⟩

lemma *mat-unit-orthonormal*:
assumes *i-lt*: $i < m$
and *j-lt*: $j < n$
and *k-lt*: $k < m$
and *l-lt*: $l < n$
shows $\langle\langle\ E[\![i,j\ ;\ m,n]\!]\ |\ E[\![k,l\ ;\ m,n]\!]\ \rangle\rangle =$
(*if* $i = k \wedge j = l$ *then 1 else 0*)
⟨*proof*⟩

Expansion in the basis $\{E_{ij}\}$, stated entrywise because matrices of different dimensions cannot be added in the library.

lemma *mat-unit-expansion*:
assumes $r <$ *dim-row x* **and** $c <$ *dim-col x*
shows x $\$\$$ $(r,c) =$
$(\sum i \in \{0..<\textit{dim-row}\ x\}.\ \sum j \in \{0..<\textit{dim-col}\ x\}.$
x $\$\$$ $(i,j) *$ (*mat-unit i j* (*dim-row x*) (*dim-col x*)) $\$\$$ $(r,c))$
⟨*proof*⟩

lemma *mat-unit-hs-expansion*:
assumes $r <$ *dim-row x* **and** $c <$ *dim-col x*
shows x $\$\$$ $(r,c) =$
$(\sum i \in \{0..<\textit{dim-row}\ x\}.\ \sum j \in \{0..<\textit{dim-col}\ x\}.$
$\langle\langle$ *mat-unit i j* (*dim-row x*) (*dim-col x*) $|\ x\ \rangle\rangle *$
(*mat-unit i j* (*dim-row x*) (*dim-col x*)) $\$\$$ $(r,c))$
⟨*proof*⟩

4.1 The trace

Definition (notes). $\mathrm{Tr} : M_{n,n} \to \mathbb{C}$, $\mathrm{Tr}(X) = \sum_{i=1}^{n} x_{ii}$; we sometimes write Tr_n.

The formal *Tr* sums the diagonal of any matrix. On matrix units, $\mathrm{Tr}(E_{ii}) = 1$ and $\mathrm{Tr}(E_{ij}) = 0$ for $i \neq j$, so the trace is the sum of the diagonal Hilbert–Schmidt coefficients.

definition *Tr* :: *complex Matrix.mat* ⇒ *complex* **where**
Tr x = (∑ *r* < *dim-row x*. *if r* < *dim-col x then x* $$ (*r*,*r*) *else 0*)

lemma *Tr-mat-unit*:
assumes *i-lt*: *i* < *m*
and *j-lt*: *j* < *n*
shows *Tr* (*E*⟦*i*,*j* ; *m*,*n*⟧) = (*if i* = *j then 1 else 0*)
⟨*proof*⟩

So $\operatorname{Tr} E_{ij} = \delta_{ij}$, and since $\langle E_{rr}|X\rangle = x_{rr}$,

$$\operatorname{Tr} X = \sum_r x_{rr} = \sum_r \langle E_{rr}|X\rangle.$$

corollary *Tr-mat-unit-diag*:
assumes *i* < *n*
shows *Tr* (*E*⟦*i*,*i* ; *n*,*n*⟧) = *1*
⟨*proof*⟩

corollary *Tr-mat-unit-offdiag*:
assumes *i* < *m* **and** *j* < *n* **and** *i* ≠ *j*
shows *Tr* (*E*⟦*i*,*j* ; *m*,*n*⟧) = *0*
⟨*proof*⟩

lemma *Tr-hs-diagonal-sum*:
shows *Tr x* =
(∑ *r* < *dim-row x*.
if r < *dim-col x*
then ⟨⟨ *E*⟦*r*,*r* ; (*dim-row x*), (*dim-col x*)⟧ | *x* ⟩⟩
else 0)
⟨*proof*⟩

corollary *Tr-hs-diagonal-sum-square*:
assumes *rows*: *dim-row x* = *n*
and *cols*: *dim-col x* = *n*
shows *Tr x* = (∑ *r*<*n*. ⟨⟨ *E*⟦*r*,*r* ; *n*,*n*⟧ | *x* ⟩⟩)
⟨*proof*⟩

The trace is linear.

lemma *Tr-add*:
assumes *rows*: *dim-row x* = *dim-row y*
and *cols*: *dim-col x* = *dim-col y*
shows *Tr* (*x* + *y*) = *Tr x* + *Tr y*
⟨*proof*⟩

lemma *Tr-smult*: *Tr* (*a* ·$_m$ *x*) = *a* ∗ *Tr x*
⟨*proof*⟩

corollary *Tr-uminus*: *Tr* (− *x*) = − *Tr x*
⟨*proof*⟩

corollary *Tr-diff*:
assumes *rows*: *dim-row x* = *dim-row y*
and *cols*: *dim-col x* = *dim-col y*
shows *Tr* (*x* − *y*) = *Tr x* − *Tr y*
⟨*proof*⟩

lemma *Tr-zero-mat* [*simp*]:

shows *Tr* (*Matrix.mat m n* (λ(*r*,*c*). *0*)) = *0*
⟨*proof*⟩

lemma *Tr-dagger*:
fixes *M* :: *complex Matrix.mat*
assumes *M-c*: *M* ∈ *carrier-mat n n*
shows *Tr* (*M*†) = *cnj* (*Tr M*)
⟨*proof*⟩

4.2 Matrix-unit algebra and cyclicity of the trace

$E^*_{ij} = E_{ji}$, $E_{ij}E_{kl} = \delta_{jk}E_{il}$, $\mathrm{Tr}(AB) = \mathrm{Tr}(BA)$, $\mathrm{Tr}(U^*AU) = \mathrm{Tr}(A)$ for unitary U, and $\langle A|B\rangle = \mathrm{Tr}(A^*B)$: the facts about the trace that the notes use from Day 3 onwards.

lemma *mat-unit-dagger*:
shows (*E*⟦*i*,*j* ; *m*,*n*⟧)† = *E*⟦*j*,*i* ; *n*,*m*⟧
⟨*proof*⟩

Multiplying by a matrix unit selects a row or a column.

lemma *unit-mult-left-entry*:
assumes *Bm*: *Bm* ∈ *carrier-mat n m* **and** *r*: *r* < *p* **and** *k*: *k* < *m* **and** *j*: *j* < *n*
shows (*E*⟦*i*,*j* ; *p*,*n*⟧ ∗ *Bm*) $$ (*r*,*k*) = (*if r* = *i then Bm* $$ (*j*,*k*) *else 0*)
⟨*proof*⟩

lemma *unit-mult-right-entry*:
assumes *Cm*: *Cm* ∈ *carrier-mat p n* **and** *r*: *r* < *p* **and** *s*: *s* < *m* **and** *i*: *i* < *n*
shows (*Cm* ∗ *E*⟦*i*,*j* ; *n*,*m*⟧) $$ (*r*,*s*) = (*if s* = *j then Cm* $$ (*r*,*i*) *else 0*)
⟨*proof*⟩

lemma *mat-unit-mult-entry*:
assumes *r-lt*: *r* < *m* **and** *c-lt*: *c* < *p* **and** *j-lt*: *j* < *n*
shows (*E*⟦*i*,*j* ; *m*,*n*⟧ ∗ *E*⟦*k*,*l* ; *n*,*p*⟧) $$ (*r*,*c*) = (*if j* = *k* ∧ *r* = *i* ∧ *c* = *l then 1 else 0*)
⟨*proof*⟩

lemma *sandwich-mat-unit-entry*:
assumes *V*: *V* ∈ *carrier-mat d n* **and** *W*: *W* ∈ *carrier-mat d n*
and *a*: *a* < *n* **and** *b*: *b* < *n* **and** *i*: *i* < *d* **and** *j*: *j* < *d*
shows (*V* ∗ *mat-unit a b n n* ∗ (*W*†)) $$ (*i*, *j*) = *V* $$ (*i*, *a*) ∗ *cnj* (*W* $$ (*j*, *b*))
⟨*proof*⟩

Writing $A = \sum_{i,j} a_{ij}E_{ij}$ and $B = \sum_{k,l} b_{kl}E_{kl}$, the product rule for matrix units gives $\mathrm{Tr}(AB) = \sum_{i,j} a_{ij}b_{ji}$, which is symmetric in A and B:

lemma *Tr-cyclic*:
assumes *AB*: *dim-col A* = *dim-row B*
and *BA*: *dim-row A* = *dim-col B*
shows *Tr* (*A* ∗ *B*) = *Tr* (*B* ∗ *A*)
⟨*proof*⟩

Hence the trace is invariant under unitary conjugation, $\mathrm{Tr}(U^*AU) = \mathrm{Tr}(AUU^*) = \mathrm{Tr}\,A$, and the Hilbert–Schmidt inner product is a trace: $\langle A|B\rangle = \sum_{i,j} \overline{a_{ij}}\, b_{ij} = \mathrm{Tr}(A^*B)$.

lemma *Tr-unitary-conjugation*:
assumes *Uu*: *unitary U*
and *Arow*: *dim-row A* = *dim-row U*
and *Acol*: *dim-col A* = *dim-row U*
shows *Tr* ((*U*†) ∗ *A* ∗ *U*) = *Tr*(*A*)
⟨*proof*⟩

lemma *hs-inner-prod-as-trace*:
assumes *rows*: *dim-row A* = *dim-row B*
and *cols*: *dim-col A* = *dim-col B*
shows ⟨⟨ *A* | *B* ⟩⟩ = *Tr* ((*A*†) ∗ *B*)
⟨*proof*⟩

corollary *hs-inner-prod-self-as-trace*:
shows ⟨⟨ *A* | *A* ⟩⟩ = *Tr* ((*A*†) ∗ *A*)
⟨*proof*⟩

The trace pairing is non-degenerate: $\mathrm{Tr}(ME_{sr}) = m_{rs}$, so if $\mathrm{Tr}(M\rho) = \mathrm{Tr}(N\rho)$ for every $\rho \in M_n$ then $M = N$.

lemma *Tr-mat-unit-mult*:
fixes *M* :: *complex Matrix.mat*
assumes *M-c*: *M* ∈ *carrier-mat n n*
and *r-lt*: *r* < *n* **and** *s-lt*: *s* < *n*
shows *Tr* (*M* ∗ *mat-unit s r n n*) = *M* $$ (*r*, *s*)
⟨*proof*⟩

lemma *mat-trace-pairing-unique*:
fixes *M N* :: *complex Matrix.mat*
assumes *M-c*: *M* ∈ *carrier-mat n n*
and *N-c*: *N* ∈ *carrier-mat n n*
and *pair-eq*: ⋀*rho*. *rho* ∈ *carrier-mat n n* ⟹ *Tr* (*M* ∗ *rho*) = *Tr* (*N* ∗ *rho*)
shows *M* = *N*
⟨*proof*⟩

4.3 Orthogonal projections and Parseval's identity

Definition (notes) (Orthogonal projections)**.** Let $V \subseteq \mathbb{C}^n$ be a subspace with orthonormal basis $\{v_1, \dots, v_k\}$ and put $P = \sum_{i=1}^{k} |v_i\rangle\langle v_i|$. Then $Px = \sum_i \langle v_i|x\rangle v_i \in V$ and $x - Px \perp V$, so every $x \in \mathbb{C}^n$ is uniquely $x = v + w$ with $v \in V$ and $w \in V^{\perp}$. Consequently P does not depend on the orthonormal basis chosen for V, $Pv = v$ for $v \in V$, $P^2 = P$ and $P = P^*$: P is *the* orthogonal projection onto V.

Theorem (notes) (Parseval)**.** If $\{v_1, \dots, v_k\}$ is an orthonormal basis of V and $v \in V$, then $\|v\|^2 = \sum_{i=1}^{k} |\langle v_i|v\rangle|^2$. For $V = \mathbb{C}^n$ this holds for every v.

Theorem (notes). If $P \in M_n$ satisfies $P^2 = P$ and $P = P^*$, and $V = \mathrm{range}(P)$, then P is the orthogonal projection onto V.

Formally, an orthonormal list *vs* of vectors in $\mathbb{C}^n$ is *orthonormal-list n vs*, the matrix with columns $v_1, \dots, v_k$ is *mat-of-cols n vs*, and $P = \sum_i |v_i\rangle\langle v_i| = VV^{\dagger}$ is *outer-sum n vs*. The results below are the projection formula, $V^{\dagger}V = I_k$, the completeness relation $VV^{\dagger} = I_n$ for a full basis, $P = P^{\dagger}$, $P^2 = P$, Parseval, the orthogonality of the residual, and the uniqueness of the decomposition. Independence of P from the basis is *outer-sum-basis-independent* below; the characterisation of Hermitian idempotents as orthogonal projections needs Hermitian matrices and is *projection-annihilates-orthogonal* on Day 4.

The outer product $|v\rangle\langle w| = v\,w^*$ of Dirac's notation.

abbreviation *outer-prod* :: *complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.mat*
(**infixr** ⊘ *75*)
where *v* ⊘ *w* ≡ |*v*⟩ ∗ *bra-vec w*

lemma *bra-vec-index-0*:
assumes *j*: *j* < *dim-vec w*

shows *bra-vec w $$ (0,j) = cnj (w $ j)*
⟨*proof*⟩

$|v\rangle\langle w| = v\, w^*$ has entries $v_i\overline{w_j}$ and sends every vector to a multiple of v: $|v\rangle\langle w|\, x = \langle w|x\rangle\, v$.

lemma *outer-prod-index*:
fixes *v w* :: *complex Matrix.vec*
assumes *i*: *i* < *dim-vec v*
and *j*: *j* < *dim-vec w*
shows (*v* ⊘ *w*) $$ (*i*,*j*) = (*v* $ *i*) * *cnj* (*w* $ *j*)
⟨*proof*⟩

lemma *outer-prod-mult-vec-index*:
fixes *v w x* :: *complex Matrix.vec*
assumes *xdim*: *dim-vec x* = *dim-vec w*
and *i*: *i* < *dim-vec v*
shows ((*v* ⊘ *w*) *$_v$ *x*) $ *i* = (*v* $ *i*) * ⟨*w* | *x*⟩
⟨*proof*⟩

lemma *outer-prod-mult-vec*:
fixes *v w x* :: *complex Matrix.vec*
assumes *xdim*: *dim-vec x* = *dim-vec w*
shows (*v* ⊘ *w*) *$_v$ *x* = ⟨*w* | *x*⟩ ·$_v$ *v*
⟨*proof*⟩

lemma *outer-prod-carrier*:
fixes *v w* :: *complex Matrix.vec*
assumes *dim-vec v* = *n* **and** *dim-vec w* = *m*
shows *v* ⊘ *w* ∈ *carrier-mat n m*
⟨*proof*⟩

Conjugating a rank-one matrix: $W|\varphi\rangle\langle\varphi|W^* = |W\varphi\rangle\langle W\varphi|$.

lemma *conj-outer-prod*:
fixes *W* :: *complex Matrix.mat* **and** *φ* :: *complex Matrix.vec*
assumes *Wc*: *W* ∈ *carrier-mat m n* **and** *φd*: *dim-vec φ* = *n*
shows *W* * (*φ* ⊘ *φ*) * (*W*†) = (*W* *$_v$ *φ*) ⊘ (*W* *$_v$ *φ*)
⟨*proof*⟩

lemma *outer-prod-zero-vec*: (*0$_v$ n* :: *complex Matrix.vec*) ⊘ (*0$_v$ n*) = *0$_m$ n n*
⟨*proof*⟩

definition *orthonormal-list* :: *nat* ⇒ *complex Matrix.vec list* ⇒ *bool* **where**
orthonormal-list n vs ≡
(∀ *v* ∈ *set vs*. *dim-vec v* = *n*) ∧
(∀ *i* < *length vs*. ∀ *j* < *length vs*.
⟨*vs* ! *i* | *vs* ! *j*⟩ = (*if i* = *j then 1 else 0*))

lemma *orthonormal-listD-dim*:
assumes *orthonormal-list n vs* **and** *i* < *length vs*
shows *dim-vec* (*vs* ! *i*) = *n*
⟨*proof*⟩

lemma *orthonormal-listD-inner*:
assumes *orthonormal-list n vs* **and** *i* < *length vs* **and** *j* < *length vs*
shows ⟨*vs* ! *i* | *vs* ! *j*⟩ = (*if i* = *j then 1 else 0*)
⟨*proof*⟩

lemma *orthonormal-listD-carrier*:
assumes *orthonormal-list n vs*

shows *set vs* $\subseteq$ *carrier-vec n*
⟨*proof*⟩

definition *outer-sum* :: *nat* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.mat* **where**
outer-sum n vs = *mat-of-cols n vs* ∗ ((*mat-of-cols n vs*)†)

lemma *outer-sum-dim-row* [*simp*]: *dim-row* (*outer-sum n vs*) = *n*
⟨*proof*⟩

lemma *outer-sum-dim-col* [*simp*]: *dim-col* (*outer-sum n vs*) = *n*
⟨*proof*⟩

$P = VV^*$ is Hermitian, and orthonormality of the columns is $V^*V = I_k$, entry by entry $(V^*V)_{ij} = \langle v_i | v_j \rangle = \delta_{ij}$.

lemma *outer-sum-hermitian*:
shows (*outer-sum n vs*)† = *outer-sum n vs*
⟨*proof*⟩

lemma *cols-mat-dagger-left*:
assumes *orth*: *orthonormal-list n vs*
shows (*mat-of-cols n vs*)† ∗ (*mat-of-cols n vs*) = 1_m (*length vs*)
⟨*proof*⟩

lemma *cols-mat-dagger-right*:
assumes *orth*: *orthonormal-list n vs*
and *full*: *length vs* = *n*
shows (*mat-of-cols n vs*) ∗ ((*mat-of-cols n vs*)†) = 1_m *n*
⟨*proof*⟩

For a full basis ($k = n$) V is square, so $V^*V = I$ forces $VV^* = I$ as well. Entrywise this is the completeness relation

$$\sum_l (v_l)_i \, \overline{(v_l)_j} = \delta_{ij}.$$

lemma *completeness-relation*:
assumes *orth*: *orthonormal-list n vs*
and *full*: *length vs* = *n*
and *i-lt*: *i* < *n* **and** *j-lt*: *j* < *n*
shows (∑ *l*<*n*. (*vs* ! *l*) $ *i* ∗ *cnj* ((*vs* ! *l*) $ *j*)) = (*if i* = *j then 1 else 0*)
⟨*proof*⟩

lemma *outer-sum-index*:
assumes *orth*: *orthonormal-list n vs*
and *i-lt*: *i* < *n* **and** *j-lt*: *j* < *n*
shows (*outer-sum n vs*) $$ (*i*,*j*) =
(∑ *l* < *length vs*. (*vs* ! *l*) $ *i* ∗ *cnj* ((*vs* ! *l*) $ *j*))
⟨*proof*⟩

P acts by $Px = \sum_l \langle v_l | x \rangle \, v_l$, and $P^2 = V(V^*V)V^* = VV^* = P$.

lemma *outer-sum-apply*:
assumes *orth*: *orthonormal-list n vs*
and *xdim*: *dim-vec x* = *n*
and *i-lt*: *i* < *n*
shows (*outer-sum n vs* ∗$_v$ *x*) $ *i* = (∑ *l* < *length vs*. ⟨*vs* ! *l* | *x*⟩ ∗ (*vs* ! *l*) $ *i*)
⟨*proof*⟩

lemma *outer-sum-idempotent*:
assumes *orth*: *orthonormal-list n vs*

shows *outer-sum n vs* ∗ *outer-sum n vs* = *outer-sum n vs*
⟨*proof*⟩

For a full basis $P = I$, so each vector is its own expansion $v = \sum_l \langle v_l|v\rangle\, v_l$; taking $\langle v|\cdot\rangle$ gives Parseval, $\|v\|^2 = \sum_l |\langle v_l|v\rangle|^2$.

lemma *parseval*:
assumes *orth*: *orthonormal-list n vs*
and *full*: *length vs* = *n*
and *vdim*: *dim-vec v* = *n*
and *i-lt*: *i* < *n*
shows *v* $ *i* = (∑ *l*<*n*. ⟨*vs* ! *l* | *v*⟩ ∗ (*vs* ! *l*) $ *i*)
⟨*proof*⟩

corollary *parseval-norm*:
assumes *orth*: *orthonormal-list n vs*
and *full*: *length vs* = *n*
and *vdim*: *dim-vec v* = *n*
shows ‖*v*‖² = (∑ *l*<*n*. (*cmod* (⟨*vs* ! *l* | *v*⟩))²)
⟨*proof*⟩

The residual $x - Px$ is orthogonal to every v_l, because $\langle v_l|Px\rangle = \langle v_l|x\rangle$; hence the orthogonal decomposition $x = Px + (x - Px)$.

lemma *inner-prod-outer-sum*:
assumes *orth*: *orthonormal-list n vs*
and *xdim*: *dim-vec x* = *n*
and *l-lt*: *l* < *length vs*
shows ⟨*vs* ! *l* | *outer-sum n vs* ∗ᵥ *x*⟩ = ⟨*vs* ! *l* | *x*⟩
⟨*proof*⟩

lemma *residual-orthogonal*:
assumes *orth*: *orthonormal-list n vs*
and *xdim*: *dim-vec x* = *n*
and *l-lt*: *l* < *length vs*
shows ⟨*vs* ! *l* | *x* − *outer-sum n vs* ∗ᵥ *x*⟩ = *0*
⟨*proof*⟩

corollary *orthogonal-decomposition*:
assumes *xdim*: *dim-vec x* = *n*
shows *x* = *outer-sum n vs* ∗ᵥ *x* + (*x* − *outer-sum n vs* ∗ᵥ *x*)
⟨*proof*⟩

Uniqueness: if $x = v + w$ with $Pv = v$ and $w \perp v_l$ for all l, then $v = Px$ and $w = x - Px$.

lemma *outer-sum-orthogonal-zero*:
assumes *orth*: *orthonormal-list n vs*
and *wdim*: *dim-vec w* = *n*
and *ortho*: ∀ *l* < *length vs*. ⟨*vs* ! *l* | *w*⟩ = *0*
shows *outer-sum n vs* ∗ᵥ *w* = *0*ᵥ *n*
⟨*proof*⟩

lemma *outer-sum-add*:
assumes *vdim*: *dim-vec v* = *n* **and** *wdim*: *dim-vec w* = *n*
shows *outer-sum n vs* ∗ᵥ (*v* + *w*) =
outer-sum n vs ∗ᵥ *v* + *outer-sum n vs* ∗ᵥ *w*
⟨*proof*⟩

lemma *orthogonal-decomposition-unique*:
assumes *orth*: *orthonormal-list n vs*

and *xdim*: *dim-vec x = n*
and *vdim*: *dim-vec v = n*
and *wdim*: *dim-vec w = n*
and *decomp*: *x = v + w*
and *v-fixed*: *outer-sum n vs ∗v v = v*
and *w-ortho*: *∀ l < length vs. ⟨vs ! l | w⟩ = 0*
shows *v = outer-sum n vs ∗v x* **and** *w = x − outer-sum n vs ∗v x*
⟨*proof*⟩

lemma *Tr-one-mat*:
Tr (1m n :: complex Matrix.mat) = of-nat n
⟨*proof*⟩

4.4 The projection does not depend on the orthonormal basis

definition *col-span :: nat ⇒ complex Matrix.vec list ⇒ complex Matrix.vec set* **where**
col-span n vs = {mat-of-cols n vs ∗v c | c. dim-vec c = length vs}

lemma *outer-sum-fixes-col-span*:
assumes *orth*: *orthonormal-list n vs* **and** *w*: *w ∈ col-span n vs*
shows *outer-sum n vs ∗v w = w*
⟨*proof*⟩

lemma *mem-col-span-self*:
assumes *∀ v ∈ set vs. dim-vec v = n* **and** *j*: *j < length vs*
shows *vs ! j ∈ col-span n vs*
⟨*proof*⟩

lemma *outer-sum-absorbs*:
assumes *ov*: *orthonormal-list n vs* **and** *ow*: *orthonormal-list n ws*
and *sub*: *set ws ⊆ col-span n vs*
shows *outer-sum n vs ∗ outer-sum n ws = outer-sum n ws*
⟨*proof*⟩

D2-8: two orthonormal bases of the same subspace give the same projection.

theorem *outer-sum-basis-independent*:
assumes *ov*: *orthonormal-list n vs* **and** *ow*: *orthonormal-list n ws*
and *span*: *col-span n vs = col-span n ws*
shows *outer-sum n vs = outer-sum n ws*
⟨*proof*⟩

5 Day 3 (31 August 2011): unitary matrices, Householder unitaries, Schur's theorem

5.1 Unitary matrices

Definition (notes). $U \in M_n$ is *unitary* if $U^*U = I$.

Theorem (notes). For $U \in M_n$ the following are equivalent: (a) U is unitary; (b) U is invertible and $U^{-1} = U^*$; (c) $UU^* = I$; (d) U^* is unitary; (e) the columns of U are orthonormal; (f) the rows of U are orthonormal; (g) $\|Ux\| = \|x\|$ for all x (isometry); (h) $\langle Ux|Uy\rangle = \langle x|y\rangle$ for all x, y.

The library predicate *unitary U* bundles (a) and (c). The lemmas below give (a)⇒(b), (d), (e), (f), (g), (h); (e)⇒(a) is *orthonormal-mat-unitary*, (g)⇒(a) and (h)⇒(a) are *unitary-of-isometry*

and *unitary-of-inner-prod-preserving* (proved on Day 4, after the polarisation lemma); (d) is self-dual and (f) follows from (e) for U^*.

(b) U is invertible with inverse U^*:

lemma *unitary-invertible*:
  assumes *unitary U*
  shows *invertible-mat U*
⟨*proof*⟩

(d) U^* is unitary:

(e), (f) the columns and the rows of U are orthonormal:

lemma *unitary-cols-orthonormal*:
  assumes *U*: *unitary U*
    and *i*: *i* < *dim-col U*
    and *j*: *j* < *dim-col U*
  shows ⟨*Matrix.col U i* | *Matrix.col U j*⟩ = (*if i* = *j then 1 else 0* :: *complex*)
  ⟨*proof*⟩

lemma *unitary-rows-orthonormal*:
  assumes *U*: *unitary U*
    and *i*: *i* < *dim-row U*
    and *j*: *j* < *dim-row U*
  shows ⟨*Matrix.row U i* | *Matrix.row U j*⟩ = (*if i* = *j then 1 else 0* :: *complex*)
  ⟨*proof*⟩

(g), (h) U is an isometry and preserves inner products:

lemma *unitary-isometry*:
  assumes *unitary U* **and** *dim-vec v* = *dim-col U*
  shows ‖*U* ∗*v v*‖ = ‖*v*‖
⟨*proof*⟩

lemma *unitary-preserves-inner-prod*:
  assumes *U*: *unitary U*
    and *u*: *dim-vec u* = *dim-col U*
    and *v*: *dim-vec v* = *dim-col U*
  shows ⟨*U* ∗*v u* | *U* ∗*v v*⟩ = ⟨*u* | *v*⟩
  ⟨*proof*⟩

Conjugation by a unitary is multiplicative.

lemma *unitary-conj-mult*:
  fixes *U X Y* :: *complex Matrix.mat*
  assumes *U*: *U* ∈ *carrier-mat k k* **and** *Uu*: *unitary U* **and** *X*: *X* ∈ *carrier-mat k k* **and** *Y*: *Y* ∈ *carrier-mat k k*
  shows (*U* ∗ *X* ∗ (*U*†)) ∗ (*U* ∗ *Y* ∗ (*U*†)) = *U* ∗ (*X* ∗ *Y*) ∗ (*U*†)
⟨*proof*⟩

5.2 Householder unitaries

Definition (notes). For $w \in \mathbb{C}^n$ with $\|w\| = 1$ let $P_w = |w\rangle\langle w|$, the projection onto $\mathrm{span}\{w\}$, and $U_w = I - 2P_w$. Then $U_w^* U_w = (I - 2P_w)^2 = I - 4P_w + 4P_w^2 = I$, so U_w is unitary: the *Householder unitary* of w.

Remark. Geometrically U_w is the reflection through the hyperplane $\{w\}^\perp$.

Formally *householder w* is defined entrywise as $(\delta_{ij} - 2w_i\overline{w_j})$ and shown to equal $I - 2|w\rangle\langle w|$.

lemma *smult-mat-mult-vec*:

fixes A :: *complex Matrix.mat* **and** x :: *complex Matrix.vec*
assumes A: $A \in$ *carrier-mat nr nc*
and x: $x \in$ *carrier-vec nc*
shows $(a \cdot_m A) *_v x = a \cdot_v (A *_v x)$
⟨*proof*⟩

lemma *dagger-of-smult*:
fixes γ :: *complex*
and U :: *complex Matrix.mat*
shows $(\gamma \cdot_m U)^\dagger = (cnj\ \gamma) \cdot_m (U^\dagger)$
⟨*proof*⟩

Scaling an operator by c scales ABA^* and A^*A by $|c|^2$.

lemma *smult-sandwich*:
fixes A :: *complex Matrix.mat*
assumes A: $A \in$ *carrier-mat d n* **and** B: $B \in$ *carrier-mat n n*
shows $(c \cdot_m A) * B * ((c \cdot_m A)^\dagger) = (c * cnj\ c) \cdot_m (A * B * (A^\dagger))$
⟨*proof*⟩

lemma *smult-dagger-mult*:
fixes A :: *complex Matrix.mat*
assumes A: $A \in$ *carrier-mat d n*
shows $((c \cdot_m A)^\dagger) * (c \cdot_m A) = (cnj\ c * c) \cdot_m ((A^\dagger) * A)$
⟨*proof*⟩

definition *householder* :: *complex Matrix.vec* ⇒ *complex Matrix.mat* **where**
householder w ≡
Matrix.mat (*dim-vec w*) (*dim-vec w*)
(λ(*i*,*j*). (*if i* = *j then 1 else 0*) − 2 ∗ (*w* $ *i*) ∗ *cnj* (*w* $ *j*))

notation *householder* (U_H)

lemma *householder-index*:
assumes $i <$ *dim-vec w* **and** $j <$ *dim-vec w*
shows (*householder w*) $$ (*i*,*j*) =
(*if i* = *j then 1 else 0*) − 2 ∗ (*w* $ *i*) ∗ *cnj* (*w* $ *j*)
⟨*proof*⟩

lemma *householder-outer-prod*:
householder w = 1_m (*dim-vec w*) − 2 $\cdot_m$ (*w* ⊘ *w*)
⟨*proof*⟩

For a unit vector w, $U_H = I - 2|w\rangle\langle w|$ is the reflection in the hyperplane $w^\perp$: $U_H x = x - 2\langle w|x\rangle\, w$ fixes $w^\perp$ and sends w to $-w$. It is Hermitian, and

$$U_H^2 = I - 4|w\rangle\langle w| + 4\,|w\rangle\langle w||w\rangle\langle w| = I - 4|w\rangle\langle w| + 4\langle w|w\rangle|w\rangle\langle w| = I,$$

so it is unitary.

lemma *householder-hermitian*:
shows (*householder w*)† = *householder w*
⟨*proof*⟩

lemma *householder-unitary*:
assumes *norm-w*: $\|w\| =$ (*1* :: *real*)
shows *unitary* (*householder w*)
⟨*proof*⟩

lemma *householder-mult-vec-index*:

fixes $w\ x$ :: *complex Matrix.vec*
assumes *xdim*: *dim-vec* x = *dim-vec* w
and i: $i <$ *dim-vec* w
shows (*householder* $w *_v x$) $ i =
$x\ \$\ i - 2 * (w\ \$\ i) * \langle w \mid x \rangle$
⟨*proof*⟩

lemma *householder-mult-vec*:
fixes $w\ x$ :: *complex Matrix.vec*
assumes *xdim*: *dim-vec* x = *dim-vec* w
shows *householder* $w *_v x$ =
$x - (2 * \langle w \mid x \rangle) \cdot_v w$
⟨*proof*⟩

Lemma (notes) (Schur)**.** Given $x, y \in \mathbb{C}^n$ with $\|x\| = \|y\|$ there are $w \in \mathbb{C}^n$ with $\|w\| = 1$ and θ such that $U_w x = e^{i\theta} y$.

The formal statement writes $e^{i\theta}$ as *Complex* (*cos* ϑ) (*sin* ϑ); the case $x = 0$ is treated separately.

lemma *schur-lemma-zero-case*:
fixes $x\ y$ :: *complex Matrix.vec*
assumes *n-pos*: $n > 0$
and *xdim*: *dim-vec* $x = n$
and *ydim*: *dim-vec* $y = n$
and *norm-eq*: $\|x\| = \|y\|$
and *x-zero-norm*: $\|x\| = 0$
shows $\exists\, w$ :: *complex Matrix.vec*. $\exists\, \vartheta$ :: *real*.
dim-vec $w = n \wedge \|w\| = 1 \wedge$
householder $w *_v x$ = *Complex* (*cos* ϑ) (*sin* ϑ) $\cdot_v y$
⟨*proof*⟩

lemma *schur-lemma*:
fixes $x\ y$ :: *complex Matrix.vec*
assumes *n-pos*: $n > 0$
and *xdim*: *dim-vec* $x = n$
and *ydim*: *dim-vec* $y = n$
and *norm-eq*: $\|x\| = \|y\|$
shows $\exists\, w$ :: *complex Matrix.vec*. $\exists\, \vartheta$ :: *real*.
dim-vec $w = n \wedge \|w\| = 1 \wedge$
householder $w *_v x$ = *Complex* (*cos* ϑ) (*sin* ϑ) $\cdot_v y$
⟨*proof*⟩

Theorem (notes) (Schur)**.** For every $A \in M_n$ there is a unitary U such that $U^* A U$ is upper triangular.

The notes prove this by induction, peeling off one eigenvector with the Householder unitary of Schur's lemma and recursing on the $(n-1) \times (n-1)$ block. The formal proof follows the same route: *schur-triangular-similar* is the similarity version available from the Jordan-Normal-Form library, and the block-matrix lemmas that follow assemble the unitary step by step.

lemma *schur-triangular-similar*:
fixes A :: *complex Matrix.mat*
assumes *sq*: $A \in$ *carrier-mat* $n\ n$
shows $\exists\, T$ :: *complex Matrix.mat*.
$T \in$ *carrier-mat* $n\ n \wedge$
upper-triangular $T \wedge$
similar-mat $A\ T$
⟨*proof*⟩

The induction works with 2×2 block matrices, whose adjoints and products are computed blockwise:

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix}^* = \begin{pmatrix} A^* & C^* \\ B^* & D^* \end{pmatrix}, \qquad \begin{pmatrix} A & 0 \\ 0 & D \end{pmatrix} \begin{pmatrix} B & 0 \\ 0 & E \end{pmatrix} = \begin{pmatrix} AB & 0 \\ 0 & DE \end{pmatrix};$$

in particular $U \oplus V$ is unitary when U and V are.

lemma *dagger-four-block-mat-complex*:
  assumes *A*: $A \in$ *carrier-mat nr1 nc1*
    and *B*: $B \in$ *carrier-mat nr1 nc2*
    and *C*: $C \in$ *carrier-mat nr2 nc1*
    and *D*: $D \in$ *carrier-mat nr2 nc2*
  shows $($*four-block-mat A B C D*$)^\dagger =$
      four-block-mat $(A^\dagger)$ $(C^\dagger)$ $(B^\dagger)$ $(D^\dagger)$
⟨*proof*⟩

lemma *dagger-zero-mat*[*simp*]:
  $(0_m\ m\ n ::$ *complex Matrix.mat*$)^\dagger = 0_m\ n\ m$
  ⟨*proof*⟩

lemma *block-diag-mult*:
  fixes *A B D E* :: *complex Matrix.mat*
  assumes *A*: $A \in$ *carrier-mat n n*
    and *B*: $B \in$ *carrier-mat n n*
    and *D*: $D \in$ *carrier-mat m m*
    and *E*: $E \in$ *carrier-mat m m*
  shows *four-block-mat A* $(0_m\ n\ m)$ $(0_m\ m\ n)$ *D* $*$
      four-block-mat B $(0_m\ n\ m)$ $(0_m\ m\ n)$ *E* $=$
      four-block-mat $(A * B)$ $(0_m\ n\ m)$ $(0_m\ m\ n)$ $(D * E)$
⟨*proof*⟩

lemma *block-diag-unitary*:
  fixes *U V* :: *complex Matrix.mat*
  assumes *U*: $U \in$ *carrier-mat n n*
    and *V*: $V \in$ *carrier-mat m m*
    and *uU*: *unitary U*
    and *uV*: *unitary V*
  shows *unitary* (*four-block-mat U* $(0_m\ n\ m)$ $(0_m\ m\ n)$ *V*)
⟨*proof*⟩

The inductive step. Over $\mathbb{C}$ the characteristic polynomial has a root, so A has an eigenvector, normalised to a unit vector v. A unitary W with first column v gives

$$W^* A W = \begin{pmatrix} \lambda & * \\ 0 & A' \end{pmatrix},$$

and if $V^* A' V = T'$ is upper triangular then so is $U^* A U$ for $U = W(1 \oplus V)$.

lemma *complex-matrix-has-eigenvector*:
  fixes *A* :: *complex Matrix.mat*
  assumes *sq*: $A \in$ *carrier-mat n n*
    and *n-pos*: $n > 0$
  shows $\exists\, e\ v.$ *eigenvector A v e*
⟨*proof*⟩

lemma *orthonormal-mat-unitary*:
  assumes *orth*: *orthonormal-list n vs*
    and *full*: *length vs* $= n$

shows *unitary* (*mat-of-cols n vs*)
⟨*proof*⟩

lemma *eigenvec-normalize*:
fixes *A* :: *complex Matrix.mat* **and** *v* :: *complex Matrix.vec*
assumes *sq*: *A* ∈ *carrier-mat n n*
and *vdim*: *v* ∈ *carrier-vec n*
and *vne*: *v* ≠ 0_v *n*
and *ev*: *A* $*_v$ *v* = *e* $\cdot_v$ *v*
defines v_1 ≡ (*1* / ‖*v*‖) $\cdot_v$ *v*
shows *A* $*_v$ v_1 = *e* $\cdot_v$ v_1 **and** ‖v_1‖ = *1*
⟨*proof*⟩

definition *vec-normalize* :: *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
vec-normalize v = (*if v* = 0_v (*dim-vec v*) *then v else* (*1* / ‖*v*‖) $\cdot_v$ *v*)

lemma *normalized-vec-norm*:
assumes *vdim*: *v* ∈ *carrier-vec n*
and *vne*: *v* ≠ 0_v *n*
shows ‖*vec-normalize v*‖ = *1*
⟨*proof*⟩

lemma *four-block-mat-mult-left-block-diag*:
fixes *U* :: *complex Matrix.mat*
and *A11* :: *complex Matrix.mat*
and *A12* :: *complex Matrix.mat*
and *A21* :: *complex Matrix.mat*
and *A22* :: *complex Matrix.mat*
assumes *U*: *U* ∈ *carrier-mat n n*
and *A11*: *A11* ∈ *carrier-mat 1 1*
and *A12*: *A12* ∈ *carrier-mat 1 n*
and *A21*: *A21* ∈ *carrier-mat n 1*
and *A22*: *A22* ∈ *carrier-mat n n*
shows
(*four-block-mat* (1_m *1* :: *complex Matrix.mat*)
(0_m *1 n* :: *complex Matrix.mat*)
(0_m *n 1* :: *complex Matrix.mat*)
U)
*
(*four-block-mat A11 A12 A21 A22*)
=
four-block-mat A11 A12 (*U* * *A21*) (*U* * *A22*)
⟨*proof*⟩

lemma *cscalar-prod-is-inner-prod-swap*:
fixes *u v* :: *complex Matrix.vec*
assumes *u* ∈ *carrier-vec n* **and** *v* ∈ *carrier-vec n*
shows *u* ·*c* *v* = ⟨*v*|*u*⟩
⟨*proof*⟩

lemma *vec-normalize-dim*[*simp*]:
dim-vec (*vec-normalize v*) = *dim-vec v*
⟨*proof*⟩

lemma *vec-normalize-carrier*[*simp*]:
assumes *v* ∈ *carrier-vec n*
shows *vec-normalize v* ∈ *carrier-vec n*
⟨*proof*⟩

lemma *vec-normalize-unit*:
assumes *vdim*: $v \in$ *carrier-vec* n **and** *vnorm*: $\|v\| = 1$
shows *vec-normalize* $v = v$
⟨*proof*⟩

Any unit vector extends to an orthonormal basis (needed for the induction).

lemma *unit-vec-extends-to-onb*:
fixes $v ::$ *complex Matrix.vec*
assumes *vdim*: $v \in$ *carrier-vec* n
and *vnorm*: $\|v\| = 1$
shows $\exists$ *vs*. *orthonormal-list* n *vs* $\wedge$ *length vs* $= n \wedge$ *vs* ! 0 $= v$
⟨*proof*⟩

lemma *schur-induct*:
$\forall A ::$ *complex Matrix.mat*.
$A \in$ *carrier-mat* n $n \longrightarrow$
$(\exists U ::$ *complex Matrix.mat*. $\exists T ::$ *complex Matrix.mat*.
$U \in$ *carrier-mat* n $n \wedge$
$T \in$ *carrier-mat* n $n \wedge$
unitary $U \wedge$
upper-triangular $T \wedge$
$(((U^\dagger) * A) * U) = T)$
⟨*proof*⟩

theorem *schur-theorem*:
fixes $A ::$ *complex Matrix.mat*
assumes *sq*: $A \in$ *carrier-mat* n n
shows $\exists U ::$ *complex Matrix.mat*. $\exists T ::$ *complex Matrix.mat*.
$U \in$ *carrier-mat* n $n \wedge$
$T \in$ *carrier-mat* n $n \wedge$
unitary $U \wedge$
upper-triangular $T \wedge$
$(((U^\dagger) * A) * U) = T$
⟨*proof*⟩

Remark. The characteristic polynomial of A is $p_A(t) = \det(tI - A)$, a polynomial of degree n.

Corollary (notes). $\mathrm{Tr}(A) = \lambda_1 + \cdots + \lambda_n$, where the λ_j are the eigenvalues of A.

Formally the eigenvalues are listed as the roots of the characteristic polynomial, $p_A(t) = \prod_j (t - \lambda_j)$, read off the diagonal of the Schur form.

corollary *trace-eq-sum-eigenvalues*:
fixes $A ::$ *complex Matrix.mat*
assumes *sq*: $A \in$ *carrier-mat* n n
obtains *eigs* :: *complex list*
where *length eigs* $= n$
and *Tr* $A =$ *sum-list eigs*
and *char-poly* $A = (\prod e \leftarrow$ *eigs*. [:$-e$, 1:])
⟨*proof*⟩

6 Day 4 (2 September 2011): Hermitian matrices; positive semidefinite and positive definite matrices

6.1 Hermitian matrices

Definition (notes). $H \in M_n$ is *Hermitian* (self-adjoint) if $H = H^*$.

Theorem (notes). The following are equivalent: (a) H is Hermitian; (b) there is a unitary U with $U^*HU = D$ diagonal with real entries; (c) $\mathbb{C}^n$ has an orthonormal basis of eigenvectors of H with real eigenvalues; (d) $\langle x|Hx\rangle \in \mathbb{R}$ for all $x \in \mathbb{C}^n$.

Formally *hermitian H* is $H^\dagger = H$; (a)⇒(b) is the spectral theorem *hermitian-spectral-decomp* below, (a)⇔(d) is *hermitian-iff-real-quad-form*, and (c) is read off from (b): the columns of U are an orthonormal eigenbasis. The proof of (d)⇒(a) needs the following lemma, which the notes note fails over $\mathbb{R}$ (take $T = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$).

definition *hermitian* :: *complex Matrix.mat* ⇒ *bool* **where**
  hermitian A ≡ $A^\dagger = A$

lemma *hermitian-square*:
  assumes *hermitian A*
  shows *dim-row A* = *dim-col A*
  ⟨*proof*⟩

lemma *hermitian-index*:
  assumes *hermitian A*
    and $i <$ *dim-row A* **and** $j <$ *dim-col A*
  shows *A* $$ (*i*,*j*) = *cnj* (*A* $$ (*j*,*i*))
⟨*proof*⟩

A matrix is determined by its quadratic form. In coordinates $\langle u|Tv\rangle = \sum_{k,l} \overline{u_k}\, T_{kl}\, v_l$, so $\langle e_p|Te_q\rangle = T_{pq}$; and with $q(z) = \langle z|Tz\rangle$ the polarization identity

$$4\langle x|Ty\rangle = \sum_{k=0}^{3} i^{-k}\, q(x + i^k y)$$

recovers every entry from the values of q. This is how q real-valued forces $T = T^*$.

lemma *inner-T-double-sum*:
  fixes *T* :: *complex Matrix.mat* **and** *u v* :: *complex Matrix.vec*
  assumes *T-carr*: *T* ∈ *carrier-mat n n*
    and *udim*: *dim-vec u* = *n* **and** *vdim*: *dim-vec v* = *n*
  shows ⟨*u* | *T* ∗$_v$ *v*⟩ =
      (∑ *k* = *0*..<*n*. ∑ *l* = *0*..<*n*. *cnj* (*u* $ *k*) ∗ *T* $$ (*k*, *l*) ∗ *v* $ *l*)
⟨*proof*⟩

lemma *inner-unit-T-unit*:
  fixes *T* :: *complex Matrix.mat*
  assumes *T-carr*: *T* ∈ *carrier-mat n n*
    and *p*: *p* < *n* **and** *q*: *q* < *n*
  shows ⟨*unit-vec n p* | *T* ∗$_v$ *unit-vec n q*⟩ = *T* $$ (*p*, *q*)
⟨*proof*⟩

lemma *sum-indicator-T-indicator*:
  fixes *T* :: *complex Matrix.mat*
  assumes *p*: *p* < *n* **and** *q*: *q* < *n*
  shows (∑ *k* = *0*..<*n*. ∑ *l* = *0*..<*n*.
        (*if k* = *p then* (*1* :: *complex*) *else 0*) ∗ *T* $$ (*k*, *l*) ∗
        (*if l* = *q then* (*1* :: *complex*) *else 0*))
    = *T* $$ (*p*, *q*)
⟨*proof*⟩

lemma *polarization-helper*:
  fixes *T* :: *complex Matrix.mat* **and** *a b c d* :: *complex*

assumes *sq*: $T \in$ *carrier-mat n n*
and *i*: $i < n$ **and** *j*: $j < n$
shows $\langle a \cdot_v$ *unit-vec n i* $+ b \cdot_v$ *unit-vec n j*
$\mid T *_v (c \cdot_v$ *unit-vec n i* $+ d \cdot_v$ *unit-vec n j*$)\rangle$
$=$ *cnj a* $*$ *c* $*$ *T $$ (i, i)* $+$ *cnj a* $*$ *d* $*$ *T $$ (i, j)*
$+$ *cnj b* $*$ *c* $*$ *T $$ (j, i)* $+$ *cnj b* $*$ *d* $*$ *T $$ (j, j)*
⟨*proof*⟩

Lemma (notes). For $T \in M_n$, if $\langle x|Tx\rangle = 0$ for all $x \in \mathbb{C}^n$, then $T = 0$.

lemma *quadratic-form-zero*:
fixes *T* :: *complex Matrix.mat*
assumes *sq*: $T \in$ *carrier-mat n n*
and *zero*: $\forall x ::$ *complex Matrix.vec. dim-vec x* $= n \longrightarrow \langle x \mid T *_v x\rangle = 0$
shows $T = 0_m$ *n n*
⟨*proof*⟩

lemma *mult-mat-vec-index*:
fixes *A* :: *'a*::{*comm-ring-1*} *Matrix.mat*
and *x* :: *'a Matrix.vec*
assumes *A*: $A \in$ *carrier-mat m n*
and *x*: $x \in$ *carrier-vec n*
and *j*: $j < m$
shows $(A *_v x)$ $ $j = (\sum i = 0..<n.$ *A $$ (j, i)* $*$ *x $ i*$)$
⟨*proof*⟩

The adjoint moves across the inner product, $\langle x|My\rangle = \langle M^\dagger x|y\rangle$: the defining property of $M^\dagger$.

lemma *inner-prod-adjoint-swap*:
fixes *M* :: *complex Matrix.mat*
and *x y* :: *complex Matrix.vec*
assumes *M*: $M \in$ *carrier-mat n m*
and *x*: *dim-vec x* $= n$
and *y*: *dim-vec y* $= m$
shows $\langle x \mid M *_v y\rangle = \langle M^\dagger *_v x \mid y\rangle$
⟨*proof*⟩

Sums and real multiples of Hermitian matrices are Hermitian; the identity is Hermitian.

lemma *hermitian-plus*:
fixes *A B* :: *complex Matrix.mat*
assumes *Aherm*: *hermitian A* **and** *Bherm*: *hermitian B*
and *Ac*: $A \in$ *carrier-mat k k* **and** *Bc*: $B \in$ *carrier-mat k k*
shows *hermitian* $(A + B)$
⟨*proof*⟩

lemma *hermitian-identity*:
hermitian $(1_m$ *k* :: *complex Matrix.mat*$)$
⟨*proof*⟩

lemma *hermitian-real-scale*:
fixes *A* :: *complex Matrix.mat*
assumes *hermitian A A* $\in$ *carrier-mat k k*
shows *hermitian* $($*complex-of-real d* $\cdot_m$ *A*$)$
⟨*proof*⟩

The converses (g)⇒(a) and (h)⇒(a) of the Day 3 theorem on unitaries, postponed until the lemma $\langle x|Tx\rangle = 0\ \forall x \Rightarrow T = 0$ was available: a square matrix that preserves lengths, or inner products, is unitary (the library proves the length-preserving case in its own matrix notation as *length-preserving-is-unitary*).

lemma *unitary-of-isometry*:
fixes U :: *complex Matrix.mat*
assumes U: $U \in$ *carrier-mat n n*
and *iso*: $\bigwedge v$ :: *complex Matrix.vec*. *dim-vec* $v = n \Longrightarrow \|U *_v v\| = \|v\|$
shows *unitary U*
⟨*proof*⟩

lemma *unitary-of-inner-prod-preserving*:
fixes U :: *complex Matrix.mat*
assumes U: $U \in$ *carrier-mat n n*
and *pres*: $\bigwedge u\ v$ :: *complex Matrix.vec*. *dim-vec* $u = n \Longrightarrow$ *dim-vec* $v = n \Longrightarrow$
$\langle U *_v u \mid U *_v v\rangle = \langle u \mid v\rangle$
shows *unitary U*
⟨*proof*⟩

lemma *hermitian-iff-real-quad-form*:
assumes *sq*: $A \in$ *carrier-mat n n*
shows *hermitian* $A \longleftrightarrow (\forall x$ :: *complex Matrix.vec*. *dim-vec* $x = n \longrightarrow \langle x \mid A *_v x\rangle \in \mathbb{R})$
⟨*proof*⟩

6.2 The spectral theorem for Hermitian matrices

(a)⇒(b) of the theorem above: by Schur, $U^*HU = T$ is upper triangular, and $T^* = (U^*HU)^* = T$ forces T to be diagonal with real entries. *diag-mat-of-real D* is the complex diagonal matrix with the real list D on its diagonal.

definition *diag-mat-of-real* :: *real list* ⇒ *complex Matrix.mat* **where**
diag-mat-of-real D = *mat-diag* (*length D*) (λ*i*. *complex-of-real* (*D* ! *i*))

lemma *dim-row-diag-mat-of-real*[*simp*]: *dim-row* (*diag-mat-of-real D*) = *length D*
⟨*proof*⟩

lemma *dim-col-diag-mat-of-real*[*simp*]: *dim-col* (*diag-mat-of-real D*) = *length D*
⟨*proof*⟩

lemma *diag-mat-of-real-carrier*[*simp*]:
diag-mat-of-real D ∈ *carrier-mat* (*length D*) (*length D*)
⟨*proof*⟩

lemma *diag-mat-of-real-index*[*simp*]:
assumes *i* < *length D* **and** *j* < *length D*
shows *diag-mat-of-real D* $$ (*i*, *j*) =
(*if i* = *j then complex-of-real* (*D* ! *i*) *else* 0)
⟨*proof*⟩

lemma *diag-mat-of-real-mult-vec-index*:
fixes *y* :: *complex Matrix.vec*
assumes *ydim*: *dim-vec y* = *length D*
and *i*: *i* < *length D*
shows (*diag-mat-of-real D* $*_v$ *y*) $ *i* = *complex-of-real* (*D* ! *i*) * *y* $ *i*
⟨*proof*⟩

For a real diagonal $D = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$, $\langle y|Dy\rangle = \sum_i \lambda_i |y_i|^2$, which is ≥ 0 when every $\lambda_i \geq 0$.

lemma *diag-mat-of-real-quad-Re-nonneg*:
fixes *y* :: *complex Matrix.vec*
assumes *ydim*: *dim-vec y* = *length D*
and *D-nonneg*: ⋀*i*. *i* < *length D* ⟹ 0 ≤ *D* ! *i*
shows 0 ≤ *Re* ⟨*y* | *diag-mat-of-real D* $*_v$ *y*⟩

⟨*proof*⟩

The spectral theorem. If $U^*HU = T$ is a Schur form of a Hermitian H, then $T^* = U^*H^*U = T$; an upper triangular matrix equal to its adjoint is diagonal with real diagonal. Hence

$$H = UDU^*, \qquad D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n),\ \lambda_i \in \mathbb{R},$$

and the columns of U are an orthonormal basis of eigenvectors, $Hu_i = \lambda_i u_i$.

theorem *hermitian-spectral-decomp*:
  fixes H :: *complex Matrix.mat*
  assumes *sq*: $H \in$ *carrier-mat n n*
    and *herm*: *hermitian H*
  shows $\exists$ (U :: *complex Matrix.mat*) (D :: *real list*).
      $U \in$ *carrier-mat n n* $\wedge$
      unitary U $\wedge$
      length D $= n \wedge$
      $(U^\dagger) * H * U =$ *diag-mat-of-real D*
⟨*proof*⟩

corollary *hermitian-real-eigenvalues*:
  fixes H :: *complex Matrix.mat*
  assumes *sq*: $H \in$ *carrier-mat n n*
    and *herm*: *hermitian H*
    and *eig*: $H *_v v = lam \cdot_v v$
    and *v-nonzero*: $v \neq 0_v\ n$
    and *vdim*: *dim-vec v* $= n$
  shows *lam* $\in \mathbb{R}$
⟨*proof*⟩

6.3 Positive semidefinite and positive definite matrices

Definition (notes). $P \in M_n$ is *positive semidefinite*, $P \geq 0$, if $\langle x|Px\rangle \geq 0$ for all $x \in \mathbb{C}^n$, and *positive definite*, $P > 0$, if $\langle x|Px\rangle > 0$ for all $x \neq 0$.

By part (d) of the previous theorem, $P \geq 0$ implies $P = P^*$. The formal predicates *psd* and *pd* carry Hermiticity as an explicit conjunct together with the real-part inequality; by *hermitian-iff-real-quad-form* this is equivalent to the notes' definition.

Theorem (notes). $P \geq 0$ if and only if $P = P^*$ and all eigenvalues of P are non-negative.

Corollary (notes). $P > 0$ if and only if $P = P^*$ and all eigenvalues of P are strictly positive.

Theorem (notes). $P \geq 0$ if and only if $P = \sum_{i=1}^m |v_i\rangle\langle v_i|$ for some vectors $v_1, \ldots, v_m$.

Key: if $P = \sum_i |v_i\rangle\langle v_i|$ then $\langle x|Px\rangle = \sum_i |\langle v_i|x\rangle|^2$.

Theorem (notes). Let $P = \sum_{i=1}^m |v_i\rangle\langle v_i| \in M_n$. Then $P > 0$ if and only if $\operatorname{span}\{v_1, \ldots, v_m\} = \mathbb{C}^n$.

The notes' form of the definition is *psd-iff-quad-form-nonneg*; the corollary on positive definite matrices is *pd-iff-pos-eigenvalues* (proved after the square root); the last theorem is formalised with "the v_i span $\mathbb{C}^n$" phrased as "no nonzero x is orthogonal to every v_i".

definition *psd* :: *complex Matrix.mat* $\Rightarrow$ *bool* **where**
  psd P $\equiv$ *hermitian P* $\wedge$
      ($\forall$ *x* :: *complex Matrix.vec*. *dim-vec x* $=$ *dim-row P* $\longrightarrow$
        Re ($\langle x \mid P *_v x\rangle$) $\geq$ *0*)

definition *pd* :: *complex Matrix.mat* ⇒ *bool* **where**
pd P ≡ *hermitian P* ∧
(∀ *x* :: *complex Matrix.vec*. *dim-vec x* = *dim-row P* ⟶
x ≠ 0_v (*dim-row P*) ⟶
Re (⟨*x* | *P* ∗$_v$ *x*⟩) > *0*)

notation *psd* (- ≥$_M$ *0*)
notation *pd* (- >$_M$ *0*)

lemma *psd-hermitian*: *psd P* ⟹ *hermitian P*
⟨*proof*⟩

The notes' definition, without the explicit Hermiticity conjunct: $P \geq 0$ iff every quadratic form $\langle x|Px\rangle$ is a non-negative real number.

lemma *psd-iff-quad-form-nonneg*:
fixes *P* :: *complex Matrix.mat*
assumes *sq*: *P* ∈ *carrier-mat n n*
shows *psd P* ⟷
(∀ *x* :: *complex Matrix.vec*. *dim-vec x* = *n* ⟶
⟨*x* | *P* ∗$_v$ *x*⟩ ∈ ℝ ∧ *0* ≤ *Re* (⟨*x* | *P* ∗$_v$ *x*⟩))
⟨*proof*⟩

lemma *psd-MdagM*:
assumes *M*: *M* ∈ *carrier-mat n m*
shows *psd* (*M*† ∗ *M*)
⟨*proof*⟩

Congruence preserves positivity: $P \geq 0$ implies $M^*PM \geq 0$, since $\langle x|M^*PMx\rangle = \langle Mx|P\,Mx\rangle$.

lemma *psd-conjugate*:
fixes *P* :: *complex Matrix.mat* **and** *M* :: *complex Matrix.mat*
assumes *P-c*: *P* ∈ *carrier-mat k k*
and *P-psd*: *psd P*
and *M-c*: *M* ∈ *carrier-mat k n*
shows *psd* (*M*† ∗ *P* ∗ *M*)
⟨*proof*⟩

$P \geq 0$ iff P is Hermitian with non-negative eigenvalues:

theorem *psd-iff-nonneg-eigenvalues*:
fixes *P* :: *complex Matrix.mat*
assumes *sq*: *P* ∈ *carrier-mat n n*
shows *psd P* ⟷
(*hermitian P* ∧
(∀ *mu* :: *complex*. ∀ *v* :: *complex Matrix.vec*.
v ≠ 0_v *n* ∧ *dim-vec v* = *n* ∧ *P* ∗$_v$ *v* = *mu* ·$_v$ *v* ⟶ *Re mu* ≥ *0*))
⟨*proof*⟩

$P \geq 0$ iff $P = \sum_i |v_i\rangle\langle v_i|$. First the direction ⇐:

lemma *outer-sum-psd*:
assumes *len*: *length vs* = *k*
and *vdim*: ∀ *v* ∈ *set vs*. *dim-vec v* = *n*
shows *psd* (*outer-sum n vs*)
⟨*proof*⟩

lemma *hermitian-spectral-decomp-eq*:
assumes *Uc*: *U* ∈ *carrier-mat n n*
and *Pcar*: *P* ∈ *carrier-mat n n*
and *unit*: *unitary U*

and *spec*: $U^\dagger * P * U = \textit{diag-mat-of-real}\ D$
shows $P = U * \textit{diag-mat-of-real}\ D * (U^\dagger)$
⟨*proof*⟩

lemma *hermitian-spectral-decomp-right*:
assumes *Uc*: $U \in \textit{carrier-mat}\ n\ n$
and *Pcar*: $P \in \textit{carrier-mat}\ n\ n$
and *unit*: *unitary* U
and *spec*: $U^\dagger * P * U = \textit{diag-mat-of-real}\ D$
shows $P * U = U * \textit{diag-mat-of-real}\ D$
⟨*proof*⟩

The direction ⇒, from the spectral theorem: $P = \sum_j \lambda_j |u_j\rangle\langle u_j| = \sum_j |\lambda_j^{1/2} u_j\rangle\langle \lambda_j^{1/2} u_j|$.

The spectral theorem for positive matrices: $P = U\,\mathrm{diag}(D)\,U^\dagger$ with $D \geq 0$.

lemma *psd-spectral-decomp-nonneg*:
fixes P :: *complex Matrix.mat*
assumes *sq*: $P \in \textit{carrier-mat}\ n\ n$
and *pos*: *psd* P
shows $\exists (U :: \textit{complex Matrix.mat})\ (D :: \textit{real list}).$
$U \in \textit{carrier-mat}\ n\ n \wedge \textit{unitary}\ U \wedge \textit{length}\ D = n \wedge$
$(\forall\, i < n.\ 0 \leq D\ !\ i) \wedge$
$P = U * \textit{diag-mat-of-real}\ D * (U^\dagger)$
⟨*proof*⟩

theorem *psd-kraus-decomp*:
fixes P :: *complex Matrix.mat*
assumes *sq*: $P \in \textit{carrier-mat}\ n\ n$
and *pos*: *psd* P
shows $\exists\, vs :: \textit{complex Matrix.vec list}.$
$\textit{length}\ vs = n \wedge$
$(\forall\, v \in \textit{set}\ vs.\ \textit{dim-vec}\ v = n) \wedge$
$\textit{outer-sum}\ n\ vs = P$
⟨*proof*⟩

$P = \sum_i |v_i\rangle\langle v_i|$ is positive definite iff the v_i span $\mathbb{C}^n$:

Entries of $\sum_i |v_i\rangle\langle v_i|$ for an arbitrary list of vectors.

lemma *outer-sum-entry-sum-list*:
assumes *vs-dim*: $\forall\, v \in \textit{set}\ vs.\ \textit{dim-vec}\ v = n$
and *i-lt*: $i < n$ **and** *j-lt*: $j < n$
shows $\textit{outer-sum}\ n\ vs\ \$\$\ (i, j) =$
$\textit{sum-list}\ (\textit{map}\ (\lambda v.\ v\ \$\ i * \textit{cnj}\ (v\ \$\ j))\ vs)$
⟨*proof*⟩

lemma *mat-of-cols-dagger-mult-vec-index*:
assumes *xdim*: $\textit{dim-vec}\ x = n$
and *vs-dim*: $\forall\, v \in \textit{set}\ vs.\ \textit{dim-vec}\ v = n$
and *i*: $i < \textit{length}\ vs$
shows $(((\textit{mat-of-cols}\ n\ vs)^\dagger) *_v x)\ \$\ i = \langle vs\ !\ i \mid x\rangle$
⟨*proof*⟩

lemma *inner-prod-outer-sum-general*:
assumes *xdim*: $\textit{dim-vec}\ x = n$
and *vs-dim*: $\forall\, v \in \textit{set}\ vs.\ \textit{dim-vec}\ v = n$
shows $\langle x \mid \textit{outer-sum}\ n\ vs *_v x\rangle =$
$\langle((\textit{mat-of-cols}\ n\ vs)^\dagger) *_v x \mid$
$((\textit{mat-of-cols}\ n\ vs)^\dagger) *_v x\rangle$

⟨*proof*⟩

lemma *diag-mat-of-real-hermitian*:
  shows *hermitian* (*diag-mat-of-real D*)
  ⟨*proof*⟩

$U\,\mathrm{diag}(D)\,U^\dagger$ is positive whenever the real entries of D are non-negative.

lemma *psd-U-diag-Udag*:
  fixes *U* :: *complex Matrix.mat* **and** *D* :: *real list*
  assumes *Uc*: *U* ∈ *carrier-mat n n*
    and *Dlen*: *length D* = *n*
    and *Dnn*: ⋀*i*. *i* < *n* ⟹ *0* ≤ *D* ! *i*
  shows *psd* (*U* ∗ *diag-mat-of-real D* ∗ (*U*†))
⟨*proof*⟩

lemma *diag-mat-of-real-sqrt-square*:
  assumes *D-nonneg*: ⋀*i*. *i* < *length D* ⟹ *0* ≤ *D* ! *i*
  shows *diag-mat-of-real* (*map sqrt D*) ∗ *diag-mat-of-real* (*map sqrt D*) = *diag-mat-of-real D*
⟨*proof*⟩

theorem *pd-iff-spanning*:
  fixes *P* :: *complex Matrix.mat*
  assumes *sq*: *P* ∈ *carrier-mat n n*
    and *decomp*: *P* = *outer-sum n vs*
    and *vs-dim*: ∀ *v* ∈ *set vs*. *dim-vec v* = *n*
  shows *pd P* ⟷
      (∀ *x* :: *complex Matrix.vec*. *dim-vec x* = *n* ⟶ *x* ≠ *0*$_v$ *n* ⟶
        (∃ *v* ∈ *set vs*. ⟨*v* | *x*⟩ ≠ *0*))
⟨*proof*⟩

6.4 The positive square root

Remark (Day 8)**.** Given $P \geq 0$ with $U^*PU = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$, $\lambda_i \geq 0$, put $P^{1/2} = U\,\mathrm{diag}(\lambda_1^{1/2}, \ldots, \lambda_n^{1/2})U^*$. Then $P^{1/2} \geq 0$ and $(P^{1/2})^2 = P$.

psd-sqrt P is defined as a choice of such a square root; *psd-sqrt-exists* is the existence statement and *psd-sqrt-props* the defining properties.

definition *psd-sqrt* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
  psd-sqrt P ≡ (*SOME Q*. *Q* ∈ *carrier-mat* (*dim-row P*) (*dim-row P*) ∧ *psd Q* ∧ *Q* ∗ *Q* = *P*)

theorem *psd-sqrt-exists*:
  fixes *P* :: *complex Matrix.mat*
  assumes *sq*: *P* ∈ *carrier-mat n n*
    and *pos*: *psd P*
  shows ∃ *Q* :: *complex Matrix.mat*.
      Q ∈ *carrier-mat n n* ∧ *psd Q* ∧ *Q* ∗ *Q* = *P*
⟨*proof*⟩
lemma *psd-sqrt-props*:
  fixes *P* :: *complex Matrix.mat*
  assumes *sq*: *P* ∈ *carrier-mat n n*
    and *pos*: *psd P*
  shows *psd-sqrt P* ∈ *carrier-mat n n*
    and *psd* (*psd-sqrt P*)
    and *psd-sqrt P* ∗ *psd-sqrt P* = *P*
⟨*proof*⟩

The corollary of Day 4: $P > 0$ iff $P = P^*$ and all eigenvalues are strictly positive.

lemma *pd-iff-pos-eigenvalues*:
fixes P :: *complex Matrix.mat*
assumes *sq*: $P \in$ *carrier-mat n n*
shows *pd P* $\longleftrightarrow$
$($*hermitian P* $\wedge$
$(\forall$ *mu* :: *complex*. $\forall$ *v* :: *complex Matrix.vec*.
$v \neq 0_v\ n \wedge$ *dim-vec v* $= n \wedge P *_v v = mu \cdot_v v \longrightarrow 0 <$ *Re mu*$))$
⟨*proof*⟩

lemma *outer-sum-singleton-index*:
assumes *r*: $r < n$ **and** *c*: $c < n$
shows *outer-sum n* $[\psi]$ \$\$ $(r, c) = \psi$ \$ $r *$ *cnj* $(\psi$ \$ $c)$
⟨*proof*⟩

Carrier of *outer-sum k vs*.

lemma *outer-sum-carrier*:
outer-sum k vs $\in$ *carrier-mat k k*
⟨*proof*⟩

lemma *outer-sum-Nil*: *outer-sum n* $[] = 0_m\ n\ n$
⟨*proof*⟩

lemma *outer-sum-singleton-eq*:
assumes *vd*: *dim-vec v* $= n$
shows *outer-sum n* $[v] = v \oslash v$
⟨*proof*⟩

Cons-recursion for *outer-sum*: adds a rank-1 piece.

lemma *outer-sum-Cons-split*:
assumes $\forall v \in$ *set* $(u \#$ *us*$)$. *dim-vec v* $= k$
shows *outer-sum k* $(u \#$ *us*$)$
$=$ *outer-sum k* $[u] +$ *outer-sum k us*
⟨*proof*⟩

lemma *outer-sum-as-foldr*:
assumes $\forall v \in$ *set vs*. *dim-vec v* $= n$
shows *outer-sum n vs* $=$ *foldr* $(\lambda v$ *acc*. $v \oslash v +$ *acc*$)$ *vs* $(0_m\ n\ n)$
⟨*proof*⟩

lemma *Tr-hermitian-Im-zero*:
assumes *Pc*: $P \in$ *carrier-mat n n* **and** *herm*: *hermitian P*
shows *Im* $($*Tr P*$) = 0$
⟨*proof*⟩

lemma *Tr-hermitian-real*:
assumes *Pc*: $P \in$ *carrier-mat n n* **and** *herm*: *hermitian P*
shows *Tr P* $=$ *complex-of-real* $($*Re* $($*Tr P*$))$
⟨*proof*⟩

A positive semidefinite matrix of trace zero is zero (the notes: "given a non-zero $P \geq 0$, $\mathrm{Tr}(P) \neq 0$").

lemma *psd-Tr-zero-imp-zero*:
assumes *Pc*: $P \in$ *carrier-mat n n* **and** *pos*: *psd P* **and** *tr0*: *Tr P* $= 0$
shows $P = 0_m\ n\ n$
⟨*proof*⟩

lemma *outer-prod-self-psd*:

assumes *dim-vec v = n*
shows *psd (v ⊘ v)*
⟨*proof*⟩

lemma *hermitian-smult-real-mat*:
fixes *M* :: *complex Matrix.mat* **and** *c* :: *real*
assumes *Mcar*: *M ∈ carrier-mat n n* **and** *Mherm*: *hermitian M*
shows *hermitian (complex-of-real c ·_m M)*
⟨*proof*⟩

lemma *psd-smult-nonneg*:
assumes *c*: *0 ≤ c*
and *A*: *psd A* **and** *Ac*: *A ∈ carrier-mat n n*
shows *psd (complex-of-real c ·_m A)*
⟨*proof*⟩

6.5 The converse directions for Hermitian matrices (D4-1)

lemma *unitary-real-diag-imp-hermitian*:
assumes *U*: *U ∈ carrier-mat n n* **and** *u*: *unitary U* **and** *H*: *H ∈ carrier-mat n n*
and *D*: *length D = n* **and** *spec*: *U† * H * U = diag-mat-of-real D*
shows *hermitian H*
⟨*proof*⟩

lemma *eigenbasis-real-imp-hermitian*:
assumes *H*: *H ∈ carrier-mat n n* **and** *orth*: *orthonormal-list n vs* **and** *len*: *length vs = n*
and *eig*: *⋀i. i < n ⟹ H *_v vs ! i = complex-of-real (lam ! i) ·_v vs ! i* **and** *lam*: *length lam = n*
shows *hermitian H*
⟨*proof*⟩

6.6 Ranges; positive definiteness and spanning (D4-8)

The notes write $R(P)$ for the range of P (here *mat-range k P*). For Hermitian P, every vector orthogonal to $\ker P$ lies in $R(P)$, by the spectral decomposition $P = UDU^*$ with the nonzero eigenvalues inverted; hence $R(\sum_l w_l w_l^*) = \mathrm{span}\{w_l\}$ (*col-span*), and so the notes' form of D4-8. Day 21 uses the same lemma for $0 \le ww^* \le P \Rightarrow w \in R(P)$.

definition *mat-range* :: *nat ⇒ complex Matrix.mat ⇒ complex Matrix.vec set* **where**
*mat-range k P = {P *_v h | h. dim-vec h = k}*

lemma *diag-mat-of-real-mult-vec*:
assumes *length D = k dim-vec v = k*
shows *diag-mat-of-real D *_v v = Matrix.vec k (λi. complex-of-real (D ! i) * v $ i)*
⟨*proof*⟩

For Hermitian P, a vector orthogonal to $\ker P$ lies in the range of P.

lemma *hermitian-perp-ker-in-range*:
assumes *P*: *P ∈ carrier-mat k k* **and** *h*: *hermitian P* **and** *w*: *dim-vec w = k*
and *perp*: *⋀x. dim-vec x = k ⟹ P *_v x = 0_v k ⟹ Quantum.inner-prod x w = 0*
shows *w ∈ mat-range k P*
⟨*proof*⟩

The notes: if $P = \sum_l w_l w_l^*$ then $R(P) = \mathrm{span}\{w_1, \ldots, w_m\}$.

theorem *outer-sum-range*:
assumes *vs*: *∀ v ∈ set vs. dim-vec v = k*
shows *mat-range k (outer-sum k vs) = col-span k vs*
⟨*proof*⟩

The notes: $P = \sum_i |v_i\rangle\langle v_i| > 0$ iff $\text{span}\{v_i\} = \mathbb{C}^n$.

theorem *pd-iff-span-all*:
  assumes *vs*: $\forall\, v \in$ *set vs*. *dim-vec v* $=$ *n*
  shows *pd* (*outer-sum n vs*) $\longleftrightarrow$ *col-span n vs* $=$ *carrier-vec n*
⟨*proof*⟩

6.7 Projections

A *projection* on $\mathbb{C}^n$ is a Hermitian idempotent, the notion of Day 2; projections are the building blocks of syndrome measurements and recovery maps (Days 30–34). Day 2's theorem that such a P is the orthogonal projection onto its range follows the basic facts.

definition *projection* :: *nat* ⇒ *complex Matrix.mat* ⇒ *bool* **where**
  projection n P ≡
    P ∈ *carrier-mat n n* ∧ *hermitian P* ∧ *P* ∗ *P* = *P*

lemma *projection-carrier*:
  projection n P ⟹ *P* ∈ *carrier-mat n n*
  ⟨*proof*⟩

lemma *projection-hermitian*:
  projection n P ⟹ *hermitian P*
  ⟨*proof*⟩

lemma *projection-idempotent*:
  projection n P ⟹ *P* ∗ *P* = *P*
  ⟨*proof*⟩

lemma *projection-one*: *projection n* (*1*$_m$ *n*)
  ⟨*proof*⟩

lemma *projection-psd*:
  assumes *projection n P*
  shows *psd P*
⟨*proof*⟩

The theorem of Day 2: a Hermitian idempotent P is the orthogonal projection onto its range, i.e. $Pv = v$ for v in the range and $Pw = 0$ for w orthogonal to the range.

lemma *projection-fixes-range*:
  fixes *P* :: *complex Matrix.mat* **and** *x* :: *complex Matrix.vec*
  assumes *P*: *projection n P* **and** *x*: *dim-vec x* $=$ *n*
  shows *P* ∗$_v$ (*P* ∗$_v$ *x*) $=$ *P* ∗$_v$ *x*
⟨*proof*⟩

lemma *projection-annihilates-orthogonal*:
  fixes *P* :: *complex Matrix.mat* **and** *w* :: *complex Matrix.vec*
  assumes *P*: *projection n P* **and** *w*: *dim-vec w* $=$ *n*
    and *orth*: ⋀*x* :: *complex Matrix.vec*. *dim-vec x* $=$ *n* ⟹ ⟨*P* ∗$_v$ *x* | *w*⟩ $=$ *0*
  shows *P* ∗$_v$ *w* $=$ *0*$_v$ *n*
⟨*proof*⟩

lemma *projection-complement-idempotent*:
  assumes *projection n P*
  shows (*1*$_m$ *n* − *P*) ∗ (*1*$_m$ *n* − *P*) $=$ *1*$_m$ *n* − *P*
⟨*proof*⟩

7 Day 5 (7 September 2011): tests for positivity; direct sums and partitioned matrices

7.1 Practical tests for positivity

Given $P = P^*$, how does one decide whether $P \geq 0$ or $P > 0$? The notes give two tests. (1) Sylvester's criterion, in terms of the leading principal submatrices $P_k = (p_{ij})_{1 \leq i,j \leq k}$ (the upper left $k \times k$ corners):

Theorem (notes) (Sylvester's criterion)**.** Let $P = P^* \in M_n$. Then $P > 0$ if and only if $\det P_k > 0$ for $k = 1, \ldots, n$.

Each $\det P_k$ is real, as P_k is Hermitian. The same test with ≥ 0 does *not* characterise $P \geq 0$: $\begin{pmatrix} 0 & 0 \\ 0 & -1 \end{pmatrix}$ has leading minors 0 and 0 but is not positive. For $n = 2$ the criterion says $\begin{pmatrix} a & b \\ \bar{b} & c \end{pmatrix} > 0$ iff $a > 0$ and $ac - |b|^2 > 0$. The notes state it without proof. It is not reproved in this development: it is imported from the AFP entry *Two Hermitian Results* and restated for *pd* and *hermitian* as `pd_iff_leading_minors_pos` (for $P \in M_n$ Hermitian, $P >_M 0$ iff $\det P_k > 0$ for all $k \leq n$) in the next section, the theory `Day05_Sylvester_Criterion`. (2) Cholesky's algorithm:

Theorem (notes). If $P = (p_{ij})$ is $n \times n$ with $P = P^*$, then $P \geq 0$ if and only if $P - (\frac{p_{i1} p_{1j}}{p_{11}})_{ij} \geq 0$.

The subtracted matrix agrees with P in the first row and column, so the difference is really an $(n-1) \times (n-1)$ matrix and the test can be repeated; when $P \geq 0$ the algorithm also writes P as a sum of rank-one matrices. The formal theorem adds the hypothesis $p_{11} > 0$, which the notes leave implicit (for $p_{11} = 0$ the quotient is undefined).

theorem *cholesky-reduction*:
fixes *P* :: *complex Matrix.mat*
assumes *sq*: *P* ∈ *carrier-mat n n*
and *herm*: *hermitian P*
and *n-pos*: *0* < *n*
and *p11-pos*: *P* $$ (*0*,*0*) > *0*
shows *psd P* ⟷
psd (*Matrix.mat* (*n*−*1*) (*n*−*1*)
(λ(*i*,*j*). *P* $$ (*i*+*1*, *j*+*1*) −
P $$ (*i*+*1*, *0*) ∗ *cnj* (*P* $$ (*j*+*1*, *0*)) / *P* $$ (*0*,*0*)))
⟨*proof*⟩

7.2 Cholesky's algorithm on the notes' two examples (D5-3)

7.2.1 One step of Cholesky's algorithm

For $P = (p_{ij})$ with $p_{11} \neq 0$ the notes subtract $R = (\frac{p_{i1} p_{1j}}{p_{11}})_{ij}$; $P - R$ vanishes in the first row and column, and what remains is the $(n-1) \times (n-1)$ matrix *chol-next P*.

definition *chol-R* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
chol-R P = *Matrix.mat* (*dim-row P*) (*dim-row P*) (λ(*i*, *j*). *P* $$ (*i*, *0*) ∗ *P* $$ (*0*, *j*) / *P* $$ (*0*, *0*))

definition *chol-next* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
chol-next P = *Matrix.mat* (*dim-row P* − *1*) (*dim-row P* − *1*) (λ(*i*, *j*). (*P* − *chol-R P*) $$ (*i* + *1*, *j* + *1*))

lemma *hermitian-entry*:
assumes *hermitian P P* ∈ *carrier-mat n n i* < *n j* < *n*
shows *P* $$ (*i*, *j*) = *cnj* (*P* $$ (*j*, *i*))

⟨*proof*⟩

The test: for $P = P^*$ with $p_{11} > 0$, $P \geq 0$ iff the next matrix is ≥ 0 (from *cholesky-reduction*).

lemma *chol-test*:
assumes *P*: $P \in$ *carrier-mat n n* **and** *h*: *hermitian P* **and** *n*: $0 < n$ **and** *p*: *P* $$ $(0, 0) > 0$
shows *psd P* $\longleftrightarrow$ *psd* (*chol-next P*)
⟨*proof*⟩

7.2.2 Small matrices entry by entry

lemma *mat3-eqI*:
assumes $A \in$ *carrier-mat 3 3* $B \in$ *carrier-mat 3 3*
and $\bigwedge i\ j.\ i \in \{0, 1, 2\} \Longrightarrow j \in \{0, 1, 2\} \Longrightarrow A\ \$\$\ (i, j) = B\ \$\$\ (i, j)$
shows $A = B$
⟨*proof*⟩

lemma *mat2-eqI*:
assumes $A \in$ *carrier-mat 2 2* $B \in$ *carrier-mat 2 2*
and $\bigwedge i\ j.\ i \in \{0, 1\} \Longrightarrow j \in \{0, 1\} \Longrightarrow A\ \$\$\ (i, j) = B\ \$\$\ (i, j)$
shows $A = B$
⟨*proof*⟩

7.2.3 Example 1: a matrix that is not positive

The notes: for $P = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 5 \\ 3 & 5 & 7 \end{pmatrix}$, $R = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{pmatrix}$ and $P - R = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & -1 & -2 \end{pmatrix}$, so $P \not\geq 0$. Each matrix is checked below; the last step is the test vector $(1, 1)$ for the remaining 2×2 block.

definition *notes-P1* :: *complex Matrix.mat* **where**
notes-P1 = *mat-of-rows-list 3* [[*1*, *2*, *3*], [*2*, *4*, *5*], [*3*, *5*, *7*]]

lemma *notes-P1-carrier*: *notes-P1* $\in$ *carrier-mat 3 3*
⟨*proof*⟩

lemma *notes-P1-hermitian*: *hermitian notes-P1*
⟨*proof*⟩

lemma *notes-P1-R*: *chol-R notes-P1* = *mat-of-rows-list 3* [[*1*, *2*, *3*], [*2*, *4*, *6*], [*3*, *6*, *9*]]
⟨*proof*⟩

lemma *notes-P1-diff*: *notes-P1* − *chol-R notes-P1* = *mat-of-rows-list 3* [[*0*, *0*, *0*], [*0*, *0*, −*1*], [*0*, −*1*, −*2*]]
⟨*proof*⟩

lemma *notes-P1-next*: *chol-next notes-P1* = *mat-of-rows-list 2* [[*0*, −*1*], [−*1*, −*2*]]
⟨*proof*⟩

lemma *not-psd-witness*:
assumes *A*: $A \in$ *carrier-mat n n* **and** *x*: *dim-vec x* = *n* **and** *neg*: *Re* (*Quantum.inner-prod x* ($A *_v$ *x*)) < *0*
shows ¬ *psd A*
⟨*proof*⟩

$\begin{pmatrix} 0 & -1 \\ -1 & -2 \end{pmatrix}$ has the value -4 at $(1, 1)$, so it is not positive.

lemma *notes-P1-next-not-psd*: ¬ *psd* (*mat-of-rows-list 2* [[*0*, −*1*], [−*1*, −*2*]])

⟨*proof*⟩

theorem *notes-example-1*: *hermitian notes-P1* ∧ ¬ *psd notes-P1*
⟨*proof*⟩

7.2.4 Example 2: a positive matrix, in two steps

The notes: for $P = \begin{pmatrix} 4 & 2 & 3 \\ 2 & 5 & 6 \\ 3 & 6 & 8 \end{pmatrix}$, $R = \frac{1}{4}\begin{pmatrix} 16 & 8 & 12 \\ 8 & 4 & 6 \\ 12 & 6 & 9 \end{pmatrix}$ and $P - R = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 4 & \frac{9}{2} \\ 0 & \frac{9}{2} & \frac{23}{4} \end{pmatrix}$. Repeating on $Q = \begin{pmatrix} 4 & \frac{9}{2} \\ \frac{9}{2} & \frac{23}{4} \end{pmatrix}$: $Q - \frac{1}{4}\begin{pmatrix} 4 \\ \frac{9}{2} \end{pmatrix}\begin{pmatrix} 4 & \frac{9}{2} \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & \frac{11}{16} \end{pmatrix}$, so $P \geq 0$.

definition *notes-P2* :: *complex Matrix.mat* **where**
notes-P2 = *mat-of-rows-list 3* [[*4*, *2*, *3*], [*2*, *5*, *6*], [*3*, *6*, *8*]]

lemma *notes-P2-carrier*: *notes-P2* ∈ *carrier-mat 3 3*
⟨*proof*⟩

lemma *notes-P2-hermitian*: *hermitian notes-P2*
⟨*proof*⟩

First step: the first column is $c = (4, 2, 3)^t$ and $p_{11} = 4$, and subtracting $\frac{1}{4}cc^t$ clears the first row and column:

$$P_2 - \frac{1}{4}\begin{pmatrix} 4 \\ 2 \\ 3 \end{pmatrix}\begin{pmatrix} 4 & 2 & 3 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 4 & 9/2 \\ 0 & 9/2 & 23/4 \end{pmatrix}.$$

lemma *notes-P2-R*: *chol-R notes-P2* = *(1/4)* ·$_m$ *mat-of-rows-list 3* [[*16*, *8*, *12*], [*8*, *4*, *6*], [*12*, *6*, *9*]]
⟨*proof*⟩

lemma *notes-P2-diff*: *notes-P2* − *chol-R notes-P2* = *mat-of-rows-list 3* [[*0*, *0*, *0*], [*0*, *4*, *9/2*], [*0*, *9/2*, *23/4*]]
⟨*proof*⟩

definition *notes-Q2* :: *complex Matrix.mat* **where**
notes-Q2 = *mat-of-rows-list 2* [[*4*, *9/2*], [*9/2*, *23/4*]]

lemma *notes-P2-next*: *chol-next notes-P2* = *notes-Q2*
⟨*proof*⟩

lemma *notes-Q2-carrier*: *notes-Q2* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *notes-Q2-hermitian*: *hermitian notes-Q2*
⟨*proof*⟩

Second step, on the remaining block Q_2:

$$Q_2 - \frac{1}{4}\begin{pmatrix} 4 \\ 9/2 \end{pmatrix}\begin{pmatrix} 4 & 9/2 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & 11/16 \end{pmatrix},$$

and the last pivot $11/16$ is positive.

lemma *notes-Q2-R*: *chol-R notes-Q2* = *(1/4)* ·$_m$ *mat-of-rows-list 2* [[*16*, *18*], [*18*, *81/4*]]
⟨*proof*⟩

lemma *notes-Q2-diff*: *notes-Q2* − *chol-R notes-Q2* = *mat-of-rows-list 2* [[*0*, *0*], [*0*, *11/16*]]

⟨*proof*⟩

The last matrix is $(\frac{11}{16}) = \frac{11}{16}\,|e\rangle\langle e|$, which is positive.

lemma *notes-Q2-next*: *chol-next notes-Q2 = complex-of-real (11/16) ·$_m$ (Matrix.vec 1 (λ-. 1) ⊘ Matrix.vec 1 (λ-. 1))*
⟨*proof*⟩

lemma *notes-Q2-next-psd*: *psd (chol-next notes-Q2)*
⟨*proof*⟩

theorem *notes-example-2*: *hermitian notes-P2 ∧ psd notes-P2*
⟨*proof*⟩

The notes' remark: the algorithm writes P as a sum of rank-one matrices, $P = \frac{1}{4}|v_1\rangle\langle v_1| + \frac{1}{4}|v_2\rangle\langle v_2| + \frac{11}{16}|e_3\rangle\langle e_3|$ with $v_1 = (4, 2, 3)$, $v_2 = (0, 4, \frac{9}{2})$, $e_3 = (0, 0, 1)$ (the first columns of P, of $P - R$ and of the last step).

lemma *outer-prod-of-list*:
*vec-of-list l ⊘ vec-of-list l = Matrix.mat (length l) (length l) (λ(i, j). l ! i * cnj (l ! j))*
⟨*proof*⟩

lemma *notes-P2-rank-one-sum*:
notes-P2 = complex-of-real (1/4) ·$_m$ (vec-of-list [4, 2, 3] ⊘ vec-of-list [4, 2, 3])
+ complex-of-real (1/4) ·$_m$ (vec-of-list [0, 4, 9/2] ⊘ vec-of-list [0, 4, 9/2])
+ complex-of-real (11/16) ·$_m$ (vec-of-list [0, 0, 1] ⊘ vec-of-list [0, 0, 1])
⟨*proof*⟩

7.3 Direct sums of vector spaces and partitioned matrices

Definition (notes). The *direct sum* of vector spaces V, W is $V \oplus W = \{(v, w) : v \in V, w \in W\}$ with componentwise operations and inner product $\langle (v_1, w_1) | (v_2, w_2) \rangle = \langle v_1 | v_2 \rangle_V + \langle w_1 | w_2 \rangle_W$.

Proposition (notes). If $v_1, \ldots, v_n$ is a basis of V and $w_1, \ldots, w_k$ a basis of W, then $(v_1, 0), \ldots, (v_n, 0), (0, w_1), \ldots$ is a basis of $V \oplus W$; hence $\dim(V \oplus W) = \dim V + \dim W$.

Key examples: $\mathbb{C}^{n+k} = \mathbb{C}^n \oplus \mathbb{C}^k$, and a linear map $T : \mathbb{C}^{n_1+k_1} \to \mathbb{C}^{n_2+k_2}$ has the block form $T = \begin{pmatrix} T_{11} & T_{12} \\ T_{21} & T_{22} \end{pmatrix}$ with $T\binom{v}{w} = \binom{T_{11}v + T_{12}w}{T_{21}v + T_{22}w}$. Formally the direct sum of vectors is the concatenation $v \oplus_v w$, and block matrices are the library's *four-block-mat*.

The direct sum of vectors is JNF's concatenation $@_v$.

abbreviation *direct-sum-vec* :: *complex Matrix.vec ⇒ complex Matrix.vec ⇒ complex Matrix.vec* (**infixr** $\oplus_v$ *65*) **where**
v ⊕$_v$ w ≡ v @$_v$ w

lemma *direct-sum-vec-right* [*simp*]:
i < dim-vec w ⟹ (v ⊕$_v$ w) $ (dim-vec v + i) = w $ i
⟨*proof*⟩

lemma *direct-sum-inner-prod*:
assumes *vd1*: *dim-vec v1 = m* **and** *vd2*: *dim-vec v2 = m*
and *wd1*: *dim-vec w1 = p* **and** *wd2*: *dim-vec w2 = p*
shows *⟨v1 ⊕$_v$ w1 | v2 ⊕$_v$ w2⟩ = ⟨v1 | v2⟩ + ⟨w1 | w2⟩*
⟨*proof*⟩

lemma *block-mat-apply*:
assumes *dim-vec v = k1* **and** *dim-vec w = k2*

and *T11* ∈ *carrier-mat n1 k1* **and** *T12* ∈ *carrier-mat n1 k2*
and *T21* ∈ *carrier-mat n2 k1* **and** *T22* ∈ *carrier-mat n2 k2*
shows *four-block-mat T11 T12 T21 T22* $*_v$ $(v \oplus_v w) =$
$(T11 *_v v + T12 *_v w) \oplus_v (T21 *_v v + T22 *_v w)$
⟨*proof*⟩

7.4 A basis of $V \oplus W$ (D5-5)

A basis of $\mathbb{C}^n$ is a list of n vectors whose column matrix is invertible. The notes: if $v_1, \ldots, v_n$ and $w_1, \ldots, w_k$ are bases of V and W then $(v_i, 0)$, $(0, w_j)$ is a basis of $V \oplus W$, so $\dim(V \oplus W) = \dim V + \dim W$. The column matrix is block diagonal.

definition *direct-sum-basis* :: *nat* ⇒ *nat* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec list* **where**
direct-sum-basis n k vs ws = *map* (λ*v*. *v* $\oplus_v$ 0_v *k*) *vs* @ *map* (λ*w*. 0_v *n* $\oplus_v$ *w*) *ws*

lemma *direct-sum-basis-block*:
assumes *vs*: ∀ *v* ∈ *set vs*. *dim-vec v* = *n length vs* = *n* **and** *ws*: ∀ *w* ∈ *set ws*. *dim-vec w* = *k length ws* = *k*
shows *mat-of-cols* (*n* + *k*) (*direct-sum-basis n k vs ws*)
= *four-block-mat* (*mat-of-cols n vs*) (0_m *n k*) (0_m *k n*) (*mat-of-cols k ws*)
⟨*proof*⟩

lemma *basis-inverse*:
fixes *fs* :: *complex Matrix.vec list*
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *inv*: *invertible-mat* (*mat-of-cols m fs*)
obtains *G* **where** *G* ∈ *carrier-mat m m G* ∗ *mat-of-cols m fs* = 1_m *m mat-of-cols m fs* ∗ *G* = 1_m *m*
⟨*proof*⟩

theorem *direct-sum-basis-is-basis*:
assumes *vs*: ∀ *v* ∈ *set vs*. *dim-vec v* = *n length vs* = *n* **and** *iv*: *invertible-mat* (*mat-of-cols n vs*)
and *ws*: ∀ *w* ∈ *set ws*. *dim-vec w* = *k length ws* = *k* **and** *iw*: *invertible-mat* (*mat-of-cols k ws*)
shows ∀ *u* ∈ *set* (*direct-sum-basis n k vs ws*). *dim-vec u* = *n* + *k*
and *length* (*direct-sum-basis n k vs ws*) = *n* + *k*
and *invertible-mat* (*mat-of-cols* (*n* + *k*) (*direct-sum-basis n k vs ws*))
⟨*proof*⟩

7.5 Sylvester's criterion (D5-1)

Sylvester's criterion, stated above, is proved in the AFP entry *Two Hermitian Results* (*Sylvester-Criterion.sylves*
with its own notions of Hermitian matrix (*Complex-Matrix.hermitian*, from QHLProver) and positive definiteness (*positive-definite*, with the quadratic form *QF* and the order on *complex*). It is not reproved here. The lemmas below show that these notions agree with *Day04-Hermitian-Matrices.hermitia*
and *pd* of the formalised notes; the theorem is then restated in those terms. Names that both libraries define (*hermitian*, *inner-prod*) are written with their theory prefix. Formally P_k is *lps P k*, and *k* ranges over $0, \ldots, n$: the case $k = 0$ adds nothing, since a 0×0 determinant is 1. The entry's imports undo the hiding of Isabelle_Marries_Dirac's gate names in `Day01_Inner_Product`, so they are hidden again here.

hide-const (**open**) *Quantum.X Quantum.Y Quantum.Z Quantum.H Quantum.S Quantum.T*

lemma *hermitian-iff-Complex-Matrix-hermitian*: *Day04-Hermitian-Matrices.hermitian A* ⟷ *Complex-Matrix.hermitia*
A
⟨*proof*⟩

lemma *quadratic-form-eq-inner-prod*:
fixes *P* :: *complex Matrix.mat* **and** *x* :: *complex Matrix.vec*

assumes *P*: $P \in$ *carrier-mat n n* **and** *x*: *dim-vec x* = *n*
shows *quadratic-form P x* = *Quantum.inner-prod x* (*P* $*_v$ *x*)
⟨*proof*⟩

lemma *pd-iff-positive-definite*:
fixes *P* :: *complex Matrix.mat*
assumes *P*: $P \in$ *carrier-mat n n*
shows *Day04-Hermitian-Matrices.pd P* $\longleftrightarrow$ *positive-definite P*
⟨*proof*⟩

Sylvester's criterion: a Hermitian $P \in M_n$ is positive definite iff $\det P_k > 0$ for every leading principal submatrix P_k, $k \le n$ (*lps*; the order is that of *complex*, so each minor is real and positive).

theorem *pd-iff-leading-minors-pos*:
fixes *P* :: *complex Matrix.mat*
assumes *P*: $P \in$ *carrier-mat n n* **and** *h*: *Day04-Hermitian-Matrices.hermitian P*
shows *Day04-Hermitian-Matrices.pd P* $\longleftrightarrow$ ($\forall\, k \le n.\ 0 <$ *Determinant.det* (*lps P k*))
⟨*proof*⟩

corollary *pd-iff-leading-minors-real-pos*:
fixes *P* :: *complex Matrix.mat*
assumes *P*: $P \in$ *carrier-mat n n* **and** *h*: *Day04-Hermitian-Matrices.hermitian P*
shows *Day04-Hermitian-Matrices.pd P* $\longleftrightarrow$
($\forall\, k \le n.$ *Im* (*Determinant.det* (*lps P k*)) = *0* $\wedge$ *0 < Re* (*Determinant.det* (*lps P k*)))
⟨*proof*⟩

8 Day 6 (9 September 2011): tensor products

Definition (notes). $B : X \times Y \to Z$ is *bilinear* if $B(x_1 + x_2, y) = B(x_1, y) + B(x_2, y)$, $B(x, y_1 + y_2) = B(x, y_1) + B(x, y_2)$ and $B(\lambda x, y) = B(x, \lambda y) = \lambda B(x, y)$.

The tensor product $X \otimes Y$ is the span of elementary tensors $x \otimes y$ with the universal property: every bilinear B factors as $B(x, y) = L_B(x \otimes y)$ for a unique linear L_B. Key relations: $(x_1 + x_2) \otimes y = x_1 \otimes y + x_2 \otimes y$, $x \otimes (y_1 + y_2) = x \otimes y_1 + x \otimes y_2$, $\lambda(x \otimes y) = (\lambda x) \otimes y = x \otimes (\lambda y)$.

Theorem (notes). If $\{e_1, \ldots, e_k\}$ and $\{f_1, \ldots, f_m\}$ are bases of X and Y, then $\{e_i \otimes f_j\}$ is a basis of $X \otimes Y$; hence $\dim(X \otimes Y) = \dim X \cdot \dim Y$.

Proposition (notes). With bases as above, every $u \in X \otimes Y$ is uniquely $u = \sum_j x_j \otimes f_j$ with $x_j \in X$, and uniquely $u = \sum_i e_i \otimes y_i$ with $y_i \in Y$; consequently $X \otimes Y \cong X \oplus \cdots \oplus X$ (m copies).

Definition (notes). The *Schmidt rank* of $u \in X \otimes Y$ is the least number of elementary tensors in an expression for u.

The formalisation uses the concrete model: $\mathbb{C}^n \otimes \mathbb{C}^m = \mathbb{C}^{nm}$ with $(v \otimes w)_{im+j} = v_i w_j$ (*tensor-vec v w* from the library). The three key relations are the bilinearity lemmas below; the product basis theorem is *tensor-onb* (Day 7); the unique decomposition along a basis of Y is not stated for vectors (its operator analogue is in theory Day15_Partial_Traces).

lemma *tensor-vec-dim*:
assumes *vd*: *dim-vec v* = *n dim-vec w* = *m*
shows *dim-vec* (*tensor-vec v w*) = *n* $*$ *m*
⟨*proof*⟩

lemma *tensor-vec-index*:

assumes *dim-vec v = n dim-vec w = m i < n * m*
shows *tensor-vec v w $ i = v $ (i div m) * w $ (i mod m)*
⟨*proof*⟩

In coordinates $(v \otimes w)_{im+j} = v_i\, w_j$, the first factor giving the major index; for $v, w \in \mathbb{C}^2$,

$$v \otimes w = (v_0 w_0,\ v_0 w_1,\ v_1 w_0,\ v_1 w_1).$$

Each entry is a product of one coordinate of each factor, so $\otimes$ is bilinear:

lemma *tensor-vec-bilinear-left*:
assumes *vd*: *dim-vec v1 = n dim-vec v2 = n dim-vec w = m*
shows *tensor-vec (v1 + v2) w = tensor-vec v1 w + tensor-vec v2 w*
⟨*proof*⟩

lemma *tensor-vec-bilinear-right*:
assumes *vd*: *dim-vec v = n dim-vec w1 = m dim-vec w2 = m*
shows *tensor-vec v (w1 + w2) = tensor-vec v w1 + tensor-vec v w2*
⟨*proof*⟩

lemma *tensor-vec-smult-left*:
assumes *vd*: *dim-vec v = n dim-vec w = m*
shows *tensor-vec (α ·$_v$ v) w = α ·$_v$ tensor-vec v w*
⟨*proof*⟩

lemma *tensor-vec-smult-right*:
assumes *vd*: *dim-vec v = n dim-vec w = m*
shows *tensor-vec v (α ·$_v$ w) = α ·$_v$ tensor-vec v w*
⟨*proof*⟩

The Schmidt rank, the least length of a representation by elementary tensors. It is *LEAST*, not *Min* of the set of lengths: that set is infinite (zero terms can be added), and *Min* is unspecified on infinite sets. Its properties (rank 1 iff a product state) are proved below.

definition *schmidt-rank* :: *complex Matrix.vec ⇒ nat ⇒ nat* **where**
schmidt-rank u m ≡
LEAST k. ∃ vs :: complex Matrix.vec list. ∃ ws :: complex Matrix.vec list.
length vs = k ∧ length ws = k ∧
(∀ i < k. dim-vec (vs ! i) = dim-vec u div m) ∧
(∀ i < k. dim-vec (ws ! i) = m) ∧
u = foldr (λ(v,w) acc. tensor-vec v w + acc)
(zip vs ws) (0$_v$ (dim-vec u))

lemma *tensor-unit-vec*:
assumes *i*: *i < n* **and** *j*: *j < m*
shows *tensor-vec (unit-vec n i) (unit-vec m j) = (unit-vec (n * m) (i * m + j) :: complex Matrix.vec)*
⟨*proof*⟩

8.1 The universal properties in the Kronecker model

D6-1: for $\mathbb{C}^n \otimes \mathbb{C}^m = \mathbb{C}^{nm}$ with $(v \otimes w)_{im+j} = v_i w_j$, every bilinear map $\beta : \mathbb{C}^n \times \mathbb{C}^m \to \mathbb{C}^p$ factors as $\beta(v, w) = L(v \otimes w)$ for a unique linear L. First for bilinear forms ($p = 1$), then coordinatewise.

definition *bilinear-on* :: *nat ⇒ nat ⇒ (complex Matrix.vec ⇒ complex Matrix.vec ⇒ complex) ⇒ bool* **where**

bilinear-on n m β ⟷
(∀ *v1 v2 w*. *dim-vec v1 = n ⟶ dim-vec v2 = n ⟶ dim-vec w = m ⟶ β (v1 + v2) w = β v1 w + β v2 w*)
∧ (∀ *v w1 w2*. *dim-vec v = n ⟶ dim-vec w1 = m ⟶ dim-vec w2 = m ⟶ β v (w1 + w2) = β v w1 + β v w2*)
∧ (∀ *a v w*. *dim-vec v = n ⟶ dim-vec w = m ⟶ β (a ·$_v$ v) w = a * β v w ∧ β v (a ·$_v$ w) = a * β v w*)

definition *linear-on* :: *nat ⇒ (complex Matrix.vec ⇒ complex) ⇒ bool* **where**
linear-on K L ⟷ (∀ *x y*. *dim-vec x = K ⟶ dim-vec y = K ⟶ L (x + y) = L x + L y*)
∧ (∀ *a x*. *dim-vec x = K ⟶ L (a ·$_v$ x) = a * L x*)

definition *trunc-vec* :: *complex Matrix.vec ⇒ nat ⇒ complex Matrix.vec* **where**
trunc-vec v k = Matrix.vec (dim-vec v) (λi. if i < k then v $ i else 0)

lemma *dim-trunc-vec* [*simp*]: *dim-vec (trunc-vec v k) = dim-vec v* ⟨*proof*⟩

lemma *trunc-vec-full*: *trunc-vec v (dim-vec v) = v* ⟨*proof*⟩

lemma *trunc-vec-Suc*: *k < dim-vec v ⟹ trunc-vec v (Suc k) = trunc-vec v k + v $ k ·$_v$ unit-vec (dim-vec v) k*
⟨*proof*⟩

lemma *trunc-vec-zero*: *trunc-vec v 0 = 0$_v$ (dim-vec v)* ⟨*proof*⟩

A linear map is determined by its values on the unit vectors.

lemma *linear-on-zero*: *linear-on K L ⟹ L (0$_v$ K) = 0*
⟨*proof*⟩

lemma *linear-on-expand*:
assumes *l*: *linear-on K L* **and** *x*: *dim-vec x = K*
shows *L x = (∑k<K. x $ k * L (unit-vec K k))*
⟨*proof*⟩

A bilinear map is determined by its values on pairs of unit vectors.

A bilinear β is determined by its values on pairs of basis vectors:

$$\beta(v,w) = \sum_{i,j} v_i w_j\, \beta(e_i, e_j) = \sum_{i,j} (v \otimes w)_{im+j}\, \beta(e_i, e_j),$$

so $L(x) = \sum_k x_k\, \beta(e_{k \div m}, e_{k \bmod m})$ is the unique linear map with $L(v \otimes w) = \beta(v,w)$.

lemma *bilinear-on-expand*:
assumes *b*: *bilinear-on n m β* **and** *v*: *dim-vec v = n* **and** *w*: *dim-vec w = m*
shows *β v w = (∑i<n. ∑j<m. (v $ i * w $ j) * β (unit-vec n i) (unit-vec m j))*
⟨*proof*⟩

lemma *sum-div-mod*:
fixes *n m* :: *nat* **and** *g* :: *nat ⇒ nat ⇒ 'a*::*comm-monoid-add*
shows *(∑k<n * m. g (k div m) (k mod m)) = (∑i<n. ∑j<m. g i j)*
⟨*proof*⟩

theorem *tensor-universal-property*:
assumes *b*: *bilinear-on n m β*
shows ∃ *L*. *linear-on (n * m) L* ∧ (∀ *v w*. *dim-vec v = n ⟶ dim-vec w = m ⟶ L (tensor-vec v w) = β v w*)
∧ (∀ *L′*. *linear-on (n * m) L′* ∧ (∀ *v w*. *dim-vec v = n ⟶ dim-vec w = m ⟶ L′ (tensor-vec v w) = β v w*)

$\longrightarrow (\forall x.\ dim\text{-}vec\ x = n * m \longrightarrow L'\ x = L\ x))$
⟨*proof*⟩

definition *bilinear-vec-on* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ (*complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec*) ⇒ *bool* **where**
bilinear-vec-on n m p β ⟷
$(\forall v\ w.\ dim\text{-}vec\ v = n \longrightarrow dim\text{-}vec\ w = m \longrightarrow dim\text{-}vec\ (\beta\ v\ w) = p)$
$\wedge\ (\forall v1\ v2\ w.\ dim\text{-}vec\ v1 = n \longrightarrow dim\text{-}vec\ v2 = n \longrightarrow dim\text{-}vec\ w = m \longrightarrow \beta\ (v1 + v2)\ w = \beta\ v1\ w + \beta\ v2\ w)$
$\wedge\ (\forall v\ w1\ w2.\ dim\text{-}vec\ v = n \longrightarrow dim\text{-}vec\ w1 = m \longrightarrow dim\text{-}vec\ w2 = m \longrightarrow \beta\ v\ (w1 + w2) = \beta\ v\ w1 + \beta\ v\ w2)$
$\wedge\ (\forall a\ v\ w.\ dim\text{-}vec\ v = n \longrightarrow dim\text{-}vec\ w = m \longrightarrow \beta\ (a \cdot_v v)\ w = a \cdot_v \beta\ v\ w \wedge \beta\ v\ (a \cdot_v w) = a \cdot_v \beta\ v\ w)$

definition *linear-vec-on* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.vec* ⇒ *complex Matrix.vec*) ⇒ *bool* **where**
linear-vec-on K p L ⟷ $(\forall x.\ dim\text{-}vec\ x = K \longrightarrow dim\text{-}vec\ (L\ x) = p)$
$\wedge\ (\forall x\ y.\ dim\text{-}vec\ x = K \longrightarrow dim\text{-}vec\ y = K \longrightarrow L\ (x + y) = L\ x + L\ y)$
$\wedge\ (\forall a\ x.\ dim\text{-}vec\ x = K \longrightarrow L\ (a \cdot_v x) = a \cdot_v L\ x)$

lemma *bilinear-vec-coord*: *bilinear-vec-on n m p β* ⟹ *r* < *p* ⟹ *bilinear-on n m* (λ*v w*. *β v w* $ *r*)
⟨*proof*⟩

lemma *linear-vec-coord*: *linear-vec-on K p L* ⟹ *r* < *p* ⟹ *linear-on K* (λ*x*. *L x* $ *r*)
⟨*proof*⟩

theorem *tensor-universal-property-vec*:
assumes *b*: *bilinear-vec-on n m p β*
shows ∃ *L*. *linear-vec-on* (*n* ∗ *m*) *p L* ∧ (∀ *v w*. *dim-vec v* = *n* ⟶ *dim-vec w* = *m* ⟶ *L* (*tensor-vec v w*) = *β v w*)
∧ (∀ *L'*. *linear-vec-on* (*n* ∗ *m*) *p L'* ∧ (∀ *v w*. *dim-vec v* = *n* ⟶ *dim-vec w* = *m* ⟶ *L'* (*tensor-vec v w*) = *β v w*)
⟶ (∀ *x*. *dim-vec x* = *n* ∗ *m* ⟶ *L' x* = *L x*))
⟨*proof*⟩

8.2 Coordinates in a tensor product

$\sum_j x_j \otimes y_j$ in $\mathbb{C}^N$, written as in *schmidt-rank*.

definition *tensor-combination* :: *nat* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec* **where**
tensor-combination N xs ys = *foldr* (λ(*x*,*y*) *acc*. *tensor-vec x y* + *acc*) (*zip xs ys*) (0_v *N*)

lemma *tensor-combination-dim* [*simp*]: *dim-vec* (*tensor-combination N xs ys*) = *N*
⟨*proof*⟩

lemma *tensor-combination-index*:
assumes ∀ *x* ∈ *set xs*. *dim-vec x* = *k* ∀ *y* ∈ *set ys*. *dim-vec y* = *m length xs* = *length ys p* < *k* ∗ *m*
shows *tensor-combination* (*k* ∗ *m*) *xs ys* $ *p* = (∑*j*<*length xs*. *xs* ! *j* $ (*p div m*) ∗ *ys* ! *j* $ (*p mod m*))
⟨*proof*⟩

A vector $u \in \mathbb{C}^k \otimes \mathbb{C}^m$ has $k\,m$ entries: the coefficient of $e_i \otimes e_l$ sits at position $i\,m + l$. Read row by row, u is a $k \times m$ matrix; for $k = 2$, $m = 3$,

$$u = (u_0, u_1, u_2, u_3, u_4, u_5) \quad \longmapsto \quad \begin{pmatrix} u_0 & u_1 & u_2 \\ u_3 & u_4 & u_5 \end{pmatrix}.$$

The columns are the parts of u along the basis of the second factor, the rows those along the

basis of the first:

$$u = (u_0, u_3) \otimes e_0 + (u_1, u_4) \otimes e_1 + (u_2, u_5) \otimes e_2 = e_0 \otimes (u_0, u_1, u_2) + e_1 \otimes (u_3, u_4, u_5).$$

definition *vec-to-mat* :: *nat* ⇒ *nat* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.mat* **where**
vec-to-mat k m u = *Matrix.mat k m* (λ(*i*, *l*). *u* $ (*i* ∗ *m* + *l*))

lemma *vec-to-mat-carrier* [*simp*]: *vec-to-mat k m u* ∈ *carrier-mat k m*
⟨*proof*⟩

lemma *vec-to-mat-dims* [*simp*]: *dim-row* (*vec-to-mat k m u*) = *k dim-col* (*vec-to-mat k m u*) = *m*
⟨*proof*⟩

lemma *vec-to-mat-index* [*simp*]: *i* < *k* ⟹ *l* < *m* ⟹ *vec-to-mat k m u* $$ (*i*, *l*) = *u* $ (*i* ∗ *m* + *l*)
⟨*proof*⟩

lemma *col-vec-to-mat*: *l* < *m* ⟹ *Matrix.col* (*vec-to-mat k m u*) *l* = *Matrix.vec k* (λ*i*. *u* $ (*i* ∗ *m* + *l*))
⟨*proof*⟩

Conversely, reading a matrix row by row gives a vector, and *vec-to-mat* undoes this:

$$\begin{pmatrix} a & b & c \\ d & e & f \end{pmatrix} \quad \longmapsto \quad (a, b, c, d, e, f).$$

definition *mat-to-vec* :: *complex Matrix.mat* ⇒ *complex Matrix.vec* **where**
mat-to-vec c = *Matrix.vec* (*dim-row c* ∗ *dim-col c*) (λ*k*. *c* $$ (*k div dim-col c*, *k mod dim-col c*))

lemma *vec-to-mat-mat-to-vec*: *c* ∈ *carrier-mat K1 K2* ⟹ *vec-to-mat K1 K2* (*mat-to-vec c*) = *c*
⟨*proof*⟩

lemma *dim-mat-to-vec*: *c* ∈ *carrier-mat K1 K2* ⟹ *dim-vec* (*mat-to-vec c*) = *K1* ∗ *K2* ⟨*proof*⟩

lemma *mat-to-vec-index*:
assumes *c*: *c* ∈ *carrier-mat K1 K2* **and** *i*: *i* < *K1* **and** *j*: *j* < *K2*
shows *mat-to-vec c* $ (*i* ∗ *K2* + *j*) = *c* $$ (*i*, *j*)
⟨*proof*⟩

For other bases the matrix is multiplied by the inverse basis matrix. If $f_1, \ldots, f_m$ is a basis of $\mathbb{C}^m$ with column matrix F and $G = F^{-1}$, then $u = \sum_j x_j \otimes f_j$ says $U = XF^T$, where U is the matrix of u and X has the columns x_j; so the x_j are the columns of UG^T. In the first factor, with H the inverse of the column matrix of $e_1, \ldots, e_k$, $u = \sum_i e_i \otimes y_i$ has y_i the rows of HU.

lemma *right-coordinates-index*:
assumes *G*: *G* ∈ *carrier-mat m m* **and** *i* < *k j* < *m*
shows *Matrix.col* (*vec-to-mat k m u* ∗ *G*^{T}) *j* $ *i* = (∑ *l*<*m*. *u* $ (*i* ∗ *m* + *l*) ∗ *G* $$ (*j*,*l*))
⟨*proof*⟩

lemma *left-coordinates-index*:
assumes *H*: *H* ∈ *carrier-mat k k* **and** *i* < *k l* < *m*
shows *Matrix.row* (*H* ∗ *vec-to-mat k m u*) *i* $ *l* = (∑ *r*<*k*. *H* $$ (*i*,*r*) ∗ *u* $ (*r* ∗ *m* + *l*))
⟨*proof*⟩

lemma *inv-entry-left*:
fixes *fs* :: *complex Matrix.vec list*
assumes *fd*: ∀*f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *G*: *G* ∈ *carrier-mat m m*
and *GF*: *G* ∗ *mat-of-cols m fs* = 1_m *m* **and** *j*: *j* < *m* **and** *j'*: *j'* < *m*
shows (∑ *l*<*m*. *fs* ! *j'* $ *l* ∗ *G* $$ (*j*,*l*)) = (*if j* = *j'* *then 1 else 0*)
⟨*proof*⟩

lemma *inv-entry-right*:
fixes *fs* :: *complex Matrix.vec list*
assumes *fd*: $\forall f \in$ *set fs. dim-vec f* $=$ *m length fs* $=$ *m* **and** *G*: *G* $\in$ *carrier-mat m m*
and *FG*: *mat-of-cols m fs* $*$ *G* $=$ 1_m *m* **and** *r*: *r* $<$ *m* **and** *l*: *l* $<$ *m*
shows $(\sum j<m.$ *fs* ! *j* $ *r* $*$ *G* $$ *(j,l))* $=$ *(if r* $=$ *l then 1 else 0)*
⟨*proof*⟩

lemma *right-coordinates-unique*:
assumes *fd*: $\forall f \in$ *set fs. dim-vec f* $=$ *m length fs* $=$ *m* **and** *G*: *G* $\in$ *carrier-mat m m*
and *GF*: *G* $*$ *mat-of-cols m fs* $=$ 1_m *m*
and *xs*: *length xs* $=$ *m* $\forall x \in$ *set xs. dim-vec x* $=$ *k* **and** *j*: *j* $<$ *m*
shows *Matrix.col* (*vec-to-mat k m* (*tensor-combination* (*k* $*$ *m*) *xs fs*) $*$ G^T) *j* $=$ *xs* ! *j*
⟨*proof*⟩

lemma *right-coordinates-exist*:
assumes *fd*: $\forall f \in$ *set fs. dim-vec f* $=$ *m length fs* $=$ *m* **and** *G*: *G* $\in$ *carrier-mat m m*
and *FG*: *mat-of-cols m fs* $*$ *G* $=$ 1_m *m* **and** *u*: *dim-vec u* $=$ *k* $*$ *m*
shows *tensor-combination* (*k* $*$ *m*) (*cols* (*vec-to-mat k m u* $*$ G^T)) *fs* $=$ *u*
⟨*proof*⟩

D6-3 (i): for a basis $f_1, \ldots, f_m$ of $\mathbb{C}^m$, every $u \in \mathbb{C}^k \otimes \mathbb{C}^m$ is $\sum_j x_j \otimes f_j$ for unique $x_1, \ldots, x_m \in \mathbb{C}^k$.

theorem *tensor-right-coordinates*:
assumes *fd*: $\forall f \in$ *set fs. dim-vec f* $=$ *m length fs* $=$ *m* **and** *inv*: *invertible-mat* (*mat-of-cols m fs*)
and *u*: *dim-vec u* $=$ *k* $*$ *m*
shows $\exists!$*xs. length xs* $=$ *m* $\wedge$ $(\forall x \in$ *set xs. dim-vec x* $=$ *k*$)$ $\wedge$ *u* $=$ *tensor-combination* (*k* $*$ *m*) *xs fs*
⟨*proof*⟩

lemma *left-coordinates-unique*:
assumes *ed*: $\forall e \in$ *set es. dim-vec e* $=$ *k length es* $=$ *k* **and** *Hm*: *Hm* $\in$ *carrier-mat k k*
and *HE*: *Hm* $*$ *mat-of-cols k es* $=$ 1_m *k*
and *ys*: *length ys* $=$ *k* $\forall y \in$ *set ys. dim-vec y* $=$ *m* **and** *i*: *i* $<$ *k*
shows *Matrix.row* (*Hm* $*$ *vec-to-mat k m* (*tensor-combination* (*k* $*$ *m*) *es ys*)) *i* $=$ *ys* ! *i*
⟨*proof*⟩

lemma *left-coordinates-exist*:
assumes *ed*: $\forall e \in$ *set es. dim-vec e* $=$ *k length es* $=$ *k* **and** *Hm*: *Hm* $\in$ *carrier-mat k k*
and *EH*: *mat-of-cols k es* $*$ *Hm* $=$ 1_m *k* **and** *u*: *dim-vec u* $=$ *k* $*$ *m*
shows *tensor-combination* (*k* $*$ *m*) *es* (*rows* (*Hm* $*$ *vec-to-mat k m u*)) $=$ *u*
⟨*proof*⟩

D6-3 (ii): likewise $u = \sum_i e_i \otimes y_i$ for unique $y_1, \ldots, y_k \in \mathbb{C}^m$.

theorem *tensor-left-coordinates*:
assumes *ed*: $\forall e \in$ *set es. dim-vec e* $=$ *k length es* $=$ *k* **and** *inv*: *invertible-mat* (*mat-of-cols k es*)
and *u*: *dim-vec u* $=$ *k* $*$ *m*
shows $\exists!$*ys. length ys* $=$ *k* $\wedge$ $(\forall y \in$ *set ys. dim-vec y* $=$ *m*$)$ $\wedge$ *u* $=$ *tensor-combination* (*k* $*$ *m*) *es ys*
⟨*proof*⟩

Corollary: $\mathbb{C}^k \otimes \mathbb{C}^m \cong \mathbb{C}^k \oplus \cdots \oplus \mathbb{C}^k$ (m times) by the linear bijection $(x_1, \ldots, x_m) \mapsto \sum_j x_j \otimes f_j$, and $\cong \mathbb{C}^m \oplus \cdots \oplus \mathbb{C}^m$ (k times) likewise; the identification depends on the chosen basis.

lemma *tensor-combination-add-left*:
assumes *l*: *length xs* $=$ *length fs length xs'* $=$ *length fs*
and *d*: $\forall x \in$ *set xs. dim-vec x* $=$ *k* $\forall x \in$ *set xs'. dim-vec x* $=$ *k* **and** *fd*: $\forall y \in$ *set fs. dim-vec y* $=$ *m*
shows *tensor-combination* (*k* $*$ *m*) (*map2* (+) *xs xs'*) *fs* $=$ *tensor-combination* (*k* $*$ *m*) *xs fs* $+$ *tensor-combination* (*k* $*$ *m*) *xs' fs*
⟨*proof*⟩

lemma *tensor-combination-smult-left*:
assumes *l*: *length xs* = *length fs* **and** *d*: $\forall\, x \in$ *set xs*. *dim-vec x* = *k* **and** *fd*: $\forall\, y \in$ *set fs*. *dim-vec y* = *m*
shows *tensor-combination* (*k* ∗ *m*) (*map* (λ*x*. *a* ·$_v$ *x*) *xs*) *fs* = *a* ·$_v$ *tensor-combination* (*k* ∗ *m*) *xs fs*
⟨*proof*⟩

corollary *tensor-iso-direct-sum*:
assumes *fd*: $\forall\, f \in$ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *inv*: *invertible-mat* (*mat-of-cols m fs*)
shows *bij-betw* (λ*xs*. *tensor-combination* (*k* ∗ *m*) *xs fs*)
{*xs*. *length xs* = *m* ∧ ($\forall\, x \in$ *set xs*. *dim-vec x* = *k*)} (*carrier-vec* (*k* ∗ *m*))
⟨*proof*⟩

8.3 The Schmidt rank (D6-4)

The notes define the Schmidt rank $\mathrm{rank}_S(u)$ of $u \in X \otimes Y$ as the least number of elementary tensors in an expression for u, and use on Day 10 that a two-photon state ψ is a product $\psi_1 \otimes \psi_2$ iff $\mathrm{rank}_S(\psi) = 1$; otherwise it is entangled. Here: every $u \in \mathbb{C}^n \otimes \mathbb{C}^m$ has an expression of length n, so the least length exists; $\mathrm{rank}_S(u) = 0$ iff $u = 0$; for $u \neq 0$, $\mathrm{rank}_S(u) = 1$ iff u is a product and u is entangled iff $\mathrm{rank}_S(u) > 1$; the Bell state has Schmidt rank 2.

A representation of $u \in \mathbb{C}^n \otimes \mathbb{C}^m$ by k elementary tensors.

definition *schmidt-repr* :: *nat* ⇒ *nat* ⇒ *complex Matrix.vec* ⇒ *nat* ⇒ *bool* **where**
schmidt-repr n m u k ⟷ (∃ *vs ws*. *length vs* = *k* ∧ *length ws* = *k* ∧
($\forall\, i < k$. *dim-vec* (*vs* ! *i*) = *n*) ∧ ($\forall\, i < k$. *dim-vec* (*ws* ! *i*) = *m*) ∧
u = *tensor-combination* (*n* ∗ *m*) *vs ws*)

lemma *schmidt-rank-eq*:
assumes *u*: *dim-vec u* = *n* ∗ *m* **and** *m*: *0* < *m*
shows *schmidt-rank u m* = (*LEAST k*. *schmidt-repr n m u k*)
⟨*proof*⟩

Every u is a sum of n elementary tensors, $u = \sum_i e_i \otimes u_i$ with $u_i = (u_{im+j})_j$, so the least length exists.

lemma *schmidt-repr-exists*:
assumes *u*: *dim-vec u* = *n* ∗ *m*
shows *schmidt-repr n m u n*
⟨*proof*⟩

lemma *schmidt-rank-repr*:
assumes *u*: *dim-vec u* = *n* ∗ *m* **and** *m*: *0* < *m*
shows *schmidt-repr n m u* (*schmidt-rank u m*) **and** ⋀*k*. *schmidt-repr n m u k* ⟹ *schmidt-rank u m* ≤ *k*
⟨*proof*⟩

lemma *schmidt-repr-0*: *schmidt-repr n m u 0* ⟷ *u* = *0*$_v$ (*n* ∗ *m*)
⟨*proof*⟩

lemma *schmidt-repr-1*:
schmidt-repr n m u 1 ⟷ (∃ *v w*. *dim-vec v* = *n* ∧ *dim-vec w* = *m* ∧ *u* = *tensor-vec v w*)
⟨*proof*⟩

theorem *schmidt-rank-zero-iff*:
assumes *u*: *dim-vec u* = *n* ∗ *m* **and** *m*: *0* < *m*
shows *schmidt-rank u m* = *0* ⟷ *u* = *0*$_v$ (*n* ∗ *m*)
⟨*proof*⟩

The notes: $\psi = \psi_1 \otimes \psi_2$ iff $\mathrm{rank}_S(\psi) = 1$ (for $\psi \neq 0$).

theorem *schmidt-rank-one-iff-product*:
assumes *u*: *dim-vec u* = *n* ∗ *m* **and** *m*: *0* < *m* **and** *u0*: *u* ≠ 0_v (*n* ∗ *m*)
shows *schmidt-rank u m* = *1* ⟷ (∃ *v w*. *dim-vec v* = *n* ∧ *dim-vec w* = *m* ∧ *u* = *tensor-vec v w*)
⟨*proof*⟩

9 Day 7 (12 September 2011): tensor products of Hilbert spaces; the postulates

Remark. For Hilbert spaces $\mathcal{H}, \mathcal{K}$ set $\langle h_1 \otimes k_1 | h_2 \otimes k_2 \rangle = \langle h_1 | h_2 \rangle_{\mathcal{H}} \langle k_1 | k_2 \rangle_{\mathcal{K}}$.

Theorem (notes). This extends to a sesquilinear form on $\mathcal{H} \otimes \mathcal{K}$ with $\langle u | u \rangle = 0$ only for $u = 0$, hence to an inner product; in finite dimensions $\mathcal{H} \otimes \mathcal{K}$ with this inner product is the Hilbert space tensor product (in infinite dimensions one completes).

Proposition (notes). If $\{h_i\}$ and $\{k_j\}$ are orthonormal bases of $\mathcal{H}$ and $\mathcal{K}$, then $\{h_i \otimes k_j\}$ is an orthonormal basis of $\mathcal{H} \otimes \mathcal{K}$.

In physics notation $e_i \otimes e_j = |ij\rangle$, and for N qubits $e_{i_1} \otimes \cdots \otimes e_{i_N} = |i_1 \cdots i_N\rangle$ with the multi-index $I = (i_1, \ldots, i_N) \in \mathbb{Z}_2^N$. In the concrete model positivity of the form is automatic; the multiplicativity of the inner product and the orthonormal product basis are proved below.

lemma *tensor-inner-prod*:
assumes *dim-vec h1* = *n dim-vec h2* = *n*
dim-vec k1 = *m dim-vec k2* = *m*
shows ⟨*tensor-vec h1 k1* | *tensor-vec h2 k2*⟩ =
⟨*h1* | *h2*⟩ ∗ ⟨*k1* | *k2*⟩
⟨*proof*⟩

lemma *length-concat-map-upt*:
length (*concat* (*map* (λ*i*. *map* (λ*j*. *f i j*) [*0*..<*m*]) [*0*..<*n*])) = *n* ∗ *m*
⟨*proof*⟩

lemma *nth-concat-map-upt*:
assumes *p-lt*: *p* < *n* ∗ *m*
shows *concat* (*map* (λ*i*. *map* (λ*j*. *f i j*) [*0*..<*m*]) [*0*..<*n*]) ! *p* =
f (*p div m*) (*p mod m*)
⟨*proof*⟩

lemma *tensor-onb*:
assumes *hs-onb*: *orthonormal-list n hs*
and *hs-len*: *length hs* = *n*
and *ks-onb*: *orthonormal-list m ks*
and *ks-len*: *length ks* = *m*
shows *orthonormal-list* (*n*∗*m*)
[*tensor-vec* (*hs* ! *i*) (*ks* ! *j*). *i* ← [*0*..<*n*], *j* ← [*0*..<*m*]]
⟨*proof*⟩

9.1 Postulates of quantum mechanics

Postulate (notes) (1)**.** To each isolated physical system corresponds a Hilbert space, the *state space*; the state of the system is completely described by a unit vector in it, the *state vector*.

Postulate (notes) (2)**.** The time evolution of a closed system from t_1 to t_2 is described by a unitary $U : \mathcal{H} \to \mathcal{H}$: a system in state ψ at t_1 is in state $U\psi$ at t_2.

Postulate (notes) (3)**.** Quantum measurements are described by a collection $\{M_m\}$ of *measurement operators* on $\mathcal{H}$. If the system is in state ψ, the probability of observing m is $p_m(\psi) = \langle\psi|M_m^* M_m\psi\rangle = \|M_m\psi\|^2$; since the probabilities sum to one, $\sum_m M_m^* M_m = I$. After observing m the state is $M_m\psi/p_m(\psi)^{1/2}$.

Formally a state is *quantum-state n ψ* (a unit vector in $\mathbb{C}^n$), evolution is *evolve U ψ*, a measurement system is a list *Ms* with *measurement-system n Ms* (all $n \times n$ with $\sum_m M_m^\dagger M_m = I$), and $p_m(\psi)$ is *meas-prob M ψ*.

definition *quantum-state* :: *nat* ⇒ *complex Matrix.vec* ⇒ *bool* **where**
quantum-state n ψ ≡ *dim-vec ψ* = *n* ∧ ‖*ψ*‖ = *1*

lemma *quantum-state-norm1*:
assumes *quantum-state n ψ*
shows ‖*ψ*‖ = *1*
⟨*proof*⟩

A state is a unit vector, and evolution is $\psi \mapsto U\psi$ with U unitary; it stays a state since $\|U\psi\|^2 = \langle\psi|U^*U\psi\rangle = \|\psi\|^2 = 1$.

definition *evolve* :: *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
evolve U ψ ≡ *U* ∗*v ψ*

lemma *unitary-preserves-state*:
assumes *unitary U* **and** *quantum-state n ψ* **and** *dim-col U* = *n*
shows *quantum-state* (*dim-row U*) (*evolve U ψ*)
⟨*proof*⟩

A measurement is a list $M_1, \ldots, M_k$ with $\sum_i M_i^* M_i = I$. Outcome i has probability

$$p_i = \|M_i\psi\|^2 = \langle\psi|M_i^* M_i\,\psi\rangle,$$

and these add up to $\langle\psi|(\sum_i M_i^* M_i)\psi\rangle = \|\psi\|^2 = 1$.

definition *measurement-system* :: *nat* ⇒ *complex Matrix.mat list* ⇒ *bool* **where**
measurement-system n Ms ≡
(∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*) ∧
foldr (λ*M acc*. (*M*†) ∗ *M* + *acc*) *Ms* (*0m n n*) = *1m n*

definition *meas-prob* :: *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *real* **where**
meas-prob M ψ ≡ ‖*M* ∗*v ψ*‖²

lemma *measurement-probs-sum-one*:
assumes *ms*: *measurement-system n Ms*
and *st*: *quantum-state n ψ*
shows (∑ *M* ← *Ms*. *meas-prob M ψ*) = *1*
⟨*proof*⟩

$p_m(\psi) = \langle\psi|M_m^* M_m\psi\rangle$, the form in which the notes state Postulate 3.

lemma *meas-prob-quad-form*:
fixes *M* :: *complex Matrix.mat* **and** *ψ* :: *complex Matrix.vec*
assumes *M*: *M* ∈ *carrier-mat n n* **and** *ψ*: *dim-vec ψ* = *n*
shows *meas-prob M ψ* = *Re* ⟨*ψ* | (*M*† ∗ *M*) ∗*v ψ*⟩
⟨*proof*⟩

After outcome m the state becomes $M_m\psi/\|M_m\psi\|$:

definition *post-meas-state* :: *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
post-meas-state M ψ ≡ (*1* / *complex-of-real* (‖*M* ∗*v ψ*‖)) ·*v* (*M* ∗*v ψ*)

Part II

An introduction to quantum computing (Days 8–17)

10 Day 8 (14 September 2011): the quantum game; projective measurements and expected values

10.1 Distinguishing states

Alice has two states ψ_1, ψ_2 known to Bob; she sends one. Can Bob build a measurement system that decides with certainty which one he received? Formally one wants M_1, M_2 with $\|M_1\psi_2\| = 0$, $\|M_1\psi_1\|^2 = 1$, $\|M_2\psi_1\| = 0$, $\|M_2\psi_2\|^2 = 1$.

Case I, $\psi_1 \perp \psi_2$: take $M_1 = |\psi_1\rangle\langle\psi_1|$, $M_2 = |\psi_2\rangle\langle\psi_2|$ and $M_3 = I - M_1 - M_2$; this distinguishes with certainty (formalised below).

Case II, $\psi_1 \not\perp \psi_2$: write $\psi_2 = \alpha\psi_1 + \beta\gamma$ with $\gamma \perp \psi_1$ a unit vector, so $\alpha \neq 0$; any measurement system with $\|M_1\psi_1\| = 1$ satisfies $|\alpha|^2 \leq \|M_1\psi_2\|^2$, so outcome 1 also occurs with probability at least $|\alpha|^2$ when ψ_2 is sent (*nonorthogonal-outcome-prob* below).

Theorem (notes). Given linearly independent states $\psi_1, \ldots, \psi_n$ there is a measurement system $M_0, \ldots, M_n$ such that if outcome $i \geq 1$ occurs then ψ_i was received.

The notes' proof takes E_i the projection onto $\mathrm{span}\{\psi_j : j \neq i\}^\perp$, $M_i = E_i/\sqrt{n}$, and $M_0 = (I - \sum_i M_i^* M_i)^{1/2}$; the remark that follows observes that the inconclusive outcome 0 then has probability at least $(n-1)/n$. Formally this is *lin-indep-states-unambiguous* below.

lemma *outer-proj-self*:
  assumes *q*: *quantum-state n ψ*
  shows *outer-prod ψ ψ ∗v ψ = ψ*
⟨*proof*⟩

lemma *outer-proj-orth-zero*:
  assumes *q1*: *quantum-state n ψ1*
    and *q2*: *quantum-state n ψ2*
    and *orth*: ⟨*ψ1* | *ψ2*⟩ = *0*
  shows *outer-prod ψ1 ψ1 ∗v ψ2 = 0v n*
⟨*proof*⟩

lemma *outer-proj-idem*:
  assumes *q*: *quantum-state n ψ*
  shows (*outer-prod ψ ψ*) ∗ (*outer-prod ψ ψ*) = *outer-prod ψ ψ*
⟨*proof*⟩

lemma *outer-proj-hermitian*:
  assumes *dim-vec ψ = n*
  shows (*outer-prod ψ ψ*)† = *outer-prod ψ ψ*
⟨*proof*⟩

Case I: orthogonal states can be distinguished perfectly.

lemma *orthogonal-states-distinguishable*:
  fixes *ψ1 ψ2* :: *complex Matrix.vec*
  assumes *q1*: *quantum-state n ψ1*
    and *q2*: *quantum-state n ψ2*
    and *orth*: ⟨*ψ1* | *ψ2*⟩ = *0*
  shows ∃ *Ms* :: *complex Matrix.mat list*.

```
    measurement-system n Ms ∧
    (∃ M1 M2. M1 ∈ set Ms ∧ M2 ∈ set Ms ∧
      meas-prob M1 ψ1 = 1 ∧ meas-prob M2 ψ1 = 0 ∧
      meas-prob M1 ψ2 = 0 ∧ meas-prob M2 ψ2 = 1)
⟨proof⟩
```

The theorem on linearly independent states. Linear independence of a list of vectors is injectivity of the matrix with those columns; the formal statement produces the measurement system $M_0, \ldots, M_k$ in the order $M_1, \ldots, M_k, M_0$ (outcome $i < k$ identifies ψ_i, outcome k is inconclusive). The proof follows the notes with one change: instead of the projections E_i onto $\{\psi_j : j \neq i\}^\perp$ it uses the dual vectors ϕ_i with $\langle\phi_i|\psi_j\rangle = \delta_{ij}$ (columns of $V(V^*V)^{-1}$) and the operators $M_i = \sqrt{c}\,|e_0\rangle\langle\phi_i|$, where $c > 0$ makes $I - c\sum_i |\phi_i\rangle\langle\phi_i|$ positive; M_0 is its square root.

```
definition lin-indep-vecs :: nat ⇒ complex Matrix.vec list ⇒ bool where
  lin-indep-vecs n vs ≡ (∀ v ∈ set vs. dim-vec v = n) ∧
    (∀ x :: complex Matrix.vec. x ∈ carrier-vec (length vs) ⟶
      mat-of-cols n vs *v x = 0v n ⟶ x = 0v (length vs))

lemma outer-prod-dagger:
  fixes v w :: complex Matrix.vec
  shows (v ⊗ w)† = w ⊗ v
⟨proof⟩

lemma foldr-MdagM-shift:
  assumes Ms-c: ∀ M ∈ set Ms. M ∈ carrier-mat n n and z: z ∈ carrier-mat n n
  shows foldr (λM acc. (M†) * M + acc) Ms z
        = foldr (λM acc. (M†) * M + acc) Ms (0m n n) + z
  ⟨proof⟩

theorem lin-indep-states-unambiguous:
  fixes ψs :: complex Matrix.vec list
  assumes qs: ∀ ψ ∈ set ψs. quantum-state n ψ
    and li: lin-indep-vecs n ψs
    and k: length ψs = k and kpos: 0 < k
  shows ∃ Ms :: complex Matrix.mat list.
        measurement-system n Ms ∧ length Ms = k + 1 ∧
        (∀ i<k. ∀ j<k. i ≠ j ⟶ meas-prob (Ms ! i) (ψs ! j) = 0) ∧
        (∀ i<k. 0 < meas-prob (Ms ! i) (ψs ! i))
⟨proof⟩
```

10.2 Projective measurements, expected values and self-adjoint operators

If a measurement $\{M_m\}$ has real outcomes λ_m, the expected value of the outcome in state ψ is $E = \sum_m p_m(\psi)\lambda_m = \langle\psi|(\sum_m \lambda_m M_m^* M_m)\psi\rangle = \langle\psi|H\psi\rangle$ with $H = \sum_m \lambda_m M_m^* M_m$ self-adjoint: expected values of observables are quadratic forms of Hermitian matrices (*expected-value-eq-quad-form* below).

Formally: a measurement whose operator is the rank-one projection $|\gamma\rangle\langle\gamma|$ has $p(\psi) = |\langle\gamma|\psi\rangle|^2$ (the proposition of Day 12); an *observable* with outcomes λ_m attached to the measurement operators M_m is $H = \sum_m \lambda_m M_m^* M_m$, which is Hermitian, and the expected outcome $\sum_m \lambda_m p_m(\psi)$ equals $\langle\psi|H\psi\rangle$.

For the rank-one operator $M = |\gamma\rangle\langle\gamma|$ of a unit vector γ, $M\psi = \langle\gamma|\psi\rangle\,\gamma$ and so $p = |\langle\gamma|\psi\rangle|^2$.

```
lemma rank-one-meas-prob:
  fixes γ ψ :: complex Matrix.vec
  assumes g: quantum-state n γ and ψ: dim-vec ψ = n
  shows meas-prob (γ ⊗ γ) ψ = (cmod ⟨γ | ψ⟩)²
```

⟨*proof*⟩

An observable assigns a real value λ_i to each outcome. Its operator $A = \sum_i \lambda_i M_i^* M_i$ is Hermitian, and the expected value is its quadratic form:

$$\sum_i \lambda_i \, p_i = \sum_i \lambda_i \langle \psi | M_i^* M_i \psi \rangle = \langle \psi | A \psi \rangle.$$

definition *observable* :: *nat* ⇒ (*real* × *complex Matrix.mat*) *list* ⇒ *complex Matrix.mat* **where**
observable n lMs ≡
foldr (λ(*l*, *M*) *acc*. *complex-of-real l* ·$_m$ ((*M*†) ∗ *M*) + *acc*) *lMs* (*0*$_m$ *n n*)

definition *expected-value* :: (*real* × *complex Matrix.mat*) *list* ⇒ *complex Matrix.vec* ⇒ *real* **where**
expected-value lMs ψ ≡ (∑ (*l*, *M*) ← *lMs*. *l* ∗ *meas-prob M ψ*)

lemma *observable-Nil* [*simp*]: *observable n* [] = *0*$_m$ *n n*
⟨*proof*⟩

lemma *observable-Cons* [*simp*]:
observable n ((*l*, *M*) # *lMs*) = *complex-of-real l* ·$_m$ ((*M*†) ∗ *M*) + *observable n lMs*
⟨*proof*⟩

lemma *observable-carrier*:
assumes ∀ (*l*, *M*) ∈ *set lMs*. *M* ∈ *carrier-mat n n*
shows *observable n lMs* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *observable-hermitian*:
assumes ∀ (*l*, *M*) ∈ *set lMs*. *M* ∈ *carrier-mat n n*
shows *hermitian* (*observable n lMs*)
⟨*proof*⟩

lemma *expected-value-eq-quad-form*:
fixes *ψ* :: *complex Matrix.vec*
assumes *ψ*: *dim-vec ψ* = *n*
and *lMs*: ∀ (*l*, *M*) ∈ *set lMs*. *M* ∈ *carrier-mat n n*
shows *expected-value lMs ψ* = *Re* ⟨*ψ* | *observable n lMs* ∗$_v$ *ψ*⟩
⟨*proof*⟩

10.3 Non-orthogonal states cannot be distinguished

lemma *meas-sum-norm*:
assumes *ms*: *measurement-system n Ms* **and** *v*: *dim-vec v* = *n*
shows (∑ *l*<*length Ms*. ‖*Ms* ! *l* ∗$_v$ *v*‖^2) = ‖*v*‖^2
⟨*proof*⟩

lemma *inner-minus-smult*:
assumes *d1*: *dim-vec u* = *n* **and** *d2*: *dim-vec w* = *n*
shows ⟨*w* − *a* ·$_v$ *u* | *w* − *a* ·$_v$ *u*⟩ = ⟨*w*|*w*⟩ − *a* ∗ ⟨*w*|*u*⟩ − *cnj a* ∗ ⟨*u*|*w*⟩ + *a* ∗ *cnj a* ∗ ⟨*u*|*u*⟩
⟨*proof*⟩

lemma *norm-sq-minus-component*:
assumes *d1*: *dim-vec u* = *n* **and** *d2*: *dim-vec w* = *n* **and** *u*: ‖*u*‖ = *1*
shows ‖*w* − ⟨*u*|*w*⟩ ·$_v$ *u*‖^2 = ‖*w*‖^2 − (*cmod* ⟨*u*|*w*⟩)^2
⟨*proof*⟩

D8-2: if outcome i certainly occurs on ψ_1, it occurs on ψ_2 with probability at least $|\langle \psi_1 | \psi_2 \rangle|^2$.

theorem *nonorthogonal-outcome-prob*:

assumes *ms*: *measurement-system n Ms* **and** *q1*: *quantum-state n ψ1* **and** *q2*: *quantum-state n ψ2*
and *i*: *i < length Ms* **and** *p1*: *meas-prob (Ms ! i) ψ1 = 1*
shows *(cmod ⟨ψ1|ψ2⟩)² ≤ meas-prob (Ms ! i) ψ2*
⟨*proof*⟩

corollary *nonorthogonal-not-distinguishable*:
assumes *ms*: *measurement-system n Ms* **and** *q1*: *quantum-state n ψ1* **and** *q2*: *quantum-state n ψ2*
and *no*: *⟨ψ1|ψ2⟩ ≠ 0* **and** *i*: *i < length Ms* **and** *p1*: *meas-prob (Ms ! i) ψ1 = 1*
shows *meas-prob (Ms ! i) ψ2 > 0*
⟨*proof*⟩

11 Day 9 (16 September 2011): positive operator-valued measures; composite systems; entanglement

11.1 Positive operator-valued measures

If two measurement systems satisfy $M_m^* M_m = \tilde{M}_m^* \tilde{M}_m$ for all m then $p_m = \tilde{p}_m$: only the operators $M_m^* M_m$ matter, and $\sum_m M_m^* M_m = I$. Conversely if $P_m \geq 0$ with $\sum_m P_m = I$ then $M_m = P_m^{1/2}$ is a measurement system.

Definition (notes). A *positive operator-valued measure* (POVM) is a set $\{P_m\}$ of positive operators with $\sum_m P_m = I$.

definition *povm* :: *nat ⇒ complex Matrix.mat list ⇒ bool* **where**
povm n Ps ≡
(∀ P ∈ set Ps. P ∈ carrier-mat n n ∧ psd P) ∧
foldr (λP acc. P + acc) Ps (0_m n n) = 1_m n

Every measurement gives a POVM: $P_i = M_i^* M_i$ is positive because $\langle x|M_i^* M_i x\rangle = \|M_i x\|^2 \geq 0$, and $\sum_i P_i = I$.

lemma *measurement-induces-povm*:
assumes *ms*: *measurement-system n Ms*
shows *povm n (map (λM. (M†) * M) Ms)*
⟨*proof*⟩

Conversely, every POVM comes from a measurement: $P_i \geq 0$ has the positive square root $M_i = P_i^{1/2}$, with $M_i^* M_i = P_i$, and $\sum_i P_i = I$ is the measurement condition.

lemma *psd-sqrt-measurement-factor*:
assumes *Pcar*: *P ∈ carrier-mat n n*
and *Ppsd*: *psd P*
shows *∃ M. M ∈ carrier-mat n n ∧ M† * M = P*
⟨*proof*⟩

lemma *psd-sqrt-measurement-factors*:
assumes *Ps*: *∀ P ∈ set Ps. P ∈ carrier-mat n n ∧ psd P*
shows *∃ Ms. (∀ M ∈ set Ms. M ∈ carrier-mat n n) ∧*
*map (λM. M† * M) Ms = Ps*
⟨*proof*⟩

lemma *povm-from-sqrt*:
assumes *pv*: *povm n Ps*
shows *∃ Ms. measurement-system n Ms ∧*
*map (λM. M† * M) Ms = Ps*
⟨*proof*⟩

lemma *povm-prob*:
assumes *pv*: *povm n Ps* **and** *st*: *quantum-state n ψ*
and *Pin*: *P ∈ set Ps*
shows *Re* (⟨*ψ* | *P* ∗*v ψ*⟩) ≥ *0*
⟨*proof*⟩

11.2 Composite systems

The notes' example: photons in states $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $\frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle)$ form the pair $\frac{1}{2}(|00\rangle + i|01\rangle + |10\rangle + i|11\rangle) \in \mathbb{C}^2 \otimes \mathbb{C}^2$, a vector of Schmidt rank 1. Two photons described by a unit vector $\psi \in \mathbb{C}^2 \otimes \mathbb{C}^2$ can be regarded as two separate subsystems iff $\psi = \psi_1 \otimes \psi_2$ iff the Schmidt rank of ψ is 1; otherwise ψ is *entangled*, which does happen in nature.

Tensor products of operators: for $R : \mathcal{H} \to \mathcal{H}$ and $T : \mathcal{K} \to \mathcal{K}$ there is a unique $R \otimes T$ with $(R \otimes T)(\sum_l h_l \otimes k_l) = \sum_l Rh_l \otimes Tk_l$, and (1) $(R_1 \otimes T_1)(R_2 \otimes T_2) = R_1R_2 \otimes T_1T_2$, (2) $(R \otimes T)^* = R^* \otimes T^*$ (both proved in Unitary_Tensor_Algebra for the Kronecker product).

Remark. If lab A has the measurement system $\{M_m\}$ on $\mathcal{H}$ and lab B has state space $\mathcal{K}$, the measurement system of the composite is $\{M_m \otimes I_{\mathcal{K}}\}$ and $p_m^{AB}(\psi \otimes \phi) = \|(M_m \otimes I)(\psi \otimes \phi)\|^2 = p_m^A(\psi)$.

Postulate (notes) (4). The state space of a composite system is the tensor product of the state spaces of its components; if the i-th component is in state ψ_i, the system is in state $\psi_1 \otimes \cdots \otimes \psi_n$.

lemma *tensor-of-states-is-state*:
assumes *quantum-state n ψ1* **and** *quantum-state m ψ2*
shows *quantum-state* (*n*∗*m*) (*tensor-vec ψ1 ψ2*)
⟨*proof*⟩

lemma *tensor-mat-vec-mult*:
assumes *A*: *A ∈ carrier-mat nr nc*
and *B*: *B ∈ carrier-mat mr mc*
and *v*: *dim-vec v* = *nc*
and *w*: *dim-vec w* = *mc*
shows (*A* ⨂ *B*) ∗*v tensor-vec v w* = *tensor-vec* (*A* ∗*v v*) (*B* ∗*v w*)
⟨*proof*⟩

lemma *tensor-mat-vec-mult-right-id*:
assumes *A*: *A ∈ carrier-mat n n*
and *v*: *dim-vec v* = *n*
and *w*: *dim-vec w* = *m*
shows (*A* ⨂ *1m m*) ∗*v tensor-vec v w* = *tensor-vec* (*A* ∗*v v*) *w*
⟨*proof*⟩

lemma *composite-measurement-prob*:
fixes *M* :: *complex Matrix.mat* **and** *ψ* :: *complex Matrix.vec* **and** *φ* :: *complex Matrix.vec*
assumes *psi-st*: *quantum-state n ψ* **and** *phi-st*: *quantum-state m φ*
and *M-sq*: *M ∈ carrier-mat n n*
shows *meas-prob* (*M* ⨂ *1m m*) (*tensor-vec ψ φ*) = *meas-prob M ψ*
⟨*proof*⟩

11.3 Entanglement

A state $\psi \in \mathcal{H}_A \otimes \mathcal{H}_B$ is *entangled* if it is not a product state $u \otimes v$. The Bell state $\gamma = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ is entangled. The notes' first application of entanglement (A and B share γ; A

measures $|0\rangle\langle 0|, |1\rangle\langle 1|$ and gets each outcome with probability $\frac{1}{2}$, after which B's measurement is determined) is *bell-measurement* below.

definition *entangled* :: *nat* ⇒ *nat* ⇒ *complex Matrix.vec* ⇒ *bool* **where**
entangled n m ψ ≡
dim-vec ψ = *n* ∗ *m* ∧
¬ (∃ *u v*. *dim-vec u* = *n* ∧ *dim-vec v* = *m* ∧ *ψ* = *tensor-vec u v*)

definition *bell-state* :: *complex Matrix.vec* **where**
bell-state ≡ (*1* / *csqrt 2*) ·$_v$
(*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 0*) +
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 1*))

lemma *bell-state-norm*:
‖*bell-state*‖ = *1*
⟨*proof*⟩

lemma *bell-state-is-quantum-state*:
quantum-state 4 bell-state
⟨*proof*⟩

lemma *bell-state-entangled*:
entangled 2 2 bell-state
⟨*proof*⟩

corollary *entangled-iff-schmidt-rank*:
assumes *u*: *dim-vec u* = *n* ∗ *m* **and** *m*: *0* < *m* **and** *u0*: *u* ≠ *0*$_v$ (*n* ∗ *m*)
shows *entangled n m u* ⟷ *1* < *schmidt-rank u m*
⟨*proof*⟩

corollary *schmidt-rank-bell-state*: *schmidt-rank bell-state 2* = *2*
⟨*proof*⟩

11.4 Measuring one half of a Bell pair

definition *e0* :: *complex Matrix.vec* **where** *e0* = *unit-vec 2 0*
definition *e1* :: *complex Matrix.vec* **where** *e1* = *unit-vec 2 1*
definition *proj0* :: *complex Matrix.mat* **where** *proj0* = *e0* ⊘ *e0*
definition *proj1* :: *complex Matrix.mat* **where** *proj1* = *e1* ⊘ *e1*

lemma *e-dims* [*simp*]: *dim-vec e0* = *2 dim-vec e1* = *2*
⟨*proof*⟩

lemma *e-inner* [*simp*]: ⟨*e0*|*e0*⟩ = *1* ⟨*e0*|*e1*⟩ = *0* ⟨*e1*|*e0*⟩ = *0* ⟨*e1*|*e1*⟩ = *1*
⟨*proof*⟩

lemma *proj-carrier* [*simp*]: *proj0* ∈ *carrier-mat 2 2 proj1* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *proj-apply* [*simp*]:
proj0 ∗$_v$ *e0* = *e0 proj0* ∗$_v$ *e1* = *0*$_v$ *2 proj1* ∗$_v$ *e0* = *0*$_v$ *2 proj1* ∗$_v$ *e1* = *e1*
⟨*proof*⟩

lemma *bell-state-e*: *bell-state* = (*1* / *csqrt 2*) ·$_v$ (*tensor-vec e0 e0* + *tensor-vec e1 e1*)
⟨*proof*⟩

lemma *tensor-zero-left*: *dim-vec w* = *2* ⟹ *tensor-vec* (*0*$_v$ *2*) *w* = *0*$_v$ *4*
⟨*proof*⟩

Measuring the first qubit of $\beta = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ with $P_0 = |0\rangle\langle 0|$, $P_1 = |1\rangle\langle 1|$:

$$(P_0 \otimes I)\beta = \tfrac{1}{\sqrt{2}}|00\rangle, \qquad (P_1 \otimes I)\beta = \tfrac{1}{\sqrt{2}}|11\rangle,$$

so each outcome has probability $\frac{1}{2}$ and leaves the second qubit in the same basis state as the first.

lemma *A-meas-bell*:
(proj0 ⨂ *1*$_m$ *2) *$*_v$ *bell-state = (1 / csqrt 2) *$\cdot_v$ *tensor-vec e0 e0*
(proj1 ⨂ *1*$_m$ *2) *$*_v$ *bell-state = (1 / csqrt 2) *$\cdot_v$ *tensor-vec e1 e1*
⟨*proof*⟩

lemma *tensor-e-norm*: ‖*tensor-vec e0 e0*‖ = *1* ‖*tensor-vec e1 e1*‖ = *1*
⟨*proof*⟩

D9-6: A measures $|0\rangle\langle 0|, |1\rangle\langle 1|$ on the shared state γ: each outcome has probability $\frac{1}{2}$, and γ collapses to $e_0 \otimes e_0$ resp. $e_1 \otimes e_1$, after which B's measurement $I \otimes M_m$ gives A's outcome with probability 1.

theorem *bell-measurement*:
meas-prob (proj0 ⨂ *1*$_m$ *2) bell-state = 1/2*
meas-prob (proj1 ⨂ *1*$_m$ *2) bell-state = 1/2*
post-meas-state (proj0 ⨂ *1*$_m$ *2) bell-state = tensor-vec e0 e0*
post-meas-state (proj1 ⨂ *1*$_m$ *2) bell-state = tensor-vec e1 e1*
meas-prob (1$_m$ *2* ⨂ *proj0) (tensor-vec e0 e0) = 1*
meas-prob (1$_m$ *2* ⨂ *proj1) (tensor-vec e1 e1) = 1*
⟨*proof*⟩

lemma *proj-explicit*:
proj0 = Matrix.mat 2 2 (λ*(i,j). if i = 0 ∧ j = 0 then 1 else 0)*
proj1 = Matrix.mat 2 2 (λ*(i,j). if i = 1 ∧ j = 1 then 1 else 0)*
⟨*proof*⟩

lemma *proj-measurement-system*: *measurement-system 2* [*proj0*, *proj1*]
⟨*proof*⟩

12 Day 10 (19 September 2011): superdense coding; classical and quantum gates

12.1 One-qubit gates

The one-qubit gates of the notes:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \text{ (NOT)}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad H = \tfrac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}.$$

H is the Hadamard gate; the notes write "$H^2 = X$" here and correct it to $H^2 = I$ on Day 11, which is what is proved below. H and S together generate all 2×2 unitaries (not formalised). Formally X, Y, Z are *Pauli-X*, *Pauli-Y*, *Pauli-Z*, H is *Hadamard* (the library's *Quantum.H*) and S is *Phase-S*.

12.1.1 The Pauli matrices

The notes use two sign conventions for Y: $\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$ on Day 10 (the library's Y) and $\begin{pmatrix} 0 & i \\ -i & 0 \end{pmatrix}$ on Day 32. *Pauli-Y* follows Day 32; both are Hermitian unitary involutions, and the Pauli identities

below carry the corresponding signs. *Pauli-X* and *Pauli-Z* are the library's *Quantum.X* and *Quantum.Z*.

definition *Pauli-X* :: *complex Matrix.mat* **where**
Pauli-X ≡ *Matrix.mat 2 2*
(λ(*i*,*j*). *if* (*i* = *0* ∧ *j* = *1*) ∨ (*i* = *1* ∧ *j* = *0*) *then 1 else 0*)

definition *Pauli-Y* :: *complex Matrix.mat* **where**
Pauli-Y ≡ *Matrix.mat 2 2*
(λ(*i*,*j*). *if i* = *0* ∧ *j* = *1 then* i
else if i = *1* ∧ *j* = *0 then* −i *else 0*)

definition *Pauli-Z* :: *complex Matrix.mat* **where**
Pauli-Z ≡ *Matrix.mat 2 2*
(λ(*i*,*j*). *if i* = *0* ∧ *j* = *0 then 1*
else if i = *1* ∧ *j* = *1 then* −*1 else 0*)

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad Y = \begin{pmatrix} 0 & i \\ -i & 0 \end{pmatrix}, \qquad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

(The notes' Y is the negative of the physicists' σ_y.) Each is Hermitian and squares to I, so each is unitary: $U^*U = U^2 = I$.

lemma *Pauli-X-carrier* [*simp*]: *Pauli-X* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *Pauli-Y-carrier* [*simp*]: *Pauli-Y* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *Pauli-Z-carrier* [*simp*]: *Pauli-Z* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *Pauli-X-dagger*: *Pauli-X*† = *Pauli-X*
⟨*proof*⟩

lemma *Pauli-X-squared*: *Pauli-X* ∗ *Pauli-X* = 1_m *2*
⟨*proof*⟩

lemma *Pauli-X-unitary*: *unitary Pauli-X*
⟨*proof*⟩

lemma *Pauli-Y-dagger*: *Pauli-Y*† = *Pauli-Y*
⟨*proof*⟩

lemma *Pauli-Y-squared*: *Pauli-Y* ∗ *Pauli-Y* = 1_m *2*
⟨*proof*⟩

lemma *Pauli-Y-unitary*: *unitary Pauli-Y*
⟨*proof*⟩

lemma *Pauli-Z-dagger*: *Pauli-Z*† = *Pauli-Z*
⟨*proof*⟩

lemma *Pauli-Z-squared*: *Pauli-Z* ∗ *Pauli-Z* = 1_m *2*
⟨*proof*⟩

lemma *Pauli-Z-unitary*: *unitary Pauli-Z*
⟨*proof*⟩

definition *Hadamard* :: *complex Matrix.mat* **where**
Hadamard ≡ *Quantum.H*

lemma *csqrt-two*: *csqrt* (*2*::*complex*) = (*sqrt 2* :: *complex*)
⟨*proof*⟩

lemma *hadamard-squared*:
Hadamard ∗ *Hadamard* = *1*$_m$ *2*
⟨*proof*⟩

lemma *hadamard-e0*:
Hadamard ∗$_v$ *unit-vec 2 0* =
(*1 / csqrt 2*) ·$_v$ (*unit-vec 2 0* + *unit-vec 2 1*)
⟨*proof*⟩

lemma *hadamard-e1*:
Hadamard ∗$_v$ *unit-vec 2 1* =
(*1 / csqrt 2*) ·$_v$ (*unit-vec 2 0* − *unit-vec 2 1*)
⟨*proof*⟩

The phase gate $S = \mathrm{diag}(1, i)$ is unitary.

definition *Phase-S* :: *complex Matrix.mat* **where**
Phase-S ≡ *Quantum.S*

lemma *Phase-S-carrier* [*simp*]: *Phase-S* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *Phase-S-dagger-S*: *Phase-S*† ∗ *Phase-S* = *1*$_m$ *2*
⟨*proof*⟩

lemma *Phase-S-S-dagger*: *Phase-S* ∗ (*Phase-S*†) = *1*$_m$ *2*
⟨*proof*⟩

lemma *Phase-S-unitary*: *unitary Phase-S*
⟨*proof*⟩

12.2 Two- and three-qubit gates

The notes tabulate the classical gates AND, OR, XOR, NAND, NOR (Peirce: NAND generates all Boolean operations; Sheffer: so does NOR). No two-bit classical gate is a quantum gate, since it maps two dimensions to one and is not reversible. The controlled NOT acts by $|a, b\rangle \mapsto |a, b \oplus a\rangle$:

$$\mathrm{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix},$$

which is unitary, and Toffoli's CCNOT (identity on the first six basis vectors, swapping $|110\rangle$ and $|111\rangle$) is a reversible gate generating all Boolean operations; it is defined below.

lemma *CNOT-unitary*: *unitary CNOT*
⟨*proof*⟩

CNOT flips the target qubit when the control is 1, $|ab\rangle \mapsto |a,\, a \oplus b\rangle$:

$$|00\rangle \mapsto |00\rangle, \quad |01\rangle \mapsto |01\rangle, \quad |10\rangle \mapsto |11\rangle, \quad |11\rangle \mapsto |10\rangle.$$

lemma *CNOT-00*:
CNOT $*_v$ (*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 0*)) =
tensor-vec (*unit-vec 2 0*) (*unit-vec 2 0*)
⟨*proof*⟩

lemma *CNOT-10*:
CNOT $*_v$ (*tensor-vec* (*unit-vec 2 1*) (*unit-vec 2 0*)) =
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 1*)
⟨*proof*⟩

lemma *CNOT-01*:
CNOT $*_v$ (*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 1*)) =
tensor-vec (*unit-vec 2 0*) (*unit-vec 2 1*)
⟨*proof*⟩

lemma *CNOT-11*:
CNOT $*_v$ (*tensor-vec* (*unit-vec 2 1*) (*unit-vec 2 1*)) =
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 0*)
⟨*proof*⟩

In the Kronecker model $|ab\rangle = e_a \otimes e_b = e_{2a+b} \in \mathbb{C}^4$:

lemma *tensor-unit-vec-2-00*:
tensor-vec (*unit-vec 2 0*) (*unit-vec 2 0*) = *unit-vec 4 0*
⟨*proof*⟩

lemma *tensor-unit-vec-2-01*:
tensor-vec (*unit-vec 2 0*) (*unit-vec 2 1*) = *unit-vec 4 1*
⟨*proof*⟩

lemma *tensor-unit-vec-2-10*:
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 0*) = *unit-vec 4 2*
⟨*proof*⟩

lemma *tensor-unit-vec-2-11*:
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 1*) = *unit-vec 4 3*
⟨*proof*⟩

definition *CCNOT* :: *complex Matrix.mat* **where**
CCNOT ≡ *Matrix.mat 8 8*
(λ(*i*,*j*). *if* (*i* = *6* ∧ *j* = *7*) ∨ (*i* = *7* ∧ *j* = *6*)
then 1
else if i = *j* ∧ *i* < *6 then 1 else 0*)

lemma *CCNOT-dagger* [*simp*]:
$CCNOT^\dagger$ = *CCNOT*
⟨*proof*⟩

lemma *CCNOT-dim-row* [*simp*]:
dim-row CCNOT = *8*
⟨*proof*⟩

lemma *CCNOT-dim-col* [*simp*]:
dim-col CCNOT = *8*
⟨*proof*⟩

lemma *CCNOT-inv* [*simp*]:
CCNOT ∗ *CCNOT* = 1_m *8*

⟨*proof*⟩

lemma *CCNOT-unitary*: *unitary CCNOT*
⟨*proof*⟩

12.3 Superdense coding

A and B share $\gamma = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. Applying nothing, $\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}$, $\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}$ or $\begin{pmatrix}0 & 1\\ -1 & 0\end{pmatrix}$ to its qubit, A turns γ into one of the four orthonormal Bell states $\frac{1}{\sqrt{2}}(|00\rangle \pm |11\rangle)$, $\frac{1}{\sqrt{2}}(|10\rangle \pm |01\rangle)$, so by sending one qubit A communicates two bits; with $2m$ shared qubits, 4^m messages. Below: the four Bell states and their orthonormality (the action table itself is not formalised).

definition *bell-basis* :: *complex Matrix.vec list* **where**
bell-basis ≡
[(*1/csqrt 2*) ·$_v$ (*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 0*) +
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 1*)),
(*1/csqrt 2*) ·$_v$ (*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 0*) −
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 1*)),
(*1/csqrt 2*) ·$_v$ (*tensor-vec* (*unit-vec 2 1*) (*unit-vec 2 0*) +
tensor-vec (*unit-vec 2 0*) (*unit-vec 2 1*)),
(*1/csqrt 2*) ·$_v$ (*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 1*) −
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 0*))]

lemma *bell-basis-orthonormal*: *orthonormal-list 4 bell-basis*
⟨*proof*⟩

Lab A and Lab B share $\gamma = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, Lab A holding the first qubit. Lab A multiplies its qubit by I, $\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}$, $\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}$ or $\begin{pmatrix}0 & 1\\ -1 & 0\end{pmatrix}$ and sends it to Lab B, which then holds $\frac{1}{\sqrt{2}}(|00\rangle \pm |11\rangle)$ or $\frac{1}{\sqrt{2}}(|01\rangle \pm |10\rangle)$: four orthonormal states, so one qubit carries two bits. The four matrices are I, Z, X and ZX (*sd-op-paulis*).

lemma *less4*: *i* < (*4*::*nat*) ⟷ *i* = *0* ∨ *i* = *1* ∨ *i* = *2* ∨ *i* = *3* ⟨*proof*⟩

lemma *tensor-I2-apply*:
fixes *A* :: *complex Matrix.mat* **and** *v* :: *complex Matrix.vec*
assumes *A*: *A* ∈ *carrier-mat 2 2* **and** *v*: *dim-vec v* = *4* **and** *i*: *i* < *4*
shows (*Tensor.tensor-mat A* (*1*$_m$ *2*) ∗$_v$ *v*) $ *i* =
(∑*j*<*4*. *A* $$ (*i div 2*, *j div 2*) ∗ (*if i mod 2* = *j mod 2 then 1 else 0*) ∗ *v* $ *j*)
⟨*proof*⟩

definition *superdense-gamma* :: *complex Matrix.vec* **where**
superdense-gamma = *bell-basis* ! *0*

Lab A applies one of the notes' four matrices to its qubit.

definition *sd-op* :: *nat* ⇒ *complex Matrix.mat* **where**
sd-op k = (*if k* = *0 then mat-of-rows-list 2* [[*1*, *0*], [*0*, *1*]]
else if k = *1 then mat-of-rows-list 2* [[*1*, *0*], [*0*, −*1*]]
else if k = *2 then mat-of-rows-list 2* [[*0*, *1*], [*1*, *0*]]
else mat-of-rows-list 2 [[*0*, *1*], [−*1*, *0*]])

lemma *sd-op-carrier*: *sd-op k* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *sd-op-paulis*:
sd-op 0 = *1*$_m$ *2 sd-op 1* = *Pauli-Z sd-op 2* = *Pauli-X sd-op 3* = *Pauli-Z* ∗ *Pauli-X*

⟨*proof*⟩

The notes' table: the four results are the four Bell states.

lemma *superdense-coding-0*: *Tensor.tensor-mat* (*sd-op 0*) (*1*$_m$ *2*) $*_v$ *superdense-gamma* = *bell-basis* ! *0*
⟨*proof*⟩

lemma *superdense-coding-1*: *Tensor.tensor-mat* (*sd-op 1*) (*1*$_m$ *2*) $*_v$ *superdense-gamma* = *bell-basis* ! *1*
⟨*proof*⟩

lemma *superdense-coding-2*: *Tensor.tensor-mat* (*sd-op 2*) (*1*$_m$ *2*) $*_v$ *superdense-gamma* = *bell-basis* ! *2*
⟨*proof*⟩

lemma *superdense-coding-3*: *Tensor.tensor-mat* (*sd-op 3*) (*1*$_m$ *2*) $*_v$ *superdense-gamma* = *bell-basis* ! *3*
⟨*proof*⟩

theorem *superdense-coding*:
assumes *k*: *k* < *4*
shows *Tensor.tensor-mat* (*sd-op k*) (*1*$_m$ *2*) $*_v$ *superdense-gamma* = *bell-basis* ! *k*
⟨*proof*⟩

corollary *superdense-coding-orthonormal*:
orthonormal-list 4 (*map* (λ*k*. *Tensor.tensor-mat* (*sd-op k*) (*1*$_m$ *2*) $*_v$ *superdense-gamma*) [*0*..<*4*])
⟨*proof*⟩

13 Day 11 (21 September 2011): gates revisited, circuit diagrams, no-cloning

The lecture opens with two corrections: $H^2 = I$ (not X), with $He_0 = (e_0 + e_1)/\sqrt{2}$, $Xe_0 = e_1$, $Xe_1 = e_0$, $He_1 = (e_0 - e_1)/\sqrt{2}$, so that X and H are reflections; and the truth tables of CNOT ($|a, b\rangle \mapsto |a, b + a\rangle$, arithmetic in $\mathbb{Z}_2$) and of CCNOT ($|a, b, c\rangle \mapsto |a, b, c + ab\rangle$). Circuit diagrams are introduced; the CNOT with control on the second wire has matrix $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$, and three alternating CNOTs compose to the swap of the two wires (*SWAP-three-CNOTs* below).

13.1 Cloning and no-cloning

CNOT clones the computational basis: $U(e_a \otimes e_0) = e_a \otimes e_a$, and more generally $|i_1 \cdots i_n\rangle \otimes |0 \cdots 0\rangle \mapsto |i_1 \cdots i_n\rangle \otimes |i_1 \cdots i_n\rangle$ for some unitary. But for $\psi = \alpha e_0 + \beta e_1$ with $\alpha\beta \neq 0$, $U(\psi \otimes e_0) \neq \psi \otimes \psi$.

Theorem (notes) (No-cloning)**.** There is no state ϕ and unitary U on $\mathcal{H} \otimes \mathcal{H}$ with $U(\psi \otimes \phi) = \psi \otimes \psi$ for all $\psi \in \mathcal{H}$.

The notes argue with $-\psi$; the formal proof (for $\dim \mathcal{H} = n \geq 2$ and unit vectors ψ) uses two states with inner product neither 0 nor 1.

definition *is-cloner* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *bool* **where**
is-cloner n U $\varphi \equiv$

$U \in$ *carrier-mat* $(n\hat{}2)$ $(n\hat{}2) \wedge$ *unitary* $U \wedge$ *dim-vec* $\varphi = n \wedge$
$(\forall \psi.$ *quantum-state* n $\psi \longrightarrow$
$U *_v$ *tensor-vec* ψ $\varphi =$ *tensor-vec* ψ $\psi)$

Two states whose inner product is neither 0 nor 1 exist once $n \geq 2$.

lemma *two-states-nontrivial-inner-prod*:
assumes n: $2 \leq n$
obtains ψ_1 ψ_2 :: *complex Matrix.vec*
where *quantum-state* n ψ_1 **and** *quantum-state* n ψ_2
and $\langle \psi_1 \mid \psi_2 \rangle \neq (0 :: real)$ **and** $\langle \psi_1 \mid \psi_2 \rangle \neq (1 :: real)$
⟨*proof*⟩

theorem *no-cloning*:
assumes *n2*: $2 \leq n$
shows $\neg$ $(\exists U$ $\varphi.$ *is-cloner* n U $\varphi)$
⟨*proof*⟩

13.2 SWAP from three CNOTs

lemma *less4-cases*: $(i::nat) < 4 \longleftrightarrow i = 0 \vee i = 1 \vee i = 2 \vee i = 3$
⟨*proof*⟩

CNOT′ is CNOT controlled by the second qubit.

definition *CNOT′* :: *complex Matrix.mat* **where**
CNOT′ = *Matrix.mat 4 4* $(\lambda(i,j).$ *if* $(i,j) \in \{(0,0),(1,3),(2,2),(3,1)\}$ *then 1 else 0*)

D11-3: SWAP = CNOT · CNOT′ · CNOT (*SWAP* is the AFP's).

theorem *SWAP-three-CNOTs*: *SWAP* = *CNOT* ∗ *CNOT′* ∗ *CNOT*
⟨*proof*⟩

lemma *SWAP-basis*:
assumes $a < 2$ $b < 2$
shows *SWAP* $*_v$ *tensor-vec* (*unit-vec 2 a*) (*unit-vec 2 b*) = *tensor-vec* (*unit-vec 2 b*) (*unit-vec 2 a*)
⟨*proof*⟩

lemma *CNOT′-basis*:
assumes $a < 2$ $b < 2$
shows *CNOT′* $*_v$ *tensor-vec* (*unit-vec 2 a*) (*unit-vec 2 b*) = *tensor-vec* (*unit-vec 2* ((*a* + *b*) *mod 2*)) (*unit-vec 2 b*)
⟨*proof*⟩

13.3 The CCNOT truth table (D11-2)

lemma *CCNOT-unit-vec*:
assumes *k*: $k < 8$
shows *CCNOT* $*_v$ *unit-vec 8 k* = *unit-vec 8* (*if k = 6 then 7 else if k = 7 then 6 else k*)
⟨*proof*⟩

On basis states $|a, b, c\rangle = e_a \otimes e_b \otimes e_c$: when $a = 0$ nothing happens; when $a = 1$, CNOT acts on b, c.

theorem *CCNOT-basis*:
assumes *a*: $a < 2$ **and** *b*: $b < 2$ **and** *c*: $c < 2$
shows *CCNOT* $*_v$ *tensor-vec* (*unit-vec 2 a*) (*tensor-vec* (*unit-vec 2 b*) (*unit-vec 2 c*)) =
tensor-vec (*unit-vec 2 a*) (*tensor-vec* (*unit-vec 2 b*) (*unit-vec 2* (*if a = 1* ∧ *b = 1 then 1 − c else c*)))
⟨*proof*⟩

corollary *CCNOT-truth-table*:
fixes e :: *nat* ⇒ *complex Matrix.vec*
defines $e \equiv \lambda a.$ *unit-vec 2 a*
shows *CCNOT* $*_v$ *tensor-vec* (*e 0*) (*tensor-vec* (*e 0*) (*e 0*)) = *tensor-vec* (*e 0*) (*tensor-vec* (*e 0*) (*e 0*))
and *CCNOT* $*_v$ *tensor-vec* (*e 0*) (*tensor-vec* (*e 0*) (*e 1*)) = *tensor-vec* (*e 0*) (*tensor-vec* (*e 0*) (*e 1*))
and *CCNOT* $*_v$ *tensor-vec* (*e 0*) (*tensor-vec* (*e 1*) (*e 0*)) = *tensor-vec* (*e 0*) (*tensor-vec* (*e 1*) (*e 0*))
and *CCNOT* $*_v$ *tensor-vec* (*e 0*) (*tensor-vec* (*e 1*) (*e 1*)) = *tensor-vec* (*e 0*) (*tensor-vec* (*e 1*) (*e 1*))
and *CCNOT* $*_v$ *tensor-vec* (*e 1*) (*tensor-vec* (*e 0*) (*e 0*)) = *tensor-vec* (*e 1*) (*tensor-vec* (*e 0*) (*e 0*))
and *CCNOT* $*_v$ *tensor-vec* (*e 1*) (*tensor-vec* (*e 0*) (*e 1*)) = *tensor-vec* (*e 1*) (*tensor-vec* (*e 0*) (*e 1*))
and *CCNOT* $*_v$ *tensor-vec* (*e 1*) (*tensor-vec* (*e 1*) (*e 0*)) = *tensor-vec* (*e 1*) (*tensor-vec* (*e 1*) (*e 1*))
and *CCNOT* $*_v$ *tensor-vec* (*e 1*) (*tensor-vec* (*e 1*) (*e 1*)) = *tensor-vec* (*e 1*) (*tensor-vec* (*e 1*) (*e 0*))
⟨*proof*⟩

14 Day 12 (23 September 2011): quantum parallelism

Since $He_0 = (e_0 + e_1)/\sqrt{2}$,

$$H^{\otimes N}(e_0 \otimes \cdots \otimes e_0) = \left(\tfrac{1}{\sqrt{2}}\right)^N \sum_{J \in \mathbb{Z}_2^N} e_J = \left(\tfrac{1}{\sqrt{2}}\right)^N \sum_J |J\rangle.$$

Only the case $N = 1$ is stated here; the general identity is *hadamard-uniform-N* below (with *tensor-pow-mat* for $H^{\otimes N}$).

Definition (notes) (Oracle)**.** For $f : \mathbb{Z}_2^N \to \mathbb{Z}_2$, the vectors $e_J \otimes e_{i+f(J)}$ form an orthonormal basis of $(\mathbb{C}^2)^{\otimes N} \otimes \mathbb{C}^2$, so there is a unitary U_f with $U_f(e_J \otimes e_i) = e_J \otimes e_{i+f(J)}$, the *oracle* for f.

Proposition (notes). For a unit vector γ, the measurement $P_0 = |\gamma\rangle\langle\gamma|$, $P_1 = I - P_0$ has $p_0(\psi) = |\langle\gamma|\psi\rangle|^2$ and $p_1(\psi) = 1 - |\langle\gamma|\psi\rangle|^2$.

Application: with $x = (He_0)^{\otimes N}$, $U_f(x \otimes e_0) = 2^{-N/2} \sum_J e_J \otimes e_{f(J)}$, and measuring against $\gamma = x \otimes e_0$ gives $\langle\gamma|U_f(x \otimes e_0)\rangle = M/2^N$ with $M = \#\{J : f(J) = 0\}$, hence $p_0 = (M/2^N)^2$: repeating the experiment estimates M (a Bernoulli trial). The existence of the oracle is formalised below; the proposition is *rank-one-meas-prob* (Day 8); the counting identity is *counting-probability* below.

definition *oracle-unitary* :: (*nat* ⇒ *nat*) ⇒ *nat* ⇒ *bool* **where**
oracle-unitary f N ≡
∃ *U* :: *complex Matrix.mat*.
U ∈ *carrier-mat* (*2^(N+1)*) (*2^(N+1)*) ∧
unitary U ∧
(∀ *J* < *2^N*. ∀ *i* < *2*.
U $*_v$ (*tensor-vec* (*unit-vec* (*2^N*) *J*) (*unit-vec 2 i*)) =
tensor-vec (*unit-vec* (*2^N*) *J*) (*unit-vec 2* ((*i* + *f J*) *mod 2*)))

theorem *oracle-exists*:
fixes *f* :: *nat* ⇒ *nat*
assumes *f-range*: ∀ *J* < *2^*(*N* :: *nat*). *f J* < *2*
shows *oracle-unitary f N*
⟨*proof*⟩

$M = \#\{J < 2^N : f(J) = 0\}$, the quantity estimated by the counting experiment.

definition *count-zeros* :: (*nat* ⇒ *nat*) ⇒ *nat* ⇒ *nat* **where**
count-zeros f N ≡ *card* {*J*. *J* < *2^N* ∧ *f J* = *0*}

14.1 $H^{\otimes N}$ on $|0\cdots0\rangle$ and the counting probability (D12-1, D12-4)

N-fold tensor powers (last factor least significant, matching $(\otimes)$).

primrec *tensor-pow-mat* :: *complex Matrix.mat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
tensor-pow-mat A 0 = 1$_m$ *1*
| *tensor-pow-mat A* (*Suc k*) = *Tensor.tensor-mat* (*tensor-pow-mat A k*) *A*

primrec *tensor-pow-vec* :: *complex Matrix.vec* ⇒ *nat* ⇒ *complex Matrix.vec* **where**
tensor-pow-vec v 0 = Matrix.vec 1 (λ-. *1*)
| *tensor-pow-vec v* (*Suc k*) = *tensor-vec* (*tensor-pow-vec v k*) *v*

lemma *tensor-pow-mat-carrier*:
assumes *A* ∈ *carrier-mat 2 2*
shows *tensor-pow-mat A k* ∈ *carrier-mat* (*2^k*) (*2^k*)
⟨*proof*⟩

lemma *tensor-pow-vec-dim*:
assumes *dim-vec v = 2*
shows *dim-vec* (*tensor-pow-vec v k*) = *2^k*
⟨*proof*⟩

By the mixed-product rule $(A\otimes B)(v\otimes w) = Av\otimes Bw$, applied k times, $A^{\otimes k}v^{\otimes k} = (Av)^{\otimes k}$. With $H|0\rangle = \frac{1}{\sqrt{2}}(1,1)$, and since $(c,c)^{\otimes N}$ is the constant vector with entries c^N,

$$H^{\otimes N}|0\cdots0\rangle = \frac{1}{\sqrt{2^N}}\sum_{J<2^N}|J\rangle.$$

lemma *tensor-pow-mat-vec*:
assumes *Ac*: *A* ∈ *carrier-mat 2 2* **and** *vd*: *dim-vec v = 2*
shows *tensor-pow-mat A k* ∗$_v$ *tensor-pow-vec v k = tensor-pow-vec* (*A* ∗$_v$ *v*) *k*
⟨*proof*⟩

lemma *tensor-pow-vec-const*:
tensor-pow-vec (*Matrix.vec 2* (λ-. *c*)) *k = Matrix.vec* (*2^k*) (λ-. *c^k*)
⟨*proof*⟩

definition *uniform-vec* :: *nat* ⇒ *complex Matrix.vec* **where**
uniform-vec N = Matrix.vec (*2^N*) (λ-. *complex-of-real* (*1 / sqrt 2*) ^ *N*)

lemma *uniform-vec-dim* [*simp*]: *dim-vec* (*uniform-vec N*) = *2^N*
⟨*proof*⟩

lemma *hadamard-e0-const*: *Hadamard* ∗$_v$ *unit-vec 2 0 = Matrix.vec 2* (λ-. *complex-of-real* (*1 / sqrt 2*))
⟨*proof*⟩

theorem *hadamard-uniform-N*:
tensor-pow-mat Hadamard N ∗$_v$ *tensor-pow-vec* (*unit-vec 2 0*) *N = uniform-vec N*
⟨*proof*⟩

lemma *inner-prod-unit-vec-2*:
assumes *k < 2*
shows ⟨*unit-vec 2 0* | *unit-vec 2 k*⟩ = (*if k = 0 then 1 else 0*)
⟨*proof*⟩

Every basis state has amplitude $2^{-N/2}$ in the uniform state, which therefore has norm $\sqrt{2^N\cdot 2^{-N}} = 1$.

lemma *inner-prod-uniform-unit*:
assumes $J < 2\hat{}N$
shows $\langle$*uniform-vec N | unit-vec* (*2^N*) *J*$\rangle$ = *complex-of-real* (*1 / sqrt 2*) ^ *N*
$\langle$*proof*$\rangle$

lemma *uniform-coeff-sq*:
complex-of-real (*1 / sqrt 2*) ^ *N* * *complex-of-real* (*1 / sqrt 2*) ^ *N* = *complex-of-real* ((*1/2*) ^ *N*)
$\langle$*proof*$\rangle$

lemma *uniform-vec-self*: $\langle$*uniform-vec N | uniform-vec N*$\rangle$ = *1*
$\langle$*proof*$\rangle$

lemma *uniform-tensor-e0-as-sum*:
tensor-vec (*uniform-vec N*) (*unit-vec 2 0*) =
vec-sum (*2^(N+1)*) {*J*. *J* < *2^N*}
(λ*J*. *complex-of-real* (*1 / sqrt 2*) ^ *N* ·$_v$ *tensor-vec* (*unit-vec* (*2^N*) *J*) (*unit-vec 2 0*))
$\langle$*proof*$\rangle$

lemma *uniform-e0-state*: *quantum-state* (*2^(N+1)*) (*tensor-vec* (*uniform-vec N*) (*unit-vec 2 0*))
$\langle$*proof*$\rangle$

The notes' computation: with $x = H^{\otimes N}|0\cdots 0\rangle$ and $\gamma = x \otimes e_0$, $p_0(U_f(x \otimes e_0)) = (M/2^N)^2$ where $M = \#\{J : f(J) = 0\}$.

theorem *counting-probability*:
fixes *f* :: *nat* ⇒ *nat* **and** *U* :: *complex Matrix.mat*
assumes *f-range*: ∀ *J* < *2^N*. *f J* < *2*
and *Uc*: *U* ∈ *carrier-mat* (*2^(N+1)*) (*2^(N+1)*)
and *orac*: ∀ *J* < *2^N*. ∀ *i* < *2*.
U *$_v$ (*tensor-vec* (*unit-vec* (*2^N*) *J*) (*unit-vec 2 i*)) =
tensor-vec (*unit-vec* (*2^N*) *J*) (*unit-vec 2* ((*i* + *f J*) *mod 2*))
shows *meas-prob* (*tensor-vec* (*uniform-vec N*) (*unit-vec 2 0*) ⊘ *tensor-vec* (*uniform-vec N*) (*unit-vec 2 0*))
(*U* *$_v$ *tensor-vec* (*uniform-vec N*) (*unit-vec 2 0*))
= (*real* (*count-zeros f N*) / *2^N*)^2
$\langle$*proof*$\rangle$

15 Day 13 (26 September 2011): ensembles and density matrices

15.1 Ensembles

Motivation: measuring $\psi = a|0\rangle + b|1\rangle$ with $M_0 = |0\rangle\langle 0|$, $M_1 = |1\rangle\langle 1|$ leaves the state $\frac{a}{|a|}|0\rangle$ with probability $|a|^2$ and $\frac{b}{|b|}|1\rangle$ with probability $|b|^2$; a later measurement $\{\tilde{M}_\alpha\}$ then has outcome probabilities $p_\alpha = |a|^2\|\tilde{M}_\alpha \frac{a}{|a|}|0\rangle\|^2 + |b|^2\|\tilde{M}_\alpha \frac{b}{|b|}|1\rangle\|^2$.

Definition (notes). An *ensemble* $\{p_i, \psi_i\}_{i=1}^L$ is a set of states with probabilities $p_i \geq 0$, $\sum_i p_i = 1$. For a measurement system $\{M_\alpha\}$ the probability of outcome α is $p_\alpha(\{p_i, \psi_i\}) = \sum_i p_i \|M_\alpha \psi_i\|^2$.

A second source of ensembles: a unit vector $\psi = \sum_l \phi_l \otimes f_l \in \mathcal{H}_A \otimes \mathcal{H}_E$ (environment $\mathcal{H}_E$ with orthonormal basis f_l) behaves, for measurements $M_\alpha \otimes I$, exactly like the ensemble $\{\|\phi_l\|^2, \phi_l/\|\phi_l\|\}$.

definition *ensemble* :: *nat* ⇒ (*real* × *complex Matrix.vec*) *list* ⇒ *bool* **where**
ensemble n E ≡
(∀ (*p*, *ψ*) ∈ *set E*. *p* ≥ *0* ∧ *quantum-state n ψ*) ∧
(∑ (*p*, -) ← *E*. *p*) = *1*

15.2 Von Neumann's density matrix

Measurements cannot distinguish ψ from $e^{i\theta}\psi$, and $|e^{i\theta}\psi\rangle\langle e^{i\theta}\psi| = |\psi\rangle\langle\psi| = (\alpha_i\overline{\alpha_j})$: the object attached to a state is the projection, not the vector. For any γ, $\|\gamma\|^2 = \operatorname{Tr}|\gamma\rangle\langle\gamma|$, and

$$p_\alpha(\psi) = \|M_\alpha\psi\|^2 = \operatorname{Tr}\big((M_\alpha^* M_\alpha)(\psi\psi^*)\big), \qquad p_\alpha(\{p_l,\psi_l\}) = \operatorname{Tr}\Big((M_\alpha^* M_\alpha)\sum_l p_l\psi_l\psi_l^*\Big).$$

Definition (notes). $P = \sum_l p_l|\psi_l\rangle\langle\psi_l|$ is the *density matrix* of the ensemble.

Formally *pure-density* ψ is $|\psi\rangle\langle\psi|$, *ensemble-density* n E is P, and *density-matrix* n ϱ is the Day 14 characterisation ($\rho \geq 0$, $\operatorname{Tr}\rho = 1$).

definition *density-matrix* :: *nat* ⇒ *complex Matrix.mat* ⇒ *bool* **where**
  density-matrix n ϱ ≡ *ϱ* ∈ *carrier-mat n n* ∧ *psd ϱ* ∧ *Tr ϱ* = *1*

definition *pure-density* :: *complex Matrix.vec* ⇒ *complex Matrix.mat* **where**
  pure-density ψ ≡
    let n = *dim-vec ψ in*
    outer-sum n [*ψ*]

$\operatorname{Tr}|\psi\rangle\langle\psi| = \sum_i \psi_i\overline{\psi_i} = \|\psi\|^2$, so a unit vector gives a density matrix.

lemma *Tr-outer-sum-single*:
  assumes *dim*: *dim-vec ψ* = *n*
  shows *Tr* (*outer-sum n* [*ψ*]) = ⟨*ψ* | *ψ*⟩
⟨*proof*⟩

lemma *pure-density-is-density-matrix*:
  assumes *quantum-state n ψ*
  shows *density-matrix n* (*pure-density ψ*)
⟨*proof*⟩

An ensemble $\{p_i,\psi_i\}$ has $\rho = \sum_i p_i|\psi_i\rangle\langle\psi_i|$. It is positive, as a non-negative combination of the positive $|\psi_i\rangle\langle\psi_i|$, and $\operatorname{Tr}\rho = \sum_i p_i\|\psi_i\|^2 = \sum_i p_i = 1$.

definition *ensemble-density* :: *nat* ⇒ (*real* × *complex Matrix.vec*) *list* ⇒ *complex Matrix.mat* **where**
  ensemble-density n E ≡
    foldr (λ(*p*,*ψ*) *acc*. *complex-of-real p* ·$_m$ *outer-sum n* [*ψ*] + *acc*)
      E (*0*$_m$ *n n*)

lemma *psd-zero-mat* [*simp*]:
  psd (*0*$_m$ *n n*)
⟨*proof*⟩

lemma *psd-add-psd*:
  assumes *A*: *psd A* **and** *B*: *psd B*
    and *Ac*: *A* ∈ *carrier-mat n n* **and** *Bc*: *B* ∈ *carrier-mat n n*
  shows *psd* (*A* + *B*)
⟨*proof*⟩

lemma *ensemble-density-Nil* [*simp*]:
  ensemble-density n [] = *0*$_m$ *n n*
  ⟨*proof*⟩

lemma *ensemble-density-Cons*:
  ensemble-density n ((*p*, *ψ*) # *E*) =

complex-of-real p $\cdot_m$ *outer-sum n* [ψ] + *ensemble-density n E*
⟨*proof*⟩

lemma *ensemble-density-carrier*:
assumes ∀ (*p*, ψ) ∈ *set E*. *quantum-state n* ψ
shows *ensemble-density n E* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *ensemble-density-psd-aux*:
assumes ∀ (*p*, ψ) ∈ *set E*. *p* ≥ *0* ∧ *quantum-state n* ψ
shows *psd* (*ensemble-density n E*)
⟨*proof*⟩

lemma *Tr-zero-mat′* [*simp*]:
Tr (*0*$_m$ *n n*) = *0*
⟨*proof*⟩

lemma *ensemble-density-Tr*:
assumes ∀ (*p*, ψ) ∈ *set E*. *quantum-state n* ψ
shows *Tr* (*ensemble-density n E*) = *complex-of-real* (∑ (*p*, -) ← *E*. *p*)
⟨*proof*⟩

theorem *ensemble-density-is-dm*:
assumes *E*: *ensemble n E*
shows *density-matrix n* (*ensemble-density n E*)
⟨*proof*⟩

15.3 Probabilities via the trace

$p_\alpha(\psi) = \|M_\alpha\psi\|^2 = \mathrm{Tr}(M_\alpha^* M_\alpha |\psi\rangle\langle\psi|)$. For an ensemble with density matrix ρ, $p_\alpha = \mathrm{Tr}(M_\alpha^* M_\alpha \rho) = \langle M_\alpha^* M_\alpha | \rho \rangle$ (*ensemble-prob-trace-eq* below).

lemma *pure-state-prob-trace*:
fixes *M* :: *complex Matrix.mat* **and** ψ :: *complex Matrix.vec*
assumes *st*: *quantum-state n* ψ
and *Msq*: *M* ∈ *carrier-mat n n*
shows *meas-prob M* ψ = *Re* (*Tr* ((*M*†) ∗ *M* ∗ *outer-sum n* [ψ]))
⟨*proof*⟩

For an ensemble: $p_\alpha(\{p_i, \psi_i\}) = \sum_i p_i \|M_\alpha \psi_i\|^2 = \mathrm{Tr}(M_\alpha^* M_\alpha \rho)$.

lemma *ensemble-prob-trace-eq*:
fixes *M* :: *complex Matrix.mat*
assumes *M*: *M* ∈ *carrier-mat n n*
and *E*: ∀ (*p*, ψ) ∈ *set E*. *quantum-state n* ψ
shows (∑ (*p*, ψ) ← *E*. *p* ∗ *meas-prob M* ψ)
= *Re* (*Tr* (((*M*†) ∗ *M*) ∗ *ensemble-density n E*))
⟨*proof*⟩

lemma *Tr-psd-Re-nonneg*:
assumes *P*: *psd P* **and** *Pc*: *P* ∈ *carrier-mat n n*
shows *0* ≤ *Re* (*Tr P*)
⟨*proof*⟩

lemma *ensemble-prob-trace*:
fixes *Ms* :: *complex Matrix.mat list*
assumes *ms*: *measurement-system n Ms*
and *dm*: *density-matrix n* ϱ

and *Min*: $M \in$ *set Ms*
shows *Re* (*Tr* (($M^\dagger$) $*$ M $*$ ϱ)) $\geq$ *0*
⟨*proof*⟩

Day 13: $|e^{i\theta}\psi\rangle\langle e^{i\theta}\psi| = |\psi\rangle\langle\psi|$; the density matrix forgets the phase.

lemma *pure-density-smult-unit*:
fixes ψ :: *complex Matrix.vec* **and** c :: *complex*
assumes c: *cmod* c = *1*
shows *pure-density* (c $\cdot_v$ ψ) = *pure-density* ψ
⟨*proof*⟩

16 Day 14 (28 September 2011): density matrices, pure states, composite ensembles

Proposition (notes). $\rho \in L(\mathcal{H}_A)$ is the density matrix of some ensemble if and only if $\rho \geq 0$ and $\operatorname{Tr}\rho = 1$.

Definition (notes). ρ is a *density matrix* when $\rho \geq 0$ and $\operatorname{Tr}\rho = 1$. The ensemble $\{\lambda_i, \psi_i\}$ of eigenvalues and an orthonormal eigenbasis is the *spectral ensemble*.

Many ensembles have the same density matrix: writing $\rho = XX^* = \sum_i c_i c_i^*$ with columns c_i gives the ensemble $\{\|c_i\|^2, c_i/\|c_i\|\}$.

theorem *density-matrix-characterisation*:
fixes ϱ :: *complex Matrix.mat*
assumes *sq*: $\varrho \in$ *carrier-mat n n*
shows *density-matrix n* $\varrho \longleftrightarrow$ *psd* $\varrho \wedge$ *Tr* ϱ = *1*
⟨*proof*⟩

lemma *ensemble-density-carrier-dim*:
shows *ensemble-density n E* $\in$ *carrier-mat n n*
⟨*proof*⟩

lemma *ensemble-density-index*:
assumes r: $r < n$ **and** c: $c < n$
shows *ensemble-density n E* $$ (*r*, *c*) =
(∑ (*p*, ψ) ← *E*. *complex-of-real p* $*$ ψ $ *r* $*$ *cnj* (ψ $ *c*))
⟨*proof*⟩

The spectral ensemble: every density matrix comes from an ensemble.

theorem *spectral-ensemble-exists*:
fixes ϱ :: *complex Matrix.mat*
assumes *dm*: *density-matrix n* ϱ
shows ∃ *E*. *ensemble n E* $\wedge$ *ensemble-density n E* = ϱ
⟨*proof*⟩

corollary *density-matrix-iff-ensemble*:
fixes ϱ :: *complex Matrix.mat*
shows *density-matrix n* $\varrho \longleftrightarrow$
(∃ *E*. *ensemble n E* $\wedge$ *ensemble-density n E* = ϱ)
⟨*proof*⟩

16.1 Pure states

If $\rho = |\psi\rangle\langle\psi| = \sum_i p_i |\psi_i\rangle\langle\psi_i|$ then for $\gamma \perp \psi$, $0 = \langle\gamma|\rho\gamma\rangle = \sum_i p_i |\langle\psi_i|\gamma\rangle|^2$, so every ψ_i with $p_i > 0$ is parallel to ψ: $\psi_i = \beta_i\psi$ with $|\beta_i| = 1$ and $|\psi_i\rangle\langle\psi_i| = |\psi\rangle\langle\psi|$. These density matrices

are the *pure states*.

definition *pure-state-density* :: *nat* ⇒ *complex Matrix.mat* ⇒ *bool* **where**
  pure-state-density n ϱ ≡
    density-matrix n ϱ ∧
    (∃ ψ. *quantum-state n ψ* ∧ *ϱ* = *pure-density ψ*)

lemma *pure-state-density-is-density*:
  assumes *pure-state-density n ϱ*
  shows *density-matrix n ϱ*
  ⟨*proof*⟩

lemma *pure-density-is-pure-state-density*:
  assumes *quantum-state n ψ*
  shows *pure-state-density n* (*pure-density ψ*)
  ⟨*proof*⟩

A companion fact not in the notes: a density matrix is pure iff it is idempotent.

theorem *pure-density-iff-idempotent*:
  fixes *ϱ* :: *complex Matrix.mat*
  assumes *dm*: *density-matrix n ϱ*
  shows *pure-state-density n ϱ* ⟷ *ϱ* ∗ *ϱ* = *ϱ*
⟨*proof*⟩

lemma *outer-sum-singleton-mult-vec*:
  assumes *ψ-dim*: *dim-vec ψ* = *n* **and** *γ-dim*: *dim-vec γ* = *n*
  shows *outer-sum n* [*ψ*] ∗$_v$ *γ* = ⟨*ψ*|*γ*⟩ ·$_v$ *ψ*
⟨*proof*⟩

lemma *ensemble-density-quad-form*:
  assumes *E-dim*: ∀ (*p*, *ψ*) ∈ *set E*. *dim-vec ψ* = *n*
    and *γ-dim*: *dim-vec γ* = *n*
  shows ⟨*γ* | *ensemble-density n E* ∗$_v$ *γ*⟩ =
      (∑ (*p*, *ψ*) ← *E*. *complex-of-real p* ∗ ⟨*ψ*|*γ*⟩ ∗ *cnj* ⟨*ψ*|*γ*⟩)
  ⟨*proof*⟩

theorem *pure-ensemble-parallel*:
  fixes *ψ* :: *complex Matrix.vec*
  assumes *st*: *quantum-state n ψ*
    and *ens*: *ensemble n E*
    and *dens*: *ensemble-density n E* = *pure-density ψ*
    and *mem*: (*p*, *ψ*$_i$) ∈ *set E*
    and *ppos*: *0* < *p*
  shows ∃ *β*. *ψ*$_i$ = *β* ·$_v$ *ψ*
⟨*proof*⟩

16.2 Composite ensembles

For ensembles $\{p_i, \psi_i\}$ in lab A and $\{q_l, \phi_l\}$ in lab B the composite is the ensemble $\{p_i q_l, \psi_i \otimes \phi_l\}$ with density matrix $\rho = \sum_{i,l} p_i q_l |\psi_i\rangle\langle\psi_i| \otimes |\phi_l\rangle\langle\phi_l|$, because $|\psi \otimes \phi\rangle\langle\psi \otimes \phi| = |\psi\rangle\langle\psi| \otimes |\phi\rangle\langle\phi|$ (checked on elementary tensors $h \otimes k$). The identity is formalised below; that $\rho_1 \otimes \rho_2$ is again a density matrix is *density-matrix-tensor* (Day 15).

lemma *tensor-pure-density*:
  fixes *ψ φ* :: *complex Matrix.vec*

assumes *ψ-dim*: *dim-vec ψ* = *m*
and *φ-dim*: *dim-vec φ* = *p*
and *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p*
shows *pure-density* (*tensor-vec ψ φ*) =
(*pure-density ψ*) ⨂ (*pure-density φ*)
⟨*proof*⟩

$|\psi \otimes \varphi\rangle\langle\psi \otimes \varphi| = |\psi\rangle\langle\psi| \otimes |\varphi\rangle\langle\varphi|$.

lemma *outer-sum-tensor*:
assumes *ψ*: *dim-vec ψ* = *n* **and** *φ*: *dim-vec φ* = *m*
shows *outer-sum* (*n* ∗ *m*) [*tensor-vec ψ φ*] = *outer-sum n* [*ψ*] ⨂ *outer-sum m* [*φ*]
⟨*proof*⟩

For ensembles $\{p_i, \psi_i\}$ on $\mathcal{H}_A$ and $\{q_j, \varphi_j\}$ on $\mathcal{H}_B$, the composite ensemble is $\{p_i q_j, \psi_i \otimes \varphi_j\}$.

definition *product-ensemble* :: (*real* × *complex Matrix.vec*) *list* ⇒ (*real* × *complex Matrix.vec*) *list* ⇒ (*real* × *complex Matrix.vec*) *list* **where**
product-ensemble EA EB = *map* (λ((*p*, *ψ*), (*q*, *φ*)). (*p* ∗ *q*, *tensor-vec ψ φ*)) (*List.product EA EB*)

lemma *sum-list-product*:
fixes *f* :: *'a* ⇒ *real* **and** *g* :: *'b* ⇒ *real*
shows (∑ (*x*, *y*) ← *List.product xs ys*. *f x* ∗ *g y*) = (∑ *x* ← *xs*. *f x*) ∗ (∑ *y* ← *ys*. *g y*)
⟨*proof*⟩

lemma *product-ensemble-ensemble*:
assumes *A*: *ensemble n EA* **and** *B*: *ensemble m EB*
shows *ensemble* (*n* ∗ *m*) (*product-ensemble EA EB*)
⟨*proof*⟩

The notes' composite density $\sum_{i,j} p_i q_j \, |\psi_i\rangle\langle\psi_i| \otimes |\varphi_j\rangle\langle\varphi_j|$ is the density of the composite ensemble, hence a density matrix.

definition *composite-density* :: *nat* ⇒ *nat* ⇒ (*real* × *complex Matrix.vec*) *list* ⇒ (*real* × *complex Matrix.vec*) *list* ⇒ *complex Matrix.mat* **where**
composite-density n m EA EB = *foldr* (λ((*p*, *ψ*), (*q*, *φ*)) *acc*.
complex-of-real (*p* ∗ *q*) ·$_m$ (*outer-sum n* [*ψ*] ⨂ *outer-sum m* [*φ*]) + *acc*) (*List.product EA EB*) (*0*$_m$ (*n* ∗ *m*) (*n* ∗ *m*))

lemma *composite-density-eq*:
assumes *A*: *ensemble n EA* **and** *B*: *ensemble m EB*
shows *composite-density n m EA EB* = *ensemble-density* (*n* ∗ *m*) (*product-ensemble EA EB*)
⟨*proof*⟩

theorem *composite-density-is-dm*:
assumes *A*: *ensemble n EA* **and** *B*: *ensemble m EB*
shows *density-matrix* (*n* ∗ *m*) (*composite-density n m EA EB*)
⟨*proof*⟩

16.3 Partial traces: motivation

For a density matrix $\rho \in L(\mathcal{H}_A \otimes \mathcal{H}_B)$ and measurements $\{M_\alpha\}$ in lab A, which act as $M_\alpha \otimes I$, $p_\alpha(\rho) = \mathrm{Tr}((M_\alpha^* M_\alpha \otimes I)\rho)$. The notes announce a density matrix $\rho_A \in L(\mathcal{H}_A)$ with $\mathrm{Tr}_{\mathcal{H}_A \otimes \mathcal{H}_B}((M^*M \otimes I)\rho) = \mathrm{Tr}_{\mathcal{H}_A}(M^* M \rho_A)$ for all M; it is constructed on Day 15.

17 Day 15 (30 September 2011): traces, tensor products of operators, partial traces

Proposition (notes). For any orthonormal basis $\{u_1, \ldots, u_n\}$ of $\mathbb{C}^n$, $\mathrm{tr}(A) = \sum_i \langle u_i | A u_i \rangle$.

theorem *trace-via-onb*:
fixes A :: *complex Matrix.mat*
assumes *sq*: $A \in$ *carrier-mat n n*
and *orth*: *orthonormal-list n vs*
and *full*: *length vs* $= n$
shows *Tr A* $= (\sum i{<}n.\ \langle vs\ !\ i \mid A *_v vs\ !\ i\rangle)$
⟨*proof*⟩

Proposition (notes). $\operatorname{tr}(R \otimes T) = \operatorname{tr}(R)\operatorname{tr}(T)$.

theorem *trace-tensor*:
fixes $R\ T$ :: *complex Matrix.mat*
assumes *Rsq*: $R \in$ *carrier-mat m m*
and *Tsq*: $T \in$ *carrier-mat p p*
and *m-pos*: $0 < m$ **and** *p-pos*: $0 < p$
shows *Tr* $(R \bigotimes T) =$ *Tr R* $*$ *Tr T*
⟨*proof*⟩

17.1 The isomorphism $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) \cong L(\mathcal{H}_A \otimes \mathcal{H}_B)$

Proposition (notes). In finite dimensions the map $\Gamma : L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) \to L(\mathcal{H}_A \otimes \mathcal{H}_B)$, $R \otimes T \mapsto R \otimes T$, is a vector-space isomorphism (both sides have dimension m^2p^2 and Γ is injective).

Formally, $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B)$ is represented in coordinates as M_{m^2,p^2} and Γ as the reindexing $M_{m^2,p^2} \to M_{mp}$; it is an explicit linear isomorphism with explicit inverse.

Postulate $4'$ (Day 16) and the composite ensembles of Day 14: tensor products of positive matrices are positive, and tensor products of density matrices are density matrices.

Products of densities are densities: if $A = \sum_i a_i a_i^*$ and $B = \sum_j b_j b_j^*$ then

$$A \otimes B = \sum_{i,j} (a_i \otimes b_j)(a_i \otimes b_j)^* \geq 0, \qquad \operatorname{Tr}(A \otimes B) = \operatorname{Tr} A \operatorname{Tr} B.$$

lemma *psd-tensor*:
fixes $A\ B$ :: *complex Matrix.mat*
assumes *A*: $A \in$ *carrier-mat m m* **and** *B*: $B \in$ *carrier-mat p p*
and *pA*: *psd A* **and** *pB*: *psd B*
shows *psd* $(A \bigotimes B)$
⟨*proof*⟩

lemma *density-matrix-tensor*:
fixes $A\ B$ :: *complex Matrix.mat*
assumes *dA*: *density-matrix m A* **and** *dB*: *density-matrix p B*
and *m*: $0 < m$ **and** *p*: $0 < p$
shows *density-matrix* $(m * p)$ $(A \bigotimes B)$
⟨*proof*⟩

proposition *L-tensor-dim*:
fixes $m\ p$ :: *nat*
shows $m\hat{}2 * p\hat{}2 = (m * p)\hat{}2$
⟨*proof*⟩

definition *operator-tensor-iso-map* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
operator-tensor-iso-map m p c ≡
Matrix.mat $(m{*}p)$ $(m{*}p)$

(λ(r,s). c $$ ((r div p) * m + (s div p), (r mod p) * p + (s mod p)))

The coordinate model: $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B)$ is M_{m^2,p^2}, the entry in row (i,i') and column (k,k') being the coefficient of $E_{ii'} \otimes E_{kk'}$. Γ moves it to position $((i,k),(i',k'))$ of an $mp \times mp$ matrix,

$$\Gamma(c)_{ip+k,\, i'p+k'} = c_{im+i',\, kp+k'},$$

so $\Gamma(\sum c\, E_{ii'} \otimes E_{kk'})$ is the same sum with $\otimes$ the Kronecker product. Being a reindexing, it is linear and bijective.

lemma *operator-tensor-iso-map-carrier* [*simp*]:
operator-tensor-iso-map m p c ∈ *carrier-mat* (*m*∗*p*) (*m*∗*p*)
⟨*proof*⟩

lemma *operator-tensor-iso-map-row-bound*:
fixes *m p r s* :: *nat*
assumes *p-pos*: *0* < *p* **and** *r* < *m*∗*p* **and** *s* < *m*∗*p*
shows (*r div p*) ∗ *m* + (*s div p*) < *m*∗*m*
⟨*proof*⟩

lemma *operator-tensor-iso-map-col-bound*:
fixes *p r s* :: *nat*
assumes *p-pos*: *0* < *p*
shows (*r mod p*) ∗ *p* + (*s mod p*) < *p*∗*p*
⟨*proof*⟩

definition *operator-tensor-iso-inv* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
operator-tensor-iso-inv m p x ≡
Matrix.mat (*m*∗*m*) (*p*∗*p*)
(λ(*a*,*b*). *x* $$ ((*a div m*) ∗ *p* + (*b div p*), (*a mod m*) ∗ *p* + (*b mod p*)))

definition *operator-tensor-coordinate-space* ::
nat ⇒ *nat* ⇒ (*complex*, *complex Matrix.mat*) *linear-structure* **where**
operator-tensor-coordinate-space m p ≡ *mat-linear-structure* (*m*∗*m*) (*p*∗*p*)

definition *bipartite-operator-space* ::
nat ⇒ *nat* ⇒ (*complex*, *complex Matrix.mat*) *linear-structure* **where**
bipartite-operator-space m p ≡ *mat-linear-structure* (*m*∗*p*) (*m*∗*p*)

lemma *operator-tensor-iso-inv-carrier* [*simp*]:
operator-tensor-iso-inv m p x ∈ *carrier-mat* (*m*∗*m*) (*p*∗*p*)
⟨*proof*⟩

lemma *operator-tensor-iso-inv-row-bound*:
fixes *m p a b* :: *nat*
assumes *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p* **and** *a* < *m*∗*m* **and** *b* < *p*∗*p*
shows (*a div m*) ∗ *p* + (*b div p*) < *m*∗*p*
⟨*proof*⟩

lemma *operator-tensor-iso-inv-col-bound*:
fixes *m p a b* :: *nat*
assumes *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p*
shows (*a mod m*) ∗ *p* + (*b mod p*) < *m*∗*p*
⟨*proof*⟩

lemma *operator-tensor-iso-map-add*:
assumes *p-pos*: *0* < *p*
and *x*: *x* ∈ *carrier-mat* (*m*∗*m*) (*p*∗*p*)
and *y*: *y* ∈ *carrier-mat* (*m*∗*m*) (*p*∗*p*)

shows *operator-tensor-iso-map m p (x + y) =*
operator-tensor-iso-map m p x + operator-tensor-iso-map m p y
⟨*proof*⟩

lemma *operator-tensor-iso-map-zero*:
assumes *p-pos*: *0 < p*
shows *operator-tensor-iso-map m p (0$_m$ (m∗m) (p∗p)) = 0$_m$ (m∗p) (m∗p)*
⟨*proof*⟩

lemma *operator-tensor-iso-map-smult*:
assumes *p-pos*: *0 < p*
and *x*: *x ∈ carrier-mat (m∗m) (p∗p)*
shows *operator-tensor-iso-map m p (c ·$_m$ x) =*
c ·$_m$ operator-tensor-iso-map m p x
⟨*proof*⟩

lemma *operator-tensor-iso-inv-add*:
assumes *m-pos*: *0 < m*
and *p-pos*: *0 < p*
and *x*: *x ∈ carrier-mat (m∗p) (m∗p)*
and *y*: *y ∈ carrier-mat (m∗p) (m∗p)*
shows *operator-tensor-iso-inv m p (x + y) =*
operator-tensor-iso-inv m p x + operator-tensor-iso-inv m p y
⟨*proof*⟩

lemma *operator-tensor-iso-inv-zero*:
assumes *m-pos*: *0 < m* **and** *p-pos*: *0 < p*
shows *operator-tensor-iso-inv m p (0$_m$ (m∗p) (m∗p)) = 0$_m$ (m∗m) (p∗p)*
⟨*proof*⟩

lemma *operator-tensor-iso-inv-smult*:
assumes *m-pos*: *0 < m*
and *p-pos*: *0 < p*
and *x*: *x ∈ carrier-mat (m∗p) (m∗p)*
shows *operator-tensor-iso-inv m p (c ·$_m$ x) =*
c ·$_m$ operator-tensor-iso-inv m p x
⟨*proof*⟩

proposition *operator-tensor-space-isomorphism*:
assumes *m-pos*: *0 < m*
and *p-pos*: *0 < p*
shows *is-isomorphism-between*
(operator-tensor-coordinate-space m p)
(bipartite-operator-space m p)
(operator-tensor-iso-map m p)
⟨*proof*⟩

corollary *operator-tensor-space-isomorphic*:
assumes *0 < m* **and** *0 < p*
shows *is-isomorphic-to*
(operator-tensor-coordinate-space m p)
(bipartite-operator-space m p)
⟨*proof*⟩

D15-3: in the coordinate model $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) = M_{m^2 \times p^2}$ the elementary tensor $R \otimes T$ is the matrix $(R_{ij}T_{kl})_{(im+j,\, kp+l)}$. The isomorphism Γ of *operator-tensor-space-isomorphism* sends it to the Kronecker product $R \otimes T$, and Γ is the only linear map that does.

definition *op-elem* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
op-elem m p Ra Tb = *Matrix.mat* (*m* ∗ *m*) (*p* ∗ *p*) (λ(*a*, *b*). *Ra* $$ (*a div m*, *a mod m*) ∗ *Tb* $$ (*b div p*, *b mod p*))

theorem *gamma-elementary*:
assumes *Ra*: *Ra* ∈ *carrier-mat m m* **and** *Tb*: *Tb* ∈ *carrier-mat p p*
shows *operator-tensor-iso-map m p* (*op-elem m p Ra Tb*) = *Ra* ⨂ *Tb*
⟨*proof*⟩

definition *unit-mat* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
unit-mat K1 K2 a b = *Matrix.mat K1 K2* (λ(*i*, *j*). *if i* = *a* ∧ *j* = *b then 1 else 0*)

lemma *unit-mat-carrier* [*simp*]: *unit-mat K1 K2 a b* ∈ *carrier-mat K1 K2* ⟨*proof*⟩

The unit matrices of $M_{m^2 \times p^2}$ are elementary tensors of unit matrices.

lemma *unit-mat-op-elem*:
assumes *a*: *a* < *m* ∗ *m* **and** *b*: *b* < *p* ∗ *p*
shows *unit-mat* (*m* ∗ *m*) (*p* ∗ *p*) *a b* = *op-elem m p* (*unit-mat m m* (*a div m*) (*a mod m*)) (*unit-mat p p* (*b div p*) (*b mod p*))
⟨*proof*⟩

A linear map on $M_{K_1 \times K_2}$ is determined by its values on the unit matrices.

definition *linear-mat-on* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
linear-mat-on K1 K2 Φ ⟷
(∀ *x y*. *x* ∈ *carrier-mat K1 K2* ⟶ *y* ∈ *carrier-mat K1 K2* ⟶ Φ (*x* + *y*) = Φ *x* + Φ *y*)
∧ (∀ *c x*. *x* ∈ *carrier-mat K1 K2* ⟶ Φ (*c* ·*m* *x*) = *c* ·*m* Φ *x*)

lemma *vec-to-mat-unit*:
assumes *k*: *k* < *K1* ∗ *K2*
shows *vec-to-mat K1 K2* (*unit-vec* (*K1* ∗ *K2*) *k*) = *unit-mat K1 K2* (*k div K2*) (*k mod K2*)
⟨*proof*⟩

lemma *linear-mat-unique*:
assumes *l1*: *linear-mat-on K1 K2* Φ **and** *l2*: *linear-mat-on K1 K2* Ψ
and *dims*: ⋀*x*. *x* ∈ *carrier-mat K1 K2* ⟹ *dim-row* (Φ *x*) = *Q1* ∧ *dim-col* (Φ *x*) = *Q2* ∧ *dim-row* (Ψ *x*) = *Q1* ∧ *dim-col* (Ψ *x*) = *Q2*
and *e*: ⋀*a b*. *a* < *K1* ⟹ *b* < *K2* ⟹ Φ (*unit-mat K1 K2 a b*) = Ψ (*unit-mat K1 K2 a b*)
and *c*: *c* ∈ *carrier-mat K1 K2*
shows Φ *c* = Ψ *c*
⟨*proof*⟩

theorem *gamma-unique*:
assumes *p0*: *0* < *p* **and** *l*: *linear-mat-on* (*m* ∗ *m*) (*p* ∗ *p*) Φ
and *dims*: ⋀*x*. *x* ∈ *carrier-mat* (*m* ∗ *m*) (*p* ∗ *p*) ⟹ Φ *x* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*)
and *e*: ⋀*Ra Tb*. *Ra* ∈ *carrier-mat m m* ⟹ *Tb* ∈ *carrier-mat p p* ⟹ Φ (*op-elem m p Ra Tb*) = *Ra* ⨂ *Tb*
and *c*: *c* ∈ *carrier-mat* (*m* ∗ *m*) (*p* ∗ *p*)
shows Φ *c* = *operator-tensor-iso-map m p c*
⟨*proof*⟩

17.2 Partial traces

Definition (notes). Identify $L(\mathcal{H}_A \otimes \mathcal{H}_B) = L(\mathcal{H}_A) \otimes L(\mathcal{H}_B)$ and define $\mathrm{tr}_B(\sum_l R_l \otimes T_l) = \sum_l \mathrm{tr}(T_l) R_l \in L(\mathcal{H}_A)$ and $\mathrm{tr}_A(\sum_l R_l \otimes T_l) = \sum_l \mathrm{tr}(R_l) T_l \in L(\mathcal{H}_B)$; also written $X^A = \mathrm{tr}_B(X)$, $X^B = \mathrm{tr}_A(X)$. Well defined by the universal property, since $(R, T) \mapsto \mathrm{tr}(T)R$ is bilinear.

The formal definitions use the block formulas of Day 16: an $mp \times mp$ matrix is indexed by pairs (i,k), $i < m$, $k < p$, with flat index $ip + k$ ($\mathcal{H}_A$ is the major factor, matching $(A \otimes B)_{(r,c)} = A_{(r \div p,\, c \div p)} B_{(r \bmod p,\, c \bmod p)}$), and $(\mathrm{tr}_B\, X)_{ij} = \sum_{k<p} X_{ip+k,\, jp+k}$, $(\mathrm{tr}_A\, X)_{ij} = \sum_{k<m} X_{kp+i,\, kp+j}$. The defining property on elementary tensors is proved below.

record *'lab qspace* =
  qdim :: *nat*
  qlabel :: *'lab*

record *'lab bipartite-space* =
  left-space :: *'lab qspace*
  right-space :: *'lab qspace*

definition *partial-trace-B* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
  partial-trace-B m p x ≡
    Matrix.mat m m (λ(*i*, *j*). ∑ *k*<*p*. *x* $$ (*i* ∗ *p* + *k*, *j* ∗ *p* + *k*))

definition *partial-trace-A* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
  partial-trace-A m p x ≡
    Matrix.mat p p (λ(*i*, *j*). ∑ *k*<*m*. *x* $$ (*k* ∗ *p* + *i*, *k* ∗ *p* + *j*))

definition *partial-trace-on* ::
  'lab qspace ⇒ *'lab bipartite-space* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
  partial-trace-on s ab x ≡
    (*if s* = *right-space ab then*
      partial-trace-B (*qdim* (*left-space ab*)) (*qdim* (*right-space ab*)) *x*
     else if s = *left-space ab then*
      partial-trace-A (*qdim* (*left-space ab*)) (*qdim* (*right-space ab*)) *x*
     else 0_m *0 0*)

notation *partial-trace-on* (*tr*_‾ - [*1000*, *1000*, *1000*] *1000*)

lemma *partial-trace-on-right*:
  fixes *S* :: *'lab qspace* **and** *AB* :: *'lab bipartite-space*
  assumes *S* = *right-space AB*
  shows $tr_S^{AB}\ x$ =
      partial-trace-B (*qdim* (*left-space AB*)) (*qdim* (*right-space AB*)) *x*
  ⟨*proof*⟩

lemma *partial-trace-on-left*:
  fixes *S* :: *'lab qspace* **and** *AB* :: *'lab bipartite-space*
  assumes *S* = *left-space AB*
    and *left-space AB* ≠ *right-space AB*
  shows $tr_S^{AB}\ x$ =
      partial-trace-A (*qdim* (*left-space AB*)) (*qdim* (*right-space AB*)) *x*
  ⟨*proof*⟩

lemma *dim-row-partial-trace-B* [*simp*]: *dim-row* (*partial-trace-B m p x*) = *m*
  ⟨*proof*⟩

lemma *dim-col-partial-trace-B* [*simp*]: *dim-col* (*partial-trace-B m p x*) = *m*
  ⟨*proof*⟩

lemma *partial-trace-B-carrier* [*simp*]:
  partial-trace-B m p x ∈ *carrier-mat m m*
  ⟨*proof*⟩

lemma *dim-row-partial-trace-A* [*simp*]: *dim-row* (*partial-trace-A m p x*) = *p*

⟨*proof*⟩

lemma *dim-col-partial-trace-A* [*simp*]: *dim-col* (*partial-trace-A m p x*) = *p*
⟨*proof*⟩

On product operators the partial traces keep one factor and trace out the other:

$$\mathrm{tr}_B(R \otimes T) = \mathrm{Tr}(T)\, R, \qquad \mathrm{tr}_A(R \otimes T) = \mathrm{Tr}(R)\, T,$$

entrywise $\sum_k R_{ij} T_{kk} = R_{ij} \,\mathrm{Tr}\, T$.

lemma *partial-trace-B-tensor*:
fixes *R T* :: *complex Matrix.mat*
assumes *Rsq*: *R* ∈ *carrier-mat m m*
and *Tsq*: *T* ∈ *carrier-mat p p*
and *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p*
shows *partial-trace-B m p* (*R* ⨂ *T*) = *Tr T* ·$_m$ *R*
⟨*proof*⟩

lemma *partial-trace-A-tensor*:
fixes *R T* :: *complex Matrix.mat*
assumes *Rsq*: *R* ∈ *carrier-mat m m*
and *Tsq*: *T* ∈ *carrier-mat p p*
and *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p*
shows *partial-trace-A m p* (*R* ⨂ *T*) = *Tr R* ·$_m$ *T*
⟨*proof*⟩

17.3 Partial traces and measurements

Writing $\rho = \sum_l R_l \otimes T_l$,

$$p_\alpha(\rho) = \mathrm{tr}((M_\alpha^* M_\alpha \otimes I)\rho) = \sum_l \mathrm{tr}(M_\alpha^* M_\alpha R_l)\,\mathrm{tr}(T_l) = \mathrm{tr}\big(M_\alpha^* M_\alpha\ \mathrm{tr}_B(\rho)\big) = p_\alpha(\rho^A) :$$

lab A only sees ρ^A, and symmetrically lab B only sees ρ^B. Check: $\mathrm{tr}(\rho^A) = \mathrm{tr}\,\rho = 1$, and $\rho \geq 0$ implies $\rho^A \geq 0$.

Key identity: measurements in lab A only see $\rho^A = \mathrm{tr}_B\,\rho$.

theorem *partial-trace-measurement*:
fixes *meas* :: *complex Matrix.mat* **and** *ϱ* :: *complex Matrix.mat*
assumes *M-sq*: *meas* ∈ *carrier-mat m m*
and *dm*: *density-matrix* (*m* ∗ *p*) *ϱ*
and *p-pos*: *0* < *p*
shows *Tr* (((*meas*† ∗ *meas*) ⨂ *1*$_m$ *p*) ∗ *ϱ*) =
Tr (*meas*† ∗ *meas* ∗ *partial-trace-B m p ϱ*)
⟨*proof*⟩

The check: the partial trace preserves the trace, Hermiticity and positivity (D15-6).

lemma *partial-trace-B-trace*:
fixes *X* :: *complex Matrix.mat*
assumes *Xsq*: *X* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*)
and *p-pos*: *0* < *p*
shows *Tr* (*partial-trace-B m p X*) = *Tr X*
⟨*proof*⟩

lemma *partial-trace-B-hermitian*:
fixes *X* :: *complex Matrix.mat*
assumes *Xsq*: *X* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*)

and *herm*: *hermitian X*
shows *hermitian* (*partial-trace-B m p X*)
⟨*proof*⟩

lemma *partial-trace-B-is-density*:
fixes *X* :: *complex Matrix.mat*
assumes *dm*: *density-matrix* (*m* ∗ *p*) *X*
and *p-pos*: *0* < *p*
shows *density-matrix m* (*partial-trace-B m p X*)
⟨*proof*⟩

17.4 The partial trace over A (D15-6)

Reordering the basis: for a bijection σ of $\{0, \ldots, N-1\}$ the matrix $(x_{\sigma(r)\sigma(s)})$ has the trace of X and is Hermitian, resp. positive, with X.

definition *perm-conj* :: *nat* ⇒ (*nat* ⇒ *nat*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
perm-conj N σ X = *Matrix.mat N N* (λ(*r*, *s*). *X* $$ (*σ r*, *σ s*))

lemma *perm-conj-carrier* [*simp*]: *perm-conj N σ X* ∈ *carrier-mat N N*
⟨*proof*⟩

lemma *perm-conj-dims* [*simp*]: *dim-row* (*perm-conj N σ X*) = *N dim-col* (*perm-conj N σ X*) = *N*
⟨*proof*⟩

Conjugation by a permutation, $X \mapsto (X_{\sigma(r)\sigma(s)})_{r,s}$, preserves the trace, Hermitian matrices and positivity.

lemma *perm-conj-trace*:
assumes *σ*: *bij-betw σ* {..<*N*} {..<*N*} **and** *X*: *X* ∈ *carrier-mat N N*
shows *Tr* (*perm-conj N σ X*) = *Tr X*
⟨*proof*⟩

lemma *perm-conj-hermitian*:
assumes *σ*: ⋀*r*. *r* < *N* ⟹ *σ r* < *N* **and** *X*: *X* ∈ *carrier-mat N N* **and** *h*: *hermitian X*
shows *hermitian* (*perm-conj N σ X*)
⟨*proof*⟩

lemma *perm-conj-psd*:
assumes *σ*: *bij-betw σ* {..<*N*} {..<*N*} **and** *X*: *X* ∈ *carrier-mat N N* **and** *ps*: *psd X*
shows *psd* (*perm-conj N σ X*)
⟨*proof*⟩

Exchanging the factors: with $\sigma(im+k) = kp+i$ ($i < p$, $k < m$), tracing out A is tracing out B after the exchange.

definition *swap-index* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *nat* **where**
swap-index m p r = (*r mod m*) ∗ *p* + *r div m*

lemma *swap-index-less*:
assumes *r* < *m* ∗ *p* **shows** *swap-index m p r* < *m* ∗ *p*
⟨*proof*⟩

lemma *swap-index-bij*: *bij-betw* (*swap-index m p*) {..<*m* ∗ *p*} {..<*m* ∗ *p*}
⟨*proof*⟩

The index permutation $r \mapsto (r \bmod m)\,p + r \div m$ exchanges the two tensor factors and turns tr_A into tr_B; the properties of tr_A follow from those of tr_B.

lemma *partial-trace-A-as-B*:

assumes X: $X \in$ *carrier-mat* $(m * p)$ $(m * p)$
shows *partial-trace-A* m p $X =$ *partial-trace-B* p m (*perm-conj* $(m * p)$ (*swap-index* m p) X)
⟨*proof*⟩

theorem *partial-trace-A-trace*:
assumes X: $X \in$ *carrier-mat* $(m * p)$ $(m * p)$ **and** m: $0 < m$
shows *Tr* (*partial-trace-A* m p X) = *Tr* X
⟨*proof*⟩

theorem *partial-trace-A-hermitian*:
assumes X: $X \in$ *carrier-mat* $(m * p)$ $(m * p)$ **and** h: *hermitian* X
shows *hermitian* (*partial-trace-A* m p X)
⟨*proof*⟩

theorem *partial-trace-A-is-density*:
assumes *dm*: *density-matrix* $(m * p)$ X **and** m: $0 < m$
shows *density-matrix* p (*partial-trace-A* m p X)
⟨*proof*⟩

18 Day 16 (3 October 2011): partial traces in block form; the postulates for density matrices

With matrix units E_{ij} of $L(\mathcal{H}_B)$, every $X \in L(\mathcal{H}_A) \otimes L(\mathcal{H}_B)$ is uniquely $X = \sum_{i,j} X_{ij} \otimes E_{ij}$ with $X_{ij} \in L(\mathcal{H}_A)$, and $\mathrm{tr}_B(X) = \sum_i X_{ii}$, $\mathrm{tr}_A(X) = (\mathrm{tr}\, X_{ij})_{ij}$; symmetrically for $X = \sum E_{kl} \otimes Y_{kl}$. Choosing a basis $f_1, \ldots, f_p$ of $\mathcal{H}_B$ identifies $\mathcal{H}_A \otimes \mathcal{H}_B \cong \mathcal{H}_A^{(p)} = \mathcal{H}_A \oplus \cdots \oplus \mathcal{H}_A$ and $L(\mathcal{H}_A \otimes \mathcal{H}_B) \cong M_p(L(\mathcal{H}_A))$, block matrices (X_{ij}) acting by $(X_{ij})(h_j)_j = (\sum_j X_{ij} h_j)_i$; in block notation $\mathrm{tr}_B((X_{ij})) = X_{11} + \cdots + X_{pp}$ and $\mathrm{tr}_A((X_{ij})) = (\mathrm{tr}\, X_{ij})$.
Reformulated postulates. $1'$: the state of a system is a density matrix $\rho \in L(\mathcal{H}_A)$. $2'$: closed evolution is $\rho \mapsto U\rho U^*$. $3'$: a measurement system $\{M_\alpha\}$ with $\sum M_\alpha^* M_\alpha = I$ yields outcome α with probability $\mathrm{tr}(M_\alpha^* M_\alpha \rho)$. $4'$: systems with densities ρ_1, ρ_2 compose to $\rho_1 \otimes \rho_2$ on $\mathcal{H}_A \otimes \mathcal{H}_B$.

18.1 $H_A \otimes H_B \cong H_A^{(p)}$ and $L(H_A \otimes H_B) \cong M_p(L(H_A))$ (D16-2)

The notes identify $H_A \otimes H_B$ with p copies of H_A through $\sum_i h_i \otimes f_i \leftrightarrow (h_1, \ldots, h_p)$, with $\|v\|^2 = \sum_i \|h_i\|^2$, and an operator X with its block matrix (X_{ij}) over $L(H_A)$, acting by $(h_j) \mapsto (\sum_j X_{ij} h_j)$, so that $X = \sum_{i,j} X_{ij} \otimes E_{ij}$, $\mathrm{tr}_B(X) = X_{11} + \cdots + X_{pp}$ and $\mathrm{tr}_A(X) = (\mathrm{Tr}\, X_{ij})$. Here $f_i = e_i$, and indices are A-major as everywhere in the development.

With $H_A = \mathbb{C}^m$, $H_B = \mathbb{C}^p$ and A-major indices $ap + i$, the part of v along e_i is $h_i = (v_{ap+i})_a \in H_A$, the i-th column of *vec-to-mat* m p v: for $m = 2$, $p = 3$, $h_0 = (v_0, v_3)$, $h_1 = (v_1, v_4)$ and $h_2 = (v_2, v_5)$.

Likewise an operator X on $\mathbb{C}^{mp}$ has the blocks $X_{ij} = (X_{ap+i,\, bp+j})_{a,b} \in L(H_A)$: the rows of X with second index i and its columns with second index j. For $m = p = 2$ the indices $0, 2$ have second index 0 and $1, 3$ have second index 1, so

$$X = \begin{pmatrix} x_{00} & x_{01} & x_{02} & x_{03} \\ x_{10} & x_{11} & x_{12} & x_{13} \\ x_{20} & x_{21} & x_{22} & x_{23} \\ x_{30} & x_{31} & x_{32} & x_{33} \end{pmatrix}, \qquad X_{00} = \begin{pmatrix} x_{00} & x_{02} \\ x_{20} & x_{22} \end{pmatrix}, \quad X_{01} = \begin{pmatrix} x_{01} & x_{03} \\ x_{21} & x_{23} \end{pmatrix}.$$

The blocks interleave because the second index runs fastest. X_{ij} carries the part of v along e_j to the part of Xv along e_i.

definition *block* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
block m p X i j = *Matrix.mat m m* (λ(*a*, *b*). *X* $$ (*a* ∗ *p* + *i*, *b* ∗ *p* + *j*))

lemma *block-carrier* [*simp*]: *block m p X i j* ∈ *carrier-mat m m*
⟨*proof*⟩

lemma *block-dims* [*simp*]: *dim-row* (*block m p X i j*) = *m dim-col* (*block m p X i j*) = *m*
⟨*proof*⟩

lemma *sum-mp-reindex*:
fixes *f* :: *nat* ⇒ *complex*
shows (∑ *r*<*m* ∗ *p*. *f r*) = (∑ *b*<*m*. ∑ *j*<*p*. *f* (*b* ∗ *p* + *j*))
⟨*proof*⟩

$H_A \otimes H_B \cong H_A^{(p)}$ isometrically: $\langle v|w\rangle = \sum_i \langle h_i|k_i\rangle$.

theorem *vec-to-mat-cols-inner-prod*:
assumes *v*: *dim-vec v* = *m* ∗ *p* **and** *w*: *dim-vec w* = *m* ∗ *p*
shows *Quantum.inner-prod v w* = (∑ *i*<*p*. *Quantum.inner-prod* (*Matrix.col* (*vec-to-mat m p v*) *i*) (*Matrix.col* (*vec-to-mat m p w*) *i*))
⟨*proof*⟩

$v = \sum_i h_i \otimes e_i$.

theorem *vec-to-mat-cols-expansion*:
assumes *v*: *dim-vec v* = *m* ∗ *p*
shows *v* = *tensor-combination* (*m* ∗ *p*) (*cols* (*vec-to-mat m p v*)) (*map* (*unit-vec p*) [*0*..<*p*])
⟨*proof*⟩

The block action: the i-th coordinate of Xv is $\sum_j X_{ij} h_j$.

theorem *block-action*:
assumes *X*: *X* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*) **and** *v*: *dim-vec v* = *m* ∗ *p* **and** *i*: *i* < *p*
shows *Matrix.col* (*vec-to-mat m p* (*X* ∗*v v*)) *i* = *Matrix.vec m* (λ*a*. ∑ *j*<*p*. (*block m p X i j* ∗*v Matrix.col* (*vec-to-mat m p v*) *j*) $ *a*)
⟨*proof*⟩

$L(H_A \otimes H_B) \cong M_p(L(H_A))$: $X = \sum_{i,j} X_{ij} \otimes E_{ij}$, and the blocks are unique.

lemma *double-delta*:
fixes *c* :: *complex* **and** *a b p q* :: *nat*
assumes *a* < *p b* < *q*
shows (∑ *i*<*p*. ∑ *j*<*q*. *if i* = *a* ∧ *j* = *b then c else 0*) = *c*
⟨*proof*⟩

lemma *mat-unit-tensor-index*:
assumes *A*: *A* ∈ *carrier-mat m m* **and** *r*: *r* < *m* ∗ *p* **and** *s*: *s* < *m* ∗ *p* **and** *i*: *i* < *p* **and** *j*: *j* < *p*
shows (*A* ⨂ *E*⟦*i*,*j* ; *p*,*p*⟧) $$ (*r*, *s*) = (*if r mod p* = *i* ∧ *s mod p* = *j then A* $$ (*r div p*, *s div p*) *else 0*)
⟨*proof*⟩

theorem *tensor-block-decomposition*:
assumes *X*: *X* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*)
shows *X* = *msum* (*m* ∗ *p*) (*m* ∗ *p*) (λ(*i*, *j*). *block m p X i j* ⨂ *E*⟦*i*,*j* ; *p*,*p*⟧) (*List.product* [*0*..<*p*] [*0*..<*p*])
⟨*proof*⟩

theorem *tensor-block-decomposition-unique*:
assumes *Y*: ⋀*i j*. *Y i j* ∈ *carrier-mat m m*
and *X*: *X* = *msum* (*m* ∗ *p*) (*m* ∗ *p*) (λ(*i*, *j*). *Y i j* ⨂ *E*⟦*i*,*j* ; *p*,*p*⟧) (*List.product* [*0*..<*p*] [*0*..<*p*])
and *i*: *i* < *p* **and** *j*: *j* < *p*

shows *Y i j = block m p X i j*
⟨*proof*⟩

The partial traces in block form (the definitions, restated): $\mathrm{tr}_B(X) = \sum_i X_{ii}$, $\mathrm{tr}_A(X) = (\mathrm{Tr}\, X_{ij})_{ij}$.

lemma *partial-trace-B-blocks*:
partial-trace-B m p X = msum m m (λ*i*. *block m p X i i*) [*0..<p*]
⟨*proof*⟩

lemma *partial-trace-A-blocks*:
partial-trace-A m p X = Matrix.mat p p (λ(*i*, *j*). *Tr* (*block m p X i j*))
⟨*proof*⟩

19 Day 17 (5 October 2011): measurement maps, quantum noise, completely positive maps

19.1 Measurement maps

Measuring ψ with $\{M_\alpha\}$ and keeping the outcome as an ensemble gives the density matrix $\sum_\alpha p_\alpha(\psi)|M_\alpha\psi/\|M_\alpha\psi\|\rangle\langle M_\alpha\psi/\|M_\alpha\psi\|| = \sum_\alpha M_\alpha(\psi\psi^*)M_\alpha^*$; for an ensemble with density ρ_1 one obtains $M(\rho_1) = \sum_\alpha M_\alpha\rho_1 M_\alpha^*$: a linear map on $L(\mathcal{H}_A)$ sending density matrices to density matrices.

Classical noise flips a bit with some probability; probability vectors evolve by stochastic matrices (non-negative entries, columns summing to one; doubly stochastic if rows do too). Quantum noise: with an environment $\mathcal{H}_E$ in state $|\phi\rangle$, the system $\rho\otimes|\phi\rangle\langle\phi|$ evolves by a unitary $U = (U_{ij})$ on $\mathcal{H}_A \otimes \mathcal{H}_E$ and lab A sees

$$\mathrm{tr}_E\big(U(\rho\otimes|\phi\rangle\langle\phi|)U^*\big) = \sum_i W_i\rho W_i^*, \qquad W_i = \sum_k U_{ik}\alpha_k, \qquad \sum_i W_i^*W_i = I.$$

Measurement by an intruder and environmental noise are maps of the same form $\rho \mapsto \sum_\alpha X_\alpha\rho X_\alpha^*$ with $\sum X_\alpha^*X_\alpha = I$. The environment computation is *environment-model* below.

Third way, axiomatic: a quantum event is a linear $\Phi : L(\mathcal{H}_A) \to L(\mathcal{H}_A)$.

Proposition (notes). Φ maps density matrices to density matrices iff Φ is positive ($\rho \geq 0 \Rightarrow \Phi(\rho) \geq 0$) and trace preserving ($\mathrm{tr}\,\Phi(X) = \mathrm{tr}\, X$); then $\Phi \otimes I_{L(\mathcal{H}_B)}$ also maps density matrices to density matrices for every $\mathcal{H}_B$.

Definition (notes). Φ is *completely positive* if $\Phi\otimes I_{L(\mathcal{H}_B)}$ is positive for every finite-dimensional $\mathcal{H}_B$.

Theorem (notes) (Choi–Kraus)**.** $\Phi : L(\mathcal{H}_A) \to L(\mathcal{H}_A)$ is completely positive and trace preserving iff $\Phi(X) = \sum_i E_i X E_i^*$ for matrices E_i with $\sum_i E_i^*E_i = I$.

The proposition is *density-preserving-iff-positive-tp* on Day 18, whose decomposition into positives it uses; complete positivity is defined below and the Choi–Kraus theorem is *choi-kraus-cptp* on Day 19.

definition *meas-map* :: *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
meas-map Ms ϱ ≡
foldr (λ*M acc*. *M* ∗ *ϱ* ∗ (*M*†) + *acc*) *Ms* (*0*$_m$ (*dim-row ϱ*) (*dim-col ϱ*))

lemma *meas-map-Nil* [*simp*]:
meas-map [] *ϱ = 0*$_m$ (*dim-row ϱ*) (*dim-col ϱ*)
⟨*proof*⟩

lemma *meas-map-Cons* [*simp*]:
meas-map (*M* # *Ms*) ϱ = *M* ∗ ϱ ∗ (*M*†) + *meas-map Ms* ϱ
⟨*proof*⟩

lemma *meas-map-carrier*:
assumes *Ms-c*: ∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*
and ϱ*-c*: ϱ ∈ *carrier-mat n n*
shows *meas-map Ms* ϱ ∈ *carrier-mat n n*
⟨*proof*⟩

If the outcome of a measurement is not recorded, $\rho \mapsto \sum_i M_i \rho M_i^*$. Each term is positive, $\langle x|M\rho M^* x\rangle = \langle M^* x|\rho\, M^* x\rangle \geq 0$, and cyclicity of the trace gives

$$\operatorname{Tr} \sum_i M_i \rho M_i^* = \operatorname{Tr}\Big((\sum_i M_i^* M_i)\rho\Big) = \operatorname{Tr} \rho,$$

so densities go to densities.

lemma *meas-map-psd*:
assumes *Ms-c*: ∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*
and ϱ*-c*: ϱ ∈ *carrier-mat n n*
and ϱ*-psd*: *psd* ϱ
shows *psd* (*meas-map Ms* ϱ)
⟨*proof*⟩

lemma *foldr-MdagM-carrier*:
assumes *Ms-c*: ∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*
shows *foldr* (λ*M acc*. (*M*†) ∗ *M* + *acc*) *Ms* (0$_m$ *n n*) ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *meas-map-trace-aux*:
assumes *Ms-c*: ∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*
and ϱ*-c*: ϱ ∈ *carrier-mat n n*
shows *Tr* (*meas-map Ms* ϱ) =
Tr (*foldr* (λ*M acc*. (*M*†) ∗ *M* + *acc*) *Ms* (0$_m$ *n n*) ∗ ϱ)
⟨*proof*⟩

lemma *meas-map-density*:
assumes *ms*: *measurement-system n Ms*
and *dm*: *density-matrix n* ϱ
shows *density-matrix n* (*meas-map Ms* ϱ)
⟨*proof*⟩

19.2 Kraus maps, positive maps, complete positivity

A *Kraus map* $\rho \mapsto \sum_i W_i \rho W_i^\dagger$ is *kraus-map-dim d Ws*; the list *Ws* is a list of *Kraus operators* (*kraus-operators n d Ws*) when $\sum_i W_i^\dagger W_i = I_n$. Positive, trace-preserving and unital maps are defined on the carrier M_n. The amplification $\Phi^{(p)} = \Phi \otimes \mathrm{id}_p$ acts on block matrices by $(X_{ab}) \mapsto (\Phi(X_{ab}))$ (*amplification n d p* Φ), and Φ is *p*-positive when $\Phi^{(p)}$ is positive, completely positive when it is *p*-positive for all *p*. The main result of the subsection: Kraus maps are completely positive.

definition *kraus-map-dim* :: *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
kraus-map-dim d Ws ϱ ≡
foldr (λ*W acc*. *W* ∗ ϱ ∗ (*W*†) + *acc*) *Ws* (0$_m$ *d d*)

definition *kraus-map* :: *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
kraus-map Ws ϱ ≡ *kraus-map-dim* (*dim-row* ϱ) *Ws* ϱ

definition *kraus-operators* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat list* ⇒ *bool* **where**
kraus-operators n d Ws ≡
(∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*) ∧
foldr (λ*W acc*. (*W*†) ∗ *W* + *acc*) *Ws* (0*m n n*) = 1*m n*

lemma *kraus-map-dim-Nil* [*simp*]:
kraus-map-dim d [] ϱ = 0*m d d*
⟨*proof*⟩

lemma *kraus-map-dim-Cons* [*simp*]:
kraus-map-dim d (*W* # *Ws*) ϱ =
W ∗ ϱ ∗ (*W*†) + *kraus-map-dim d Ws* ϱ
⟨*proof*⟩

lemma *kraus-map-dim-two*:
assumes *V*: *V* ∈ *carrier-mat d n* **and** *W*: *W* ∈ *carrier-mat d n* **and** ϱ: ϱ ∈ *carrier-mat n n*
shows *kraus-map-dim d* [*V*, *W*] ϱ = *V* ∗ ϱ ∗ (*V*†) + *W* ∗ ϱ ∗ (*W*†)
⟨*proof*⟩

lemma *kraus-map-dim-carrier*:
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and ϱ*-c*: ϱ ∈ *carrier-mat n n*
shows *kraus-map-dim d Ws* ϱ ∈ *carrier-mat d d*
⟨*proof*⟩

At the identity a Kraus map gives $\sum_i W_i W_i^*$; so it is unital iff $\sum_i W_i W_i^* = I_d$.

lemma *kraus-map-dim-one*:
assumes ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
shows *kraus-map-dim d Ws* (1*m n*)
= *foldr* (λ*W acc*. *W* ∗ (*W*†) + *acc*) *Ws* (0*m d d*)
⟨*proof*⟩

The same computation shows that every Kraus map $\Phi(\rho) = \sum_i W_i \rho W_i^*$, with $W_i \in M_{d,n}$, sends positive matrices to positive matrices.

lemma *kraus-map-dim-psd*:
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and ϱ*-c*: ϱ ∈ *carrier-mat n n*
and ϱ*-psd*: *psd* ϱ
shows *psd* (*kraus-map-dim d Ws* ϱ)
⟨*proof*⟩

definition *positive-map* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
positive-map n d Φ ≡
(∀ *A*. *A* ∈ *carrier-mat n n* ⟶ Φ *A* ∈ *carrier-mat d d*) ∧
(∀ *P*. *P* ∈ *carrier-mat n n* ⟶ *psd P* ⟶ *psd* (Φ *P*))

lemma *kraus-map-dim-positive-map*:
assumes *Ws*: *kraus-operators n d Ws*
shows *positive-map n d* (*kraus-map-dim d Ws*)
⟨*proof*⟩

definition *trace-preserving* :: *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
trace-preserving n Φ ≡
∀ *X*. *X* ∈ *carrier-mat n n* ⟶ *Tr* (Φ *X*) = *Tr X*

definition *unital-map* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
unital-map n d Φ ≡ Φ (1*m n*) = 1*m d*

A $pn \times pn$ matrix is a $p \times p$ matrix of $n \times n$ blocks, $(X_{ab})_{rc} = X_{an+r,\, bn+c}$:

$$X = \begin{pmatrix} X_{00} & \cdots & X_{0,p-1} \\ \vdots & & \vdots \\ X_{p-1,0} & \cdots & X_{p-1,p-1} \end{pmatrix}.$$

These blocks are contiguous; the blocks of Day 16 (*block*) are interleaved.

definition *block-of-mat* :: *nat* ⇒ *complex Matrix.mat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
block-of-mat n x a b ≡
Matrix.mat n n (λ(*r*,*c*). *x* $$ (*a*∗*n* + *r*, *b*∗*n* + *c*))

lemma *block-of-mat-carrier* [*simp*]:
block-of-mat n X a b ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *block-of-mat-index*:
assumes *r* < *n* **and** *c* < *n*
shows *block-of-mat n X a b* $$ (*r*,*c*) = *X* $$ (*a*∗*n* + *r*, *b*∗*n* + *c*)
⟨*proof*⟩

lemma *block-of-mat-single*:
assumes *X*: *X* ∈ *carrier-mat n n*
shows *block-of-mat n X 0 0* = *X*
⟨*proof*⟩

A $pn \times pn$ matrix is the sum of its blocks in place: $x = \sum_{a,b} E_{ab} \otimes x_{ab}$.

lemma *block-of-mat-decomposition*:
assumes *x*: *x* ∈ *carrier-mat* (*p*∗*n*) (*p*∗*n*) **and** *n*: *0* < *n*
shows *x* = *msum* (*p*∗*n*) (*p*∗*n*) (λ(*a*, *b*). *E*⟦*a*,*b* ; *p*,*p*⟧ ⨂ *block-of-mat n x a b*) (*List.product* [*0*..<*p*] [*0*..<*p*])
⟨*proof*⟩

The amplification $\Phi^{(p)} = \mathrm{id}_p \otimes \Phi$ applies Φ block by block, $\Phi^{(p)}((X_{ab})) = (\Phi(X_{ab}))$. Φ is *p-positive* if $\Phi^{(p)}$ is positive, and *completely positive* if it is p-positive for every p.

definition *amplification* :: *nat* ⇒ *nat* ⇒ *nat* ⇒
(*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
amplification n d p Φ *x* ≡
Matrix.mat (*p*∗*d*) (*p*∗*d*)
(λ(*i*,*j*). Φ (*block-of-mat n x* (*i div d*) (*j div d*)) $$
(*i mod d*, *j mod d*))

lemma *amplification-carrier* [*simp*]:
amplification n d p Φ *X* ∈ *carrier-mat* (*p*∗*d*) (*p*∗*d*)
⟨*proof*⟩

lemma *amplification-dims* [*simp*]:
dim-row (*amplification n d p* Φ *x*) = *p*∗*d dim-col* (*amplification n d p* Φ *x*) = *p*∗*d*
⟨*proof*⟩

lemma *amplification-index*:
assumes *i* < *p*∗*d* **and** *j* < *p*∗*d*
shows *amplification n d p* Φ *X* $$ (*i*,*j*) =
Φ (*block-of-mat n X* (*i div d*) (*j div d*)) $$
(*i mod d*, *j mod d*)
⟨*proof*⟩

So $\Phi^{(p)}(x) = \sum_{a,b} E_{ab} \otimes \Phi(x_{ab})$, and $\Phi^{(p)}$ is linear in Φ.

lemma *amplification-blocks*:
assumes Φ: $\bigwedge a\ b.\ a < p \Longrightarrow b < p \Longrightarrow \Phi\ (\textit{block-of-mat}\ n\ x\ a\ b) \in \textit{carrier-mat}\ d\ d$ **and** d: $0 < d$
shows *amplification n d p* Φ *x*
$= \textit{msum}\ (p{*}d)\ (p{*}d)\ (\lambda(a,\ b).\ E[\![a,b\ ;\ p,p]\!] \bigotimes \Phi\ (\textit{block-of-mat}\ n\ x\ a\ b))\ (\textit{List.product}\ [0..{<}p]\ [0..{<}p])$
⟨*proof*⟩

lemma *amplification-zero-map*:
assumes *dpos*: $0 < d$
shows *amplification n d p* $(\lambda\varrho.\ 0_m\ d\ d)\ x = 0_m\ (p{*}d)\ (p{*}d)$
⟨*proof*⟩

lemma *amplification-add*:
assumes *dpos*: $0 < d$
and F: $\bigwedge a\ b.\ F\ (\textit{block-of-mat}\ n\ x\ a\ b) \in \textit{carrier-mat}\ d\ d$
and G: $\bigwedge a\ b.\ G\ (\textit{block-of-mat}\ n\ x\ a\ b) \in \textit{carrier-mat}\ d\ d$
shows *amplification n d p* $(\lambda\varrho.\ F\ \varrho + G\ \varrho)\ x =$ *amplification n d p F x* + *amplification n d p G x*
⟨*proof*⟩

The amplification of a Kraus map is a Kraus map, with operators $I_p \otimes W_i$:

$$\Phi^{(p)}(X) = \sum_i (I_p \otimes W_i)\, X\, (I_p \otimes W_i)^*,$$

since $(I_p \otimes W)X(I_p \otimes W)^*$ has the blocks $WX_{ab}W^*$. The index computations below check this entry by entry.

lemma *tensor-id-kraus-carrier*:
assumes W: $W \in \textit{carrier-mat}\ d\ n$
shows $(1_m\ p \bigotimes W) \in \textit{carrier-mat}\ (p{*}d)\ (p{*}n)$
⟨*proof*⟩

lemma *tensor-id-kraus-list-carrier*:
assumes *Ws*: $\forall\, W \in \textit{set}\ Ws.\ W \in \textit{carrier-mat}\ d\ n$
shows $\forall\, L \in \textit{set}\ (\textit{map}\ (\lambda W.\ 1_m\ p \bigotimes W)\ Ws).$
$L \in \textit{carrier-mat}\ (p{*}d)\ (p{*}n)$
⟨*proof*⟩

Block by block, $(I \otimes W)(E_{ab} \otimes x_{ab})(I \otimes W)^* = E_{ab} \otimes Wx_{ab}W^*$, so the amplification of $X \mapsto WXW^*$ is conjugation by $I_p \otimes W$.

lemma *tensor-unit-sandwich*:
assumes W: $W \in \textit{carrier-mat}\ d\ n$ **and** B: $B \in \textit{carrier-mat}\ n\ n$
shows $(1_m\ p \bigotimes W) * (E[\![a,b\ ;\ p,p]\!] \bigotimes B) * ((1_m\ p \bigotimes W)^\dagger) = E[\![a,b\ ;\ p,p]\!] \bigotimes (W * B * (W^\dagger))$
⟨*proof*⟩

lemma *amplification-sandwich*:
assumes W: $W \in \textit{carrier-mat}\ d\ n$ **and** x: $x \in \textit{carrier-mat}\ (p{*}n)\ (p{*}n)$ **and** n: $0 < n$ **and** d: $0 < d$
shows *amplification n d p* $(\lambda\varrho.\ W * \varrho * (W^\dagger))\ x = (1_m\ p \bigotimes W) * x * ((1_m\ p \bigotimes W)^\dagger)$
⟨*proof*⟩

lemma *amplification-kraus-map-dim*:
assumes *Ws*: $\forall\, W \in \textit{set}\ Ws.\ W \in \textit{carrier-mat}\ d\ n$
and x: $x \in \textit{carrier-mat}\ (p{*}n)\ (p{*}n)$
and *dpos*: $0 < d$
and *npos*: $0 < n$
shows *amplification n d p* (*kraus-map-dim d Ws*) *x* =
kraus-map-dim $(p{*}d)\ (\textit{map}\ (\lambda W.\ 1_m\ p \bigotimes W)\ Ws)\ x$
⟨*proof*⟩

definition *p-positive* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
p-positive n d p Φ ≡
∀ *X* :: *complex Matrix.mat*. *X* ∈ *carrier-mat* (*p*∗*n*) (*p*∗*n*) ⟶ *psd X* ⟶
psd (*amplification n d p* Φ *X*)

definition *completely-positive* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
completely-positive n d Φ ≡ ∀ *p* > *0*. *p-positive n d p* Φ

So every $\Phi^{(p)}$ is a Kraus map, hence positive: Kraus maps are completely positive.

theorem *kraus-is-cp*:
assumes *Ws*: *kraus-operators n d Ws*
and *npos*: *0* < *n*
and *dpos*: *0* < *d*
shows *completely-positive n d* (*kraus-map-dim d Ws*)
⟨*proof*⟩

lemma *kraus-map-dim-append*:
fixes *Ws Vs* :: *complex Matrix.mat list*
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and ϱ*-c*: ϱ ∈ *carrier-mat n n*
shows *kraus-map-dim d* (*Ws* @ *Vs*) ϱ
= *kraus-map-dim d Ws* ϱ + *kraus-map-dim d Vs* ϱ
⟨*proof*⟩

Since *amplification* only evaluates Φ on $n \times n$ blocks, two maps that agree on *carrier-mat n n* yield the same amplification, hence the same notion of complete positivity.

lemma *amplification-cong*:
fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Xm* :: *complex Matrix.mat*
assumes *eq*: ⋀*r*. *r* ∈ *carrier-mat n n* ⟹ *Phi r* = *Psi r*
shows *amplification n d p Phi Xm* = *amplification n d p Psi Xm*
⟨*proof*⟩

lemma *completely-positive-cong*:
fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes ⋀*r*. *r* ∈ *carrier-mat n n* ⟹ *Phi r* = *Psi r*
shows *completely-positive n d Phi* ⟷ *completely-positive n d Psi*
⟨*proof*⟩

Entry-wise formula for *kraus-map-dim* as a sum over the list.

lemma *kraus-map-dim-entry-sum-list*:
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and *rho-c*: ϱ ∈ *carrier-mat n n*
and *i-lt*: *i* < *d* **and** *j-lt*: *j* < *d*
shows *kraus-map-dim d Ws* ϱ $$ (*i*, *j*) =
sum-list (*map* (λ*W*. (*W* ∗ ϱ ∗ (*W*†)) $$ (*i*, *j*)) *Ws*)
⟨*proof*⟩

The sum $\sum_i W_i^* W_i$ over a Kraus list.

definition *sum-wdag-w* :: *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* **where**
sum-wdag-w n Ws ≡
foldr (λ*W acc*. (*W*†) ∗ *W* + *acc*) *Ws* (*0*$_m$ *n n*)

lemma *sum-wdag-w-Nil* [*simp*]: *sum-wdag-w n* [] = *0*$_m$ *n n*

⟨*proof*⟩

lemma *sum-wdag-w-Cons*:
sum-wdag-w n (*W # Ws*) = (*W*†) ∗ *W* + *sum-wdag-w n Ws*
⟨*proof*⟩

lemma *sum-wdag-w-two*:
assumes *V*: *V* ∈ *carrier-mat d n* **and** *W*: *W* ∈ *carrier-mat d n*
shows *sum-wdag-w n* [*V*, *W*] = (*V*†) ∗ *V* + (*W*†) ∗ *W*
⟨*proof*⟩

lemma *sum-wdag-w-carrier*:
assumes ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
shows *sum-wdag-w n Ws* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *sum-wdag-w-dims* [*simp*]: *dim-row* (*sum-wdag-w n Ws*) = *n dim-col* (*sum-wdag-w n Ws*) = *n*
⟨*proof*⟩

lemma *kraus-operators-altdef*:
kraus-operators n d Ws ⟷
(∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*) ∧ *sum-wdag-w n Ws* = *1*$_m$ *n*
⟨*proof*⟩

lemma *Tr-kraus-map-dim-as-wdagw*:
fixes ϱ :: *complex Matrix.mat*
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and *X-c*: ϱ ∈ *carrier-mat n n*
shows *Tr* (*kraus-map-dim d Ws* ϱ) = *Tr* (*sum-wdag-w n Ws* ∗ ϱ)
⟨*proof*⟩

Its entries are $\left(\sum_i W_i^* W_i\right)_{ab} = \sum_i \sum_k \overline{(W_i)_{ka}}\,(W_i)_{kb}$; trace preservation then gives $\sum_i W_i^* W_i = I_n$.

lemma *sum-wdag-w-entry*:
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and *a-lt*: *a* < *n* **and** *b-lt*: *b* < *n*
shows *sum-wdag-w n Ws* $$ (*a*, *b*) =
sum-list (*map* (λ*W*. ((*W*†) ∗ *W*) $$ (*a*, *b*)) *Ws*)
⟨*proof*⟩

Pairing with a matrix unit picks out an entry: $\mathrm{Tr}(XE_{ab}) = x_{ba}$.

19.3 The environment model of noise

Day 17, the environment model of noise: a state ρ of lab A meets an environment in the state $|\varphi\rangle$, the pair evolves by a unitary U on $\mathbb{C}^m \otimes \mathbb{C}^p$, and lab A sees $\mathrm{tr}_E(U(\rho \otimes |\varphi\rangle\langle\varphi|)U^*) = \sum_i W_i \rho W_i^*$ with $\sum_i W_i^* W_i = I$. The index of $\mathbb{C}^m \otimes \mathbb{C}^p$ is $a\,p + i$ (lab index a, environment index i), and W_i is the matrix with entries $\sum_k U_{(a,i),(c,k)} \varphi_k$, i.e. the notes' $W_i = \sum_k U_{ik} \alpha_k$ for the blocks U_{ik} of U.

definition *env-kraus* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
env-kraus m p U φ *i* = *Matrix.mat m m* (λ(*a*,*c*). ∑ *k*<*p*. *U* $$ (*a* ∗ *p* + *i*, *c* ∗ *p* + *k*) ∗ φ $ *k*)

The Kraus operators of the environment model are $W_i = (I \otimes \langle e_i|)\,U\,(I \otimes |\varphi\rangle)$, entrywise $(W_i)_{ac} = \sum_k U_{ap+i,\,cp+k}\,\varphi_k$, and

$$\mathrm{tr}_E(U(\rho \otimes |\varphi\rangle\langle\varphi|)U^*) = \sum_i W_i \rho W_i^*.$$

lemma *env-kraus-carrier* [*simp*]: *env-kraus m p U φ i ∈ carrier-mat m m*
⟨*proof*⟩

lemma *env-kraus-dims* [*simp*]: *dim-row* (*env-kraus m p U φ i*) = *m dim-col* (*env-kraus m p U φ i*) = *m*
⟨*proof*⟩

lemma *env-kraus-index*: *a* < *m* ⟹ *c* < *m* ⟹ *env-kraus m p U φ i* $$ (*a*,*c*) = (∑ *k*<*p*. *U* $$ (*a* ∗ *p* + *i*, *c* ∗ *p* + *k*) ∗ *φ* $ *k*)
⟨*proof*⟩

lemma *sum-split-mp*:
fixes *f* :: *nat* ⇒ *'a*::*comm-monoid-add*
shows (∑ *x* < *m* ∗ *p*. *f x*) = (∑ *c*<*m*. ∑ *k*<*p*. *f* (*c* ∗ *p* + *k*))
⟨*proof*⟩

lemma *tensor-outer-index*:
fixes *ϱ* :: *complex Matrix.mat* **and** *φ* :: *complex Matrix.vec*
assumes *ϱ*: *ϱ* ∈ *carrier-mat m m* **and** *φ*: *dim-vec φ* = *p*
and *c*: *c* < *m* **and** *d*: *d* < *m* **and** *k*: *k* < *p* **and** *l*: *l* < *p*
shows *Tensor.tensor-mat ϱ* (*φ* ⊘ *φ*) $$ (*c* ∗ *p* + *k*, *d* ∗ *p* + *l*) = *ϱ* $$ (*c*,*d*) ∗ (*φ* $ *k* ∗ *cnj* (*φ* $ *l*))
⟨*proof*⟩

The entries of $U(\rho \otimes |\varphi\rangle\langle\varphi|)U^*$.

lemma *env-entry*:
fixes *U ϱ* :: *complex Matrix.mat* **and** *φ* :: *complex Matrix.vec*
assumes *U*: *U* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*) **and** *ϱ*: *ϱ* ∈ *carrier-mat m m* **and** *φ*: *dim-vec φ* = *p*
and *x*: *x* < *m* ∗ *p* **and** *y*: *y* < *m* ∗ *p*
shows (*U* ∗ *Tensor.tensor-mat ϱ* (*φ* ⊘ *φ*) ∗ (*U*†)) $$ (*x*,*y*)
= (∑ *d*<*m*. ∑ *c*<*m*. (∑ *k*<*p*. *U* $$ (*x*, *c* ∗ *p* + *k*) ∗ *φ* $ *k*) ∗ *ϱ* $$ (*c*,*d*)
∗ *cnj* (∑ *l*<*p*. *U* $$ (*y*, *d* ∗ *p* + *l*) ∗ *φ* $ *l*))
⟨*proof*⟩

theorem *environment-model*:
fixes *U ϱ* :: *complex Matrix.mat* **and** *φ* :: *complex Matrix.vec*
assumes *U*: *U* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*) **and** *Uu*: *unitary U*
and *φ*: *dim-vec φ* = *p* **and** *φ1*: ⟨*φ*|*φ*⟩ = *1* **and** *ϱ*: *ϱ* ∈ *carrier-mat m m*
defines *Ws* ≡ *map* (*env-kraus m p U φ*) [*0*..<*p*]
shows *partial-trace-B m p* (*U* ∗ *Tensor.tensor-mat ϱ* (*φ* ⊘ *φ*) ∗ (*U*†)) = *kraus-map-dim m Ws ϱ*
and *kraus-operators m m Ws*
⟨*proof*⟩

Part III

Completely positive maps (Days 18–23)

20 Day 18 (7 October 2011): the theory of completely positive maps

Proposition (notes). Every $X \in M_n$ is $X = (P_1 - P_2) + i(P_3 - P_4)$ with $P_i \geq 0$: $\operatorname{span}(M_n^+) = M_n$.

(Write $X = H + iK$ with H, K Hermitian and split each spectral decomposition into its non-negative and negative eigenvalues.)

Definition (notes). A linear $\Phi : M_n \to M_d$ is *positive* if $P \geq 0$ implies $\Phi(P) \geq 0$; it is *p-positive* if $\Phi^{(p)} : M_p(M_n) \to M_p(M_d)$, $(X_{ij}) \mapsto (\Phi(X_{ij}))$, is positive, and *completely positive* if it is p-positive for every p.

Proposition (notes). A linear $\Phi : M_n \to M_d$ maps density matrices to density matrices iff it is positive and trace preserving.

Example (notes): the transpose $\Phi(A) = A^t$ on M_2 is positive but not 2-positive, since $\Phi^{(2)}$ sends the positive matrix $(E_{ij})_{i,j}$ to a matrix of determinant -1. The decomposition into positives is formalised below; the proposition is *density-preserving-iff-positive-tp* and the example *transpose-not-2-positive*, both below.

lemma *hermitian-decomp-psd*:
  fixes *H* :: *complex Matrix.mat*
  assumes *sq*: *H* ∈ *carrier-mat n n*
    and *herm*: *hermitian H*
  shows ∃ *P1 P2* :: *complex Matrix.mat*.
      P1 ∈ *carrier-mat n n* ∧ *P2* ∈ *carrier-mat n n* ∧
      psd P1 ∧ *psd P2* ∧
      H = *P1* − *P2*
⟨*proof*⟩

lemma *matrix-decomp-psd*:
  fixes *X* :: *complex Matrix.mat*
  assumes *sq*: *X* ∈ *carrier-mat n n*
  shows ∃ *P1 P2 P3 P4* :: *complex Matrix.mat*.
      (∀ *i* ∈ {*0..3*}. [*P1*,*P2*,*P3*,*P4*] ! *i* ∈ *carrier-mat n n* ∧ *psd* ([*P1*,*P2*,*P3*,*P4*] ! *i*)) ∧
      X = (*P1* − *P2*) + i ·$_m$ (*P3* − *P4*)
⟨*proof*⟩

definition *choi-seed* :: *nat* ⇒ *complex Matrix.mat* **where**
  choi-seed n ≡
    Matrix.mat (*n*∗*n*) (*n*∗*n*)
      (λ(*r*,*c*). *mat-unit* (*r div n*) (*c div n*) *n n* $$ (*r mod n*, *c mod n*))

lemma *choi-seed-carrier* [*simp*]:
  choi-seed n ∈ *carrier-mat* (*n*∗*n*) (*n*∗*n*)
  ⟨*proof*⟩

20.1 Densities to densities iff positive and trace-preserving (D17-4, D18-2)

theorem *density-preserving-iff-positive-tp*:
  fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
  assumes *linear*: ⋀*A B*. *A* ∈ *carrier-mat n n* ⟹ *B* ∈ *carrier-mat n n* ⟹
        Φ (*A* + *B*) = Φ *A* + Φ *B*
    and *smult*: ⋀*c A*. *A* ∈ *carrier-mat n n* ⟹ Φ (*c* ·$_m$ *A*) = *c* ·$_m$ Φ *A*
    and *range*: ∀ *A*. *A* ∈ *carrier-mat n n* ⟶ Φ *A* ∈ *carrier-mat d d*
    and *npos*: *0* < *n*
  shows (∀ ϱ. *density-matrix n* ϱ ⟶ *density-matrix d* (Φ ϱ)) ⟷
      (*positive-map n d* Φ ∧ *trace-preserving n* Φ)
⟨*proof*⟩

20.2 The transpose is positive but not 2-positive (D18-4)

lemma *transpose-positive-map*: *positive-map n n* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*)
⟨*proof*⟩

The notes' witness: $X = (E_{ij}) \in M_2(M_2)$ is positive (it is $|w\rangle\langle w|$ for $w = e_0 \otimes e_0 + e_1 \otimes e_1$) while $\Phi^{(2)}(X)$ has the eigenvector $(0, 1, -1, 0)$ with eigenvalue -1 (the notes compute $\det \Phi^{(2)}(X) = -1$ instead).

lemma *choi-seed-2-outer*: *choi-seed 2 = outer-sum 4* [*Matrix.vec 4* (λ*i*. *if i = 0 ∨ i = 3 then 1 else 0*)]
⟨*proof*⟩

lemma *choi-seed-2-psd*: *psd* (*choi-seed 2*)
⟨*proof*⟩

lemma *transpose-not-2-positive*:
¬ *p-positive 2 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*)
⟨*proof*⟩

corollary *transpose-not-completely-positive*:
¬ *completely-positive 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*)
⟨*proof*⟩

21 Day 19 (10 October 2011): Choi's theorem

Theorem (notes) (Choi, 1975)**.** For linear $\Phi : M_n \to M_d$ the following are equivalent: (1) Φ is completely positive; (2) Φ is n-positive; (3) $P_\Phi = (\Phi(E_{ij}))_{i,j} \geq 0$ in $M_n(M_d)$; (4) there are $d \times n$ matrices $B_1, \ldots, B_K$ with $\Phi(X) = \sum_k B_k X B_k^*$.

Notes: (1) $\sum_k B_k X B_k^*$ is a *Choi–Kraus representation*; (2) P_Φ is the *Choi–Jamiołkowski matrix*; (3) $CP(M_n, M_d) \leftrightarrow M_n(M_d)^+$ via $\Phi \leftrightarrow P_\Phi$; (4) Φ is trace preserving iff $\mathrm{tr}(\Phi(E_{ij})) = \delta_{ij}$; (5) Φ is unital iff $\sum_i \Phi(E_{ii}) = I_d$.
Formally P_Φ is *choi-matrix n d Φ* (an $nd \times nd$ matrix built from the blocks $\Phi(E_{ij})$); *choi-theorem* is (1)⇔(3), *kraus-rep-of-psd-choi* is (3)⇒(4), and *kraus-is-cp* is (4)⇒(1). The step (2)⇒(3) is the observation that $P_\Phi = \Phi^{(n)}((E_{ij}))$ with $(E_{ij}) \geq 0$ (*choi-matrix-as-amplification*). Linearity of Φ is assumed on the carrier M_n (additivity and homogeneity hypotheses). The trace-preserving case is *choi-kraus-cptp*; notes (4) and (5) in Choi-matrix form are *choi-trace-preserving-iff* and *choi-unital-iff* below.

definition *choi-matrix* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* **where**
choi-matrix n d Φ ≡
Matrix.mat (*n*∗*d*) (*n*∗*d*)
(λ(*r*,*c*). *Φ* (*mat-unit* (*r div d*) (*c div d*) *n n*) $$ (*r mod d*, *c mod d*))

notation *choi-matrix* (P_Φ)

The seed $\sum_{a,b} E_{ab} \otimes E_{ab} \in M_n(M_n)$ has the matrix unit E_{ab} as its (a, b) block, so the Choi matrix is the amplification of Φ applied to it:

$$P_\Phi = \big(\Phi(E_{ab})\big)_{a,b} = \Phi^{(n)}\Big(\sum_{a,b} E_{ab} \otimes E_{ab}\Big).$$

The seed is $|\xi\rangle\langle\xi|$ with $\xi = \sum_a e_a \otimes e_a$, hence positive; so $P_\Phi \geq 0$ when Φ is completely positive.

lemma *choi-seed-block*:
assumes *npos*: *n* > *0*
and *a*: *a* < *n*
and *b*: *b* < *n*
shows *block-of-mat n* (*choi-seed n*) *a b* = *mat-unit a b n n*

⟨*proof*⟩

lemma *choi-matrix-as-amplification*:
  assumes *npos*: $n > 0$
    and *dpos*: $d > 0$
  shows *choi-matrix n d* Φ = *amplification n d n* Φ (*choi-seed n*)
⟨*proof*⟩

lemma *choi-psd-if-cp*:
  assumes *npos*: $n > 0$
    and *dpos*: $d > 0$
    and *cp*: *completely-positive n d* Φ
  shows *psd* (*choi-matrix n d* Φ)
⟨*proof*⟩

lemma *kraus-cp-no-tp*:
  assumes *Ws-c*: $\forall$ *W* $\in$ *set Ws. W* $\in$ *carrier-mat d n*
    and *npos*: $0 < n$
    and *dpos*: $0 < d$
  shows *completely-positive n d* (*kraus-map-dim d Ws*)
⟨*proof*⟩

21.1 From a positive Choi matrix to a Kraus representation

The notes' proof of (3)⇒(4): write $P_\Phi = \sum_k v_k v_k^*$ with $v_k \in \mathbb{C}^{nd}$, cut each v_k into n blocks $w_1^k, \ldots, w_n^k \in \mathbb{C}^d$, and let $B_k = [w_1^k| \cdots |w_n^k]$; then $B_k E_{ij} B_k^* = w_i^k (w_j^k)^*$ and $\Phi(X) = \sum_{ij} x_{ij} \Phi(E_{ij}) = \sum_k B_k X B_k^*$ by linearity.

Complete positivity from a Kraus list without the trace-preservation constraint: the argument is the same as ⟦*kraus-operators ?n ?d ?Ws*; $0 <$ *?n*; $0 <$ *?d*⟧ ⟹ *completely-positive ?n ?d* (*kraus-map-dim ?d ?Ws*), which only uses the carrier hypothesis.

Linearity of the Kraus-sum map in its matrix argument.

lemma *kraus-map-dim-add*:
  assumes *A-c*: *A* $\in$ *carrier-mat n n* **and** *B-c*: *B* $\in$ *carrier-mat n n*
    and *Ws-c*: $\forall$ *W* $\in$ *set Ws. W* $\in$ *carrier-mat d n*
  shows *kraus-map-dim d Ws* (*A* + *B*) =
      kraus-map-dim d Ws A + *kraus-map-dim d Ws B*
  ⟨*proof*⟩

lemma *kraus-map-dim-smult*:
  assumes *A-c*: *A* $\in$ *carrier-mat n n*
    and *Ws-c*: $\forall$ *W* $\in$ *set Ws. W* $\in$ *carrier-mat d n*
  shows *kraus-map-dim d Ws* (*c* $\cdot_m$ *A*) = *c* $\cdot_m$ *kraus-map-dim d Ws A*
  ⟨*proof*⟩

lemma *kraus-map-dim-zero* [*simp*]:
  assumes *Ws-c*: $\forall$ *W* $\in$ *set Ws. W* $\in$ *carrier-mat d n*
  shows *kraus-map-dim d Ws* (0_m *n n*) = 0_m *d d*
  ⟨*proof*⟩

Enumeration of $[0..<n] \times [0..<n]$ as a list of pairs.

definition *mat-unit-pairs* :: *nat* ⇒ (*nat* × *nat*) *list* **where**
  mat-unit-pairs n ≡
    concat (*map* (λ*a*. *map* (*Pair a*) [*0*..<*n*]) [*0*..<*n*])

lemma *mat-unit-pairs-set*:

set (*mat-unit-pairs n*) = {*p*. *fst p* < *n* ∧ *snd p* < *n*}
⟨*proof*⟩

lemma *mat-unit-pairs-distinct*: *distinct* (*mat-unit-pairs n*)
⟨*proof*⟩

The combination $\sum_{a,b} A_{ab}\, f(E_{ab})$.

primrec *pair-comb* ::
(*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒
nat ⇒ *nat* ⇒ (*nat* × *nat*) *list* ⇒ *complex Matrix.mat* **where**
pair-comb f A n d [] = 0_m *d d* |
pair-comb f A n d (*p* # *ps*) =
A $$ *p* ·$_m$ *f* (*mat-unit* (*fst p*) (*snd p*) *n n*) + *pair-comb f A n d ps*

lemma *pair-comb-carrier*:
assumes *f-car*: ⋀*a b*. *a* < *n* ⟹ *b* < *n* ⟹ *f* (*mat-unit a b n n*) ∈ *carrier-mat d d*
and *ps-valid*: ∀ *p* ∈ *set ps*. *fst p* < *n* ∧ *snd p* < *n*
shows *pair-comb f A n d ps* ∈ *carrier-mat d d*
⟨*proof*⟩

lemma *pair-comb-entry*:
assumes *f-car*: ⋀*a b*. *a* < *n* ⟹ *b* < *n* ⟹ *f* (*mat-unit a b n n*) ∈ *carrier-mat d d*
and *ps-valid*: ∀ *p* ∈ *set ps*. *fst p* < *n* ∧ *snd p* < *n*
and *r-lt*: *r* < *d* **and** *c-lt*: *c* < *d*
shows *pair-comb f A n d ps* $$ (*r*,*c*) =
(∑ *p* ← *ps*. *A* $$ *p* ∗ *f* (*mat-unit* (*fst p*) (*snd p*) *n n*) $$ (*r*,*c*))
⟨*proof*⟩

For the identity-like choice $f = id$, the pair combination reconstructs A when ps enumerates all basis positions.

lemma *pair-comb-id-reconstructs*:
assumes *A-car*: *A* ∈ *carrier-mat n n*
shows *pair-comb* (λ*X*. *X*) *A n n* (*mat-unit-pairs n*) = *A*
⟨*proof*⟩

For linear $f : M_n \to M_d$, $f(A) = \sum_{a,b} A_{ab}\, f(E_{ab})$.

lemma *linear-eq-pair-comb*:
fixes *f* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *f-add*: ⋀*X Y*. *X* ∈ *carrier-mat n n* ⟹ *Y* ∈ *carrier-mat n n* ⟹
f (*X* + *Y*) = *f X* + *f Y*
and *f-smult*: ⋀*c X*. *X* ∈ *carrier-mat n n* ⟹ *f* (*c* ·$_m$ *X*) = *c* ·$_m$ *f X*
and *f-car*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *f X* ∈ *carrier-mat d d*
and *A-car*: *A* ∈ *carrier-mat n n*
and *npos*: *n* > *0* **and** *dpos*: *d* > *0*
shows *f A* = *pair-comb f A n d* (*mat-unit-pairs n*)
⟨*proof*⟩

$(WE_{ab}W^*)_{ij} = W_{ia}\,\overline{W_{jb}}$.

lemma *conjugation-mat-unit-entry*:
assumes *Wc*: *W* ∈ *carrier-mat d n*
and *a-lt*: *a* < *n* **and** *b-lt*: *b* < *n*
and *i-lt*: *i* < *d* **and** *j-lt*: *j* < *d*
shows (*W* ∗ *mat-unit a b n n* ∗ (*W*†)) $$ (*i*, *j*) =
W $$ (*i*, *a*) ∗ *cnj* (*W* $$ (*j*, *b*))
⟨*proof*⟩

From a positive Choi matrix to Kraus operators: factor $P_\Phi = VV^*$ and reshape each column of V into a $d \times n$ matrix W_k; then $\Phi(X) = \sum_k W_k X W_k^*$ on all of M_n.

lemma *kraus-rep-of-psd-choi*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow$ $B \in$ *carrier-mat n n* $\Longrightarrow$
$\Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow$ $\Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow$ $\Phi\ A \in$ *carrier-mat d d*
and *npos*: $n > 0$
and *dpos*: $d > 0$
and *psdP*: *psd* (*choi-matrix n d* Φ)
shows $\exists$ *Ws* :: *complex Matrix.mat list*.
length Ws $= n * d \wedge$
$(\forall W \in$ *set Ws*. $W \in$ *carrier-mat d n*$) \wedge$
$(\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow$ $\Phi\ A =$ *kraus-map-dim d Ws A*$)$
⟨*proof*⟩

theorem *choi-theorem*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow$ $B \in$ *carrier-mat n n* $\Longrightarrow$
$\Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow$ $\Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow$ $\Phi\ A \in$ *carrier-mat d d*
and *npos*: $n > 0$
and *dpos*: $d > 0$
shows *completely-positive n d* $\Phi \longleftrightarrow$
psd (*choi-matrix n d* Φ)
⟨*proof*⟩

lemma *Tr-mult-mat-unit*:
assumes *M-c*: $M \in$ *carrier-mat n n*
and *a-lt*: $a < n$ **and** *b-lt*: $b < n$
shows *Tr* $(M *$ *mat-unit a b n n*$) = M$ \$\$ (b, a)
⟨*proof*⟩

theorem *choi-kraus-cptp*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow$ $B \in$ *carrier-mat n n* $\Longrightarrow$
$\Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow$ $\Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow$ $\Phi\ A \in$ *carrier-mat d d*
and *npos*: $n > 0$
and *dpos*: $d > 0$
shows (*completely-positive n d* $\Phi \wedge$ *trace-preserving n* Φ) $\longleftrightarrow$
($\exists$ *Ws* :: *complex Matrix.mat list*.
kraus-operators n d Ws $\wedge$
$(\forall X.\ X \in$ *carrier-mat n n* $\longrightarrow$ $\Phi\ X =$ *kraus-map-dim d Ws X*$)$)
⟨*proof*⟩

lemma *kraus-lin-on*:
assumes $\forall W \in$ *set Ws*. $W \in$ *carrier-mat d n*
shows *lin-on n d* (*carrier-mat n n*) (*kraus-map-dim d Ws*)
⟨*proof*⟩

lemma *sum-wdag-w-append*:
fixes *Ws Vs* :: *complex Matrix.mat list*
assumes *Ws-c*: $\forall W \in$ *set Ws*. $W \in$ *carrier-mat d n*
and *Vs-c*: $\forall V \in$ *set Vs*. $V \in$ *carrier-mat d n*
shows *sum-wdag-w n* (*Ws* @ *Vs*)
$=$ *sum-wdag-w n Ws* $+$ *sum-wdag-w n Vs*

⟨*proof*⟩

lemma *kraus-is-trace-preserving*:
fixes *Ws* :: *complex Matrix.mat list*
assumes *Ws*: *kraus-operators n d Ws*
shows *trace-preserving n* (*kraus-map-dim d Ws*)
⟨*proof*⟩

21.2 Composition of Kraus maps

definition *kraus-compose* :: *complex Matrix.mat list* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat list* **where**
kraus-compose As Es = *concat* (*map* (λ*E*. *map* (λ*A*. *A* ∗ *E*) *As*) *Es*)

lemma *kraus-compose-Nil* [*simp*]: *kraus-compose As* [] = []
⟨*proof*⟩

lemma *kraus-compose-Cons*: *kraus-compose As* (*E* # *Es*) = *map* (λ*A*. *A* ∗ *E*) *As* @ *kraus-compose As Es*
⟨*proof*⟩

lemma *length-kraus-compose* [*simp*]: *length* (*kraus-compose As Es*) = *length Es* ∗ *length As*
⟨*proof*⟩

Composing Kraus maps multiplies their operators: for $\mathcal{E}(X) = \sum_i E_i X E_i^*$ and $\mathcal{A}(Y) = \sum_l A_l Y A_l^*$,

$$(\mathcal{A} \circ \mathcal{E})(X) = \sum_{i,l} (A_l E_i)\, X\, (A_l E_i)^*,$$

and if both are trace preserving then $\sum_{i,l} (A_l E_i)^* (A_l E_i) = \sum_i E_i^* \big(\sum_l A_l^* A_l\big) E_i = \sum_i E_i^* E_i = I$.

lemma *kraus-compose-nth*:
assumes *i*: *i* < *length Es* **and** *l*: *l* < *length As*
shows *kraus-compose As Es* ! (*i* ∗ *length As* + *l*) = *As* ! *l* ∗ *Es* ! *i*
⟨*proof*⟩

lemma *kraus-compose-carrier*:
assumes ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat m d* **and** ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat d n*
shows ∀ *W* ∈ *set* (*kraus-compose As Es*). *W* ∈ *carrier-mat m n*
⟨*proof*⟩

lemma *kraus-map-dim-map-mult-right*:
assumes *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat m d* **and** *Ec*: *E* ∈ *carrier-mat d n* **and** *Xc*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim m* (*map* (λ*A*. *A* ∗ *E*) *As*) *M* = *kraus-map-dim m As* (*E* ∗ *M* ∗ (*E*†))
⟨*proof*⟩

lemma *kraus-map-dim-compose*:
assumes *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat m d* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat d n*
and *Xc*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim m* (*kraus-compose As Es*) *M* = *kraus-map-dim m As* (*kraus-map-dim d Es M*)
⟨*proof*⟩

lemma *sum-wdag-w-map-mult-right*:
assumes *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat m d* **and** *Ec*: *E* ∈ *carrier-mat d n*
shows *sum-wdag-w n* (*map* (λ*A*. *A* ∗ *E*) *As*) = (*E*†) ∗ *sum-wdag-w d As* ∗ *E*
⟨*proof*⟩

lemma *sum-wdag-w-compose*:

assumes *As-c*: $\forall\, A \in$ *set As*. $A \in$ *carrier-mat m d* **and** *Es-c*: $\forall\, E \in$ *set Es*. $E \in$ *carrier-mat d n*
and *As-tp*: *sum-wdag-w d As* $= 1_m\ d$
shows *sum-wdag-w n* (*kraus-compose As Es*) = *sum-wdag-w n Es*
⟨*proof*⟩

lemma *kraus-operators-compose*:
assumes *As*: *kraus-operators d m As* **and** *Es*: *kraus-operators n d Es*
shows *kraus-operators n m* (*kraus-compose As Es*)
⟨*proof*⟩

21.3 Vectorisation of Kraus operators

The vectorisation of $V \in M_{d,n}$ stacks its columns: the vector in $\mathbb{C}^{nd}$ with entry $V_{r \bmod d,\, r \div d}$ at position r.

definition *vec-of-kraus* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec*
where
vec-of-kraus d n V = *Matrix.vec* (*n*d*) (λ*r*. *V* $$ (*r mod d*, *r div d*))

lemma *vec-of-kraus-dim* [*simp*]: *dim-vec* (*vec-of-kraus d n V*) = *n*d*
⟨*proof*⟩

lemma *vec-of-kraus-index*:
assumes *r-lt*: *r* < *n*d*
shows *vec-of-kraus d n V* $ *r* = *V* $$ (*r mod d*, *r div d*)
⟨*proof*⟩

At position $ad + i$ it holds V_{ia}.

lemma *vec-of-kraus-combined-index*:
assumes *V-c*: *V* ∈ *carrier-mat d n*
and *a-lt*: *a* < *n* **and** *i-lt*: *i* < *d* **and** *dpos*: *0* < *d*
shows *vec-of-kraus d n V* $ (*a*d* + *i*) = *V* $$ (*i*, *a*)
⟨*proof*⟩

21.4 Choi matrix of a Kraus map

The Choi matrix of a Kraus map is the outer sum of the vectorised Kraus operators:

$$\Phi(X) = \sum_i V_i X V_i^* \quad \Longrightarrow \quad P_\Phi = \sum_i v_i v_i^*, \qquad v_i = \mathrm{vec}(V_i),$$

so its range is spanned by the v_i.

lemma *kraus-map-dim-mat-unit-entry*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *a-lt*: *a* < *n* **and** *b-lt*: *b* < *n*
and *i-lt*: *i* < *d* **and** *j-lt*: *j* < *d*
shows (*kraus-map-dim d Vs* (*mat-unit a b n n*)) $$ (*i*, *j*)
= (∑ *k*<*length Vs*. *Vs*!*k* $$ (*i*, *a*) * *cnj* (*Vs*!*k* $$ (*j*, *b*)))
⟨*proof*⟩

lemma *choi-matrix-kraus-eq*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *npos*: *0* < *n* **and** *dpos*: *0* < *d*
shows *choi-matrix n d* (*kraus-map-dim d Vs*)
= *outer-sum* (*n*d*) (*map* (*vec-of-kraus d n*) *Vs*)
⟨*proof*⟩

21.5 Choi matrix is linear in the map

The Choi matrix depends linearly on the map: $P_{\Phi-\Psi} = P_\Phi - P_\Psi$.

lemma *choi-matrix-carrier* [*simp*]:
choi-matrix n d Φ ∈ *carrier-mat* (*n*∗*d*) (*n*∗*d*)
⟨*proof*⟩

The Choi matrix is linear in the map.

lemma *choi-matrix-smult*:
assumes *F*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *F X* ∈ *carrier-mat d d* **and** *d*: *0* < *d*
shows *choi-matrix n d* (λ*X*. *a* ·$_m$ *F X*) = *a* ·$_m$ *choi-matrix n d F*
⟨*proof*⟩

lemma *choi-matrix-lincomb*:
assumes *F*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *F X* ∈ *carrier-mat d d*
and *G*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *G X* ∈ *carrier-mat d d* **and** *d*: *0* < *d*
shows *choi-matrix n d* (λ*X*. *a* ·$_m$ *F X* + *b* ·$_m$ *G X*) = *a* ·$_m$ *choi-matrix n d F* + *b* ·$_m$ *choi-matrix n d G*
⟨*proof*⟩

The Choi matrix agrees on maps that agree on the carrier.

lemma *choi-matrix-cong*:
assumes *agree*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Phi X* = *Psi X*
shows *choi-matrix n d Phi* = *choi-matrix n d Psi*
⟨*proof*⟩

21.6 Choi matrix determines a linear CP map on carrier

If two maps that are linear on carrier have the same Choi matrix and the same target carrier, they agree pointwise on carrier.

lemma *map-eq-from-choi-when-linear*:
fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *Phi-add*: ⋀*X Y*. *X* ∈ *carrier-mat n n* ⟹ *Y* ∈ *carrier-mat n n*
⟹ *Phi* (*X* + *Y*) = *Phi X* + *Phi Y*
and *Phi-smult*: ⋀*c X*. *X* ∈ *carrier-mat n n* ⟹ *Phi* (*c* ·$_m$ *X*) = *c* ·$_m$ *Phi X*
and *Phi-car*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Phi X* ∈ *carrier-mat d d*
and *Psi-add*: ⋀*X Y*. *X* ∈ *carrier-mat n n* ⟹ *Y* ∈ *carrier-mat n n*
⟹ *Psi* (*X* + *Y*) = *Psi X* + *Psi Y*
and *Psi-smult*: ⋀*c X*. *X* ∈ *carrier-mat n n* ⟹ *Psi* (*c* ·$_m$ *X*) = *c* ·$_m$ *Psi X*
and *Psi-car*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Psi X* ∈ *carrier-mat d d*
and *choi-eq*: *choi-matrix n d Phi* = *choi-matrix n d Psi*
and *npos*: *n* > *0* **and** *dpos*: *d* > *0*
and *A-c*: *A* ∈ *carrier-mat n n*
shows *Phi A* = *Psi A*
⟨*proof*⟩

21.7 Choi's theorem with (2) isolated (D19-1)

For $f : \{0,\dots,r-1\} \to \{0,\dots,n-1\}$ the block matrix $(E_{f(a)f(b)})_{a,b} \in M_r(M_n)$. The notes' examples are $f = (i,j)$ with $r = 2$, giving $\begin{pmatrix} E_{ii} & E_{ij} \\ E_{ji} & E_{jj} \end{pmatrix}$, and $f = \mathrm{id}$ with $r = n$, giving (E_{ij}).

definition *unit-blocks* :: *nat* ⇒ *nat* ⇒ (*nat* ⇒ *nat*) ⇒ *complex Matrix.mat* **where**
unit-blocks r n f = *Matrix.mat* (*r* ∗ *n*) (*r* ∗ *n*)
(λ(*x*, *y*). *if x mod n* = *f* (*x div n*) ∧ *y mod n* = *f* (*y div n*) *then 1 else 0*)

lemma *unit-blocks-block*:
assumes $a < r$ $b < r$ $f\ a < n$ $f\ b < n$
shows *block-of-mat n* (*unit-blocks r n f*) *a b* = *mat-unit* (*f a*) (*f b*) *n n*
⟨*proof*⟩

definition *unit-blocks-vec* :: *nat* ⇒ *nat* ⇒ (*nat* ⇒ *nat*) ⇒ *complex Matrix.vec* **where**
unit-blocks-vec r n f = *Matrix.vec* (*r* ∗ *n*) (λ*x*. *if x mod n* = *f* (*x div n*) *then 1 else 0*)

The matrix with blocks $E_{f(a)f(b)}$ is $|u\rangle\langle u|$ for $u = \sum_a e_a \otimes e_{f(a)}$, hence positive.

lemma *unit-blocks-outer*: *unit-blocks r n f* = *unit-blocks-vec r n f* ⊘ *unit-blocks-vec r n f*
⟨*proof*⟩

lemma *unit-blocks-psd*: *psd* (*unit-blocks r n f*)
⟨*proof*⟩

lemma *choi-seed-unit-blocks*: *choi-seed n* = *unit-blocks n n id*
⟨*proof*⟩

lemma *choi-seed-psd*: *psd* (*choi-seed n*)
⟨*proof*⟩

lemma *choi-psd-if-n-positive*:
assumes *n*: *0* < *n* **and** *d*: *0* < *d* **and** *pp*: *p-positive n d n* Φ
shows *psd* (*choi-matrix n d* Φ)
⟨*proof*⟩

theorem *choi-four-conditions*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *linear*: ⋀*A B*. *A* ∈ *carrier-mat n n* ⟹ *B* ∈ *carrier-mat n n* ⟹ Φ (*A* + *B*) = Φ *A* + Φ *B*
and *smult*: ⋀*c A*. *A* ∈ *carrier-mat n n* ⟹ Φ (*c* ·*m A*) = *c* ·*m* Φ *A*
and *range*: ∀ *A*. *A* ∈ *carrier-mat n n* ⟶ Φ *A* ∈ *carrier-mat d d*
and *n*: *0* < *n* **and** *d*: *0* < *d*
shows (*completely-positive n d* Φ ⟷ *p-positive n d n* Φ)
∧ (*p-positive n d n* Φ ⟷ *psd* (*choi-matrix n d* Φ))
∧ (*psd* (*choi-matrix n d* Φ) ⟷
(∃ *Ws*. (∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*) ∧ (∀ *A*. *A* ∈ *carrier-mat n n* ⟶ Φ *A* = *kraus-map-dim d Ws A*)))
⟨*proof*⟩

21.8 Trace preservation and unitality in Choi-matrix form (D19-3)

Day 19, notes (4) and (5) after Choi's theorem: write $P_\Phi = (R_{ij})$ with $R_{ij} = \Phi(E_{ij}) \in M_d$. Then Φ is trace preserving iff $\mathrm{Tr}\, R_{ij} = \delta_{ij}$, and unital iff $\sum_i R_{ii} = I_d$; with *choi-theorem* this characterises CPTP and UCP maps by their Choi matrix. Linearity on M_n is *lin-on n d* (*carrier-mat n n*) Φ.

lemma *choi-block*:
assumes *range*: ∀ *A*. *A* ∈ *carrier-mat n n* ⟶ Φ *A* ∈ *carrier-mat d d* **and** *i*: *i* < *n* **and** *j*: *j* < *n*
shows *block-of-mat d* (*choi-matrix n d* Φ) *i j* = Φ (*E*⟦*i*,*j* ; *n*,*n*⟧)
⟨*proof*⟩

lemma *msum-cong*: (⋀*x*. *x* ∈ *set xs* ⟹ *g x* = *h x*) ⟹ *msum r c g xs* = *msum r c h xs*
⟨*proof*⟩

lemma *Tr-msum*:
assumes ∀ *x* ∈ *set xs*. *g x* ∈ *carrier-mat d d*
shows *Tr* (*msum d d g xs*) = (∑ *x*←*xs*. *Tr* (*g x*))
⟨*proof*⟩

lemma *Tr-msum-pairs*:
assumes $\bigwedge i\ j.\ i < k \Longrightarrow j < k \Longrightarrow g\ i\ j \in carrier\text{-}mat\ r\ r$
shows $Tr\ (msum\ r\ r\ (\lambda(i,\ j).\ g\ i\ j)\ (List.product\ [0..<k]\ [0..<k])) = (\sum i{<}k.\ \sum j{<}k.\ Tr\ (g\ i\ j))$
⟨*proof*⟩

In matrix units $A = \sum_{i,j} a_{ij} E_{ij}$ and $I = \sum_i E_{ii}$, so a linear Φ satisfies $\Phi(A) = \sum_{i,j} a_{ij}\,\Phi(E_{ij}) = \sum_{i,j} a_{ij} R_{ij}$, the R_{ij} being the blocks of P_Φ.

lemma *unit-expansion*:
assumes *A*: $A \in carrier\text{-}mat\ n\ n$
shows $A = msum\ n\ n\ (\lambda(i,j).\ A\ \$\$\ (i,j)\ \cdot_m\ E[\![i,j\ ;\ n,n]\!])\ (List.product\ [0..<n]\ [0..<n])$
⟨*proof*⟩

lemma *id-expansion*: $1_m\ n = msum\ n\ n\ (\lambda i.\ E[\![i,i\ ;\ n,n]\!])\ [0..<n]$
⟨*proof*⟩

lemma *lin-expansion*:
assumes *L*: $lin\text{-}on\ n\ d\ (carrier\text{-}mat\ n\ n)\ \Phi$ **and** *A*: $A \in carrier\text{-}mat\ n\ n$
shows $\Phi\ A = msum\ d\ d\ (\lambda(i,j).\ A\ \$\$\ (i,j)\ \cdot_m\ \Phi\ (E[\![i,j\ ;\ n,n]\!]))\ (List.product\ [0..<n]\ [0..<n])$
⟨*proof*⟩

D19-3, note (4): trace preservation in Choi-matrix form.

theorem *choi-trace-preserving-iff*:
assumes *L*: $lin\text{-}on\ n\ d\ (carrier\text{-}mat\ n\ n)\ \Phi$
shows $trace\text{-}preserving\ n\ \Phi \longleftrightarrow$
$(\forall\, i < n.\ \forall\, j < n.\ Tr\ (block\text{-}of\text{-}mat\ d\ (choi\text{-}matrix\ n\ d\ \Phi)\ i\ j) = (if\ i = j\ then\ 1\ else\ 0))$
⟨*proof*⟩

D19-3, note (5): unitality in Choi-matrix form, $\sum_i R_{ii} = I_d$.

theorem *choi-unital-iff*:
assumes *L*: $lin\text{-}on\ n\ d\ (carrier\text{-}mat\ n\ n)\ \Phi$
shows $unital\text{-}map\ n\ d\ \Phi \longleftrightarrow msum\ d\ d\ (\lambda i.\ block\text{-}of\text{-}mat\ d\ (choi\text{-}matrix\ n\ d\ \Phi)\ i\ i)\ [0..<n] = 1_m\ d$
⟨*proof*⟩

lemma *lin-on-choi-hyps*:
assumes *L*: $lin\text{-}on\ n\ d\ (carrier\text{-}mat\ n\ n)\ \Phi$
shows $\bigwedge A\ B.\ A \in carrier\text{-}mat\ n\ n \Longrightarrow B \in carrier\text{-}mat\ n\ n \Longrightarrow \Phi\ (A + B) = \Phi\ A + \Phi\ B$
and $\bigwedge c\ A.\ A \in carrier\text{-}mat\ n\ n \Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and $\forall\, A.\ A \in carrier\text{-}mat\ n\ n \longrightarrow \Phi\ A \in carrier\text{-}mat\ d\ d$
⟨*proof*⟩

Notes (4) and (5) with Choi's theorem: CPTP iff $P_\Phi \geq 0$ and $\operatorname{Tr} R_{ij} = \delta_{ij}$; UCP iff $P_\Phi \geq 0$ and $\sum_i R_{ii} = I_d$.

corollary *choi-cptp-iff*:
assumes *L*: $lin\text{-}on\ n\ d\ (carrier\text{-}mat\ n\ n)\ \Phi$ **and** *n*: $n > 0$ **and** *d*: $d > 0$
shows $completely\text{-}positive\ n\ d\ \Phi \wedge trace\text{-}preserving\ n\ \Phi \longleftrightarrow$
$psd\ (choi\text{-}matrix\ n\ d\ \Phi) \wedge$
$(\forall\, i < n.\ \forall\, j < n.\ Tr\ (block\text{-}of\text{-}mat\ d\ (choi\text{-}matrix\ n\ d\ \Phi)\ i\ j) = (if\ i = j\ then\ 1\ else\ 0))$
⟨*proof*⟩

corollary *choi-ucp-iff*:
assumes *L*: $lin\text{-}on\ n\ d\ (carrier\text{-}mat\ n\ n)\ \Phi$ **and** *n*: $n > 0$ **and** *d*: $d > 0$
shows $completely\text{-}positive\ n\ d\ \Phi \wedge unital\text{-}map\ n\ d\ \Phi \longleftrightarrow$
$psd\ (choi\text{-}matrix\ n\ d\ \Phi) \wedge msum\ d\ d\ (\lambda i.\ block\text{-}of\text{-}mat\ d\ (choi\text{-}matrix\ n\ d\ \Phi)\ i\ i)\ [0..<n] = 1_m\ d$
⟨*proof*⟩

22 Day 20 (12 October 2011): positive functionals, adjoints, Choi rank

Proposition (notes). For linear $\phi : M_n \to \mathbb{C}$: ϕ positive $\Leftrightarrow$ ϕ completely positive $\Leftrightarrow$ $(\phi(E_{ij})) \in M_n^+$.

Proposition (notes). Let $\Phi : M_n \to M_d$ with Hilbert–Schmidt adjoint $\Phi^* : M_d \to M_n$ ($\langle Y|X\rangle = \mathrm{tr}(Y^*X)$). (1) If $\Phi(X) = \sum_i A_i X A_i^*$ then $\Phi^*(Y) = \sum_i A_i^* Y A_i$. (2) Φ is completely positive iff Φ^* is. (3) Φ is unital and completely positive iff Φ^* is completely positive and trace preserving.

Definition (notes). The *Choi rank* of a completely positive Φ is $\mathrm{cr}(\Phi) = \min\{L : \Phi(X) = \sum_{l=1}^{L} B_l X B_l^*\}$.

Theorem (notes) (Choi)**.** $\mathrm{cr}(\Phi) = \mathrm{rank}(P_\Phi)$.

The functional proposition, the adjoint proposition (for Kraus maps) and the definition of the Choi rank are formalised below; the rank theorem is *choi-rank-eq-rank*, proved on Day 22 from the linear independence of minimal Kraus representations (its lemma $\mathrm{rank}((E_{ij})) = 1$ is not needed), and the bound $\mathrm{cr}(\Phi) \le nd$ is below.

22.1 The adjoint of a Kraus map

For $\Phi(X) = \sum_i A_i X A_i^\dagger$ with $A_i \in M_{d,n}$, the adjoint is $\Phi^*(Y) = \sum_i A_i^\dagger Y A_i$ (*kraus-adjoint-dim n As*; the output dimension n is kept explicit for the empty list).

definition *kraus-adjoint-dim* ::
  nat ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
  kraus-adjoint-dim n As y ≡
    foldr (λ*A acc*. (*A*†) ∗ *y* ∗ *A* + *acc*) *As* (0*m n n*)

lemma *kraus-adjoint-dim-Nil* [*simp*]:
  fixes *Y* :: *complex Matrix.mat*
  shows *kraus-adjoint-dim n* [] *Y* = 0*m n n*
  ⟨*proof*⟩

lemma *kraus-adjoint-dim-Cons* [*simp*]:
  fixes *Y* :: *complex Matrix.mat*
  shows *kraus-adjoint-dim n* (*A* # *As*) *Y* =
        (*A*†) ∗ *Y* ∗ *A* + *kraus-adjoint-dim n As Y*
  ⟨*proof*⟩

lemma *kraus-adjoint-dim-carrier*:
  fixes *Y* :: *complex Matrix.mat*
  assumes *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat d n*
    and *Y-c*: *Y* ∈ *carrier-mat d d*
  shows *kraus-adjoint-dim n As Y* ∈ *carrier-mat n n*
  ⟨*proof*⟩

lemma *kraus-adjoint-dim-as-kraus-map-dim*:
  fixes *Y* :: *complex Matrix.mat*
  assumes *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat d n*
    and *Y-c*: *Y* ∈ *carrier-mat d d*
  shows *kraus-adjoint-dim n As Y* =
        kraus-map-dim n (*map* (λ*A*. (*A*†)) *As*) *Y*
  ⟨*proof*⟩

The adjoint identity $\langle Y|\Phi(X)\rangle = \langle \Phi^*(Y)|X\rangle$, by induction on the Kraus list and cyclicity of the trace.

lemma *hs-inner-prod-right-zero*:
$\langle\langle\ y \mid 0_m\ m\ n\ \rangle\rangle = 0$
⟨*proof*⟩

lemma *hs-inner-prod-kraus-term*:
fixes $A\ X\ Y$ :: *complex Matrix.mat*
assumes *A-c*: $A \in$ *carrier-mat d n*
and *X-c*: $X \in$ *carrier-mat n n*
and *Y-c*: $Y \in$ *carrier-mat d d*
shows $\langle\langle\ Y \mid A * X * (A^\dagger)\ \rangle\rangle = \langle\langle\ (A^\dagger) * Y * A \mid X\ \rangle\rangle$
⟨*proof*⟩

theorem *kraus-adjoint-correct*:
fixes *As* :: *complex Matrix.mat list*
and *X* :: *complex Matrix.mat*
and *Y* :: *complex Matrix.mat*
assumes *As-c*: $\forall A \in$ *set As*. $A \in$ *carrier-mat d n*
and *X-c*: $X \in$ *carrier-mat n n*
and *Y-c*: $Y \in$ *carrier-mat d d*
shows $\langle\langle\ Y \mid$ *kraus-map-dim d As X* $\rangle\rangle$
$= \langle\langle$ *kraus-adjoint-dim n As Y* $\mid X\ \rangle\rangle$
⟨*proof*⟩

22.2 Complete positivity and the UCP/CPTP duality

Φ^* is the Kraus map of the adjoint operators, hence completely positive, and Φ is unital iff Φ^* is trace preserving: $\sum_i A_i A_i^* = I$ in both cases, using that $\mathrm{tr}(BY) = \mathrm{tr}(Y)$ for all Y forces $B = I$.

theorem *adjoint-cp-from-kraus*:
assumes *As-c*: $\forall A \in$ *set As*. $A \in$ *carrier-mat d n*
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *completely-positive d n* (*kraus-adjoint-dim n As*)
⟨*proof*⟩

lemma *kraus-map-dim-one-eq-sum-wdag-w-dagger*:
fixes *As* :: *complex Matrix.mat list*
assumes *As-c*: $\forall A \in$ *set As*. $A \in$ *carrier-mat d n*
shows *kraus-map-dim d As* $(1_m\ n)$
$=$ *sum-wdag-w d* (*map* $(\lambda A.\ (A^\dagger))$ *As*)
⟨*proof*⟩

theorem *cp-unital-iff-adjoint-tp*:
assumes *As-c*: $\forall A \in$ *set As*. $A \in$ *carrier-mat d n*
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *unital-map n d* (*kraus-map-dim d As*)
$\longleftrightarrow$ *trace-preserving d* (*kraus-adjoint-dim n As*)
⟨*proof*⟩

22.3 Positive functionals are completely positive

The case $d = 1$ of Choi's theorem: for $P = (P_{ij}) \in M_r(M_n)^+$ and $v = (\alpha_i) \in \mathbb{C}^r$, $\langle v|(\phi(P_{ij}))v\rangle = \phi(\sum_{ij} \overline{\alpha_i}\alpha_j P_{ij}) = \phi(Y^*PY) \geq 0$ with Y the block column $(\alpha_i I)_i$.

lemma *phi-entry-double-sum*:

fixes φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *f-add*: $\bigwedge X\ Y.\ X \in$ *carrier-mat n n* $\Longrightarrow\ Y \in$ *carrier-mat n n*
$\Longrightarrow \varphi\ (X + Y) = \varphi\ X + \varphi\ Y$
and *f-smult*: $\bigwedge c\ X.\ X \in$ *carrier-mat n n* $\Longrightarrow \varphi\ (c \cdot_m X) = c \cdot_m \varphi\ X$
and *f-car*: $\bigwedge X.\ X \in$ *carrier-mat n n* $\Longrightarrow \varphi\ X \in$ *carrier-mat d d*
and *M-c*: $M \in$ *carrier-mat n n*
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
and *npos*: $0 < n$ **and** *dpos*: $0 < d$
shows $\varphi\ M$ \$\$ $(r,\ c) =$
$(\sum i{<}n.\ \sum j{<}n.\ M$ \$\$ $(i,\ j) * \varphi\ ($*mat-unit i j n n*$)$ \$\$ $(r,\ c))$
⟨*proof*⟩

theorem *positive-scalar-is-cp*:
fixes φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow\ B \in$ *carrier-mat n n*
$\Longrightarrow \varphi\ (A + B) = \varphi\ A + \varphi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow \varphi\ (c \cdot_m A) = c \cdot_m \varphi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \varphi\ A \in$ *carrier-mat 1 1*
and *pos*: *positive-map n 1* φ
and *npos*: $0 < n$
shows *completely-positive n 1* φ
⟨*proof*⟩

22.4 Choi rank

choi-rank n d Φ is the least length of a Kraus list representing Φ on the carrier. The theorem $\mathrm{cr}(\Phi) = \mathrm{rank}(P_\Phi)$ is *choi-rank-eq-rank* (Day 22); the bound $\mathrm{cr}(\Phi) \le nd$ follows from the Kraus-rank bound of Day 21.

definition *choi-rank* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *nat* **where**
choi-rank n d $\Phi \equiv$
Inf { *length As* | *As* :: *complex Matrix.mat list*.
$(\forall A \in$ *set As*. $A \in$ *carrier-mat d n*$)$
$\wedge\ (\forall X.\ X \in$ *carrier-mat n n* $\longrightarrow \Phi\ X =$ *kraus-map-dim d As X*$)$ }

23 Day 21 (14 October 2011): the Kraus rank bound; unitary freedom of representations

Corollary (notes). A completely positive $\Phi : M_n \to M_d$ has a Kraus representation $\Phi(X) = \sum_{l=1}^{K} A_l X A_l^*$ with $K \le nd$.

The notes then record a conjecture (every CP map is an average of n UCP maps of Choi rank $\le d$) and develop the uniqueness theory:

Lemma (notes). If $T : \mathbb{C}^r \to \mathbb{C}^m$ is one-to-one then T^* is onto.

Proposition (notes). If $v_1, \dots, v_r \in \mathbb{C}^k$ are linearly independent then so are $\{v_i v_j^*\}_{i,j} \subseteq M_k$.

Lemma (notes). If $0 \le ww^* \le P$ then $w \in \mathrm{range}(P)$; hence for $P = \sum_l w_l w_l^* \ge 0$, $\mathrm{range}(P) = \mathrm{span}\{w_l\}$.

Theorem (notes). If $P = \sum_{j=1}^{r} v_j v_j^* = \sum_{l=1}^{m} w_l w_l^*$ with $\mathrm{rank}\, P = r$, there is $U = (\alpha_{ij})_{m \times r}$ with $w_i = \sum_j \alpha_{ij} v_j$ and $U^*U = I_r$.

Theorem (notes) (Choi 2)**.** If Φ is completely positive with $\mathrm{cr}(\Phi) = r$ and $\Phi(X) = \sum_{j=1}^{r} V_j X V_j^* = \sum_{l=1}^{m} W_l X W_l^*$, then $W_i = \sum_j \alpha_{ij} V_j$ for a unique $U = (\alpha_{ij})$ with $U^*U = I_r$.

Formalised below: the Kraus rank bound, unitary freedom (D21-3, D21-5, D21-6) and the ranges of positive matrices (D21-4), whose kernel form ($\ker A \subseteq \ker B$ when $0 \le B \le A$, *loewner-ker-subset*) Day 23 uses to prove Choi's Theorem 3 without the unitary-freedom theorem. The conjecture on averages (D21-2) is the next theory.

corollary *kraus-rank-bound*:
  fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
  assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow B \in$ *carrier-mat n n*
    $\Longrightarrow \Phi\ (A + B) = \Phi\ A + \Phi\ B$
    and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
    and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \Phi\ A \in$ *carrier-mat d d*
    and *cp*: *completely-positive n d* Φ
    and *npos*: $0 < n$
    and *dpos*: $0 < d$
  shows $\exists$ *As* :: *complex Matrix.mat list*.
      length As $\le n * d \wedge$
      $(\forall A \in$ *set As*. $A \in$ *carrier-mat d n*$) \wedge$
      $(\forall X.\ X \in$ *carrier-mat n n* $\longrightarrow \Phi\ X =$ *kraus-map-dim d As X*$)$
⟨*proof*⟩

$\mathrm{cr}(\Phi) \le nd$: the defining set of the Choi rank is non-empty.

corollary *choi-rank-le*:
  fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
  assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow B \in$ *carrier-mat n n*
    $\Longrightarrow \Phi\ (A + B) = \Phi\ A + \Phi\ B$
    and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
    and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \Phi\ A \in$ *carrier-mat d d*
    and *cp*: *completely-positive n d* Φ
    and *npos*: $0 < n$
    and *dpos*: $0 < d$
  shows *choi-rank n d* $\Phi \le n * d$
⟨*proof*⟩

23.1 Unitary freedom of rank-one decompositions and Kraus representations (D21-3, D21-5, D21-6)

23.1.1 The Gram matrix of an independent list is invertible; dual vectors

lemma *gram-det-nonzero*:
  assumes *li*: *lin-indep-vecs n vs*
  shows *det* $((\textit{mat-of-cols n vs})^\dagger * \textit{mat-of-cols n vs}) \neq 0$
⟨*proof*⟩

lemma *gram-inverse*:
  assumes *li*: *lin-indep-vecs n vs*
  obtains *B* **where** $B \in$ *carrier-mat* (*length vs*) (*length vs*)
    and $((\textit{mat-of-cols n vs})^\dagger * \textit{mat-of-cols n vs}) * B = 1_m\ (\textit{length vs})$
    and $B * ((\textit{mat-of-cols n vs})^\dagger * \textit{mat-of-cols n vs}) = 1_m\ (\textit{length vs})$
    and $B^\dagger = B$
⟨*proof*⟩

Dual vectors (the notes' "T^* is onto"): for independent $v_1, \ldots, v_k$ there are $\varphi_1, \ldots, \varphi_k$ with $\langle \varphi_i | v_j \rangle = \delta_{ij}$.

lemma *dual-vectors-exist*:
  assumes *li*: *lin-indep-vecs n vs*
  shows $\exists \varphi s$ :: *complex Matrix.vec list*. *length* $\varphi s =$ *length vs* $\wedge$
      $(\forall \varphi \in$ *set* φs. *dim-vec* $\varphi = n) \wedge$
      $(\forall i <$ *length vs*. $\forall j <$ *length vs*. $\langle \varphi s\ !\ i \mid vs\ !\ j \rangle =$ (*if* $i = j$ *then 1 else 0*)$)$

⟨*proof*⟩

23.1.2 D21-3: independent vectors give independent rank-one matrices

Linear independence over $\mathbb{C}$ of a list of $d \times n$ matrices.

definition *lin-indep-complex-mat* ::
  nat ⇒ *nat* ⇒ *complex Matrix.mat list* ⇒ *bool*
where
  lin-indep-complex-mat d n Ws ≡
    (∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*) ∧
    (∀ *c* :: *nat* ⇒ *complex*.
      foldr (λ*k acc*. *c k* ·$_m$ (*Ws*!*k*) + *acc*) [*0*..<*length Ws*] (*0*$_m$ *d n*)
        = *0*$_m$ *d n*
      ⟶ (∀ *k* < *length Ws*. *c k* = *0*))

Linear independence passes to the adjoints: $\sum_k c_k W_k^* = 0$ gives $\sum_k \bar{c}_k W_k = 0$.

lemma *lin-indep-complex-mat-dagger*:
  assumes *li*: *lin-indep-complex-mat d n Ws*
  shows *lin-indep-complex-mat n d* (*map* (λ*W*. *W*†) *Ws*)
  ⟨*proof*⟩

lemma *lin-indep-complex-mat-msum*:
  lin-indep-complex-mat d n Ws ⟷ (∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*) ∧
    (∀ *c*. *msum d n* (λ*k*. *c k* ·$_m$ (*Ws* ! *k*)) [*0*..<*length Ws*] = *0*$_m$ *d n* ⟶ (∀ *k* < *length Ws*. *c k* = *0*))
  ⟨*proof*⟩

definition *outer-prods* :: *complex Matrix.vec list* ⇒ *complex Matrix.mat list* **where**
  outer-prods vs = *map* (λ*k*. *vs* ! (*k div length vs*) ⊘ *vs* ! (*k mod length vs*)) [*0*..<*length vs* ∗ *length vs*]

lemma *length-outer-prods* [*simp*]: *length* (*outer-prods vs*) = *length vs* ∗ *length vs*
  ⟨*proof*⟩

lemma *outer-prods-nth*:
  assumes *k* < *length vs* ∗ *length vs*
  shows *outer-prods vs* ! *k* = *vs* ! (*k div length vs*) ⊘ *vs* ! (*k mod length vs*)
  ⟨*proof*⟩

lemma *inner-prod-cnj-delta*:
  assumes *dim-vec* φ = *n* **and** *dim-vec v* = *n* **and** ⟨φ | *v*⟩ = (*if P then 1 else 0*)
  shows ⟨*v* | φ⟩ = (*if P then 1 else 0*)
  ⟨*proof*⟩

proposition *outer-prods-lin-indep*:
  assumes *li*: *lin-indep-vecs n vs*
  shows *lin-indep-complex-mat n n* (*outer-prods vs*)
⟨*proof*⟩

23.1.3 Isometric mixing of columns preserves VV^*

lemma *dagger-transpose-mult*:
  assumes *Uc*: *U* ∈ *carrier-mat m r* **and** *UU*: (*U*†) ∗ *U* = *1*$_m$ *r*
  shows *transpose-mat U* ∗ ((*transpose-mat U*)†) = *1*$_m$ *r*
⟨*proof*⟩

lemma *isometry-mix-outer*:
  assumes *Vc*: *V* ∈ *carrier-mat n r* **and** *Uc*: *U* ∈ *carrier-mat m r* **and** *UU*: (*U*†) ∗ *U* = *1*$_m$ *r*
  shows (*V* ∗ *transpose-mat U*) ∗ ((*V* ∗ *transpose-mat U*)†) = *V* ∗ (*V*†)

⟨*proof*⟩

lemma *dagger-mult-zero-imp-zero*:
fixes Y :: *complex Matrix.mat*
assumes *Yc*: $Y \in$ *carrier-mat p q* **and** *YY*: $(Y^\dagger) * Y = 0_m\ q\ q$
shows $Y = 0_m\ p\ q$
⟨*proof*⟩

23.1.4 D21-5: two rank-one decompositions are related by an isometry

If $P = \sum_{j<r} v_j v_j^* = \sum_{l<m} w_l w_l^*$ with $v_1, \ldots, v_r$ linearly independent (the notes assume rank $P = r$, which is equivalent), then $w_l = \sum_j \alpha_{lj} v_j$ with $U = (\alpha_{lj})$ an $m \times r$ isometry, $U^*U = I_r$. Matrix form: $W = VU^t$.

theorem *rank-one-decomp-isometry*:
assumes *li*: *lin-indep-vecs n vs*
and *wsd*: $\forall\, w \in$ *set ws. dim-vec w = n*
and *same*: *outer-sum n vs = outer-sum n ws*
shows $\exists\, U$ :: *complex Matrix.mat. U* $\in$ *carrier-mat* (*length ws*) (*length vs*) $\wedge$
$(U^\dagger) * U = 1_m$ (*length vs*) $\wedge$
*mat-of-cols n ws = mat-of-cols n vs * transpose-mat U*
⟨*proof*⟩

corollary *rank-one-decomp-isometry-cols*:
assumes *li*: *lin-indep-vecs n vs*
and *wsd*: $\forall\, w \in$ *set ws. dim-vec w = n*
and *same*: *outer-sum n vs = outer-sum n ws*
shows $\exists\, U$ :: *complex Matrix.mat. U* $\in$ *carrier-mat* (*length ws*) (*length vs*) $\wedge$
$(U^\dagger) * U = 1_m$ (*length vs*) $\wedge$
$(\forall\, l <$ *length ws. ws ! l = mat-of-cols n vs* $*_v$ *Matrix.row U l*)
⟨*proof*⟩

23.1.5 D21-6 (Choi 2): unitary freedom of Kraus representations

lemma *vec-of-kraus-mat-of-cols-index*:
assumes *Vs-c*: $\forall\, V \in$ *set Vs. V* $\in$ *carrier-mat d n* **and** *dpos*: $0 < d$
and *i*: $i < d$ **and** *a*: $a < n$ **and** *k*: $k <$ *length Vs*
shows *mat-of-cols* $(n*d)$ (*map* (*vec-of-kraus d n*) *Vs*) \$\$ $(a*d + i, k) =$ *Vs ! k* \$\$ (i, a)
⟨*proof*⟩

lemma *lin-indep-complex-mat-vec-of-kraus*:
assumes *li*: *lin-indep-complex-mat d n Vs* **and** *dpos*: $0 < d$
shows *lin-indep-vecs* $(n*d)$ (*map* (*vec-of-kraus d n*) *Vs*)
⟨*proof*⟩

Choi's Theorem 2: if $\Phi(X) = \sum_{j<r} V_j X V_j^* = \sum_{l<m} W_l X W_l^*$ with $V_1, \ldots, V_r$ linearly independent (the notes: $r = \mathrm{cr}(\Phi)$), then $W_l = \sum_j \alpha_{lj} V_j$ with $U = (\alpha_{lj})$ an isometry, $U^*U = I_r$.

theorem *choi-2-unitary-freedom*:
assumes *npos*: $0 < n$ **and** *dpos*: $0 < d$
and *Ws-c*: $\forall\, W \in$ *set Ws. W* $\in$ *carrier-mat d n*
and *li*: *lin-indep-complex-mat d n Vs*
and *same*: $\forall\, X.\ X \in$ *carrier-mat n n* $\longrightarrow$ *kraus-map-dim d Vs X = kraus-map-dim d Ws X*
shows $\exists\, U$ :: *complex Matrix.mat. U* $\in$ *carrier-mat* (*length Ws*) (*length Vs*) $\wedge$
$(U^\dagger) * U = 1_m$ (*length Vs*) $\wedge$
$(\forall\, l <$ *length Ws. Ws ! l =*
foldr $(\lambda j\ acc.\ U$ \$\$ $(l,j) \cdot_m$ (*Vs ! j*) $+$ *acc*) [*0..<length Vs*] $(0_m\ d\ n))$
⟨*proof*⟩

23.2 Ranges of positive matrices (D21-4)

If $A \geq B \geq 0$ then $\ker A \subseteq \ker B$: for $Ax = 0$, $0 = \langle x|Ax\rangle = \langle x|Bx\rangle + \langle x|(A-B)x\rangle$ with both terms ≥ 0, so $\langle x|Bx\rangle = 0$; writing $B = NN^*$ this is $\|N^*x\|^2 = 0$, hence $Bx = N(N^*x) = 0$.

lemma *loewner-ker-subset*:
fixes *A B* :: *complex Matrix.mat* **and** *x* :: *complex Matrix.vec*
assumes *A-c*: *A* ∈ *carrier-mat k k*
and *B-c*: *B* ∈ *carrier-mat k k*
and *B-psd*: *psd B*
and *diff-psd*: *psd* (*A* − *B*)
and *x-dim*: *dim-vec x* = *k*
and *Ax-zero*: *A* ∗$_v$ *x* = 0$_v$ *k*
shows *B* ∗$_v$ *x* = 0$_v$ *k*
⟨*proof*⟩

The notes: if $0 \leq ww^* \leq P$ then $w \in R(P)$.

theorem *dominated-rank-one-in-range*:
assumes *P*: *P* ∈ *carrier-mat k k* **and** *Pp*: *psd P* **and** *w*: *dim-vec w* = *k*
and *dom*: *psd* (*P* − *outer-sum k* [*w*])
shows *w* ∈ *mat-range k P*
⟨*proof*⟩

23.3 The conjecture on averages of unital maps (D21-2)

The notes ask whether every unital completely positive $\Phi : M_n \to M_d$ is the average of n unital completely positive maps of Choi rank at most d (*average-conjecture*). This is the strong Ruskai–Audenaert conjecture for the dual channel, open in general. Proved here: Fourier mixing settles every map whose Kraus operators split into n groups with vanishing cross sums (*fourier-split*); this covers measurement maps (*ra-measurement*), and Horn's lemma (*horn-lemma*, the case $d = 1$) covers maps with diagonal values (*ra-cq*). The notes' form (i) is false as written (*notes-form-i-false*).

lemma *conj-entry*:
assumes *W*: *W* ∈ *carrier-mat d n* **and** *X*: *X* ∈ *carrier-mat n n* **and** *a*: *a* < *d* **and** *b*: *b* < *d*
shows (*W* ∗ *X* ∗ (*W*†)) $$ (*a*, *b*) = (∑ *p*<*n*. ∑ *q*<*n*. *W* $$ (*a*, *p*) ∗ *X* $$ (*p*, *q*) ∗ *cnj* (*W* $$ (*b*, *q*)))
⟨*proof*⟩

lemma *sum-list-concat-upt*:
sum-list (*map f* (*concat* (*map* (λ*m*. *map* (*g m*) [0..<*d*]) [0..<*n*]))) = (∑ *m*<*n*. ∑ *j*<*d*. *f* (*g m j*))
⟨*proof*⟩

Characters of $\mathbb{Z}_n$.

definition *omega-pow* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex* **where**
omega-pow n l m = *cis* (2 ∗ *pi* ∗ *real l* ∗ *real m* / *real n*)

With $\omega = e^{2\pi i/n}$, *omega-pow n l m* is ω^{lm}. The characters are orthogonal,

$$\sum_{l<n} \omega^{lm}\,\overline{\omega^{lm'}} = n\,\delta_{mm'},$$

so the Fourier mixing $x \mapsto \left(\sum_m \omega^{lm} x_m\right)_l$ multiplies inner products by n.

lemma *omega-pow-norm* [*simp*]: *omega-pow n l m* ∗ *cnj* (*omega-pow n l m*) = 1
⟨*proof*⟩

lemma *omega-pow-orth*:
assumes *m*: *m* < *n* **and** *m'*: *m'* < *n*

shows $(\sum l{<}n.\ \mathit{omega\text{-}pow}\ n\ l\ m * \mathit{cnj}\ (\mathit{omega\text{-}pow}\ n\ l\ m')) = (\mathit{if}\ m = m'\ \mathit{then}\ \mathit{of\text{-}nat}\ n\ \mathit{else}\ 0)$
⟨*proof*⟩

lemma *omega-pow-mix*:
$(\sum l{<}n.\ (\sum m{<}n.\ \mathit{omega\text{-}pow}\ n\ l\ m * x\ m) * \mathit{cnj}\ (\sum m{<}n.\ \mathit{omega\text{-}pow}\ n\ l\ m * y\ m)) = \mathit{of\text{-}nat}\ n * (\sum m{<}n.\ x\ m * \mathit{cnj}\ (y\ m))$
⟨*proof*⟩

lemma *kraus-concat-entry*:
assumes g: $\bigwedge m\ j.\ m < k \Longrightarrow j < d \Longrightarrow g\ m\ j \in \mathit{carrier\text{-}mat}\ d\ n$ **and** X: $X \in \mathit{carrier\text{-}mat}\ n\ n$ **and** a: $a < d$ **and** b: $b < d$
shows $\mathit{kraus\text{-}map\text{-}dim}\ d\ (\mathit{concat}\ (\mathit{map}\ (\lambda m.\ \mathit{map}\ (g\ m)\ [0..{<}d])\ [0..{<}k]))\ X\ \$\$\ (a,\ b) = (\sum m{<}k.\ \sum j{<}d.\ (g\ m\ j * X * ((g\ m\ j)^\dagger))\ \$\$\ (a,\ b))$
⟨*proof*⟩

lemma *kraus-concat-carrier*:
assumes g: $\bigwedge m\ j.\ m < k \Longrightarrow j < d \Longrightarrow g\ m\ j \in \mathit{carrier\text{-}mat}\ d\ n$ **and** X: $X \in \mathit{carrier\text{-}mat}\ n\ n$
shows $\mathit{kraus\text{-}map\text{-}dim}\ d\ (\mathit{concat}\ (\mathit{map}\ (\lambda m.\ \mathit{map}\ (g\ m)\ [0..{<}d])\ [0..{<}k]))\ X \in \mathit{carrier\text{-}mat}\ d\ d$
⟨*proof*⟩

lemma *prod-dagger-entry*:
assumes A: $A \in \mathit{carrier\text{-}mat}\ d\ n$ **and** B: $B \in \mathit{carrier\text{-}mat}\ d\ n$ **and** a: $a < d$ **and** b: $b < d$
shows $(A * (B^\dagger))\ \$\$\ (a,\ b) = (\sum p{<}n.\ A\ \$\$\ (a,\ p) * \mathit{cnj}\ (B\ \$\$\ (b,\ p)))$
⟨*proof*⟩

D21-2 in the notes' main form: a unital $\Phi : M_n \to M_d$ is the average of n unital completely positive maps of Choi rank at most d.

definition *ucp-average-split* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
$\mathit{ucp\text{-}average\text{-}split}\ n\ d\ \Phi \longleftrightarrow (\exists\, Bs.\ \mathit{length}\ Bs = n \wedge (\forall\, l{<}n.\ \mathit{length}\ (Bs\ !\ l) \le d \wedge (\forall\, B \in \mathit{set}\ (Bs\ !\ l).\ B \in \mathit{carrier\text{-}mat}\ d\ n)$
$\quad \wedge\ \mathit{kraus\text{-}map\text{-}dim}\ d\ (Bs\ !\ l)\ (1_m\ n) = 1_m\ d) \wedge (\forall\, X \in \mathit{carrier\text{-}mat}\ n\ n.\ \Phi\ X = (1\ /\ \mathit{of\text{-}nat}\ n) \cdot_m \mathit{kraus\text{-}map\text{-}dim}\ d\ (\mathit{concat}\ Bs)\ X))$

The conjecture itself, for maps given by Kraus operators (every completely positive map is one). It is the strong Ruskai–Audenaert conjecture for the dual channel $M_d \to M_n$, open in general; the cases below are the ones settled in the literature by direct constructions.

definition *average-conjecture* :: *bool* **where**
$\mathit{average\text{-}conjecture} \longleftrightarrow (\forall\, n\ d\ As.\ 0 < n \longrightarrow (\forall\, A \in \mathit{set}\ As.\ A \in \mathit{carrier\text{-}mat}\ d\ n) \longrightarrow \mathit{kraus\text{-}map\text{-}dim}\ d\ As\ (1_m\ n) = 1_m\ d$
$\quad \longrightarrow \mathit{ucp\text{-}average\text{-}split}\ n\ d\ (\mathit{kraus\text{-}map\text{-}dim}\ d\ As))$

definition *fourier-mix* :: *nat* ⇒ *nat* ⇒ (*nat* ⇒ *nat* ⇒ *complex Matrix.mat*) ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
$\mathit{fourier\text{-}mix}\ n\ d\ A\ l\ j = \mathit{Matrix.mat}\ d\ n\ (\lambda(a,\ p).\ \sum m{<}n.\ \mathit{omega\text{-}pow}\ n\ l\ m * A\ m\ j\ \$\$\ (a,\ p))$

lemma *fourier-mix-carrier* [*simp*]: $\mathit{fourier\text{-}mix}\ n\ d\ A\ l\ j \in \mathit{carrier\text{-}mat}\ d\ n$ ⟨*proof*⟩

lemma *fourier-mix-index*: $a < d \Longrightarrow p < n \Longrightarrow \mathit{fourier\text{-}mix}\ n\ d\ A\ l\ j\ \$\$\ (a,\ p) = (\sum m{<}n.\ \mathit{omega\text{-}pow}\ n\ l\ m * A\ m\ j\ \$\$\ (a,\ p))$
⟨*proof*⟩

If the Kraus operators fall into n groups of d with $\sum_j A_{mj} A^*_{m'j} = 0$ for $m \neq m'$, Fourier mixing $B_{lj} = \sum_m \omega^{lm} A_{mj}$ gives the decomposition.

theorem *fourier-split*:
assumes n: $0 < n$ **and** A: $\bigwedge m\ j.\ m < n \Longrightarrow j < d \Longrightarrow A\ m\ j \in \mathit{carrier\text{-}mat}\ d\ n$
and u: $\mathit{kraus\text{-}map\text{-}dim}\ d\ (\mathit{concat}\ (\mathit{map}\ (\lambda m.\ \mathit{map}\ (A\ m)\ [0..{<}d])\ [0..{<}n]))\ (1_m\ n) = 1_m\ d$

and *orth*: $\bigwedge m\ m'\ a\ b.\ m < n \Longrightarrow m' < n \Longrightarrow m \neq m' \Longrightarrow a < d \Longrightarrow b < d \Longrightarrow (\sum j{<}d.\ (A\ m\ j * ((A\ m'\ j)^\dagger))\ \$\$\ (a,\ b)) = 0$
shows *ucp-average-split n d* (*kraus-map-dim d* (*concat* (*map* ($\lambda m.$ *map* (*A m*) [*0*..<*d*]) [*0*..<*n*])))
⟨*proof*⟩

23.3.1 Positive matrices as sums of d rank-one terms

lemma *udu-entry*:
assumes *U*: $U \in$ *carrier-mat d d* **and** *D*: *length D* $= d$ **and** *a*: $a < d$ **and** *b*: $b < d$
shows $(U * \textit{diag-mat-of-real}\ D * (U^\dagger))\ \$\$\ (a,\ b) = (\sum k{<}d.\ U\ \$\$\ (a,\ k) * \textit{complex-of-real}\ (D\ !\ k) * \textit{cnj}\ (U\ \$\$\ (b,\ k)))$
⟨*proof*⟩

lemma *psd-eig*:
assumes *Q*: $Q \in$ *carrier-mat d d psd Q*
shows $\exists\, U\ D.\ U \in \textit{carrier-mat}\ d\ d \wedge \textit{length}\ D = d \wedge (\forall k{<}d.\ 0 \leq D\ !\ k) \wedge (U^\dagger) * U = 1_m\ d$
$\wedge\ (\forall a{<}d.\ \forall b{<}d.\ Q\ \$\$\ (a,\ b) = (\sum k{<}d.\ U\ \$\$\ (a,\ k) * \textit{complex-of-real}\ (D\ !\ k) * \textit{cnj}\ (U\ \$\$\ (b,\ k))))$
⟨*proof*⟩

lemma *psd-vecs*:
assumes *Q*: $Q \in$ *carrier-mat d d psd Q*
shows $\exists\, q.\ \forall a{<}d.\ \forall b{<}d.\ Q\ \$\$\ (a,\ b) = (\sum k{<}d.\ q\ k\ a * \textit{cnj}\ (q\ k\ b))$
⟨*proof*⟩

lemma *ucp-average-split-cong*: $(\bigwedge X.\ X \in \textit{carrier-mat}\ n\ n \Longrightarrow \Phi\ X = \Psi\ X) \Longrightarrow \textit{ucp-average-split}\ n\ d\ \Psi \Longrightarrow \textit{ucp-average-split}\ n\ d\ \Phi$
⟨*proof*⟩

lemma *delta-sum2*:
fixes *n i* :: *nat*
assumes *i*: $i < n$
shows $(\sum p{<}n.\ \sum p'{<}n.\ (\textit{if}\ p = i\ \textit{then}\ x\ \textit{else}\ 0) * f\ p\ p' * \textit{cnj}\ (\textit{if}\ p' = i\ \textit{then}\ y\ \textit{else}\ 0)) = x * f\ i\ i * \textit{cnj}\ (y :: \textit{complex})$
⟨*proof*⟩

23.3.2 Measurement maps

$\Phi(X) = \sum_i X_{ii} Q_i$ with $Q_i \geq 0$, $\sum_i Q_i = I$ (dual to a channel with classical output): D21-2 holds.

definition *povm-map* :: *nat* ⇒ *nat* ⇒ (*nat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
povm-map n d Q X = *Matrix.mat d d* ($\lambda(a,\ b).\ \sum i{<}n.\ X\ \$\$\ (i,\ i) * Q\ i\ \$\$\ (a,\ b)$)

theorem *ra-measurement*:
assumes *n*: $0 < n$ **and** *Q*: $\bigwedge i.\ i < n \Longrightarrow Q\ i \in \textit{carrier-mat}\ d\ d \wedge \textit{psd}\ (Q\ i)$
and *QI*: $\bigwedge a\ b.\ a < d \Longrightarrow b < d \Longrightarrow (\sum i{<}n.\ Q\ i\ \$\$\ (a,\ b)) = 1_m\ d\ \$\$\ (a,\ b)$
shows *ucp-average-split n d* (*povm-map n d Q*)
⟨*proof*⟩

23.3.3 The case $d = 1$: Horn's lemma

Every density matrix on $\mathbb{C}^n$ is the average of n pure states (Fourier mixing of an eigenbasis).

theorem *horn-lemma*:
assumes *ϱ*: *density-matrix n ϱ* **and** *n*: $0 < n$
shows $\exists \varphi.\ (\forall l{<}n.\ \textit{dim-vec}\ (\varphi\ l) = n \wedge \|\varphi\ l\| = 1)$
$\wedge\ (\forall a{<}n.\ \forall b{<}n.\ \varrho\ \$\$\ (a,\ b) = (1\ /\ \textit{of-nat}\ n) * (\sum l{<}n.\ \varphi\ l\ \$\ a * \textit{cnj}\ (\varphi\ l\ \$\ b)))$
⟨*proof*⟩

lemma *delta-pair*:
fixes c c' d :: *nat*
assumes $c < d$
shows $(\sum a{<}d.\ \textit{if } c = a \wedge c' = a \textit{ then } f\ a \textit{ else } 0) = (\textit{if } c = c' \textit{ then } f\ c \textit{ else } (0{::}\textit{complex}))$
⟨*proof*⟩

23.3.4 Maps with diagonal values

$\Phi(X) = \mathrm{diag}(\mathrm{tr}(R_1 X), \ldots, \mathrm{tr}(R_d X))$ with density matrices R_a (dual to a channel with classical input): D21-2 holds, by Horn's lemma for each R_a.

definition *cq-map* :: *nat* ⇒ *nat* ⇒ (*nat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
cq-map n d Rh X = *Matrix.mat d d* (λ(*a*, *b*). *if a* = *b then* $(\sum p{<}n.\ \sum q{<}n.$ *Rh a* $\$\$$ (*q*, *p*) ∗ *X* $\$\$$ (*p*, *q*)) *else 0*)

theorem *ra-cq*:
assumes *n*: *0* < *n* **and** *Rh*: $\bigwedge a.\ a < d \Longrightarrow$ *density-matrix n* (*Rh a*)
shows *ucp-average-split n d* (*cq-map n d Rh*)
⟨*proof*⟩

23.3.5 The notes' form (i) is false as written

Form (i) asks for n trace-preserving maps $M_n \to M_d$ of Choi rank at most d. For $n = 2$, $d = 1$ there is no trace-preserving map $M_2 \to \mathbb{C}$ of Choi rank at most one, while the trace is trace preserving. (The equivalent dual form has the dimensions exchanged: maps $M_d \to M_n$.)

lemma *no-tp-rank-one*: *length Ws* ≤ *1* ⟹ ¬ *kraus-operators 2 1 Ws*
⟨*proof*⟩

theorem *notes-form-i-false*:
¬ (∀ *Ws*. *kraus-operators 2 1 Ws* ⟶ (∃ *Bs*. *length Bs* = *2* ∧ (∀ *l*<*2*. *length* (*Bs* ! *l*) ≤ *1* ∧ *kraus-operators 2 1* (*Bs* ! *l*))
∧ (∀ *X* ∈ *carrier-mat 2 2*. *kraus-map-dim 1 Ws X* = (*1* / *2*) ·$_m$ *kraus-map-dim 1* (*concat Bs*) *X*)))
⟨*proof*⟩

23.4 The conjecture D21-2 for $n = 2$

Every unital completely positive $\Phi : M_2 \to M_d$ is the average of two unital maps of Choi rank at most d (*ra-two*; Bacon, Smolin and Wolf). The off-diagonal block of the Choi matrix is $R_1 K R_2^*$ with $R_i R_i^*$ the diagonal blocks and K a contraction (*wide-factor*, *bessel-rows*), and a contraction is the average of two unitaries (*contraction-avg*, through a singular value decomposition, *svd-square*).

23.4.1 Completing an orthonormal family to an orthonormal basis

lemma *onb-complete*:
fixes *d* :: *nat* **and** *J* :: *nat set* **and** *w* :: *nat* ⇒ *nat* ⇒ *complex*
assumes *J*: *J* ⊆ {..<*d*}
and *on*: $\bigwedge k\ k'.\ k \in J \Longrightarrow k' \in J \Longrightarrow (\sum a{<}d.\ \textit{cnj}\ (w\ k\ a) * w\ k'\ a) = (\textit{if } k = k' \textit{ then } 1 \textit{ else } 0)$
shows ∃ *W*. *W* ∈ *carrier-mat d d* ∧ (*W*†) ∗ *W* = *1*$_m$ *d* ∧ (∀ *k*∈*J*. ∀ *a*<*d*. *W* $\$\$$ (*a*, *k*) = *w k a*)
⟨*proof*⟩

23.4.2 Singular value decomposition

lemma *unitary-sq-right*: *V* ∈ *carrier-mat d d* ⟹ (*V*†) ∗ *V* = *1*$_m$ *d* ⟹ *V* ∗ (*V*†) = *1*$_m$ *d*
⟨*proof*⟩

lemma *svd-square*:
assumes *Km*: *Km* ∈ *carrier-mat d d*
shows ∃ *W V*. *W* ∈ *carrier-mat d d* ∧ (*W*†) ∗ *W* = *1*$_m$ *d* ∧ *V* ∈ *carrier-mat d d* ∧ (*V*†) ∗ *V* = *1*$_m$ *d*
∧ (∀ *a*<*d*. ∀ *b*<*d*. *Km* $$ (*a*, *b*) = (∑ *k*<*d*. *W* $$ (*a*, *k*) ∗ *complex-of-real* (*sqrt* (∑ *c*<*d*. (*cmod* ((*Km* ∗ *V*) $$ (*c*, *k*)))^2)) ∗ *cnj* (*V* $$ (*b*, *k*))))
⟨*proof*⟩

23.4.3 A contraction is the average of two unitaries

definition *phase-diag* :: *nat* ⇒ (*nat* ⇒ *real*) ⇒ *complex Matrix.mat* **where**
phase-diag d ϑ = *Matrix.mat d d* (λ(*i*, *j*). *if i* = *j then cis* (*ϑ i*) *else 0*)

lemma *phase-diag-carrier* [*simp*]: *phase-diag d ϑ* ∈ *carrier-mat d d* ⟨*proof*⟩

A contraction is the average of two unitaries. Write $K = W\Sigma V^*$ with W, V unitary and $\Sigma = \mathrm{diag}(\sigma_i)$, $0 \le \sigma_i \le 1$, and $\sigma_i = \cos\theta_i = \frac{1}{2}(e^{i\theta_i} + e^{-i\theta_i})$. Then

$$K = \tfrac{1}{2}(WD_\theta V^* + WD_{-\theta}V^*), \qquad D_\theta = \mathrm{diag}(e^{i\theta_1}, \ldots, e^{i\theta_d}),$$

and both $WD_{\pm\theta}V^*$ are unitary.

lemma *phase-diag-unitary*: (*phase-diag d ϑ*)† ∗ *phase-diag d ϑ* = *1*$_m$ *d*
⟨*proof*⟩

lemma *wdv-entry*:
assumes *W*: *W* ∈ *carrier-mat d d* **and** *V*: *V* ∈ *carrier-mat d d* **and** *a*: *a* < *d* **and** *b*: *b* < *d*
shows (*W* ∗ *phase-diag d ϑ* ∗ (*V*†)) $$ (*a*, *b*) = (∑ *k*<*d*. *W* $$ (*a*, *k*) ∗ *cis* (*ϑ k*) ∗ *cnj* (*V* $$ (*b*, *k*)))
⟨*proof*⟩

lemma *unitary-triple*:
assumes *W*: *W* ∈ *carrier-mat d d* (*W*†) ∗ *W* = *1*$_m$ *d* **and** *V*: *V* ∈ *carrier-mat d d* (*V*†) ∗ *V* = *1*$_m$ *d*
and *D*: *D* ∈ *carrier-mat d d* (*D*†) ∗ *D* = *1*$_m$ *d*
shows (*W* ∗ *D* ∗ (*V*†))† ∗ (*W* ∗ *D* ∗ (*V*†)) = *1*$_m$ *d*
⟨*proof*⟩

theorem *contraction-avg*:
assumes *Km*: *Km* ∈ *carrier-mat d d* **and** *ct*: ⋀*x*. *dim-vec x* = *d* ⟹ ‖*Km* ∗$_v$ *x*‖ ≤ ‖*x*‖
shows ∃ *U1 U2*. *U1* ∈ *carrier-mat d d* ∧ *U2* ∈ *carrier-mat d d* ∧ (*U1*†) ∗ *U1* = *1*$_m$ *d* ∧ (*U2*†) ∗ *U2* = *1*$_m$ *d*
∧ (∀ *a*<*d*. ∀ *b*<*d*. *Km* $$ (*a*, *b*) = (*U1* $$ (*a*, *b*) + *U2* $$ (*a*, *b*)) / *2*)
⟨*proof*⟩

23.4.4 Bessel's inequality for rows that are orthonormal or zero

lemma *cmod-sq-sum*: *complex-of-real* (∑ *k*∈*A*. (*cmod* (*f k*))^2) = (∑ *k*∈*A*. *f k* ∗ *cnj* (*f k*))
⟨*proof*⟩

lemma *bessel-rows*:
fixes *s* :: *nat* ⇒ *nat* ⇒ *complex* **and** *y* :: *nat* ⇒ *complex* **and** *d K* :: *nat*
assumes *on*: ⋀*j j'*. *j* < *d* ⟹ *j'* < *d* ⟹ (∑ *k*<*K*. *s j k* ∗ *cnj* (*s j' k*)) = (*if j* = *j'* ∧ *j* ∈ *J then 1 else 0*)
shows (∑ *j*<*d*. (*cmod* (∑ *k*<*K*. *s j k* ∗ *y k*))^2) ≤ (∑ *k*<*K*. (*cmod* (*y k*))^2)
⟨*proof*⟩

23.4.5 Factoring a wide matrix

$M = RS$ with $RR^* = MM^*$ and the rows of S orthonormal or zero.

lemma *wide-factor*:
fixes $m :: nat \Rightarrow nat \Rightarrow complex$ **and** $d\ Kn :: nat$
shows $\exists R\ S\ J.\ R \in carrier\text{-}mat\ d\ d$
$\wedge\ (\forall k{<}Kn.\ \forall a{<}d.\ m\ k\ a = (\sum j{<}d.\ R\ \$\$\ (a,\ j) * S\ j\ k))$
$\wedge\ (\forall a{<}d.\ \forall b{<}d.\ (\sum j{<}d.\ R\ \$\$\ (a,\ j) * cnj\ (R\ \$\$\ (b,\ j))) = (\sum k{<}Kn.\ m\ k\ a * cnj\ (m\ k\ b)))$
$\wedge\ (\forall j{<}d.\ \forall j'{<}d.\ (\sum k{<}Kn.\ S\ j\ k * cnj\ (S\ j'\ k)) = (if\ j = j' \wedge j \in J\ then\ 1\ else\ 0))$
⟨*proof*⟩

23.4.6 The case $n = 2$

definition *kraus-gram* :: *complex Matrix.mat list* ⇒ *nat* ⇒ *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex* **where**
kraus-gram Fs p q i i' = $(\sum k{<}length\ Fs.\ Fs\ !\ k\ \$\$\ (i,\ p) * cnj\ (Fs\ !\ k\ \$\$\ (i',\ q)))$

lemma *kraus2-entry*:
assumes *F*: $\forall A \in set\ Fs.\ A \in carrier\text{-}mat\ d\ 2$ **and** *X*: $X \in carrier\text{-}mat\ 2\ 2$ **and** *i*: $i < d$ **and** *i'*: $i' < d$
shows *kraus-map-dim d Fs X* \$\$ $(i,\ i') = (\sum p{<}2.\ \sum q{<}2.\ X\ \$\$\ (p,\ q) * kraus\text{-}gram\ Fs\ p\ q\ i\ i')$
⟨*proof*⟩

lemma *kraus-gram-map*:
kraus-gram $(map\ f\ [0..{<}d])\ p\ q\ i\ i' = (\sum j{<}d.\ f\ j\ \$\$\ (i,\ p) * cnj\ (f\ j\ \$\$\ (i',\ q)))$
⟨*proof*⟩

lemma *mult-entry3*:
assumes *A*: $A \in carrier\text{-}mat\ d\ d$ **and** *B*: $B \in carrier\text{-}mat\ d\ d$ **and** *C*: $C \in carrier\text{-}mat\ d\ d$ **and** *i*: $i < d$ **and** *i'*: $i' < d$
shows $(A * B * (C^\dagger))\ \$\$\ (i,\ i') = (\sum j{<}d.\ \sum m{<}d.\ A\ \$\$\ (i,\ j) * B\ \$\$\ (j,\ m) * cnj\ (C\ \$\$\ (i',\ m)))$
⟨*proof*⟩

Every unital $\Phi : M_2 \to M_d$ is the average of two unital maps of Choi rank at most d (Bacon–Smolin–Wolf): the off-diagonal Choi block is $R_1 K R_2^*$ with K a contraction, and $K = \frac{1}{2}(U_1 + U_2)$.

theorem *ra-two*:
assumes *As*: $\forall A \in set\ As.\ A \in carrier\text{-}mat\ d\ 2$ **and** *u*: *kraus-map-dim d As* $(1_m\ 2) = 1_m\ d$
shows *ucp-average-split 2 d* (*kraus-map-dim d As*)
⟨*proof*⟩

24 Day 22 (17 October 2011): minimal Kraus representations, convex sets, UCP and CPTP

Proposition (notes). If Φ is completely positive, $\mathrm{cr}(\Phi) = r$ and $\Phi(X) = \sum_{i=1}^{r} V_i^* X V_i$, then $V_1, \dots, V_r$ are linearly independent.

Definition (notes). $C \subseteq V$ is *convex* if $tv_1 + (1-t)v_2 \in C$ for $v_1, v_2 \in C$, $0 \le t \le 1$; $v \in C$ is *extreme* if $v = \frac{1}{2}(v_1 + v_2)$ with $v_1, v_2 \in C$ forces $v = v_1 = v_2$. The *convex hull* of E is the set of convex combinations of points of E; a map between convex sets is *affine* if it preserves convex combinations.

Theorem (notes) (Krein–Milman)**.** A compact convex subset of a finite-dimensional real vector space is the convex hull of its extreme points.

Examples: $UCP(M_n, M_d)$, the unital completely positive maps, is convex ($P_{t\Phi_1 + (1-t)\Phi_2} = tP_{\Phi_1} + (1-t)P_{\Phi_2} \ge 0$) and compact (closed; bounded because $0 \le \Phi(E_{ii}) \le I$ gives $|r_{ij}| \le 1$ for the entries of P_Φ); $CPTP(M_d, M_n)$ is convex and compact, being the image of $UCP(M_n, M_d)$ under the affine bijection $\Gamma(\Phi) = \Phi^*$.

Theorem (notes) (Choi 3)**.** If $\Phi : M_n \to M_d$ is unital and completely positive, $\Phi(X) = \sum_{i=1}^{r} V_i^* X V_i$ with $r = \mathrm{cr}(\Phi)$, then Φ is an extreme point of $UCP(M_n, M_d)$ iff $\{V_i^* V_j : 1 \le i, j \le r\}$ is linearly independent in M_d.

Corollary (notes). If Φ is extreme then $\mathrm{cr}(\Phi) \le d$. If $\Phi : M_d \to M_n$ is CPTP with $\Phi(X) = \sum_{i=1}^{r} V_i X V_i^*$, $r = \mathrm{cr}(\Phi)$, then Φ is extreme in $CPTP(M_d, M_n)$ iff $\{V_i^* V_j\}$ is linearly independent.

Below: convexity of UCP and CPTP, the proposition (*minimal-kraus-rep-lin-indep*), with it Choi's rank theorem of Day 20, and the affine bijection Γ; compactness is the next theory. Choi's Theorem 3 and its corollary $\mathrm{cr}(\Phi) \le d$ (*extreme-CPTP-choi-rank-le*) are proved on Day 23, in the CPTP form. Krein–Milman is available in HOL-Analysis (*Krein-Milman-Minkowski*).

24.1 Convex sets and extreme points

definition *convex-set-mat* :: *complex Matrix.mat set* ⇒ *bool* **where**
convex-set-mat C ≡
∀ *A* ∈ *C*. ∀ *B* ∈ *C*. ∀ *t* :: *real*. *0* ≤ *t* ∧ *t* ≤ *1* ⟶
complex-of-real t ·$_m$ *A*
+ (*1* − *complex-of-real t*) ·$_m$ *B* ∈ *C*

definition *extreme-point-mat* ::
complex Matrix.mat set ⇒ *complex Matrix.mat* ⇒ *bool* **where**
extreme-point-mat C v ≡
v ∈ *C* ∧
¬ (∃ *v1* ∈ *C*. ∃ *v2* ∈ *C*. *v1* ≠ *v* ∧ *v* ≠ *v2* ∧
v = (*1*/*2* :: *complex*) ·$_m$ *v1* + (*1*/*2* :: *complex*) ·$_m$ *v2*)

definition *affine-map-mat* ::
(*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
affine-map-mat tmap ≡
∀ *A B*. ∀ *t* :: *real*. *0* ≤ *t* ∧ *t* ≤ *1* ⟶
tmap (*complex-of-real t* ·$_m$ *A*
+ (*1* − *complex-of-real t*) ·$_m$ *B*) =
complex-of-real t ·$_m$ *tmap A*
+ (*1* − *complex-of-real t*) ·$_m$ *tmap B*

24.2 The convex sets UCP and CPTP

definition *UCP* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) *set* **where**
UCP n d ≡ {Φ. *completely-positive n d* Φ ∧ *unital-map n d* Φ}

definition *CPTP* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) *set* **where**
CPTP n d ≡ {Φ. *completely-positive n d* Φ ∧ *trace-preserving n* Φ}

lemma *amplification-smult*:
assumes *dpos*: *0* < *d*
and *F-block*: ⋀*a b*. *F* (*block-of-mat n x a b*) ∈ *carrier-mat d d*
shows *amplification n d p* (λϱ. *c* ·$_m$ *F* ϱ) *x*
= *c* ·$_m$ *amplification n d p F x*
⟨*proof*⟩

UCP is convex. The range hypotheses are explicit because *completely-positive* does not constrain the values of Φ_i off the carrier.

lemma *UCP-convex*:
fixes t :: *real*
assumes *t-nn*: $0 \leq t$
and *t-le*: $t \leq 1$
and *Φ1-in*: $\Phi 1 \in UCP\ n\ d$
and *Φ2-in*: $\Phi 2 \in UCP\ n\ d$
and *r1*: $\forall X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow \Phi 1\ X \in carrier\text{-}mat\ d\ d$
and *r2*: $\forall X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow \Phi 2\ X \in carrier\text{-}mat\ d\ d$
and *dpos*: $0 < d$
shows $(\lambda X.\ complex\text{-}of\text{-}real\ t \cdot_m \Phi 1\ X + (1 - complex\text{-}of\text{-}real\ t) \cdot_m \Phi 2\ X) \in UCP\ n\ d$
⟨*proof*⟩

CPTP is convex.

lemma *CPTP-convex*:
fixes t :: *real*
assumes *t-nn*: $0 \leq t$
and *t-le*: $t \leq 1$
and *Φ1-in*: $\Phi 1 \in CPTP\ n\ d$
and *Φ2-in*: $\Phi 2 \in CPTP\ n\ d$
and *r1*: $\forall X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow \Phi 1\ X \in carrier\text{-}mat\ d\ d$
and *r2*: $\forall X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow \Phi 2\ X \in carrier\text{-}mat\ d\ d$
and *dpos*: $0 < d$
shows $(\lambda X.\ complex\text{-}of\text{-}real\ t \cdot_m \Phi 1\ X + (1 - complex\text{-}of\text{-}real\ t) \cdot_m \Phi 2\ X) \in CPTP\ n\ d$
⟨*proof*⟩

lemma *kraus-is-cptp*:
fixes *Ws* :: *complex Matrix.mat list*
assumes *Ws*: *kraus-operators n d Ws*
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *kraus-map-dim d Ws* $\in$ *CPTP n d*
⟨*proof*⟩

24.3 A minimal Kraus representation is linearly independent (D22-1)

24.3.1 Mixing a Kraus list by an isometry does not change the map

definition *kraus-mix* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat list* **where**
kraus-mix d n U Vs =
map (λ*l*. *foldr* (λ*j acc*. *U* $$ (*l*,*j*) ·$_m$ (*Vs* ! *j*) + *acc*) [*0*..<*length Vs*] (0_m *d n*)) [*0*..<*dim-row U*]

lemma *kraus-mix-length* [*simp*]: *length* (*kraus-mix d n U Vs*) = *dim-row U*
⟨*proof*⟩

lemma *kraus-mix-nth*:
assumes $l <$ *dim-row U*
shows *kraus-mix d n U Vs* ! *l* = *foldr* (λ*j acc*. *U* $$ (*l*,*j*) ·$_m$ (*Vs* ! *j*) + *acc*) [*0*..<*length Vs*] (0_m *d n*)
⟨*proof*⟩

lemma *kraus-mix-carrier*:
assumes $\forall$ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
shows $\forall$ *W* ∈ *set* (*kraus-mix d n U Vs*). *W* ∈ *carrier-mat d n*
⟨*proof*⟩

Mixing a Kraus list by a matrix $U = (u_{lj})$ gives $W_l = \sum_j u_{lj} V_j$. When $U^*U = I$ the two lists

define the same map:

$$\sum_l W_l X W_l^* = \sum_{j,j'} \Big(\sum_l \overline{u_{lj'}}\, u_{lj} \Big) V_j X V_{j'}^* = \sum_j V_j X V_j^*.$$

lemma *kraus-mix-entry*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n* **and** *l*: *l* < *dim-row U* **and** *i*: *i* < *d* **and** *a*: *a* < *n*
shows *kraus-mix d n U Vs* ! *l* $$ (*i*,*a*) = (∑ *j* = *0*..<*length Vs*. *U* $$ (*l*,*j*) ∗ *Vs* ! *j* $$ (*i*,*a*))
⟨*proof*⟩

lemma *kraus-mix-mat-of-cols*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n* **and** *Uc*: *U* ∈ *carrier-mat m* (*length Vs*) **and** *dpos*: *0* < *d*
shows *mat-of-cols* (*n*∗*d*) (*map* (*vec-of-kraus d n*) (*kraus-mix d n U Vs*))
= *mat-of-cols* (*n*∗*d*) (*map* (*vec-of-kraus d n*) *Vs*) ∗ *transpose-mat U*
⟨*proof*⟩

lemma *kraus-mix-same-map*:
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *Uc*: *U* ∈ *carrier-mat m* (*length Vs*) **and** *UU*: (*U*†) ∗ *U* = *1*$_m$ (*length Vs*)
and *Xc*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim d* (*kraus-mix d n U Vs*) *M* = *kraus-map-dim d Vs M*
⟨*proof*⟩

24.3.2 A unitary with a prescribed first row

lemma *transpose-unitary-isometry*:
assumes *Mc*: *M* ∈ *carrier-mat r r* **and** *MM*: *M* ∗ (*M*†) = *1*$_m$ *r*
shows ((*transpose-mat M*)†) ∗ *transpose-mat M* = *1*$_m$ *r*
⟨*proof*⟩

lemma *isometry-with-first-row*:
assumes *ud*: *dim-vec u* = *r* **and** *un*: ‖*u*‖ = *1*
shows ∃ *U* :: *complex Matrix.mat*. *U* ∈ *carrier-mat r r* ∧ (*U*†) ∗ *U* = *1*$_m$ *r* ∧ (∀ *j* < *r*. *U* $$ (*0*,*j*) = *u* $ *j*)
⟨*proof*⟩

24.3.3 D22-1

lemma *kraus-map-dim-zero-head*:
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n* **and** *Xc*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim d* (*0*$_m$ *d n* # *Ws*) *M* = *kraus-map-dim d Ws M*
⟨*proof*⟩

If $\Phi(X) = \sum_{j<r} V_j X V_j^*$ with $r = \mathrm{cr}(\Phi)$, then $V_0, \ldots, V_{r-1}$ are linearly independent: a dependency $\sum_j c_j V_j = 0$ would allow a unitary change of representation whose first Kraus operator is 0, giving a representation with $r - 1$ terms.

theorem *minimal-kraus-rep-lin-indep*:
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *minimal*: *choi-rank n d* (*kraus-map-dim d Vs*) = *length Vs*
shows *lin-indep-complex-mat d n Vs*
⟨*proof*⟩

24.4 Choi's rank theorem $\mathrm{cr}(\Phi) = \mathrm{rank}\, P_\Phi$ (D20-4)

A linearly independent list of vectors in $\mathbb{C}^n$ has at most n members. The bridge is to JNF's *vec-space*: kernel-triviality of *mat-of-cols* is JNF's *lin-indpt* (via *vec-space.lin-depE*), and *li-le-dim*

bounds the cardinality.

lemma *lin-indep-vecs-distinct*:
  assumes *li*: *lin-indep-vecs n vs*
  shows *distinct vs*
⟨*proof*⟩

lemma *lin-indep-vecs-length-le*:
  assumes *li*: *lin-indep-vecs n vs*
  shows *length vs* ≤ *n*
⟨*proof*⟩

Flattening a matrix into a vector carries linear independence of matrices to *lin-indep-vecs*, so a linearly independent family in M_n has at most n^2 members.

lemma *lin-indep-mats-vecs*:
  assumes *li*: *lin-indep-complex-mat n n Bs*
  shows *lin-indep-vecs* (*n*∗*n*) (*map mat-to-vec Bs*)
⟨*proof*⟩

lemma *lin-indep-mats-length-le*:
  assumes *lin-indep-complex-mat n n Bs*
  shows *length Bs* ≤ *n*∗*n*
  ⟨*proof*⟩

Rank is JNF's *vec-space.rank*, the dimension of the column space.

abbreviation *mat-rank* :: *nat* ⇒ *complex Matrix.mat* ⇒ *nat* **where**
  mat-rank m A ≡ *vec-space.rank m A*

24.4.1 $\operatorname{rank}(\sum_{l<L} w_l w_l^*) \le L$

lemma *rank-outer-prod-le-1*:
  assumes *vd*: *dim-vec v* = *m*
  shows *mat-rank m* (*v* ⊘ *v*) ≤ *1*
⟨*proof*⟩

lemma *rank-foldr-outer-le*:
  assumes *vsd*: ∀ *v* ∈ *set vs*. *dim-vec v* = *m*
  shows *mat-rank m* (*foldr* (λ*v acc*. *v* ⊘ *v* + *acc*) *vs* (*0*$_m$ *m m*)) ≤ *length vs*
  ⟨*proof*⟩

lemma *rank-outer-sum-le*:
  assumes *vsd*: ∀ *v* ∈ *set vs*. *dim-vec v* = *m*
  shows *mat-rank m* (*outer-sum m vs*) ≤ *length vs*
  ⟨*proof*⟩

24.4.2 Independent vectors inside the column space bound the rank from below

lemma *mult-mat-vec-in-span*:
  assumes *Ac*: *A* ∈ *carrier-mat m k* **and** *xc*: *x* ∈ *carrier-vec k*
  shows *A* ∗$_v$ *x* ∈ *module.span class-ring* (*module-vec TYPE*(*complex*) *m*) (*set* (*cols A*))
⟨*proof*⟩

lemma *rank-ge-indep-in-colspace*:
  assumes *Ac*: *A* ∈ *carrier-mat m k* **and** *li*: *lin-indep-vecs m ws*
    and *insp*: ⋀*w*. *w* ∈ *set ws* ⟹ *w* ∈ *module.span class-ring* (*module-vec TYPE*(*complex*) *m*) (*set* (*cols A*))
  shows *length ws* ≤ *mat-rank m A*
⟨*proof*⟩

24.4.3 The theorem

lemma *choi-rank-attained*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *linear*: ⋀*A B*. *A* ∈ *carrier-mat n n* ⟹ *B* ∈ *carrier-mat n n* ⟹ Φ (*A* + *B*) = Φ *A* + Φ *B*
and *smult*: ⋀*c A*. *A* ∈ *carrier-mat n n* ⟹ Φ (*c* ·$_m$ *A*) = *c* ·$_m$ Φ *A*
and *range*: ∀ *A*. *A* ∈ *carrier-mat n n* ⟶ Φ *A* ∈ *carrier-mat d d*
and *cp*: *completely-positive n d* Φ
and *npos*: *0* < *n* **and** *dpos*: *0* < *d*
shows ∃ *As* :: *complex Matrix.mat list*. *length As* = *choi-rank n d* Φ ∧
(∀ *A* ∈ *set As*. *A* ∈ *carrier-mat d n*) ∧
(∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ *X* = *kraus-map-dim d As X*)
⟨*proof*⟩

theorem *choi-rank-eq-rank*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *linear*: ⋀*A B*. *A* ∈ *carrier-mat n n* ⟹ *B* ∈ *carrier-mat n n* ⟹ Φ (*A* + *B*) = Φ *A* + Φ *B*
and *smult*: ⋀*c A*. *A* ∈ *carrier-mat n n* ⟹ Φ (*c* ·$_m$ *A*) = *c* ·$_m$ Φ *A*
and *range*: ∀ *A*. *A* ∈ *carrier-mat n n* ⟶ Φ *A* ∈ *carrier-mat d d*
and *cp*: *completely-positive n d* Φ
and *npos*: *0* < *n* **and** *dpos*: *0* < *d*
shows *choi-rank n d* Φ = *mat-rank* (*n*∗*d*) (*choi-matrix n d* Φ)
⟨*proof*⟩

24.5 The adjoint map, an affine bijection from UCP onto CPTP (D22-5)

The notes: UCP(M_n, M_d) and CPTP(M_d, M_n) are convex, and $\Gamma(\Phi) = \Phi^*$ is an affine bijection between them with affine inverse, so the two convex sets are affinely isomorphic. The adjoint was available only for Kraus maps (*kraus-adjoint-dim*); here it is defined for every linear Φ. Maps are total functions on matrices, so Γ is a bijection up to agreement on M_n, resp. M_d: Γ sends linear UCP maps to linear CPTP maps and back, and $\Gamma(\Gamma(\Phi)) = \Phi$ on M_n. The set *UCP* does not record linearity, which the notes' maps have, so it is assumed.

For linear $\Phi : M_n \to M_d$, $\Phi^* : M_d \to M_n$ is determined by $\langle Y|\Phi(X)\rangle = \langle \Phi^*(Y)|X\rangle$; taking $X = E_{ij}$ gives $\Phi^*(Y)_{ij} = \langle \Phi(E_{ij})|Y\rangle$.

definition *map-adjoint* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
map-adjoint n d Φ *Y* = *Matrix.mat n n* (λ(*i*, *j*). ⟨⟨ Φ (*E*⟦*i*,*j* ; *n*,*n*⟧) | *Y* ⟩⟩)

$\Phi^* : M_d \to M_n$ is determined by $\langle \Phi^*(Y)|X\rangle = \langle Y|\Phi(X)\rangle$. Since $\Phi(X) = \sum_{i,j} x_{ij}\Phi(E_{ij})$ for linear Φ, it suffices that

$$\Phi^*(Y)_{ij} = \langle \Phi(E_{ij})|Y\rangle,$$

which is the definition.

lemma *map-adjoint-carrier* [*simp*]: *map-adjoint n d* Φ *Y* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *map-adjoint-dims* [*simp*]: *dim-row* (*map-adjoint n d* Φ *Y*) = *n dim-col* (*map-adjoint n d* Φ *Y*) = *n*
⟨*proof*⟩

lemma *lin-entry*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *X*: *X* ∈ *carrier-mat n n* **and** *a*: *a* < *d* **and** *b*: *b* < *d*
shows Φ *X* $$ (*a*, *b*) = (∑*i*<*n*. ∑*j*<*n*. *X* $$ (*i*, *j*) ∗ Φ (*E*⟦*i*,*j* ; *n*,*n*⟧) $$ (*a*, *b*))
⟨*proof*⟩

theorem *map-adjoint-correct*:

assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *X*: *X* ∈ *carrier-mat n n* **and** *Y*: *Y* ∈ *carrier-mat d d*
shows ⟨⟨ *Y* | Φ *X* ⟩⟩ = ⟨⟨ *map-adjoint n d* Φ *Y* | *X* ⟩⟩
⟨*proof*⟩

lemma *hs-inner-prod-mat-unit*:
assumes *A*: *A* ∈ *carrier-mat n n* **and** *i*: *i* < *n* **and** *j*: *j* < *n*
shows ⟨⟨ *A* | *E*⟦*i*,*j* ; *n*,*n*⟧ ⟩⟩ = *cnj* (*A* $$ (*i*, *j*))
⟨*proof*⟩

lemma *hs-inner-prod-ext*:
assumes *A*: *A* ∈ *carrier-mat n n* **and** *B*: *B* ∈ *carrier-mat n n*
and *eq*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ ⟨⟨ *A* | *X* ⟩⟩ = ⟨⟨ *B* | *X* ⟩⟩
shows *A* = *B*
⟨*proof*⟩

lemma *map-adjoint-lin*: *lin-on d n* (*carrier-mat d d*) (*map-adjoint n d* Φ)
⟨*proof*⟩

lemma *map-adjoint-cong*:
assumes ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ Φ *X* = Ψ *X*
shows *map-adjoint n d* Φ = *map-adjoint n d* Ψ
⟨*proof*⟩

For a Kraus map the adjoint is *kraus-adjoint-dim*.

lemma *map-adjoint-kraus*:
assumes *As*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat d n* **and** *Y*: *Y* ∈ *carrier-mat d d*
shows *map-adjoint n d* (*kraus-map-dim d As*) *Y* = *kraus-adjoint-dim n As Y*
⟨*proof*⟩

$\Phi^{**} = \Phi$.

theorem *map-adjoint-adjoint*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *X*: *X* ∈ *carrier-mat n n*
shows *map-adjoint d n* (*map-adjoint n d* Φ) *X* = Φ *X*
⟨*proof*⟩

lemma *cp-kraus-form*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *cp*: *completely-positive n d* Φ **and** *n*: *0* < *n* **and** *d*: *0* < *d*
shows ∃ *Ws*. (∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*) ∧ (∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ *X* = *kraus-map-dim d Ws X*)
⟨*proof*⟩

lemma *trace-preserving-cong*:
assumes ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ Φ *X* = Ψ *X*
shows *trace-preserving n* Φ ⟷ *trace-preserving n* Ψ
⟨*proof*⟩

$\Gamma(\Phi) = \Phi^*$ maps the unital CP maps $M_n \to M_d$ to the CPTP maps $M_d \to M_n$ and back.

theorem *adjoint-ucp-imp-cptp*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *U*: Φ ∈ *UCP n d* **and** *n*: *0* < *n* **and** *d*: *0* < *d*
shows *map-adjoint n d* Φ ∈ *CPTP d n*
⟨*proof*⟩

theorem *adjoint-cptp-imp-ucp*:
assumes *L*: *lin-on d n* (*carrier-mat d d*) Ψ **and** *C*: Ψ ∈ *CPTP d n* **and** *n*: *0* < *n* **and** *d*: *0* < *d*
shows *map-adjoint d n* Ψ ∈ *UCP n d*
⟨*proof*⟩

Γ is affine (for real weights), and $\Gamma^{-1} = \Gamma$ by *map-adjoint-adjoint*.

lemma *hs-inner-prod-left-real-comb*:
fixes *t s* :: *real*
assumes *A*: $A \in$ *carrier-mat d d* **and** *B*: $B \in$ *carrier-mat d d* **and** *Y*: $Y \in$ *carrier-mat d d*
shows $\langle\langle$ *complex-of-real t* $\cdot_m$ *A* + *complex-of-real s* $\cdot_m$ *B* | *Y* $\rangle\rangle$
= *complex-of-real t* $*$ $\langle\langle$ *A* | *Y* $\rangle\rangle$ + *complex-of-real s* $*$ $\langle\langle$ *B* | *Y* $\rangle\rangle$
⟨*proof*⟩

theorem *map-adjoint-affine*:
fixes *t* :: *real*
assumes *L1*: *lin-on n d* (*carrier-mat n n*) *Φ1* **and** *L2*: *lin-on n d* (*carrier-mat n n*) *Φ2* **and** *Y*: $Y \in$ *carrier-mat d d*
shows *map-adjoint n d* (λ*X*. *complex-of-real t* $\cdot_m$ *Φ1 X* + *complex-of-real* (1 − *t*) $\cdot_m$ *Φ2 X*) *Y*
= *complex-of-real t* $\cdot_m$ *map-adjoint n d Φ1 Y* + *complex-of-real* (1 − *t*) $\cdot_m$ *map-adjoint n d Φ2 Y*
⟨*proof*⟩

24.6 UCP(M_n, M_d) is compact (D22-4)

The notes: UCP(M_n, M_d) is convex and compact, closed because the conditions are closed and bounded because $\Phi(I) = I$ forces $0 \le \Phi(E_{ii}) \le I$. The space $L(M_n, M_d)$ carries no topology in this development; the Choi matrix is a linear bijection onto M_{nd}, so the statement is made in Choi coordinates: the Choi matrices of linear UCP maps are exactly the positive P whose diagonal blocks sum to I_d (*ucp-choi-image*, with the inverse Choi map *choi-inv*), and that set is compact (*compact-ucp-choi-set*): it is closed, and it lies in d times the compact set of unit-trace positive matrices.

24.6.1 Typed PSD predicate

definition *psd-lmat* :: (*complex*, *'k* :: *finite*, *'k*) *lmat* ⇒ *bool* **where**
psd-lmat M ⟷ *psd* (*unwrap-lm M*)

lemma *psd-lmat-unwrap* [*iff*]:
psd-lmat (*M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*) ⟷ *psd* (*unwrap-lm M*)
⟨*proof*⟩

24.6.2 The positive matrices form a closed set

Sub-lemmas for the closedness of PSD: the Hermitian half and the nonneg-quadratic-form half.

lemma *closed-entry-hermitian-eq*:
fixes *i j* :: *nat*
assumes *ij*: *i* < *CARD*(*'k* :: *finite*) *j* < *CARD*(*'k*)
shows *closed* {*M* :: (*complex*, *'k*, *'k*) *lmat*.
unwrap-lm M $$ (*i*, *j*) = *cnj* (*unwrap-lm M* $$ (*j*, *i*))}
⟨*proof*⟩

lemma *closed-hermitian-unwrap-lmat*:
closed {*M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*. *hermitian* (*unwrap-lm M*)}
⟨*proof*⟩

lemma *closed-nonneg-quad-form-lmat*:
fixes *x* :: *complex Matrix.vec*
assumes *xdim*: *dim-vec x* = *CARD*(*'k* :: *finite*)
shows *closed* {*M* :: (*complex*, *'k*, *'k*) *lmat*.
Re (⟨*x* | *unwrap-lm M* $*_v$ *x*⟩) ≥ *0*}

⟨*proof*⟩

lemma *closed-psd-lmat*:
closed {*M* :: (*complex*, 'k :: *finite*, 'k) *lmat*. *psd-lmat M*}
⟨*proof*⟩

24.6.3 Trace on lmat

definition *Tr-lmat* :: (*complex*, 'k :: *finite*, 'k) *lmat* ⇒ *complex* **where**
Tr-lmat M = (∑ *i* = *0*..<*CARD*('k). *unwrap-lm M* $$ (*i*, *i*))

lemma *continuous-on-Tr-lmat*:
fixes *S* :: (*complex*, 'k :: *finite*, 'k) *lmat set*
shows *continuous-on S Tr-lmat*
⟨*proof*⟩

lemma *Tr-lmat-add*:
fixes *M N* :: (*complex*, 'k :: *finite*, 'k) *lmat*
shows *Tr-lmat* (*M* + *N*) = *Tr-lmat M* + *Tr-lmat N*
⟨*proof*⟩

lemma *Tr-lmat-scaleR*:
fixes *M* :: (*complex*, 'k :: *finite*, 'k) *lmat* **and** *a* :: *real*
shows *Tr-lmat* (*a* *$_R$ *M*) = *a* *$_R$ *Tr-lmat M*
⟨*proof*⟩

lemma *Tr-lmat-zero* [*simp*]:
Tr-lmat (*0* :: (*complex*, 'k :: *finite*, 'k) *lmat*) = *0*
⟨*proof*⟩

lemma *Tr-lmat-sum-scaleR*:
fixes *T* :: 'a *set* **and** *u* :: 'a ⇒ *real*
and *f* :: 'a ⇒ (*complex*, 'k :: *finite*, 'k) *lmat*
assumes *finite T*
shows *Tr-lmat* (∑ *t* ∈ *T*. *u t* *$_R$ *f t*) = (∑ *t* ∈ *T*. *u t* *$_R$ *Tr-lmat* (*f t*))
⟨*proof*⟩

lemma *closed-unit-trace-lmat*:
closed {*M* :: (*complex*, 'k :: *finite*, 'k) *lmat*. *Tr-lmat M* = *1*}
⟨*proof*⟩

24.6.4 Compactness of the unit-trace PSD slice

For $\sigma \geq 0$ with $\operatorname{Tr}\sigma = 1$ and eigenvalues $\lambda_i \geq 0$, $\|\sigma\|_{HS}^2 = \sum_i \lambda_i^2 \leq \left(\sum_i \lambda_i\right)^2 = 1$.

lemma *psd-entry-bound-by-diag*:
fixes *M* :: *complex Matrix.mat*
assumes *Mcar*: *M* ∈ *carrier-mat n n*
and *Mpsd*: *psd M*
and *i*: *i* < *n* **and** *j*: *j* < *n*
shows (*norm* (*M* $$ (*i*, *j*)))^2 ≤ *Re* (*M* $$ (*i*, *i*)) * *Re* (*M* $$ (*j*, *j*))
⟨*proof*⟩

lemma *norm-lmat-le-of-psd-unit-trace*:
fixes *M* :: (*complex*, 'k :: *finite*, 'k) *lmat*
assumes *psd*: *psd-lmat M* **and** *tr*: *Tr-lmat M* = *1*
shows *norm M* ≤ *1*
⟨*proof*⟩

lemma *compact-unit-trace-psd-lmat*:
compact {*M* :: (*complex*, 'k :: *finite*, 'k) *lmat*. *psd-lmat M* ∧ *Tr-lmat M* = *1*}
⟨*proof*⟩

24.6.5 UCP maps in Choi coordinates

In Choi coordinates a linear map $\Phi : M_n \to M_d$ is UCP iff $P_\Phi \geq 0$ and $\sum_i R_{ii} = I_d$ (*choi-ucp-iff*); the set of such P is compact. Dimensions are cardinalities of finite types, $n = |'n|$, $d = |'d|$, so that the typed matrices (a Euclidean space, *Lmat-Euclidean*) and the closed and compact sets of positive matrices above can be used.

definition *ucp-choi-set* :: (*complex*, 'n :: *finite* × 'd :: *finite*, 'n × 'd) *lmat set* **where**
ucp-choi-set = {*M*. *psd-lmat M* ∧ (∀ *a* < *CARD*('d). ∀ *b* < *CARD*('d).
(∑ *i*<*CARD*('n). *unwrap-lm M* $$ (*i* ∗ *CARD*('d) + *a*, *i* ∗ *CARD*('d) + *b*)) = (*if a* = *b then 1 else 0*))}

lemma *closed-block-sum-eq*:
assumes *a*: *a* < *CARD*('d :: *finite*) **and** *b*: *b* < *CARD*('d)
shows *closed* {*M* :: (*complex*, 'n :: *finite* × 'd, 'n × 'd) *lmat*.
(∑ *i*<*CARD*('n). *unwrap-lm M* $$ (*i* ∗ *CARD*('d) + *a*, *i* ∗ *CARD*('d) + *b*)) = *c*}
⟨*proof*⟩

lemma *closed-ucp-choi-set*: *closed* (*ucp-choi-set* :: (*complex*, 'n :: *finite* × 'd :: *finite*, 'n × 'd) *lmat set*)
⟨*proof*⟩

lemma *ucp-choi-set-trace*:
assumes *M* ∈ (*ucp-choi-set* :: (*complex*, 'n :: *finite* × 'd :: *finite*, 'n × 'd) *lmat set*)
shows *Tr-lmat M* = *of-nat CARD*('d)
⟨*proof*⟩

theorem *compact-ucp-choi-set*: *compact* (*ucp-choi-set* :: (*complex*, 'n :: *finite* × 'd :: *finite*, 'n × 'd) *lmat set*)
⟨*proof*⟩

The inverse of the Choi map: $\Phi_P(X) = \sum_{i,j} x_{ij} P_{ij}$ with P_{ij} the (i,j) block.

definition *choi-inv* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
choi-inv n d P X = *Matrix.mat d d* (λ(*a*, *b*). ∑ *i*<*n*. ∑ *j*<*n*. *X* $$ (*i*, *j*) ∗ *P* $$ (*i* ∗ *d* + *a*, *j* ∗ *d* + *b*))

lemma *choi-inv-lin*: *lin-on n d* (*carrier-mat n n*) (*choi-inv n d P*)
⟨*proof*⟩

lemma *choi-matrix-choi-inv*:
assumes *P*: *P* ∈ *carrier-mat* (*n* ∗ *d*) (*n* ∗ *d*)
shows *choi-matrix n d* (*choi-inv n d P*) = *P*
⟨*proof*⟩

lemma *block-sum-entry*:
assumes *P*: *P* ∈ *carrier-mat* (*n* ∗ *d*) (*n* ∗ *d*) **and** *a*: *a* < *d* **and** *b*: *b* < *d*
shows *msum d d* (λ*i*. *block-of-mat d P i i*) [*0*..<*n*] $$ (*a*, *b*) = (∑ *i*<*n*. *P* $$ (*i* ∗ *d* + *a*, *i* ∗ *d* + *b*))
⟨*proof*⟩

UCP in Choi coordinates is exactly *ucp-choi-set*, so it is compact.

theorem *ucp-choi-image*:
(λΦ. *wrap-lm* (*choi-matrix CARD*('n) *CARD*('d) Φ) :: (*complex*, 'n :: *finite* × 'd :: *finite*, 'n × 'd) *lmat*)

‘ {Φ. *lin-on CARD('n) CARD('d) (carrier-mat CARD('n) CARD('n)) Φ ∧ Φ ∈ UCP CARD('n) CARD('d)*}
= *ucp-choi-set*
⟨*proof*⟩

corollary *compact-ucp*:
compact ((λΦ. *wrap-lm* (*choi-matrix CARD('n) CARD('d) Φ*) :: (*complex*, *'n* :: *finite* × *'d* :: *finite*, *'n* × *'d*) *lmat*)
‘ {Φ. *lin-on CARD('n) CARD('d) (carrier-mat CARD('n) CARD('n)) Φ ∧ Φ ∈ UCP CARD('n) CARD('d)*})
⟨*proof*⟩

25 Day 23 (19 October 2011): Choi's Theorem 3, extreme points of UCP and CPTP

Day 22 stated and Day 23 proves:

Theorem (notes) (Choi 3)**.** If $\Phi : M_n \to M_d$ is unital and completely positive, $\Phi(X) = \sum_{i=1}^{r} V_i^* X V_i$ with $r = \mathrm{cr}(\Phi)$, then Φ is extreme in $UCP(M_n, M_d)$ if and only if $\{V_i^* V_j : 1 \le i, j \le r\} \subseteq M_d$ is linearly independent.

Corollary (notes). If $\Phi : M_d \to M_n$ is completely positive and trace preserving, $\Phi(X) = \sum_{i=1}^{r} V_i X V_i^*$ with $r = \mathrm{cr}(\Phi)$, then Φ is extreme in $CPTP(M_d, M_n)$ iff $\{V_i^* V_j\}$ is linearly independent (apply the theorem to Φ^*).

The notes' proof. ($\Rightarrow$) If $\sum_{ij} \lambda_{ij} V_i^* V_j = 0$ it suffices to treat Hermitian $\mu = (\mu_{ij})$ with $-I \le \mu \le I$: writing $I \pm \mu = (\alpha_{ij})^*(\alpha_{ij})$ and $W_i = \sum_j \alpha_{ij} V_j$ shows that $\psi_\pm(X) = \sum_i V_i^* X V_i \pm \sum_{ij} \mu_{ij} V_i^* X V_j$ are UCP with $\Phi = \frac{1}{2}(\psi_+ + \psi_-)$, so extremality gives $\psi_+ = \Phi$, and Choi 2 (unitary freedom) forces $I + \mu = U^*U = I$, i.e. $\mu = 0$. ($\Leftarrow$) If $\Phi = \frac{1}{2}(\psi_1 + \psi_2)$ with $\psi_k(X) = \sum_p W_p^* X W_p$, Choi 2 gives $W_p/\sqrt{2} = \sum_j \alpha_{pj} V_j$ with $U^*U = I_r$; unitality of ψ_1 and linear independence of the $V_i^* V_j$ force $U_1^* U_1 = \frac{1}{2} I_r$, whence $\psi_1 = \Phi = \psi_2$. Examples: $X \mapsto U^* X U$ is extreme; $\frac{1}{2}(U_1^* X U_1 + U_2^* X U_2)$, $X \mapsto \mathrm{tr}(X) I_d$ and $X \mapsto \mathrm{diag}(X)$ are not.
The formalisation proves the corollary first, for maps $\Phi(X) = \sum_i V_i X V_i^*$ with a fixed linearly independent Kraus list (what $r = \mathrm{cr}(\Phi)$ gives, Day 22), and the theorem from it by duality. Everything goes through the quadratic Kraus maps $\psi_T(X) = \sum_{i,j} T_{ij} V_i X V_j^*$ and their Choi matrices VTV^*. The direction ($\Rightarrow$) is the notes' perturbation argument; for ($\Leftarrow$) a CP Radon–Nikodym theorem takes the place of Choi 2. The examples are in the next theory.

25.1 Extreme points

A point of a set S of maps is extreme if it is not the midpoint of two other points of S. Maps are compared on M_n: the two points are linear on M_n, and one of them differs from Φ at some matrix (then so does the other). This defines the extreme points of $\mathrm{CPTP}(M_n, M_d)$ and of $\mathrm{UCP}(M_n, M_d)$ at once.

definition *midpoint-pair* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) *set* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*)
⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
midpoint-pair n d S Φ Φ₁ Φ₂ ⟷
Φ₁ ∈ *S* ∧ Φ₂ ∈ *S* ∧ *lin-on n d* (*carrier-mat n n*) Φ₁ ∧ *lin-on n d* (*carrier-mat n n*) Φ₂ ∧

$(\exists\, X \in \mathit{carrier\text{-}mat}\ n\ n.\ \Phi_1\ X \neq \Phi\ X) \wedge$
$(\forall\, X \in \mathit{carrier\text{-}mat}\ n\ n.\ \Phi\ X = (1/2 :: \mathit{complex}) \cdot_m \Phi_1\ X + (1/2 :: \mathit{complex}) \cdot_m \Phi_2\ X)$

definition *extreme-point* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) *set* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
extreme-point n d S Φ ⟷ Φ ∈ *S* ∧ ¬ (∃ Φ₁ Φ₂. *midpoint-pair n d S* Φ Φ₁ Φ₂)

abbreviation *extreme-CPTP* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where *extreme-CPTP n d* ≡ *extreme-point n d* (*CPTP n d*)

abbreviation *extreme-UCP* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where *extreme-UCP n d* ≡ *extreme-point n d* (*UCP n d*)

lemma *midpoint-pair-cong*:
assumes ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ Φ *X* = Ψ *X*
shows *midpoint-pair n d S* Φ Φ₁ Φ₂ ⟷ *midpoint-pair n d S* Ψ Φ₁ Φ₂
⟨*proof*⟩

A Kraus representation of a channel, and what it gives.

definition *cptp-kraus-rep* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒
complex Matrix.mat list ⇒ *bool*
where
cptp-kraus-rep n d Φ *Vs* ≡
kraus-operators n d Vs ∧
(∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ *X* = *kraus-map-dim d Vs X*)

lemma *cptp-kraus-rep-facts*:
assumes *cptp-kraus-rep n d* Φ *Vs*
shows ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n sum-wdag-w n Vs* = 1_m *n*
⋀*X*. *X* ∈ *carrier-mat n n* ⟹ Φ *X* = *kraus-map-dim d Vs X*
⋀*X*. *X* ∈ *carrier-mat n n* ⟹ Φ *X* ∈ *carrier-mat d d*
⟨*proof*⟩

25.2 Quadratic Kraus maps

The quadratic Kraus map of a list $V_1, \ldots, V_k$ and a matrix $T \in M_k$ is

$$\psi_T(X) = \sum_{i,j} T_{ij}\, V_i X V_j^*.$$

It is linear in X and in T, and ψ_I is the Kraus map of the list.

definition *kraus-quad-map* ::
nat ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat*
where
kraus-quad-map d Vs T X = *msum d d* (λ(*i*, *j*). *T* $$ (*i*, *j*) ·$_m$ (*Vs* ! *i* ∗ *X* ∗ ((*Vs* ! *j*)†)))
(*List.product* [*0*..<*length Vs*] [*0*..<*length Vs*])

lemma *kraus-quad-map-dims* [*simp*]:
dim-row (*kraus-quad-map d Vs T X*) = *d dim-col* (*kraus-quad-map d Vs T X*) = *d*
⟨*proof*⟩

lemma *kraus-list-nth*:
assumes ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n* **and** *i* < *length Vs*
shows *Vs* ! *i* ∈ *carrier-mat d n dim-row* (*Vs* ! *i*) = *d dim-col* (*Vs* ! *i*) = *n*
⟨*proof*⟩

lemma *kraus-quad-term-carrier*:
assumes *Vs*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$ **and** *X*: $X \in carrier\text{-}mat\ n\ n$
and *i*: $i < length\ Vs$ **and** *j*: $j < length\ Vs$
shows $Vs\ !\ i * X * ((Vs\ !\ j)^\dagger) \in carrier\text{-}mat\ d\ d$
⟨*proof*⟩

lemma *kraus-quad-map-carrier*:
assumes *Vs*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$ **and** *X*: $X \in carrier\text{-}mat\ n\ n$
shows $kraus\text{-}quad\text{-}map\ d\ Vs\ T\ X \in carrier\text{-}mat\ d\ d$
⟨*proof*⟩

lemma *kraus-quad-map-index*:
assumes *Vs*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$ **and** *X*: $X \in carrier\text{-}mat\ n\ n$
and *r*: $r < d$ **and** *c*: $c < d$
shows $kraus\text{-}quad\text{-}map\ d\ Vs\ T\ X\ \$\$\ (r,\ c)$
$= (\sum i{<}length\ Vs.\ \sum j{<}length\ Vs.\ T\ \$\$\ (i,\ j) * (Vs\ !\ i * X * ((Vs\ !\ j)^\dagger))\ \$\$\ (r,\ c))$
⟨*proof*⟩

lemma *kraus-quad-map-lin-on*:
assumes *Vs*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
shows $lin\text{-}on\ n\ d\ (carrier\text{-}mat\ n\ n)\ (kraus\text{-}quad\text{-}map\ d\ Vs\ T)$
⟨*proof*⟩

lemma *kraus-quad-map-coeff*:
assumes *Vs*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$ **and** *X*: $X \in carrier\text{-}mat\ n\ n$
and *S*: $S \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$ **and** *T*: $T \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
shows $kraus\text{-}quad\text{-}map\ d\ Vs\ (a \cdot_m S + b \cdot_m T)\ X$
$= a \cdot_m kraus\text{-}quad\text{-}map\ d\ Vs\ S\ X + b \cdot_m kraus\text{-}quad\text{-}map\ d\ Vs\ T\ X$
⟨*proof*⟩

lemma *kraus-quad-map-smult*:
assumes *Vs*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$ **and** *X*: $X \in carrier\text{-}mat\ n\ n$
and *T*: $T \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
shows $kraus\text{-}quad\text{-}map\ d\ Vs\ (a \cdot_m T)\ X = a \cdot_m kraus\text{-}quad\text{-}map\ d\ Vs\ T\ X$
⟨*proof*⟩

lemma *kraus-quad-map-one*:
assumes *Vs*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$ **and** *X*: $X \in carrier\text{-}mat\ n\ n$
shows $kraus\text{-}quad\text{-}map\ d\ Vs\ (1_m\ (length\ Vs))\ X = kraus\text{-}map\text{-}dim\ d\ Vs\ X$
⟨*proof*⟩

25.2.1 The Choi matrix

The Choi matrix of ψ_T is VTV^*, where the columns of V are the vectorised V_i.

definition *kraus-vec-mat* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat*
where $kraus\text{-}vec\text{-}mat\ d\ n\ Vs = mat\text{-}of\text{-}cols\ (n{*}d)\ (map\ (vec\text{-}of\text{-}kraus\ d\ n)\ Vs)$

lemma *kraus-vec-mat-carrier* [*simp*]: $kraus\text{-}vec\text{-}mat\ d\ n\ Vs \in carrier\text{-}mat\ (n{*}d)\ (length\ Vs)$
⟨*proof*⟩

lemma *kraus-vec-mat-dims* [*simp*]:
$dim\text{-}row\ (kraus\text{-}vec\text{-}mat\ d\ n\ Vs) = n{*}d\ dim\text{-}col\ (kraus\text{-}vec\text{-}mat\ d\ n\ Vs) = length\ Vs$
⟨*proof*⟩

lemma *kraus-vec-mat-index*:
assumes $r < n{*}d$ **and** $k < length\ Vs$
shows $kraus\text{-}vec\text{-}mat\ d\ n\ Vs\ \$\$\ (r,\ k) = Vs\ !\ k\ \$\$\ (r\ mod\ d,\ r\ div\ d)$

⟨*proof*⟩

lemma *choi-matrix-kraus-quad-map*:
assumes *Vs*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n* **and** *T*: *T* ∈ *carrier-mat* (*length Vs*) (*length Vs*)
and *n*: *0* < *n* **and** *d*: *0* < *d*
shows *choi-matrix n d* (*kraus-quad-map d Vs T*) = *kraus-vec-mat d n Vs* ∗ *T* ∗ ((*kraus-vec-mat d n Vs*)†)
⟨*proof*⟩

So ψ_T is completely positive for $T \geq 0$, by Choi's theorem (Day 19).

lemma *kraus-quad-map-cp*:
assumes *Vs*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n* **and** *T*: *T* ∈ *carrier-mat* (*length Vs*) (*length Vs*)
and *psdT*: *psd T* **and** *n*: *0* < *n* **and** *d*: *0* < *d*
shows *completely-positive n d* (*kraus-quad-map d Vs T*)
⟨*proof*⟩

If the V_i are linearly independent, so are the columns of V, and $\psi_T = 0$ on M_n only for $T = 0$.

lemma *kraus-quad-map-eq-zero*:
assumes *Vs*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n* **and** *li*: *lin-indep-complex-mat d n Vs*
and *A*: *A* ∈ *carrier-mat* (*length Vs*) (*length Vs*) **and** *n*: *0* < *n* **and** *d*: *0* < *d*
and *z*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *kraus-quad-map d Vs A X* = 0_m *d d*
shows *A* = 0_m (*length Vs*) (*length Vs*)
⟨*proof*⟩

25.2.2 The pair sum and the trace

The pair sum $P_T = \sum_{i,j} T_{ij} V_j^* V_i \in M_n$. By cyclicity $\mathrm{Tr}(V_i \rho V_j^*) = \mathrm{Tr}(V_j^* V_i \rho)$, so $\mathrm{Tr}\,\psi_T(\rho) = \mathrm{Tr}(P_T \rho)$, and ψ_T is trace preserving iff $P_T = I_n$.

definition *kraus-pair-sum* :: *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat*
where
kraus-pair-sum n Vs A = *msum n n* (λ(*i*, *j*). *A* $$ (*i*, *j*) ·$_m$ ((*Vs* ! *j*)† ∗ *Vs* ! *i*))
(*List.product* [*0*..<*length Vs*] [*0*..<*length Vs*])

lemma *kraus-pair-sum-dims* [*simp*]:
dim-row (*kraus-pair-sum n Vs A*) = *n dim-col* (*kraus-pair-sum n Vs A*) = *n*
⟨*proof*⟩

lemma *kraus-pair-term-carrier*:
assumes *Vs*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n* **and** *i*: *i* < *length Vs* **and** *j*: *j* < *length Vs*
shows (*Vs* ! *j*)† ∗ *Vs* ! *i* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *kraus-pair-sum-index*:
assumes *Vs*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n* **and** *r*: *r* < *n* **and** *c*: *c* < *n*
shows *kraus-pair-sum n Vs A* $$ (*r*, *c*)
= (∑ *i*<*length Vs*. ∑ *j*<*length Vs*. *A* $$ (*i*, *j*) ∗ ((*Vs* ! *j*)† ∗ *Vs* ! *i*) $$ (*r*, *c*))
⟨*proof*⟩

lemma *kraus-pair-sum-coeff*:
assumes *Vs*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *S*: *S* ∈ *carrier-mat* (*length Vs*) (*length Vs*) **and** *T*: *T* ∈ *carrier-mat* (*length Vs*) (*length Vs*)
shows *kraus-pair-sum n Vs* (*a* ·$_m$ *S* + *b* ·$_m$ *T*) = *a* ·$_m$ *kraus-pair-sum n Vs S* + *b* ·$_m$ *kraus-pair-sum n Vs T*
⟨*proof*⟩

lemma *kraus-pair-sum-one*:

assumes *Vs*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
shows *kraus-pair-sum n Vs* $(1_m\ (length\ Vs)) = sum\text{-}wdag\text{-}w\ n\ Vs$
⟨*proof*⟩

lemma *kraus-pair-sum-dagger*:
assumes *Vs*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$ **and** *A*: $A \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
shows *kraus-pair-sum n Vs* $(A^\dagger) = (kraus\text{-}pair\text{-}sum\ n\ Vs\ A)^\dagger$
⟨*proof*⟩

lemma *Tr-kraus-quad-map*:
assumes *Vs*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$ **and** *rho*: $\varrho \in carrier\text{-}mat\ n\ n$
shows $Tr\ (kraus\text{-}quad\text{-}map\ d\ Vs\ T\ \varrho) = Tr\ (kraus\text{-}pair\text{-}sum\ n\ Vs\ T * \varrho)$
⟨*proof*⟩

lemma *kraus-quad-map-tp-iff*:
assumes *Vs*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
shows $trace\text{-}preserving\ n\ (kraus\text{-}quad\text{-}map\ d\ Vs\ T) \longleftrightarrow kraus\text{-}pair\text{-}sum\ n\ Vs\ T = 1_m\ n$
⟨*proof*⟩

25.2.3 The products of a Kraus list

The products $V_j^* V_i \in M_n$ for all pairs (i, j), the one at position $ik + j$ being $V_j^* V_i$. The pair sum is their combination with the coefficients T_{ij}: independent products determine T from P_T, and a dependence among them is a non-zero T with $P_T = 0$.

definition *kraus-products* ::
complex Matrix.mat list ⇒ *complex Matrix.mat list*
where
kraus-products Vs ≡
concat (*map* (λ*i*. *map* (λ*j*. (*Vs*!*j*)† ∗ *Vs*!*i*) [*0*..<*length Vs*])
[*0*..<*length Vs*])

lemma *kraus-products-carrier*:
assumes $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
shows $\forall\, B \in set\ (kraus\text{-}products\ Vs).\ B \in carrier\text{-}mat\ n\ n$
⟨*proof*⟩

lemma *length-kraus-products*:
$length\ (kraus\text{-}products\ Vs) = length\ Vs * length\ Vs$
⟨*proof*⟩

lemma *kraus-products-nth*:
assumes *i-lt*: $i < length\ Vs$ **and** *j-lt*: $j < length\ Vs$
shows $kraus\text{-}products\ Vs\ !\ (i * length\ Vs + j) = (Vs!j)^\dagger * Vs!i$
⟨*proof*⟩

lemma *pair-index-lt*:
fixes *i j k* :: *nat*
assumes $i < k\ j < k$
shows $i * k + j < k * k\ (i * k + j)\ div\ k = i\ (i * k + j)\ mod\ k = j$
⟨*proof*⟩

lemma *kraus-pair-sum-products*:
assumes *Vs*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
shows *kraus-pair-sum n Vs A*
$= msum\ n\ n\ (\lambda\kappa.\ A\ \$\$\ (\kappa\ div\ length\ Vs, \kappa\ mod\ length\ Vs) \cdot_m (kraus\text{-}products\ Vs\ !\ \kappa))$
$[0..<length\ (kraus\text{-}products\ Vs)]$
⟨*proof*⟩

lemma *kraus-pair-sum-eq-zero*:
assumes *Vs*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n* **and** *li*: *lin-indep-complex-mat n n* (*kraus-products Vs*)
and *A*: $A \in$ *carrier-mat* (*length Vs*) (*length Vs*) **and** *z*: *kraus-pair-sum n Vs A* $= 0_m$ *n n*
shows $A = 0_m$ (*length Vs*) (*length Vs*)
⟨*proof*⟩

lemma *kraus-pair-sum-dependence*:
assumes *Vs*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n* **and** *dep*: $\neg$ *lin-indep-complex-mat n n* (*kraus-products Vs*)
shows $\exists\, A \in$ *carrier-mat* (*length Vs*) (*length Vs*). $A \neq 0_m$ (*length Vs*) (*length Vs*) $\wedge$ *kraus-pair-sum n Vs A* $= 0_m$ *n n*
⟨*proof*⟩

25.3 Extreme maps have independent products

The notes' direction ($\Rightarrow$), by contraposition. A dependence among the products gives a non-zero Hermitian H with $P_H = 0$. For small $\epsilon > 0$ the maps $\Phi_\pm = \psi_{I \pm \epsilon H}$ are completely positive, as $I \pm \epsilon H \geq 0$, and trace preserving, as $P_{I \pm \epsilon H} = I$; they differ from $\Phi = \psi_I$, as $\psi_H \neq 0$, and $\Phi = \frac{1}{2}(\Phi_+ + \Phi_-)$.

A dependence can be taken Hermitian: $P_{A^*} = P_A^*$, and one of $A + A^*$, $i(A - A^*)$ is non-zero.

lemma *kraus-pair-sum-hermitian-witness*:
assumes *Vs*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n* **and** *A*: $A \in$ *carrier-mat* (*length Vs*) (*length Vs*)
and *nz*: $A \neq 0_m$ (*length Vs*) (*length Vs*) **and** *z*: *kraus-pair-sum n Vs A* $= 0_m$ *n n*
shows $\exists\, H \in$ *carrier-mat* (*length Vs*) (*length Vs*). *hermitian H* $\wedge$ $H \neq 0_m$ (*length Vs*) (*length Vs*)
$\wedge$ *kraus-pair-sum n Vs H* $= 0_m$ *n n*
⟨*proof*⟩

For Hermitian H, $I + tH \geq 0$ when $|t|$ is small: if $H = UDU^*$, then $I + tH = U(I + tD)U^*$, and $\epsilon = 1/(1 + \sum_i |d_i|)$ works.

lemma *small-eps-psd*:
fixes *A* :: *complex Matrix.mat*
assumes *A-c*: $A \in$ *carrier-mat k k* **and** *herm*: *hermitian A*
shows $\exists\, \varepsilon > 0.\ \forall\, d ::$ *real*. $|d| \leq \varepsilon \longrightarrow$ *psd* (1_m *k* + *complex-of-real d* $\cdot_m$ *A*)
⟨*proof*⟩

lemma *not-extreme-CPTP-of-herm-witness*:
assumes *n*: $0 < n$ **and** *d*: $0 < d$ **and** *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *li*: *lin-indep-complex-mat d n Vs*
and *H*: $H \in$ *carrier-mat* (*length Vs*) (*length Vs*) *hermitian H H* $\neq 0_m$ (*length Vs*) (*length Vs*)
kraus-pair-sum n Vs H $= 0_m$ *n n*
shows $\neg$ *extreme-CPTP n d* Φ
⟨*proof*⟩

theorem *extreme-CPTP-imp-products-indep*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: $0 < n$ **and** *dpos*: $0 < d$
and Φ-*in*: $\Phi \in$ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *ext*: *extreme-CPTP n d* Φ
shows *lin-indep-complex-mat n n* (*kraus-products Vs*)
⟨*proof*⟩

25.4 A CP Radon–Nikodym theorem

The notes prove ($\Leftarrow$) with Choi 2. Here Choi matrices replace it: if $0 \le P_\Psi \le P_\Phi = VV^*$, then $P_\Psi = VTV^*$ with $T \ge 0$, so $\Psi = \psi_T$. The factorisation of dominated positive matrices uses the Moore–Penrose pseudoinverse.

25.4.1 Dominated positive matrices

The pseudoinverse of a real diagonal: $d_i^+ = 1/d_i$ for $d_i \neq 0$ and $d_i^+ = 0$ otherwise.

definition *pinv-list* :: *real list* ⇒ *real list* **where**
pinv-list D = *map* (λ*d*. *if d* = *0 then 0 else 1*/*d*) *D*

lemma *pinv-list-length* [*simp*]: *length* (*pinv-list D*) = *length D*
⟨*proof*⟩

lemma *pinv-list-index*:
assumes *i* < *length D*
shows *pinv-list D* ! *i* = (*if D* ! *i* = *0 then 0 else 1* / (*D* ! *i*))
⟨*proof*⟩

lemma *diag-pinv-diag-eq*:
fixes *D* :: *real list*
shows *diag-mat-of-real D* ∗ *diag-mat-of-real* (*pinv-list D*) ∗ *diag-mat-of-real D* = *diag-mat-of-real D*
⟨*proof*⟩

For Hermitian $A = UDU^*$ set $A^+ = UD^+U^*$; then $AA^+A = UDD^+DU^* = A$.

lemma *A-Aplus-A-eq*:
fixes *U* :: *complex Matrix.mat* **and** *D* :: *real list*
assumes *Uc*: *U* ∈ *carrier-mat k k*
and *UU*: *unitary U*
and *Dlen*: *length D* = *k*
shows (*U* ∗ *diag-mat-of-real D* ∗ (*U*†))
∗ (*U* ∗ *diag-mat-of-real* (*pinv-list D*) ∗ (*U*†))
∗ (*U* ∗ *diag-mat-of-real D* ∗ (*U*†))
= *U* ∗ *diag-mat-of-real D* ∗ (*U*†)
⟨*proof*⟩

If $0 \le B \le A$ then $B = AA^+B$: with $Q = A^+A$, $AQ = A$, so $x - Qx \in \ker A \subseteq \ker B$, whence $B = BQ$, and taking adjoints $B = QB = AA^+B$.

lemma *A-Aplus-B-eq-B*:
fixes *U* :: *complex Matrix.mat* **and** *D* :: *real list* **and** *B* :: *complex Matrix.mat*
assumes *Uc*: *U* ∈ *carrier-mat k k*
and *UU*: *unitary U*
and *Dlen*: *length D* = *k*
and *Dnn*: ⋀*i*. *i* < *k* ⟹ *0* ≤ *D* ! *i*
and *B-c*: *B* ∈ *carrier-mat k k*
and *B-psd*: *psd B*
and *dom-psd*: *psd* ((*U* ∗ *diag-mat-of-real D* ∗ (*U*†)) − *B*)
shows (*U* ∗ *diag-mat-of-real D* ∗ (*U*†))
∗ (*U* ∗ *diag-mat-of-real* (*pinv-list D*) ∗ (*U*†)) ∗ *B* = *B*
⟨*proof*⟩

Dominated positive matrices factor through the dominating frame: if $0 \le B \le VV^*$ then

$$B = VTV^*, \qquad T = V^*A^+BA^+V \ge 0, \quad A = VV^*.$$

lemma *dominated-outer-sum-param*:

fixes *vs* :: *complex Matrix.vec list*
and *B* :: *complex Matrix.mat*
assumes *vs-dim*: $\forall\, v \in$ *set vs*. *dim-vec v* = *k*
and *k-pos*: *0* < *k*
and *B-c*: *B* ∈ *carrier-mat k k*
and *B-psd*: *psd B*
and *B-dom*: *psd* (*outer-sum k vs* − *B*)
shows ∃ *T*. *T* ∈ *carrier-mat* (*length vs*) (*length vs*) ∧ *psd T* ∧
B = (*mat-of-cols k vs*) ∗ *T* ∗ ((*mat-of-cols k vs*)†)
⟨*proof*⟩

25.4.2 Completely positive maps below a Kraus map

If $0 \le P_\Psi \le P_\Phi$ for a linear Ψ, where Φ is the Kraus map of the V_i, then $\Psi = \psi_T$ for some $T \ge 0$.

theorem *cp-radon-nikodym*:
assumes *Vs*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n* **and** *L*: *lin-on n d* (*carrier-mat n n*) Ψ
and *P*: *psd* (*choi-matrix n d* Ψ) **and** *dom*: *psd* (*choi-matrix n d* (*kraus-map-dim d Vs*) − *choi-matrix n d* Ψ)
and *n*: *0* < *n* **and** *d*: *0* < *d*
shows ∃ *T* ∈ *carrier-mat* (*length Vs*) (*length Vs*). *psd T* ∧ (∀ *X* ∈ *carrier-mat n n*. Ψ *X* = *kraus-quad-map d Vs T X*)
⟨*proof*⟩

25.5 Independent products give extreme maps

The notes' direction (⇐). If $\Phi = \frac{1}{2}(\Phi_1 + \Phi_2)$, then $0 \le \frac{1}{2}P_{\Phi_1} \le P_\Phi$, so $\frac{1}{2}\Phi_1 = \psi_S$ with $S \ge 0$ and $\Phi_1 = \psi_{2S}$. Trace preservation gives $P_{2S} = I = P_I$, independence of the products $2S = I$, i.e. $\Phi_1 = \Phi$.

theorem *products-indep-imp-extreme-CPTP*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and Φ-*in*: Φ ∈ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *lin-indep*: *lin-indep-complex-mat n n* (*kraus-products Vs*)
shows *extreme-CPTP n d* Φ
⟨*proof*⟩

25.6 Choi's Theorem 3

For a linearly independent Kraus list,

$$\Phi \text{ extreme in } \mathrm{CPTP}(M_n, M_d) \iff \{V_j^* V_i\}_{i,j} \text{ linearly independent.}$$

By unitary freedom (*choi-2-unitary-freedom*, Day 21) any two such lists of Φ differ by a unitary, so the condition does not depend on the list.

theorem *choi-theorem-3-cptp-iff*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and Φ-*in*: Φ ∈ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
shows *extreme-CPTP n d* Φ ⟷ *lin-indep-complex-mat n n* (*kraus-products Vs*)
⟨*proof*⟩

25.6.1 Unital maps (D23-1)

The notes state the theorem for UCP maps. It follows by the duality of Day 22: $\Gamma(\Phi) = \Phi^*$ carries midpoint decompositions of a UCP map to midpoint decompositions of its adjoint and back, and the Kraus operators of Φ^* are the adjoints W_i^*, independent with the W_i.

lemma *midpoint-pair-adjoint*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *mid*: *midpoint-pair n d S* Φ Φ_1 Φ_2
and *S*: ⋀Ψ. *lin-on n d* (*carrier-mat n n*) Ψ ⟹ Ψ ∈ *S* ⟹ *map-adjoint n d* Ψ ∈ *S'*
shows *midpoint-pair d n S'* (*map-adjoint n d* Φ) (*map-adjoint n d* Φ_1) (*map-adjoint n d* Φ_2)
⟨*proof*⟩

theorem *extreme-UCP-iff-adjoint*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *U*: Φ ∈ *UCP n d* **and** *n*: *0* < *n* **and** *d*: *0* < *d*
shows *extreme-UCP n d* Φ ⟷ *extreme-CPTP d n* (*map-adjoint n d* Φ)
⟨*proof*⟩

The adjoints of the Kraus operators of a unital map are Kraus operators of its adjoint.

lemma *kraus-operators-dagger-of-unital*:
assumes *Wc*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n* **and** *un*: *kraus-map-dim d Ws* (1_m *n*) = 1_m *d*
shows *kraus-operators d n* (*map* (λ*W*. $W^\dagger$) *Ws*)
⟨*proof*⟩

Choi's Theorem 3: a unital CP map $\Phi(X) = \sum_i W_i X W_i^*$ with linearly independent W_i is extreme iff the products $W_j W_i^*$ are linearly independent (*kraus-products* of the adjoints). The notes write $\Phi(X) = \sum_i V_i^* X V_i$, i.e. $V_i = W_i^*$, and the products $V_i^* V_j$.

theorem *choi-theorem-3-ucp*:
assumes *n*: *0* < *n* **and** *d*: *0* < *d* **and** *U*: Φ ∈ *UCP n d*
and *Wc*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and *rep*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ Φ *X* = *kraus-map-dim d Ws X*
and *li*: *lin-indep-complex-mat d n Ws*
shows *extreme-UCP n d* Φ ⟷ *lin-indep-complex-mat d d* (*kraus-products* (*map* (λ*W*. $W^\dagger$) *Ws*))
⟨*proof*⟩

25.6.2 An extreme channel has Choi rank at most n (D22-6)

Corollary to Choi's Theorem 3 (D22-6): an extreme CPTP map $M_n \to M_d$ with a minimal Kraus representation $(V_i)_{i<r}$ has $r^2 \le n^2$, i.e. $\mathrm{cr}(\Phi) \le n$.

corollary *extreme-CPTP-kraus-length-le*:
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and Φ-*in*: Φ ∈ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *ext*: *extreme-CPTP n d* Φ
shows *length Vs* ≤ *n*
⟨*proof*⟩

corollary *extreme-CPTP-choi-rank-le*:
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and Φ-*in*: Φ ∈ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *ext*: *extreme-CPTP n d* Φ
shows *choi-rank n d* Φ ≤ *n*
⟨*proof*⟩

25.7 Extreme and non-extreme maps (D23-4)

Day 23, examples after Choi's Theorem 3, in the CPTP form and with the Kraus convention $\Phi(X) = \sum_i W_i X W_i^*$ of the formalisation (the notes write $U^* X U$; replace U by U^*).

25.7.1 Linear dependence tests

lemma *not-indep-of-dup*:
assumes *a*: $a <$ *length Bs* **and** *b*: $b <$ *length Bs* **and** *ab*: $a \neq b$ **and** *eq*: *Bs* ! $a =$ *Bs* ! b
shows ¬ *lin-indep-complex-mat n n Bs*
⟨*proof*⟩

lemma *not-indep-of-zero*:
assumes *a*: $a <$ *length Bs* **and** *z*: *Bs* ! $a = 0_m$ *n n*
shows ¬ *lin-indep-complex-mat n n Bs*
⟨*proof*⟩

lemma *single-lin-indep*:
assumes *W*: $W \in$ *carrier-mat d n* **and** *r*: $r < d$ **and** *s*: $s < n$ **and** *nz*: *W* $$ (*r*,*s*) $\neq 0$
shows *lin-indep-complex-mat d n* [*W*]
⟨*proof*⟩

lemma *single-mat-lin-indep*:
assumes *B*: $B \in$ *carrier-mat n n* **and** *r*: $r < n$ **and** *s*: $s < n$ **and** *nz*: *B* $$ (*r*,*s*) $\neq 0$
shows *lin-indep-complex-mat n n* [*B*]
⟨*proof*⟩

25.7.2 Example 1: unitary conjugation is extreme

theorem *unitary-conjugation-extreme*:
assumes *U*: $U \in$ *carrier-mat n n* **and** *Uu*: *unitary U* **and** *n*: $0 < n$
shows *extreme-CPTP n n* (*kraus-map-dim n* [*U*])
⟨*proof*⟩

25.7.3 Example 2: an average of two unitary conjugations

definition *avg2* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat list* **where**
avg2 U1 U2 = [(*1 / csqrt 2*) ·$_m$ *U1*, (*1 / csqrt 2*) ·$_m$ *U2*]

lemma *half-sq*: *cnj* (*1 / csqrt 2*) ∗ (*1 / csqrt 2*) = *1/2*
⟨*proof*⟩

lemma *smult-prod*:
fixes *A B* :: *complex Matrix.mat*
assumes *A* ∈ *carrier-mat m k B* ∈ *carrier-mat k l*
shows (*a* ·$_m$ *A*) ∗ (*b* ·$_m$ *B*) = (*a* ∗ *b*) ·$_m$ (*A* ∗ *B*)
⟨*proof*⟩

lemma *smult-triple*:
fixes *A X B* :: *complex Matrix.mat*
assumes *A* ∈ *carrier-mat m k X* ∈ *carrier-mat k k B* ∈ *carrier-mat k l*
shows (*a* ·$_m$ *A*) ∗ *X* ∗ (*b* ·$_m$ *B*) = (*a* ∗ *b*) ·$_m$ (*A* ∗ *X* ∗ *B*)
⟨*proof*⟩

The average of two unitary conjugations has the Kraus operators $W_k = \frac{1}{\sqrt{2}} U_k$: $\sum_k W_k X W_k^* = \frac{1}{2}(U_1 X U_1^* + U_2 X U_2^*)$, and $\sum_k W_k^* W_k = \frac{1}{2}(I + I) = I$.

lemma *avg2-products*:
assumes *U1*: *U1* ∈ *carrier-mat n n unitary U1* **and** *U2*: *U2* ∈ *carrier-mat n n unitary U2*

shows $(avg2\ U1\ U2\ !\ 0)^\dagger * avg2\ U1\ U2\ !\ 0 = (1/2) \cdot_m 1_m\ n$
and $(avg2\ U1\ U2\ !\ 1)^\dagger * avg2\ U1\ U2\ !\ 1 = (1/2) \cdot_m 1_m\ n$
⟨*proof*⟩

lemma *avg2-kraus*:
assumes *U1*: $U1 \in carrier\text{-}mat\ n\ n\ unitary\ U1$ **and** *U2*: $U2 \in carrier\text{-}mat\ n\ n\ unitary\ U2$
shows *kraus-operators n n* (*avg2 U1 U2*)
⟨*proof*⟩

lemma *avg2-map*:
assumes *U1*: $U1 \in carrier\text{-}mat\ n\ n$ **and** *U2*: $U2 \in carrier\text{-}mat\ n\ n$ **and** *M*: $M \in carrier\text{-}mat\ n\ n$
shows $kraus\text{-}map\text{-}dim\ n\ (avg2\ U1\ U2)\ M = (1/2) \cdot_m (U1 * M * (U1^\dagger) + U2 * M * (U2^\dagger))$
⟨*proof*⟩

The notes' claim needs U_1, U_2 linearly independent: then the products $V_i^* V_j$ contain $\frac{1}{2}I$ twice and Φ is not extreme.

theorem *avg2-not-extreme*:
assumes *U1*: $U1 \in carrier\text{-}mat\ n\ n\ unitary\ U1$ **and** *U2*: $U2 \in carrier\text{-}mat\ n\ n\ unitary\ U2$
and *n*: $0 < n$ **and** *ind*: *lin-indep-complex-mat n n* (*avg2 U1 U2*)
shows ¬ *extreme-CPTP n n* (*kraus-map-dim n* (*avg2 U1 U2*))
⟨*proof*⟩

Without that hypothesis the claim fails: for $U_2 = \lambda U_1$ with $|\lambda| = 1$ the map is $X \mapsto U_1 X U_1^*$, which is extreme.

theorem *avg2-proportional-extreme*:
assumes *U1*: $U1 \in carrier\text{-}mat\ n\ n\ unitary\ U1$ **and** *lam*: $cmod\ z = 1$ **and** *n*: $0 < n$
shows *extreme-CPTP n n* (*kraus-map-dim n* (*avg2 U1* ($z \cdot_m U1$)))
⟨*proof*⟩

25.7.4 Example 3: $X \mapsto \mathrm{Tr}(X)\, I_d$

As written, $\Phi(X) = \mathrm{Tr}(X) I_d$ is neither unital ($\Phi(I_n) = n I_d$) nor trace preserving ($\mathrm{Tr}\,\Phi(X) = d\,\mathrm{Tr}\,X$) unless $n = 1$ resp. $d = 1$. The CPTP normalisation is $\Phi(X) = \mathrm{Tr}(X) I_d/d$, with the nd Kraus operators $E_{ab}/\sqrt{d}$; these are linearly independent, so $\mathrm{cr}(\Phi) = nd > n$ for $d \geq 2$ and Φ is not extreme (D22-6).

definition *tr-kraus* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat list* **where**
tr-kraus n d = *map* (λ(*a*,*b*). *complex-of-real* (*1 / sqrt* (*real d*)) ·$_m$ *E*⟦*a*,*b* ; *d*,*n*⟧) (*List.product* [*0*..<*d*] [*0*..<*n*])

lemma *tr-kraus-carrier*: ∀ *W* ∈ *set* (*tr-kraus n d*). *W* ∈ *carrier-mat d n*
⟨*proof*⟩

lemma *tr-kraus-length*: *length* (*tr-kraus n d*) = *d* * *n*
⟨*proof*⟩

lemma *tr-scalar*:
assumes *d*: *0* < *d*
shows *cnj* (*complex-of-real* (*1 / sqrt* (*real d*))) * *complex-of-real* (*1 / sqrt* (*real d*)) = *1 / of-nat d*
and *complex-of-real* (*1 / sqrt* (*real d*)) * *cnj* (*complex-of-real* (*1 / sqrt* (*real d*))) = *1 / of-nat d*
⟨*proof*⟩

lemma *tr-kraus-operators*:
assumes *d*: *0* < *d*
shows *kraus-operators n d* (*tr-kraus n d*)
⟨*proof*⟩

$X \mapsto \frac{\operatorname{Tr} X}{d} I_d$ has the dn Kraus operators $\frac{1}{\sqrt{d}} E_{ab}$:

$$\frac{1}{d} \sum_{a,b} E_{ab} X E_{ba} = \frac{1}{d} \sum_{a} \Big(\sum_{b} x_{bb} \Big) E_{aa} = \frac{\operatorname{Tr} X}{d}\, I_d.$$

lemma *tr-kraus-map*:
assumes *d*: $0 < d$ **and** *M*: $M \in$ *carrier-mat n n*
shows *kraus-map-dim d* (*tr-kraus n d*) *M* = (*Tr M* / *of-nat d*) $\cdot_m$ 1_m *d*
⟨*proof*⟩

lemma *tr-kraus-lin-indep*:
assumes *d*: $0 < d$ **and** *n*: $0 < n$
shows *lin-indep-complex-mat d n* (*tr-kraus n d*)
⟨*proof*⟩

theorem *tr-map-not-extreme*:
assumes *n*: $0 < n$ **and** *d*: $2 \leq d$
shows ¬ *extreme-CPTP n d* (*kraus-map-dim d* (*tr-kraus n d*))
⟨*proof*⟩

25.7.5 Example 4: the diagonal map

definition *diag-kraus* :: *nat* ⇒ *complex Matrix.mat list* **where**
diag-kraus n = *map* (λ*i*. *E*⟦*i*,*i* ; *n*,*n*⟧) [*0*..<*n*]

lemma *diag-kraus-carrier*: ∀ *W* ∈ *set* (*diag-kraus n*). *W* ∈ *carrier-mat n n*
⟨*proof*⟩

The diagonal map has the Kraus operators E_{ii}, and $\sum_i E_{ii} X E_{ii} = \operatorname{diag}(x_{11}, \ldots, x_{nn})$. The E_{ii} are independent, but for $n \geq 2$ the products $E_{ii}^* E_{jj} = \delta_{ij} E_{ii}$ include 0, so they are dependent and the map is not extreme.

lemma *diag-kraus-map*:
assumes *M*: $M \in$ *carrier-mat n n*
shows *kraus-map-dim n* (*diag-kraus n*) *M* = *Matrix.mat n n* (λ(*r*,*s*). *if r* = *s then M* $$ (*r*,*r*) *else 0*)
⟨*proof*⟩

lemma *diag-kraus-operators*: *kraus-operators n n* (*diag-kraus n*)
⟨*proof*⟩

lemma *diag-kraus-lin-indep*: *lin-indep-complex-mat n n* (*diag-kraus n*)
⟨*proof*⟩

theorem *diag-not-extreme*:
assumes *n*: $2 \leq n$
shows ¬ *extreme-CPTP n n* (*kraus-map-dim n* (*diag-kraus n*))
⟨*proof*⟩

Part IV

Operator systems, Arveson's theorems and entanglement (Days 24–28)

26 Day 24 (24 October 2011): operator systems

Day 24: operator systems $S \subseteq M_n$ (subspaces containing I and closed under adjoints), the matrices $M_p(S)$ over them, and completely positive maps on S. An element of $M_p(M_n)$ is a $pn \times pn$ matrix whose (a, b) block is *block-of-mat n X a b*; the ampliation $\Phi^{(p)}$ is *amplification n d p* Φ (Day 19).

definition *operator-system* :: *nat* ⇒ *complex Matrix.mat set* ⇒ *bool* **where**
operator-system n OS ⟷ *OS* ⊆ *carrier-mat n n* ∧ 1_m *n* ∈ *OS* ∧ (∀ *A* ∈ *OS*. $A^\dagger$ ∈ *OS*)
∧ (∀ *A* ∈ *OS*. ∀ *B* ∈ *OS*. *A* + *B* ∈ *OS*) ∧ (∀ *A* ∈ *OS*. ∀ *c*. *c* ·$_m$ *A* ∈ *OS*)

definition *Mp* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat set* ⇒ *complex Matrix.mat set* **where**
Mp p n OS = {*A* ∈ *carrier-mat* (*p* ∗ *n*) (*p* ∗ *n*). ∀ *a* < *p*. ∀ *b* < *p*. *block-of-mat n A a b* ∈ *OS*}

definition *p-positive-on* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
p-positive-on n d p OS Φ ⟷ (∀ *A* ∈ *Mp p n OS*. *psd A* ⟶ *psd* (*amplification n d p* Φ *A*))

definition *cp-on* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
cp-on n d OS Φ ⟷ (∀ *p* > *0*. *p-positive-on n d p OS* Φ)

lemma *block-of-mat-dims* [*simp*]: *dim-row* (*block-of-mat n A a b*) = *n dim-col* (*block-of-mat n A a b*) = *n*
⟨*proof*⟩

An operator system is a subspace $S \subseteq M_n$ with $I \in S$ and $S^* = S$. $M_p(S)$ consists of the $p \times p$ block matrices with every block in S, and Φ is p-positive on S if $\Phi^{(p)}$ sends the positive elements of $M_p(S)$ to positive matrices.

lemma *operator-systemD*:
assumes *operator-system n OS*
shows *OS* ⊆ *carrier-mat n n* 1_m *n* ∈ *OS* ⋀*A*. *A* ∈ *OS* ⟹ $A^\dagger$ ∈ *OS*
⋀*A B*. *A* ∈ *OS* ⟹ *B* ∈ *OS* ⟹ *A* + *B* ∈ *OS* ⋀*A c*. *A* ∈ *OS* ⟹ *c* ·$_m$ *A* ∈ *OS*
⟨*proof*⟩

lemma *operator-system-zero*:
assumes *OS*: *operator-system n OS*
shows 0_m *n n* ∈ *OS*
⟨*proof*⟩

lemma *operator-system-diff*:
assumes *OS*: *operator-system n OS* **and** *A*: *A* ∈ *OS* **and** *B*: *B* ∈ *OS*
shows *A* − *B* ∈ *OS*
⟨*proof*⟩

For $S = M_n$ these are the notions of Days 17–18:

lemma *operator-system-full*: *operator-system n* (*carrier-mat n n*)
⟨*proof*⟩

lemma *Mp-full*: *Mp p n* (*carrier-mat n n*) = *carrier-mat* (*p* ∗ *n*) (*p* ∗ *n*)

⟨*proof*⟩

lemma *cp-on-full*: *cp-on n d* (*carrier-mat n n*) Φ ⟷ *completely-positive n d* Φ
⟨*proof*⟩

26.1 $M_p(OS)$ is an operator system

lemma *block-of-mat-add*:
assumes $A \in$ *carrier-mat* $(p * n)$ $(p * n)$ $B \in$ *carrier-mat* $(p * n)$ $(p * n)$ $a < p$ $b < p$
shows *block-of-mat n* $(A + B)$ *a b* = *block-of-mat n A a b* + *block-of-mat n B a b*
⟨*proof*⟩

lemma *block-of-mat-smult*:
assumes $A \in$ *carrier-mat* $(p * n)$ $(p * n)$ $a < p$ $b < p$
shows *block-of-mat n* $(k \cdot_m A)$ *a b* = $k \cdot_m$ *block-of-mat n A a b*
⟨*proof*⟩

lemma *block-of-mat-dagger*:
assumes $A \in$ *carrier-mat* $(p * n)$ $(p * n)$ $a < p$ $b < p$
shows *block-of-mat n* $(A^\dagger)$ *a b* = (*block-of-mat n A b a*)†
⟨*proof*⟩

lemma *block-of-mat-one*:
assumes $a < p$ $b < p$
shows *block-of-mat n* $(1_m\ (p * n))$ *a b* = (*if a* = *b then* 1_m *n else* 0_m *n n*)
⟨*proof*⟩

The note of Day 24: $M_p(S)$ is again an operator system.

lemma *Mp-operator-system*:
assumes *OS*: *operator-system n OS*
shows *operator-system* $(p * n)$ (*Mp p n OS*)
⟨*proof*⟩

26.2 Every element is a combination of four positives

lemma *herm-bound*:
assumes *Hmc*: *Hm* ∈ *carrier-mat n n* **and** *Hmh*: *hermitian Hm*
shows ∃ *c*::*real*. *0* ≤ *c* ∧ *psd* (*complex-of-real c* $\cdot_m$ 1_m *n* − *Hm*) ∧ *psd* (*complex-of-real c* $\cdot_m$ 1_m *n* + *Hm*)
⟨*proof*⟩

Proposition (Day 24): $X = (P_1 - P_2) + i(P_3 - P_4)$ with $P_k \in S^+$.

proposition *operator-system-four-positives*:
assumes *OS*: *operator-system n OS* **and** *A*: *A* ∈ *OS*
shows ∃ *P1 P2 P3 P4*. *P1* ∈ *OS* ∧ *P2* ∈ *OS* ∧ *P3* ∈ *OS* ∧ *P4* ∈ *OS* ∧ *psd P1* ∧ *psd P2* ∧ *psd P3* ∧ *psd P4*
∧ *A* = (*P1* − *P2*) + i $\cdot_m$ (*P3* − *P4*)
⟨*proof*⟩

27 Day 25 (26 October 2011): the Hahn–Banach theorem

Definition (notes). Let W be a normed space. A linear functional $f : W \to \mathbb{C}$ is *bounded* if $|f(w)| \le C\|w\|$ for some constant C; the least such C is the norm $\|f\| = \sup\{|f(w)| : \|w\| \le 1\}$.

Theorem (notes) (Hahn–Banach)**.** Let W be a normed space, $V \subseteq W$ a subspace and $g : V \to \mathbb{C}$ a bounded linear functional. Then there is a bounded linear functional $f : W \to \mathbb{C}$ with $\|f\| = \|g\|$ and $f(v) = g(v)$ for all $v \in V$.

In the notes the theorem is used (Day 26) to extend the state S_Φ from $M_d(S)$ to $M_d(M_n)$ in the proof of Arveson's extension theorem, which is not formalised. Here it is proved for $W = (\mathbb{C}\text{-valued } m \times n \text{ matrices})$ with the Hilbert–Schmidt norm: HOL's Hahn–Banach gives a real-linear norm-preserving extension of $\Re g$, and the Bohnenblust–Sobczyk construction $f(x) = F(x) - iF(ix)$ recovers a complex-linear extension with the same bound.

27.1 Bounded complex-linear functionals on lmat

definition *clinear-functional-on* ::
  (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
  ⇒ ((*complex*, *'m*, *'n*) *lmat* ⇒ *complex*) ⇒ *bool*
where
  clinear-functional-on V g ≡
    (∀ *x* ∈ *V*. ∀ *y* ∈ *V*. *g* (*x* + *y*) = *g x* + *g y*) ∧
    (∀ *x* ∈ *V*. ∀ *c*. *g* (*cmult-lmat c x*) = *c* ∗ *g x*)

definition *bounded-clinear-functional-on* ::
  (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
  ⇒ ((*complex*, *'m*, *'n*) *lmat* ⇒ *complex*) ⇒ *bool*
where
  bounded-clinear-functional-on V g ≡
    clinear-functional-on V g ∧
    (∃ *C* ≥ *0*. ∀ *x* ∈ *V*. *cmod* (*g x*) ≤ *C* ∗ *hs-norm-lmat x*)

27.2 Real-part functional and linearform/continuous instances

definition *re-part-lmat* ::
  ((*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ *complex*)
  ⇒ ((*complex*, *'m*, *'n*) *lmat* ⇒ *real*)
where
  re-part-lmat g x = *Re* (*g x*)

lemma *linearform-re-part-lmat*:
  fixes *V* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
    and *g* :: (*complex*, *'m*, *'n*) *lmat* ⇒ *complex*
  assumes *V-sub*: *Subspace.subspace V UNIV*
    and *g-clin*: *clinear-functional-on V g*
  shows *linearform V* (*re-part-lmat g*)
⟨*proof*⟩

lemma *continuous-re-part-lmat*:
  fixes *V* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
    and *g* :: (*complex*, *'m*, *'n*) *lmat* ⇒ *complex*
  assumes *V-sub*: *Subspace.subspace V UNIV*
    and *g-bnd*: *bounded-clinear-functional-on V g*
  shows *continuous V* (*re-part-lmat g*) *hs-norm-lmat*
⟨*proof*⟩

27.3 Application of HOL/Hahn_Banach (norm-preserving extension)

Apply HOL/Hahn_Banach to extend the real-linearform *re-part-lmat g* to all of UNIV preserving the function norm.

theorem *hahn-banach-re-part-lmat*:
  fixes *V* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
    and *g* :: (*complex*, *'m*, *'n*) *lmat* ⇒ *complex*
  assumes *V-sub*: *Subspace.subspace V* (*UNIV* :: (*complex*, *'m*, *'n*) *lmat set*)
    and *g-bnd*: *bounded-clinear-functional-on V g*

shows
$\exists F :: (complex, 'm, 'n)\ lmat \Rightarrow real.$
$linearform\ UNIV\ F$
$\wedge\ continuous\ UNIV\ F\ hs\text{-}norm\text{-}lmat$
$\wedge\ (\forall x \in V.\ F\ x = re\text{-}part\text{-}lmat\ g\ x)$
⟨*proof*⟩

27.4 Bohnenblust-Sobczyk: complex extension from real extension

Given a real-linear extension F of *re-part-lmat* g on all of UNIV, define the complex-linear extension $f\ x = F\ x - i \cdot F\ (i \cdot x)$.

definition *bs-complex-extension* ::
$((complex, 'm :: finite, 'n :: finite)\ lmat \Rightarrow real)$
$\Rightarrow ((complex, 'm, 'n)\ lmat \Rightarrow complex)$
where
bs-complex-extension F x =
complex-of-real $(F\ x) - \mathrm{i} *$ *complex-of-real* $(F\ (cmult\text{-}lmat\ \mathrm{i}\ x))$

lemma *bs-complex-extension-add*:
fixes $F :: (complex, 'm :: finite, 'n :: finite)\ lmat \Rightarrow real$
assumes *F-lin*: *linearform UNIV F*
shows *bs-complex-extension F* $(x + y)$
$= bs\text{-}complex\text{-}extension\ F\ x + bs\text{-}complex\text{-}extension\ F\ y$
⟨*proof*⟩

lemma *bs-complex-extension-real-smult*:
fixes $F :: (complex, 'm :: finite, 'n :: finite)\ lmat \Rightarrow real$
assumes *F-lin*: *linearform UNIV F*
shows *bs-complex-extension F* $(Vector\text{-}Space.prod\ a\ x)$
$= complex\text{-}of\text{-}real\ a * bs\text{-}complex\text{-}extension\ F\ x$
⟨*proof*⟩

lemma *bs-complex-extension-i-smult*:
fixes $F :: (complex, 'm :: finite, 'n :: finite)\ lmat \Rightarrow real$
assumes *F-lin*: *linearform UNIV F*
shows *bs-complex-extension F* $(cmult\text{-}lmat\ \mathrm{i}\ x)$
$= \mathrm{i} * bs\text{-}complex\text{-}extension\ F\ x$
⟨*proof*⟩

lemma *bs-complex-extension-csmult*:
fixes $F :: (complex, 'm :: finite, 'n :: finite)\ lmat \Rightarrow real$
assumes *F-lin*: *linearform UNIV F*
shows *bs-complex-extension F* $(cmult\text{-}lmat\ c\ x)$
$= c * bs\text{-}complex\text{-}extension\ F\ x$
⟨*proof*⟩

lemma *bs-complex-extension-clinear*:
fixes $F :: (complex, 'm :: finite, 'n :: finite)\ lmat \Rightarrow real$
assumes *F-lin*: *linearform UNIV F*
shows *clinear-functional-on UNIV* $(bs\text{-}complex\text{-}extension\ F)$
⟨*proof*⟩

The exact Bohnenblust-Sobczyk bound: any C bound on F lifts verbatim to a bound on *bs-complex-extension F*. This is the standard $e\hat{}\{-i\vartheta\}$-rotation argument used to prove the complex Hahn-Banach theorem from the real one.

lemma *bs-complex-extension-tight-bound*:
fixes $F :: (complex, 'm :: finite, 'n :: finite)\ lmat \Rightarrow real$

assumes *F-lin*: *linearform UNIV F*
and *F-C*: $\bigwedge x.\ |F\ x| \leq C * \textit{hs-norm-lmat}\ x$
shows *cmod* (*bs-complex-extension F x*) $\leq C * \textit{hs-norm-lmat}\ x$
⟨*proof*⟩

lemma *bs-complex-extension-bounded*:
fixes $F :: (\textit{complex}, 'm :: \textit{finite}, 'n :: \textit{finite})\ \textit{lmat} \Rightarrow \textit{real}$
assumes *F-lin*: *linearform UNIV F*
and *F-cont*: *continuous UNIV F hs-norm-lmat*
shows $\exists\, C \geq 0.\ \forall x.\ \textit{cmod}\ (\textit{bs-complex-extension}\ F\ x) \leq C * \textit{hs-norm-lmat}\ x$
⟨*proof*⟩

lemma *bs-complex-extension-extends*:
fixes $V :: (\textit{complex}, 'm :: \textit{finite}, 'n :: \textit{finite})\ \textit{lmat set}$
and $g :: (\textit{complex}, 'm, 'n)\ \textit{lmat} \Rightarrow \textit{complex}$
and $F :: (\textit{complex}, 'm, 'n)\ \textit{lmat} \Rightarrow \textit{real}$
assumes *g-clin*: *clinear-functional-on V g*
and *V-clos-i*: $\forall x \in V.\ \textit{cmult-lmat}\ \mathrm{i}\ x \in V$
and *F-ext*: $\forall x \in V.\ F\ x = \textit{re-part-lmat}\ g\ x$
shows $\forall x \in V.\ \textit{bs-complex-extension}\ F\ x = g\ x$
⟨*proof*⟩

27.5 The matrix-space Hahn-Banach theorem

Combine all the steps: a bounded complex-linear functional on a complex-linear subspace V of (*complex*, *'m*, *'n*) *lmat* extends to a bounded complex-linear functional on all of (*complex*, *'m*, *'n*) *lmat* (without claiming norm preservation; for the application to Arveson Extension this is enough together with the bound chase).

theorem *matrix-hahn-banach-lmat*:
fixes $V :: (\textit{complex}, 'm :: \textit{finite}, 'n :: \textit{finite})\ \textit{lmat set}$
and $g :: (\textit{complex}, 'm, 'n)\ \textit{lmat} \Rightarrow \textit{complex}$
assumes *V-csub*: *cmat-subspace-lmat V*
and *g-bnd*: *bounded-clinear-functional-on V g*
shows $\exists f :: (\textit{complex}, 'm, 'n)\ \textit{lmat} \Rightarrow \textit{complex}.$
bounded-clinear-functional-on UNIV f
$\wedge\ (\forall x \in V.\ f\ x = g\ x)$
⟨*proof*⟩

The explicit-bound form of the matrix Hahn-Banach theorem: any C-bound on $|\textit{Re} \circ g|$ over the subspace V lifts verbatim to a C-bound on $\textit{cmod} \circ f$ over the whole space.

The proof unfolds HOL/Hahn_Banach's locale-local *fn-norm* to its *Sup*-based definition, bounds the supremum on the subspace side by C via *g-re-C*, transports the bound to the extension side via *norm-Hahn-Banach*'s norm-preservation conclusion, then uses *bs-complex-extension-tight-bound* to turn the bound on the real extension into a bound on the complex extension.

theorem *matrix-hahn-banach-lmat-with-bound*:
fixes $V :: (\textit{complex}, 'm :: \textit{finite}, 'n :: \textit{finite})\ \textit{lmat set}$
and $g :: (\textit{complex}, 'm, 'n)\ \textit{lmat} \Rightarrow \textit{complex}$
and $C :: \textit{real}$
assumes *V-csub*: *cmat-subspace-lmat V*
and *g-clin*: *clinear-functional-on V g*
and *g-re-C*: $\forall x \in V.\ |\textit{Re}\ (g\ x)| \leq C * \textit{hs-norm-lmat}\ x$
and *C-nn*: $0 \leq C$
shows $\exists f :: (\textit{complex}, 'm, 'n)\ \textit{lmat} \Rightarrow \textit{complex}.$
clinear-functional-on UNIV f
$\wedge\ (\forall x \in V.\ f\ x = g\ x)$
$\wedge\ (\forall x.\ \textit{cmod}\ (f\ x) \leq C * \textit{hs-norm-lmat}\ x)$

⟨*proof*⟩

27.6 Arveson's correspondence between maps and functionals

Days 24–25: Arveson's correspondence between maps $\Phi : S \to M_d$ and functionals $S_\Phi : M_d(S) \to \mathbb{C}$, $S_\Phi((A_{ij})) = \frac{1}{d}\sum_{i,j} \Phi(A_{ij})_{ij}$, with inverse $f \mapsto \Phi_f$, $\Phi_f(M) = d\,(f(E_{ij} \otimes M))_{ij}$.

27.6.1 Finite sums of matrices

lemma *msum-in-os*:
assumes *OS*: *operator-system n OS* **and** *g*: $\forall\, x \in$ *set xs*. *g x* $\in$ *OS*
shows *msum n n g xs* $\in$ *OS*
⟨*proof*⟩

lemma *lin-on-zero*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ
shows Φ (*0*$_m$ *n n*) = *0*$_m$ *d d*
⟨*proof*⟩

lemma *lin-on-msum*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** *g*: $\forall\, x \in$ *set xs*. *g x* $\in$ *OS*
shows Φ (*msum n n g xs*) = *msum d d* (λ*x*. Φ (*g x*)) *xs*
⟨*proof*⟩

27.6.2 $E_{ij} \otimes M$ and the block decomposition

definition *unit-block* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
unit-block d n i j M = *Matrix.mat* (*d* ∗ *n*) (*d* ∗ *n*)
(λ(*r*,*c*). *if r div n* = *i* ∧ *c div n* = *j then M* $$ (*r mod n*, *c mod n*) *else 0*)

lemma *unit-block-carrier* [*simp*]: *unit-block d n i j M* ∈ *carrier-mat* (*d* ∗ *n*) (*d* ∗ *n*)
⟨*proof*⟩

lemma *unit-block-dims* [*simp*]: *dim-row* (*unit-block d n i j M*) = *d* ∗ *n dim-col* (*unit-block d n i j M*) = *d* ∗ *n*
⟨*proof*⟩

unit-block d n i j M is $E_{ij} \otimes M$.

lemma *unit-block-tensor*:
assumes *M*: *M* ∈ *carrier-mat n n* **and** *i*: *i* < *d* **and** *j*: *j* < *d*
shows *unit-block d n i j M* = *Tensor.tensor-mat* (*Matrix.mat d d* (λ(*a*,*b*). *if a* = *i* ∧ *b* = *j then 1 else 0*)) *M*
⟨*proof*⟩

lemma *block-unit-block*:
assumes *M*: *M* ∈ *carrier-mat n n* **and** *a*: *a* < *d* **and** *b*: *b* < *d*
shows *block-of-mat n* (*unit-block d n i j M*) *a b* = (*if a* = *i* ∧ *b* = *j then M else 0*$_m$ *n n*)
⟨*proof*⟩

lemma *unit-block-Mp*:
assumes *OS*: *operator-system n OS* **and** *M*: *M* ∈ *OS* **and** *i*: *i* < *d* **and** *j*: *j* < *d*
shows *unit-block d n i j M* ∈ *Mp d n OS*
⟨*proof*⟩

lemma *block-decomposition*:
assumes *A*: *A* ∈ *carrier-mat* (*d* ∗ *n*) (*d* ∗ *n*)

shows $A = msum\ (d * n)\ (d * n)\ (\lambda(i,j).\ unit\text{-}block\ d\ n\ i\ j\ (block\text{-}of\text{-}mat\ n\ A\ i\ j))\ (List.product\ [0..<d]\ [0..<d])$
⟨*proof*⟩

27.6.3 The correspondence

definition *arveson-functional* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒ *complex* **where**
arveson-functional n d Φ A = $(\sum i{<}d.\ \sum j{<}d.\ \Phi\ (block\text{-}of\text{-}mat\ n\ A\ i\ j)\ \$\$\ (i,j))\ /\ of\text{-}nat\ d$

definition *arveson-map* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
arveson-map n d f M = *Matrix.mat d d* $(\lambda(i,j).\ of\text{-}nat\ d * f\ (unit\text{-}block\ d\ n\ i\ j\ M))$

definition *lin-fun-on* :: *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex*) ⇒ *bool* **where**
lin-fun-on TS f ⟷ $(\forall A \in TS.\ \forall B \in TS.\ f\ (A + B) = f\ A + f\ B) \wedge (\forall A \in TS.\ \forall c.\ f\ (c\ \cdot_m\ A) = c * f\ A)$

definition *positive-fun-on* :: *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex*) ⇒ *bool* **where**
positive-fun-on TS f ⟷ $(\forall A \in TS.\ psd\ A \longrightarrow 0 \le f\ A)$

For $A = (A_{ij}) \in M_d(S)$, the functional of a map $\Phi : S \to M_d$ and the map of a functional f on $M_d(S)$ are

$$S_\Phi(A) = \frac{1}{d} \sum_{i,j<d} \Phi(A_{ij})_{ij}, \qquad \Phi_f(M)_{ij} = d\, f(E_{ij} \otimes M),$$

and they are mutually inverse (*arveson-functional-of-map*, *arveson-map-of-functional*).

lemma *amplification-idx*:
assumes *r*: $r < q$ **and** *s*: $s < q$ **and** *i*: $i < d$ **and** *j*: $j < d$
shows *amplification n d q Φ A* $\$\$\ (r * d + i,\ s * d + j) = \Phi\ (block\text{-}of\text{-}mat\ n\ A\ r\ s)\ \$\$\ (i,j)$
⟨*proof*⟩

lemma *arveson-map-index*: $i < d \Longrightarrow j < d \Longrightarrow$ *arveson-map n d f M* $\$\$\ (i,j) = of\text{-}nat\ d * f\ (unit\text{-}block\ d\ n\ i\ j\ M)$
⟨*proof*⟩

lemma *arveson-map-carrier* [*simp*]: *arveson-map n d f M* ∈ *carrier-mat d d*
⟨*proof*⟩

lemma *arveson-map-dims* [*simp*]: *dim-row* (*arveson-map n d f M*) = *d dim-col* (*arveson-map n d f M*) = *d*
⟨*proof*⟩

lemma *lin-fun-zero*:
assumes *TS*: *operator-system N TS* **and** *f*: *lin-fun-on TS f*
shows $f\ (0_m\ N\ N) = 0$
⟨*proof*⟩

lemma *lin-fun-msum*:
assumes *TS*: *operator-system N TS* **and** *f*: *lin-fun-on TS f* **and** *g*: $\forall x \in set\ xs.\ g\ x \in TS$
shows $f\ (msum\ N\ N\ g\ xs) = (\sum x{\leftarrow}xs.\ f\ (g\ x))$
⟨*proof*⟩

$S_{\Phi_f} = f$ on $M_d(S)$.

theorem *arveson-functional-of-map*:
assumes *OS*: *operator-system n OS* **and** *f*: *lin-fun-on* (*Mp d n OS*) *f* **and** *A*: *A* ∈ *Mp d n OS* **and** *d*: $0 < d$
shows *arveson-functional n d* (*arveson-map n d f*) *A* = *f A*

⟨*proof*⟩

$\Phi_{S_\Phi} = \Phi$ on S.

theorem *arveson-map-of-functional*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** *M*: $M \in OS$ **and** *d*: $0 < d$
shows *arveson-map n d* (*arveson-functional n d* Φ) *M* = Φ *M*
⟨*proof*⟩

lemma *unit-block-add*:
assumes *M*: $M \in$ *carrier-mat n n* **and** *M'*: $M' \in$ *carrier-mat n n*
shows *unit-block d n i j* (*M* + *M'*) = *unit-block d n i j M* + *unit-block d n i j M'*
⟨*proof*⟩

lemma *unit-block-smult*:
assumes *M*: $M \in$ *carrier-mat n n*
shows *unit-block d n i j* (*c* ·$_m$ *M*) = *c* ·$_m$ *unit-block d n i j M*
⟨*proof*⟩

lemma *arveson-map-lin*:
assumes *OS*: *operator-system n OS* **and** *f*: *lin-fun-on* (*Mp d n OS*) *f*
shows *lin-on n d OS* (*arveson-map n d f*)
⟨*proof*⟩

lemma *arveson-functional-lin*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ
shows *lin-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
⟨*proof*⟩

lemma *quad-split*:
fixes *M* :: *complex Matrix.mat* **and** *w* :: *complex Matrix.vec*
assumes *M*: $M \in$ *carrier-mat* (*q* ∗ *d*) (*q* ∗ *d*) **and** *w*: *dim-vec w* = *q* ∗ *d*
shows ⟨*w* | *M* ∗$_v$ *w*⟩ = (∑ *r*<*q*. ∑ *i*<*d*. ∑ *s*<*q*. ∑ *j*<*d*.
cnj (*w* $ (*r* ∗ *d* + *i*)) ∗ *M* $$ (*r* ∗ *d* + *i*, *s* ∗ *d* + *j*) ∗ *w* $ (*s* ∗ *d* + *j*))
⟨*proof*⟩

27.6.4 Theorem (Arveson): completely positive iff d-positive iff S_Φ positive

lemma *if-zero-mult*: (*if P then x else 0*) ∗ (*y*::*complex*) = (*if P then x* ∗ *y else 0*)
⟨*proof*⟩

lemma *mult-if-zero*: (*y*::*complex*) ∗ (*if P then x else 0*) = (*if P then y* ∗ *x else 0*)
⟨*proof*⟩

lemma *cnj-if-zero*: *cnj* (*if P then x else 0*) = (*if P then cnj x else 0*)
⟨*proof*⟩

lemma *nonneg-of-real-quad*:
fixes *z* :: *complex*
assumes *z* ∈ ℝ **and** 0 ≤ *Re z* **and** *d*: 0 < *d*
shows 0 ≤ *z* / *of-nat d*
⟨*proof*⟩

lemma *diag-quad-amplification*:
fixes *d n* :: *nat* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **and** *A* :: *complex Matrix.mat*
defines *v* ≡ *Matrix.vec* (*d* ∗ *d*) (λ*x*. *if x div d* = *x mod d then 1 else* (*0*::*complex*))
shows ⟨*v* | *amplification n d d* Φ *A* ∗$_v$ *v*⟩ = (∑ *i*<*d*. ∑ *j*<*d*. Φ (*block-of-mat n A i j*) $$ (*i*,*j*))
⟨*proof*⟩

lemma *dpos-imp-functional-pos*:
assumes *OS*: *operator-system n OS* **and** *d*: *0 < d* **and** *P*: *p-positive-on n d d OS* Φ
shows *positive-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
⟨*proof*⟩

lemma *conj-tensor-index*:
fixes *w* :: *complex Matrix.vec*
assumes *s*: *s < q* **and** *k*: *k < d* **and** *b'*: *b' < n* **and** *b*: *b < n*
defines *V* ≡ *Tensor.tensor-mat* (*Matrix.mat q d* (λ(*r*,*t*). *w* $ (*r* * *d* + *t*))) (*1*$_m$ *n*)
shows *V* $$ (*s* * *n* + *b'*, *k* * *n* + *b*) = (*if b' = b then w* $ (*s* * *d* + *k*) *else 0*)
⟨*proof*⟩

lemma *conj-block*:
fixes *A* :: *complex Matrix.mat* **and** *w* :: *complex Matrix.vec*
assumes *A*: *A* ∈ *carrier-mat* (*q* * *n*) (*q* * *n*) **and** *i*: *i < d* **and** *j*: *j < d* **and** *a*: *a < n* **and** *b*: *b < n*
defines *V* ≡ *Tensor.tensor-mat* (*Matrix.mat q d* (λ(*r*,*t*). *w* $ (*r* * *d* + *t*))) (*1*$_m$ *n*)
shows ((*V*†) * *A* * *V*) $$ (*i* * *n* + *a*, *j* * *n* + *b*)
= (∑ *r*<*q*. ∑ *s*<*q*. *cnj* (*w* $ (*r* * *d* + *i*)) * *w* $ (*s* * *d* + *j*) * *A* $$ (*r* * *n* + *a*, *s* * *n* + *b*))
⟨*proof*⟩

lemma *functional-pos-imp-cp*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** *d*: *0 < d*
and *Pos*: *positive-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
shows *cp-on n d OS* Φ
⟨*proof*⟩

Theorem (Arveson, Day 24): for linear $\Phi : S \to M_d$, completely positive $\iff$ d-positive $\iff$ S_Φ positive.

theorem *arveson-cp-iff*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** *d*: *0 < d*
shows *cp-on n d OS* Φ ⟷ *p-positive-on n d d OS* Φ
and *p-positive-on n d d OS* Φ ⟷ *positive-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
⟨*proof*⟩

Theorem (Arveson correspondence, Day 25): $\Phi \mapsto S_\Phi$ maps $CP(S, M_d)$ onto the positive functionals on $M_d(S)$, with inverse $f \mapsto \Phi_f$; it is affine and sends unital maps to states.

theorem *arveson-correspondence*:
assumes *OS*: *operator-system n OS* **and** *d*: *0 < d*
shows *lin-on n d OS* Φ ⟹ *cp-on n d OS* Φ ⟷ *positive-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
and *lin-fun-on* (*Mp d n OS*) *f* ⟹ *cp-on n d OS* (*arveson-map n d f*) ⟷ *positive-fun-on* (*Mp d n OS*) *f*
and *lin-on n d OS* Φ ⟹ *M* ∈ *OS* ⟹ *arveson-map n d* (*arveson-functional n d* Φ) *M* = Φ *M*
and *lin-fun-on* (*Mp d n OS*) *f* ⟹ *A* ∈ *Mp d n OS* ⟹ *arveson-functional n d* (*arveson-map n d f*) *A* = *f A*
⟨*proof*⟩

lemma *arveson-functional-affine*:
assumes *OS*: *operator-system n OS* **and** *L1*: *lin-on n d OS* Φ **and** *L2*: *lin-on n d OS* Ψ **and** *A*: *A* ∈ *Mp d n OS*
shows *arveson-functional n d* (λ*M*. *t* ·$_m$ Φ *M* + (*1* − *t*) ·$_m$ Ψ *M*) *A*
= *t* * *arveson-functional n d* Φ *A* + (*1* − *t*) * *arveson-functional n d* Ψ *A*
⟨*proof*⟩

lemma *arveson-functional-unital*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** *u*: Φ (*1*$_m$ *n*) = *1*$_m$ *d* **and** *d*: *0 < d*
shows *arveson-functional n d* Φ (*1*$_m$ (*d* * *n*)) = *1*
⟨*proof*⟩

27.7 A unital functional is positive iff it has norm one (D25-4)

Day 25 proposition (D25-4): for an operator system $S \subseteq M_n$ and a linear $f : S \to \mathbb{C}$ with $f(I) = 1$, f is positive iff $\|f\| = 1$. The operator norm is $\|A\| = \sup\{\|Av\| : \|v\| \le 1\}$ and, as in the notes, $\|f\|$ is defined for bounded f. The proof differs from the notes in one step: for $P \ge 0$ the bound $\|P - sI + itI\|^2 \le s^2 + t^2$ ($s = \|P\|/2$) comes from $P^2 \le \|P\|\,P$, which uses the square root of P, instead of from the spectrum of P. Cauchy–Schwarz is *cauchy-schwarz-ineq* of Isabelle_Marries_Dirac.

definition *op-norm* :: *nat* ⇒ *complex Matrix.mat* ⇒ *real* **where**
op-norm n A = *Sup* {‖*A* ∗*v v*‖ | *v*. *dim-vec v* = *n* ∧ ‖*v*‖ ≤ *1*}

definition *fun-norm* :: *nat* ⇒ *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex*) ⇒ *real* **where**
fun-norm n TS f = *Sup* {*cmod* (*f A*) | *A*. *A* ∈ *TS* ∧ *op-norm n A* ≤ *1*}

27.7.1 Vectors

lemma *vnorm-nonneg*: ‖*v*‖ ≥ *0*
⟨*proof*⟩

lemma *cauchy-schwarz-norm*:
assumes *dim-vec v* = *dim-vec w*
shows *cmod* ⟨*v*|*w*⟩ ≤ ‖*v*‖ ∗ ‖*w*‖
⟨*proof*⟩

lemma *inner-add-smult-expand*:
assumes *dim-vec u* = *dim-vec x*
shows ⟨*u* + *z* ·*v x* | *u* + *z* ·*v x*⟩ = ⟨*u*|*u*⟩ + *z* ∗ ⟨*u*|*x*⟩ + *cnj z* ∗ ⟨*x*|*u*⟩ + *cnj z* ∗ *z* ∗ ⟨*x*|*x*⟩
⟨*proof*⟩

lemma *smult-mat-vec*: *A* ∈ *carrier-mat n m* ⟹ *dim-vec v* = *m* ⟹ (*c* ·*m A*) ∗*v v* = *c* ·*v* (*A* ∗*v v*)
⟨*proof*⟩

lemma *entry-le-vnorm*: *i* < *dim-vec v* ⟹ *cmod* (*v* $ *i*) ≤ ‖*v*‖
⟨*proof*⟩

lemma *vnorm-le-sum*: ‖*w*‖ ≤ (∑ *i*<*dim-vec w*. *cmod* (*w* $ *i*))
⟨*proof*⟩

lemma *vnorm-zero-iff*: *dim-vec v* = *n* ⟹ ‖*v*‖ = *0* ⟷ *v* = *0v n*
⟨*proof*⟩

27.7.2 The operator norm

The operator norm $\|A\| = \sup\{\|Av\| : \|v\| \le 1\}$ is finite because $\|Av\| \le \big(\sum_{i,k} |a_{ik}|\big)\|v\|$. It satisfies $\|Av\| \le \|A\|\,\|v\|$, $\|I\| = 1$ and $\|cA\| \le |c|\,\|A\|$, and vanishes only at $A = 0$.

lemma *mult-vec-norm-bound*:
assumes *A*: *A* ∈ *carrier-mat n n* **and** *v*: *dim-vec v* = *n*
shows ‖*A* ∗*v v*‖ ≤ (∑ *i*<*n*. ∑ *k*<*n*. *cmod* (*A* $$ (*i*,*k*))) ∗ ‖*v*‖
⟨*proof*⟩

lemma *op-norm-bdd*:
assumes *A*: *A* ∈ *carrier-mat n n*
shows *bdd-above* {‖*A* ∗*v v*‖ | *v*. *dim-vec v* = *n* ∧ ‖*v*‖ ≤ *1*}

⟨*proof*⟩

lemma *mult-zero-vec'*: $A \in$ *carrier-mat n n* $\Longrightarrow$ $A *_v 0_v\ n = 0_v\ n$
⟨*proof*⟩

lemma *op-norm-ge*:
assumes *A*: $A \in$ *carrier-mat n n* **and** *v*: *dim-vec v* $= n$ $\|v\| \leq 1$
shows $\|A *_v v\| \leq$ *op-norm n A*
⟨*proof*⟩

lemma *op-norm-nonneg*:
assumes *A*: $A \in$ *carrier-mat n n*
shows $0 \leq$ *op-norm n A*
⟨*proof*⟩

lemma *op-norm-mult*:
assumes *A*: $A \in$ *carrier-mat n n* **and** *v*: *dim-vec v* $= n$
shows $\|A *_v v\| \leq$ *op-norm n A* $* \|v\|$
⟨*proof*⟩

lemma *op-norm-le*:
assumes *A*: $A \in$ *carrier-mat n n* **and** *c*: $0 \leq c$ **and** *b*: $\bigwedge v$. *dim-vec v* $= n \Longrightarrow \|A *_v v\| \leq c * \|v\|$
shows *op-norm n A* $\leq c$
⟨*proof*⟩

lemma *op-norm-one*:
assumes *n*: $0 < n$
shows *op-norm n* $(1_m\ n) = 1$
⟨*proof*⟩

lemma *op-norm-smult-le*:
assumes *A*: $A \in$ *carrier-mat n n*
shows *op-norm n* $(c \cdot_m A) \leq$ *cmod c* $*$ *op-norm n A*
⟨*proof*⟩

lemma *op-norm-zero-imp*:
assumes *A*: $A \in$ *carrier-mat n n* **and** *z*: *op-norm n A* $= 0$
shows $A = 0_m\ n\ n$
⟨*proof*⟩

27.7.3 Unital functionals

context
fixes *n* :: *nat* **and** *TS* :: *complex Matrix.mat set* **and** *f* :: *complex Matrix.mat* $\Rightarrow$ *complex*
assumes *OS*: *operator-system n TS* **and** *L*: *lin-fun-on TS f* **and** *f1*: $f\ (1_m\ n) = 1$
begin

lemma *TSc*: $A \in TS \Longrightarrow A \in$ *carrier-mat n n* ⟨*proof*⟩

lemma *lin2*: $A \in TS \Longrightarrow B \in TS \Longrightarrow f\ (a \cdot_m A + b \cdot_m B) = a * f\ A + b * f\ B$
⟨*proof*⟩

lemma *comb-in*: $A \in TS \Longrightarrow B \in TS \Longrightarrow a \cdot_m A + b \cdot_m B \in TS$
⟨*proof*⟩

lemma *f-zero*: $f\ (0_m\ n\ n) = 0$
⟨*proof*⟩

A Hermitian $A \in S$ with $|\Re\langle x|Ax\rangle| \leq c\|x\|^2$ satisfies $cI \pm A \geq 0$, so a positive f has $f(A)$ real

with $|f(A)| \leq c$.

lemma *herm-bound*:
assumes *pos*: *positive-fun-on TS f* **and** *A*: $A \in$ *TS hermitian A* **and** *c*: $0 \leq c$
and *b*: $\bigwedge x.$ *dim-vec* $x = n \Longrightarrow |Re\ \langle x|A *_v x\rangle| \leq c * \|x\|^2$
shows $Im\ (f\ A) = 0$ **and** $|Re\ (f\ A)| \leq c$
⟨*proof*⟩

lemma *adjoint-inner*:
assumes *A*: $A \in$ *carrier-mat n n* **and** *x*: *dim-vec* $x = n$
shows $\langle x|A^\dagger *_v x\rangle = cnj\ \langle x|A *_v x\rangle$
⟨*proof*⟩

($\Rightarrow$): a positive unital f has $|f(A)| \leq \|A\|$.

lemma *positive-imp-bound*:
assumes *pos*: *positive-fun-on TS f* **and** *A*: $A \in$ *TS*
shows *cmod* $(f\ A) \leq$ *op-norm n A*
⟨*proof*⟩

($\Leftarrow$): if $|f(A)| \leq \|A\|$ on S then f is positive.

lemma *sq-le-norm-quad*:
assumes *P*: $P \in$ *carrier-mat n n psd P* **and** *x*: *dim-vec* $x = n$
shows $\|P *_v x\|^2 \leq$ *op-norm n P* $*$ $Re\ \langle x|P *_v x\rangle$
⟨*proof*⟩

lemma *bound-imp-positive*:
assumes *n*: $0 < n$ **and** *b*: $\forall A \in$ *TS*. *cmod* $(f\ A) \leq$ *op-norm n A*
shows *positive-fun-on TS f*
⟨*proof*⟩

lemma *fun-norm-one-iff*:
assumes *n*: $0 < n$ **and** *bdd*: $\exists C.\ \forall A \in$ *TS*. *cmod* $(f\ A) \leq C *$ *op-norm n A*
shows *fun-norm n TS f* $= 1 \longleftrightarrow (\forall A \in$ *TS*. *cmod* $(f\ A) \leq$ *op-norm n A*$)$
⟨*proof*⟩

D25-4.

theorem *unital-positive-iff-norm-one*:
assumes *n*: $0 < n$ **and** *bdd*: $\exists C.\ \forall A \in$ *TS*. *cmod* $(f\ A) \leq C *$ *op-norm n A*
shows *positive-fun-on TS f* $\longleftrightarrow$ *fun-norm n TS f* $= 1$
⟨*proof*⟩

end

28 Day 26 (28 October 2011): Arveson's extension theorem; separability

Arveson's extension theorem. The notes extend the state S_Φ by Hahn–Banach (norm one) and use that a unital functional of norm one is positive. Here the positive functional S_Φ on the operator system $M_d(S)$ is extended to a positive functional on $M_d(M_n)$ directly, by separating the positive definite cone from the kernel of S_Φ (*separating-hyperplane-set-0* in the Euclidean space of typed matrices); the extension is $X \mapsto \mathrm{Tr}(DX)$ with $D \geq 0$. Dimensions are cardinalities of finite types, as in *Day22-UCP-Compact*.

28.1 Hermitian parts and traces of products

lemma *operator-system-herm-parts*:
assumes *OS*: *operator-system n OS* **and** *A*: $A \in OS$
shows $\exists\, Hp\ Kp.\ Hp \in OS \wedge Kp \in OS \wedge$ *hermitian Hp* $\wedge$ *hermitian Kp* $\wedge\ A = Hp + \mathrm{i} \cdot_m Kp$
⟨*proof*⟩

Traces of products: $\mathrm{Tr}(WM) \in \mathbb{R}$ for Hermitian W, M; $\mathrm{Tr}(A|v\rangle\langle v|) = \langle v|Av\rangle$; and for $D, P \geq 0$, writing $P = \sum_k v_k v_k^*$, $\mathrm{Tr}(DP) = \sum_k \langle v_k|Dv_k\rangle \geq 0$.

lemma *Tr-herm-mult-real*:
assumes *Wc*: $W \in$ *carrier-mat n n* **and** *Mc*: $M \in$ *carrier-mat n n* **and** *Wh*: *hermitian W* **and** *Mh*: *hermitian M*
shows *Tr* $(W * M) \in \mathbb{R}$
⟨*proof*⟩

lemma *Tr-mult-outer*:
fixes A :: *complex Matrix.mat* **and** v :: *complex Matrix.vec*
assumes *A*: $A \in$ *carrier-mat n n* **and** *v*: *dim-vec v* $= n$
shows *Tr* $(A * (v \oslash v)) = \langle v \mid A *_v v\rangle$
⟨*proof*⟩

lemma *Tr-psd-mult-nonneg*:
assumes *Dc*: $D \in$ *carrier-mat n n* **and** *Pc*: $P \in$ *carrier-mat n n* **and** *Dp*: *psd D* **and** *Pp*: *psd P*
shows $0 \leq$ *Tr* $(D * P)$
⟨*proof*⟩

lemma *herm-entry*:
assumes $M \in$ *carrier-mat n n hermitian M* $i < n\ j < n$
shows M \$\$ $(i,j) =$ *cnj* $(M$ \$\$ $(j,i))$
⟨*proof*⟩

lemma *herm-real-comb*:
assumes *Ac*: $A \in$ *carrier-mat n n* **and** *Bc*: $B \in$ *carrier-mat n n* **and** *Ah*: *hermitian A* **and** *Bh*: *hermitian B*
shows *hermitian* (*complex-of-real u* $\cdot_m$ *A* + *complex-of-real v* $\cdot_m$ *B*)
⟨*proof*⟩

Two facts about real inequalities: if $c + tb \geq 0$ for every $t \in \mathbb{R}$ then $b = 0$; if $x + \epsilon y \geq 0$ for every $\epsilon > 0$ then $x \geq 0$.

lemma *lin-t-zero*:
fixes $b\ c$ :: *real*
assumes $\forall t.\ 0 \leq c + t * b$
shows $b = 0$
⟨*proof*⟩

lemma *eps-nonneg*:
fixes $x\ y$ :: *real*
assumes $\forall \varepsilon{>}0.\ 0 \leq x + \varepsilon * y$
shows $0 \leq x$
⟨*proof*⟩

lemma *psd-one-mat*: *psd* $(1_m\ n)$
⟨*proof*⟩

28.2 Matrices as points of the Euclidean space *lmat*

lemma *wrap-lm-add*:
fixes $M\ M'$ :: *complex Matrix.mat*

assumes M: $M \in$ *carrier-mat CARD('k::finite) CARD('k)* **and** M': $M' \in$ *carrier-mat CARD('k) CARD('k)*
shows *(wrap-lm (M + M') :: (complex, 'k, 'k) lmat) = wrap-lm M + wrap-lm M'*
⟨*proof*⟩

lemma *wrap-lm-scale*:
fixes M :: *complex Matrix.mat*
assumes M: $M \in$ *carrier-mat CARD('k::finite) CARD('k)*
shows *(wrap-lm (complex-of-real r ·$_m$ M) :: (complex, 'k, 'k) lmat) = r *$_R$ wrap-lm M*
⟨*proof*⟩

lemma *wrap-lm-zero*: *(wrap-lm (0$_m$ CARD('k::finite) CARD('k)) :: (complex, 'k, 'k) lmat) = 0*
⟨*proof*⟩

28.3 Real functionals on typed matrices and Hermitian matrices

The real inner product on *(complex, 'm, 'n) lmat* is the real part of the JNF Hilbert-Schmidt inner product on the underlying matrices. For square matrices it is also $Re\ (Tr\ (M^\dagger * N))$.

lemma *inner-lmat-eq-re-hs-inner-prod*:
fixes *M N :: (complex, 'm :: finite, 'n :: finite) lmat*
shows *inner M N = Re ⟨⟨ unwrap-lm M | unwrap-lm N ⟩⟩*
⟨*proof*⟩

The inner-product-to-Re-trace bridge for hermitian arguments, extracted as a separate lemma so we can reuse it for the strict separation theorem. This is the same hermitisation construction that drives *entanglement-witness-hermitian*.

lemma *hermitian-witness-from-lmat*:
fixes *a :: (complex, 'k :: finite, 'k) lmat*
shows *∃ W :: complex Matrix.mat.*
W ∈ carrier-mat (CARD('k)) (CARD('k))
∧ hermitian W
∧ (∀ M :: (complex, 'k, 'k) lmat.
hermitian (unwrap-lm M)
*⟶ Re (Tr (W * unwrap-lm M)) = inner a M)*
⟨*proof*⟩

28.4 Extension of positive functionals

A positive functional on an operator system $T \subseteq M_N$ is $X \mapsto \mathrm{Tr}(DX)$ on T for some $D \geq 0$; hence it extends to a positive functional on M_N.

theorem *positive-functional-extension*:
fixes *TS :: complex Matrix.mat set* **and** *f :: complex Matrix.mat ⇒ complex*
assumes *TS*: *operator-system CARD('k::finite) TS* **and** *f*: *lin-fun-on TS f* **and** *fpos*: *positive-fun-on TS f*
shows *∃ D. D ∈ carrier-mat CARD('k) CARD('k) ∧ psd D ∧ (∀ A ∈ TS. f A = Tr (D * A))*
⟨*proof*⟩

28.5 Arveson's extension theorem

theorem *arveson-extension*:
fixes *OS :: complex Matrix.mat set* **and** *Φ :: complex Matrix.mat ⇒ complex Matrix.mat*
assumes *OS*: *operator-system CARD('n::finite) OS* **and** *L*: *lin-on CARD('n) CARD('d::finite) OS Φ*
and *CP*: *cp-on CARD('n) CARD('d) OS Φ*
shows *∃ Ψ. lin-on CARD('n) CARD('d) (carrier-mat CARD('n) CARD('n)) Ψ*
∧ completely-positive CARD('n) CARD('d) Ψ ∧ (∀ M ∈ OS. Ψ M = Φ M)
⟨*proof*⟩

Corollary: a completely positive map on an operator system has a Kraus form.

corollary *arveson-kraus*:
fixes *OS* :: *complex Matrix.mat set* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *OS*: *operator-system CARD('n::finite) OS* **and** *L*: *lin-on CARD('n) CARD('d::finite) OS* Φ
and *CP*: *cp-on CARD('n) CARD('d) OS* Φ
shows ∃ *Ws*. (∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat CARD('d) CARD('n)*) ∧ (∀ *M* ∈ *OS*. Φ *M* = *kraus-map-dim CARD('d) Ws M*)
⟨*proof*⟩

28.6 Factoring a CP map through a unital or a trace-preserving one (D26-1)

The i-th block embedding $B_i : \mathbb{C}^n \to \mathbb{C}^{kn}$. The B_i are isometries with orthogonal ranges, $\sum_i B_i B_i^* = I$, so a Kraus representation $\Phi(X) = \sum_i W_i X W_i^*$ gives $\Phi(X) = V^* \psi(X) V$ with $\psi(X) = \sum_i B_i X B_i^*$ unital and $V = \sum_i B_i W_i^*$.

definition *blk-emb* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
blk-emb k n i = *Matrix.mat* (*k* ∗ *n*) *n* (λ(*x*, *y*). *if x div n* = *i* ∧ *x mod n* = *y then 1 else 0*)

lemma *blk-emb-carrier* [*simp*]: *blk-emb k n i* ∈ *carrier-mat* (*k* ∗ *n*) *n*
⟨*proof*⟩

lemma *blk-emb-dims* [*simp*]: *dim-row* (*blk-emb k n i*) = *k* ∗ *n dim-col* (*blk-emb k n i*) = *n*
⟨*proof*⟩

lemma *blk-cond*:
fixes *x y i n* :: *nat*
assumes *y*: *y* < *n*
shows (*x div n* = *i* ∧ *x mod n* = *y*) ⟷ *x* = *i* ∗ *n* + *y*
⟨*proof*⟩

$J_i = e_i \otimes I_n : \mathbb{C}^n \to \mathbb{C}^k \otimes \mathbb{C}^n$ embeds $\mathbb{C}^n$ as the i-th block, and the blocks fill the space: $\sum_i J_i J_i^* = I_{kn}$.

lemma *blk-emb-gram-entry*:
assumes *x*: *x* < *k* ∗ *n* **and** *y*: *y* < *k* ∗ *n*
shows (*blk-emb k n i* ∗ ((*blk-emb k n i*)†)) $$ (*x*, *y*) =
(*if x div n* = *i* ∧ *y div n* = *i* ∧ *x mod n* = *y mod n then 1 else 0*)
⟨*proof*⟩

lemma *blk-emb-gram*:
foldr (λ*B acc*. *B* ∗ (*B*†) + *acc*) (*map* (*blk-emb k n*) [*0*..<*k*]) (*0*$_m$ (*k* ∗ *n*) (*k* ∗ *n*)) = *1*$_m$ (*k* ∗ *n*)
⟨*proof*⟩

lemma *kraus-compose-single*: *kraus-compose* [*A*] *Es* = *map* (λ*E*. *A* ∗ *E*) *Es*
⟨*proof*⟩

V has the blocks W_i^* stacked vertically, so $V^* B_i = W_i$.

definition *kraus-stack-dag* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* **where**
kraus-stack-dag n d Ws = *Matrix.mat* (*length Ws* ∗ *n*) *d* (λ(*x*, *c*). *cnj* (*Ws* ! (*x div n*) $$ (*c*, *x mod n*)))

lemma *kraus-stack-dag-blk*:
assumes *Wc*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n* **and** *i*: *i* < *length Ws*
shows (*kraus-stack-dag n d Ws*)† ∗ *blk-emb* (*length Ws*) *n i* = *Ws* ! *i*
⟨*proof*⟩

Proposition (Day 26): a CP map $\Phi : S \to M_d$ on an operator system with $\Phi(I) = P$ is $\Phi(X) = V^* \psi(X) V$ with ψ unital and completely positive (here even on all of M_n) and $V^* V = P$. The notes compress Φ to the range of P; here the Kraus form of *arveson-kraus* is used.

theorem *cp-unital-factor*:
fixes *OS* :: *complex Matrix.mat set* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *OS*: *operator-system CARD('n::finite) OS* **and** *L*: *lin-on CARD('n) CARD('d::finite) OS* Φ
and *CP*: *cp-on CARD('n) CARD('d) OS* Φ
shows ∃ *r V ψ*. *0* < *r* ∧ *V* ∈ *carrier-mat r CARD('d)* ∧ *ψ* ∈ *UCP CARD('n) r*
∧ *lin-on CARD('n) r* (*carrier-mat CARD('n) CARD('n)*) *ψ*
∧ (∀ *X* ∈ *OS*. Φ *X* = (*V*†) ∗ *ψ X* ∗ *V*) ∧ (*V*†) ∗ *V* = Φ (*1*$_m$ *CARD('n)*)
⟨*proof*⟩

V with the blocks W_i stacked vertically, so $B_i^* V = W_i$.

definition *kraus-stack* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* **where**
kraus-stack n d Ws = *Matrix.mat* (*length Ws* ∗ *d*) *n* (λ(*x*, *c*). *Ws* ! (*x div d*) $$ (*x mod d*, *c*))

lemma *kraus-stack-blk*:
assumes *Wc*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n* **and** *i*: *i* < *length Ws*
shows (*blk-emb* (*length Ws*) *d i*)† ∗ *kraus-stack n d Ws* = *Ws* ! *i*
⟨*proof*⟩

Proposition (Day 26): a CP map $\Phi : M_n \to M_d$ is $\Phi(X) = \psi(VXV^*)$ with $\psi : M_r \to M_d$ completely positive and trace preserving. The notes dualise the previous proposition; here $\psi(Y) = \sum_i B_i^* Y B_i$ and $V = \sum_i B_i W_i$ directly.

theorem *cp-tp-factor*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *L*: *lin-on CARD('n::finite) CARD('d::finite)* (*carrier-mat CARD('n) CARD('n)*) Φ
and *CP*: *completely-positive CARD('n) CARD('d)* Φ
shows ∃ *r V ψ*. *0* < *r* ∧ *V* ∈ *carrier-mat r CARD('n)* ∧ *ψ* ∈ *CPTP r CARD('d)*
∧ *lin-on r CARD('d)* (*carrier-mat r r*) *ψ*
∧ (∀ *X* ∈ *carrier-mat CARD('n) CARD('n)*. Φ *X* = *ψ* (*V* ∗ *X* ∗ (*V*†)))
⟨*proof*⟩

28.7 Separability

Entanglement revisited. Ensembles $\{\psi_i, p_i\}$ on $\mathcal{H}_A$ and $\{\phi_j, q_j\}$ on $\mathcal{H}_B$ compose to the density matrix $\sum_{i,j} p_i q_j |\psi_i\rangle\langle\psi_i| \otimes |\phi_j\rangle\langle\phi_j|$.

Definition (notes). A density matrix $P \in L(\mathcal{H}_A \otimes \mathcal{H}_B)$ is *separable* if $P = \sum_l p_l |\psi_l\rangle\langle\psi_l| \otimes |\phi_l\rangle\langle\phi_l|$ with unit vectors $\psi_l \in \mathcal{H}_A$, $\phi_l \in \mathcal{H}_B$, $p_l \geq 0$, $\sum p_l = 1$; otherwise P is *entangled*.

Are there entangled density matrices, and how can one tell? Days 27 and 28 answer with positive maps and convex cones.

The matrices are typed (*lmat*, dimensions given by finite types), so that the topology and the separation theorems of HOL-Analysis apply: *separable-cone-lmat* is the convex cone generated by the tensor products of positive matrices (the notes' *Sep* of Day 27), and *Sprime* is the set of product density matrices. For a density matrix, membership in the cone is the definition above, a convex combination of product density matrices (*separable-density-iff-in-convex-hull-Sprime*, proved on Day 28 with the topology).

28.7.1 Typed tensor product

Two square matrices with type-level dimensions $'m$, $'p$ tensor to a square matrix with type-level dimension $'m \times 'p$.

lemma *tensor-mat-carrier-lmat*:
fixes *A B* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat* (*CARD('m* :: *finite)*) (*CARD('m)*)

and B: $B \in$ *carrier-mat* (*CARD*('*p* :: *finite*)) (*CARD*('*p*))
shows $A \otimes B \in$ *carrier-mat* (*CARD*('*m* × '*p*)) (*CARD*('*m* × '*p*))
⟨*proof*⟩

definition *tensor-lmat* ::
(*complex*, '*m* :: *finite*, '*m*) *lmat* ⇒ (*complex*, '*p* :: *finite*, '*p*) *lmat*
⇒ (*complex*, '*m* × '*p*, '*m* × '*p*) *lmat* **where**
tensor-lmat σ τ = *wrap-lm* (*unwrap-lm* $\sigma \otimes$ *unwrap-lm* τ)

lemma *unwrap-lm-tensor-lmat* [*simp*]:
fixes σ :: (*complex*, '*m* :: *finite*, '*m*) *lmat*
and τ :: (*complex*, '*p* :: *finite*, '*p*) *lmat*
shows *unwrap-lm* (*tensor-lmat* σ τ :: (*complex*, '*m* × '*p*, '*m* × '*p*) *lmat*)
= *unwrap-lm* $\sigma \otimes$ *unwrap-lm* τ
⟨*proof*⟩

28.7.2 The separable cone

The set of *separable matrices*: all nonneg combinations of tensor products of typed PSD matrices. Equivalently, the convex conic hull of the product PSD set.

definition *separable-lmat-gen* :: (*complex*, '*m* :: *finite* × '*p* :: *finite*,
'*m* × '*p*) *lmat set* **where**
separable-lmat-gen =
{*tensor-lmat* σ τ | σ τ.
(*psd-lmat* σ :: *bool*) ∧ (*psd-lmat* τ :: *bool*)}

definition *separable-cone-lmat* :: (*complex*, '*m* :: *finite* × '*p* :: *finite*,
'*m* × '*p*) *lmat set* **where**
separable-cone-lmat = *convex-cone hull separable-lmat-gen*

Sanity: the cone contains 0, is convex, and is conic.

lemma *separable-cone-lmat-convex-cone*:
convex-cone (*separable-cone-lmat*
:: (*complex*, '*m* :: *finite* × '*p* :: *finite*, '*m* × '*p*) *lmat set*)
⟨*proof*⟩

lemma *zero-in-separable-cone-lmat*:
(*0* :: (*complex*, '*m* :: *finite* × '*p* :: *finite*, '*m* × '*p*) *lmat*)
∈ *separable-cone-lmat*
⟨*proof*⟩

28.7.3 The unit-trace product slice

definition *Sprime* :: (*complex*, '*m* :: *finite* × '*p* :: *finite*, '*m* × '*p*) *lmat set*
where
Sprime =
{ϱ. ∃(σ :: (*complex*, '*m*, '*m*) *lmat*) (τ :: (*complex*, '*p*, '*p*) *lmat*).
ϱ = *tensor-lmat* σ τ
∧ *psd-lmat* σ ∧ *Tr-lmat* σ = *1*
∧ *psd-lmat* τ ∧ *Tr-lmat* τ = *1*}

lemma *Sprime-image*:
(*Sprime* :: (*complex*, '*m* :: *finite* × '*p* :: *finite*, '*m* × '*p*) *lmat set*)
= (λ(σ, τ). *tensor-lmat* σ τ) ‘
({σ :: (*complex*, '*m*, '*m*) *lmat*. *psd-lmat* σ ∧ *Tr-lmat* σ = *1*}
× {τ :: (*complex*, '*p*, '*p*) *lmat*. *psd-lmat* τ ∧ *Tr-lmat* τ = *1*})
⟨*proof*⟩

29 Day 27 (31 October 2011): positive maps detect entanglement; convex cones

For $R \in M_r(M_n)$ (block size n), $\Phi^{(r)}(R) \geq 0$ for every positive Φ when R is separable (D27-1), so a positive map with $\Phi^{(r)}(R) \not\geq 0$ shows that R is entangled; the transpose does so for the notes' examples (D27-2). A functional f on $M_r(M_n)$ is non-negative on the separable matrices iff Φ_f is positive (D27-3). That every entangled R is detected by some positive map needs convex cones:

Theorem (notes). Let V be a finite-dimensional real vector space, $K \subseteq V$ closed and convex, $y \notin K$. Then there is a linear $f : V \to \mathbb{R}$ and $\alpha \in \mathbb{R}$ with $f(y) < \alpha \leq f(K)$; if C is a closed cone and $w \notin C$ there is f with $f(w) < 0 \leq f(C)$.

This is HOL-Analysis' *separating-hyperplane-closed-point*; Day 28 applies it to the separable cone, once the cone is shown to be closed.

29.1 Tensor products with a matrix in the first factor

lemma *tensor-add-first*:
fixes *A B P* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat r r* **and** *B*: *B* ∈ *carrier-mat r r* **and** *P*: *P* ∈ *carrier-mat n n* **and** *n*: *0* < *n*
shows *Tensor.tensor-mat* (*A* + *B*) *P* = *Tensor.tensor-mat A P* + *Tensor.tensor-mat B P*
⟨*proof*⟩

lemma *tensor-zero-first*:
fixes *P* :: *complex Matrix.mat*
assumes *P*: *P* ∈ *carrier-mat n n* **and** *n*: *0* < *n*
shows *Tensor.tensor-mat* (*0*$_m$ *r r*) *P* = *0*$_m$ (*r* ∗ *n*) (*r* ∗ *n*)
⟨*proof*⟩

lemma *tensor-msum-first*:
fixes *P* :: *complex Matrix.mat*
assumes *g*: ∀ *x* ∈ *set xs*. *g x* ∈ *carrier-mat r r* **and** *P*: *P* ∈ *carrier-mat n n* **and** *n*: *0* < *n*
shows *Tensor.tensor-mat* (*msum r r g xs*) *P* = *msum* (*r* ∗ *n*) (*r* ∗ *n*) (λ*x*. *Tensor.tensor-mat* (*g x*) *P*) *xs*
⟨*proof*⟩

29.2 The quadratic form of Φ_f

definition *rank1* :: *complex Matrix.vec* ⇒ *complex Matrix.mat* **where**
rank1 w = *Matrix.mat* (*dim-vec w*) (*dim-vec w*) (λ(*i*,*j*). *cnj* (*w* $ *i*) ∗ *w* $ *j*)

lemma *rank1-carrier* [*simp*]: *rank1 w* ∈ *carrier-mat* (*dim-vec w*) (*dim-vec w*)
⟨*proof*⟩

lemma *rank1-dims* [*simp*]: *dim-row* (*rank1 w*) = *dim-vec w dim-col* (*rank1 w*) = *dim-vec w*
⟨*proof*⟩

rank1 w has entries $\overline{w_i}\, w_j$: it is $|u\rangle\langle u|$ for $u = \overline{w}$, hence positive.

lemma *rank1-outer*: *rank1* (*conjugate u*) = *u* ⊘ *u*
⟨*proof*⟩

lemma *rank1-psd*: *psd* (*rank1 w*)
⟨*proof*⟩

lemma *quad-arveson-map*:
fixes f :: *complex Matrix.mat* ⇒ *complex* **and** w :: *complex Matrix.vec*
assumes f: *lin-fun-on* (*carrier-mat* $(r * n)$ $(r * n)$) f **and** w: *dim-vec* $w = r$ **and** P: $P \in$ *carrier-mat* n n **and** n: $0 < n$
shows $\langle w \mid$ *arveson-map* n r f $P *_v w\rangle$ = *of-nat* $r * f$ (*Tensor.tensor-mat* (*rank1* w) P)
⟨*proof*⟩

29.3 $f(\mathrm{Sep}) \geq 0$ iff Φ_f is positive (D27-3)

theorem *positive-map-iff-sep-nonneg*:
fixes f :: *complex Matrix.mat* ⇒ *complex*
assumes f: *lin-fun-on* (*carrier-mat* $(r * n)$ $(r * n)$) f **and** r: $0 < r$ **and** n: $0 < n$
shows *positive-map* n r (*arveson-map* n r f)
$\longleftrightarrow$ $(\forall Q\ P.\ Q \in$ *carrier-mat* r $r \longrightarrow$ *psd* $Q \longrightarrow P \in$ *carrier-mat* n $n \longrightarrow$ *psd* P
$\longrightarrow 0 \leq f$ (*Tensor.tensor-mat* Q P))
⟨*proof*⟩

29.4 Positive maps keep separable matrices positive (D27-1)

lemma *block-tensor*:
fixes Q P :: *complex Matrix.mat*
assumes Q: $Q \in$ *carrier-mat* r r **and** P: $P \in$ *carrier-mat* n n **and** a: $a < r$ **and** b: $b < r$
shows *block-of-mat* n (*Tensor.tensor-mat* Q P) a b = Q $\$\$$ (a,b) $\cdot_m$ P
⟨*proof*⟩

lemma *amplification-tensor*:
fixes Q P :: *complex Matrix.mat* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes L: *lin-on* n d (*carrier-mat* n n) Φ **and** Q: $Q \in$ *carrier-mat* r r **and** P: $P \in$ *carrier-mat* n n
and d: $0 < d$ **and** n: $0 < n$
shows *amplification* n d r Φ (*Tensor.tensor-mat* Q P) = *Tensor.tensor-mat* Q (Φ P)
⟨*proof*⟩

lemma *amplification-lin*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes L: *lin-on* n d (*carrier-mat* n n) Φ **and** A: $A \in$ *carrier-mat* $(p * n)$ $(p * n)$ **and** B: $B \in$ *carrier-mat* $(p * n)$ $(p * n)$
shows *amplification* n d p Φ $(A + B)$ = *amplification* n d p Φ A + *amplification* n d p Φ B
and *amplification* n d p Φ $(c \cdot_m A)$ = $c \cdot_m$ *amplification* n d p Φ A
⟨*proof*⟩

theorem *positive-map-separable*:
fixes R :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m \times 'p$) *lmat* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes L: *lin-on CARD*($'p$) d (*carrier-mat CARD*($'p$) *CARD*($'p$)) Φ **and** *pos*: *positive-map CARD*($'p$) d Φ
and d: $0 < d$ **and** R: $R \in$ *separable-cone-lmat*
shows *psd* (*amplification CARD*($'p$) d *CARD*($'m$) Φ (*unwrap-lm* R))
⟨*proof*⟩

29.5 Entangled matrices detected by the transpose (D27-2)

lemma *sum-two-support*:
fixes g :: *nat* ⇒ *complex*
assumes $p \neq q$ $p < m$ $q < m$ $\bigwedge x.\ x < m \Longrightarrow x \neq p \Longrightarrow x \neq q \Longrightarrow g\ x = 0$
shows $(\sum x = 0..<m.\ g\ x) = g\ p + g\ q$
⟨*proof*⟩

lemma *transpose-lin-on*: *lin-on* n n (*carrier-mat* n n) (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*)

⟨*proof*⟩

The partial transpose of $(E_{f(a)f(b)})$ is $(E_{f(b)f(a)})$; its quadratic form at $e_0 \otimes e_{f(1)} - e_1 \otimes e_{f(0)}$ is -2.

lemma *unit-blocks-partial-transpose-not-psd*:
assumes *r*: $2 \le r$ **and** *f*: $f\ 0 < n\ f\ 1 < n\ f\ 0 \neq f\ 1$
shows ¬ *psd* (*amplification n n r* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) (*unit-blocks r n f*))
⟨*proof*⟩

theorem *unit-blocks-entangled*:
fixes *R* :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m \times 'p$) *lmat*
assumes *R*: *unwrap-lm R* = *unit-blocks CARD*($'m$) *CARD*($'p$) *f*
and *r*: $2 \le$ *CARD*($'m$) **and** *f*: $f\ 0 <$ *CARD*($'p$) $f\ 1 <$ *CARD*($'p$) $f\ 0 \neq f\ 1$
shows *psd* (*unwrap-lm R*) ∧ *R* ∉ *separable-cone-lmat*
⟨*proof*⟩

The notes' first example: $\begin{pmatrix} E_{ii} & E_{ij} \\ E_{ji} & E_{jj} \end{pmatrix} \in M_2(M_n)$, $i \neq j$.

corollary *entangled-example-2*:
fixes *R* :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m \times 'p$) *lmat*
assumes *m*: *CARD*($'m$) = 2 **and** *ij*: $i <$ *CARD*($'p$) $j <$ *CARD*($'p$) $i \neq j$
and *R*: *unwrap-lm R* = *unit-blocks 2 CARD*($'p$) (λ*a*. *if a* = 0 *then i else j*)
shows *block-of-mat CARD*($'p$) (*unwrap-lm R*) 0 0 = *E*⟦*i*,*i* ; *CARD*($'p$),*CARD*($'p$)⟧
and *block-of-mat CARD*($'p$) (*unwrap-lm R*) 0 1 = *E*⟦*i*,*j* ; *CARD*($'p$),*CARD*($'p$)⟧
and *block-of-mat CARD*($'p$) (*unwrap-lm R*) 1 0 = *E*⟦*j*,*i* ; *CARD*($'p$),*CARD*($'p$)⟧
and *block-of-mat CARD*($'p$) (*unwrap-lm R*) 1 1 = *E*⟦*j*,*j* ; *CARD*($'p$),*CARD*($'p$)⟧
and *psd* (*unwrap-lm R*) ∧ *R* ∉ *separable-cone-lmat*
⟨*proof*⟩

The second: $(E_{ij}) \in M_n(M_n)$, $n \ge 2$, which is *choi-seed*.

corollary *entangled-example-choi-seed*:
fixes *R* :: (*complex*, $'p$ :: *finite* × $'p$, $'p \times 'p$) *lmat*
assumes *n*: $2 \le$ *CARD*($'p$) **and** *R*: *unwrap-lm R* = *choi-seed CARD*($'p$)
shows *psd* (*unwrap-lm R*) ∧ *R* ∉ *separable-cone-lmat*
⟨*proof*⟩

30 Day 28 (2 November 2011): entanglement witnesses; the Horodecki–Peres criterion

The separable matrices form a closed cone C in the real vector space of Hermitian matrices; an entangled R is separated from C by a real functional, which complexifies to $\tilde{f}(H + iK) = f(H) + if(K)$, and $\tilde{f}$ corresponds to a Hermitian matrix W with $\mathrm{Tr}(WR) < 0 \le \mathrm{Tr}(W\sigma)$ for all separable σ: an *entanglement witness*. The notes state the criterion through the positive maps Φ_f of Day 27 (D28-3). Formalised below: (i) every non-separable Hermitian ρ has a witness; (ii) density matrices have a witness with a strict margin; (iii) (Horodecki) ρ is separable iff every Hermitian W that is non-negative on the cone is non-negative on ρ; the positive-map criterion; and, for two qubits, the Horodecki–Peres criterion (D28-5). The universal-witness question is not formalised.

Formally we work in the typed matrix space (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*, a real inner-product space (Lmat_Euclidean), and use HOL-Analysis' *separating-hyperplane-closed-point*. Steps: the positive matrices form a closed set; the unit-trace positive matrices form a compact set; the tensor product is continuous, so the product density matrices *Sprime* form a compact

set; its convex hull is compact and does not contain 0, and the separable cone is the cone over it, hence closed. Separation then yields a real functional, which is converted into a Hermitian witness W with $\Re \operatorname{Tr}(W\rho) < 0 \leq \Re \operatorname{Tr}(W\sigma)$ on the cone.

30.1 Tensor continuity

The typed tensor product is jointly continuous in both factors.

lemma *continuous-on-tensor-lmat*:
  fixes T :: ((*complex*, $'m$ :: *finite*, $'m$) *lmat*
    × (*complex*, $'p$ :: *finite*, $'p$) *lmat*) *set*
  shows *continuous-on* T ($\lambda(\sigma, \tau)$. *tensor-lmat* σ τ
    :: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*)
⟨*proof*⟩

lemma *compact-Sprime*:
  compact (*Sprime* :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m \times 'p$) *lmat set*)
⟨*proof*⟩

Reindex $\{0..<n*q\}$ via (div, mod) into $\{0..<n\} \times \{0..<q\}$.

lemma *sum-div-mod-split*:
  fixes f :: *nat* ⇒ *nat* ⇒ $'a$ :: *comm-monoid-add*
  assumes *q-pos*: $0 < q$
  shows ($\sum a = 0..<n.\ \sum b = 0..<q.\ f\ a\ b$)
    = ($\sum i = 0..<n * q.\ f\ (i\ div\ q)\ (i\ mod\ q)$)
⟨*proof*⟩

lemma *Tr-lmat-tensor*:
  fixes σ :: (*complex*, $'m$ :: *finite*, $'m$) *lmat*
    and τ :: (*complex*, $'p$ :: *finite*, $'p$) *lmat*
  shows *Tr-lmat* (*tensor-lmat* σ τ :: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*)
    = *Tr-lmat* σ ∗ *Tr-lmat* τ
⟨*proof*⟩

30.2 Closedness of the separable cone

Why the separable cone is closed. Let $S' = \{\sigma \otimes \tau :\ \sigma, \tau \geq 0,\ \operatorname{Tr}\sigma = \operatorname{Tr}\tau = 1\}$.

1. S' is compact: the positive matrices of trace 1 form a closed and bounded set ($\|\sigma\|_{HS} \leq \operatorname{Tr}\sigma = 1$), and $\otimes$ is continuous.

2. Its convex hull K is compact, and $0 \notin K$ since every element of K has trace 1.

3. For $\sigma, \tau \geq 0$, $\sigma \otimes \tau = \operatorname{Tr}\sigma\ \operatorname{Tr}\tau \cdot \frac{\sigma}{\operatorname{Tr}\sigma} \otimes \frac{\tau}{\operatorname{Tr}\tau}$ if both traces are positive, and $\sigma \otimes \tau = 0$ otherwise (a positive matrix of trace 0 is 0).

4. So the separable cone is $\{t\,k :\ t \geq 0,\ k \in K\}$, and the cone over a compact convex set not containing 0 is closed.

The convex cone hulls of *separable-lmat-gen* and *Sprime* coincide: every *tensor-lmat* σ τ with σ, τ PSD can be rescaled into *Sprime* by dividing by traces (when nonzero) or replaced by 0 when a trace is zero.

The hard inclusion uses step 3 above: positivity is preserved by non-negative scaling, a positive matrix of trace 0 is 0, and $\otimes$ is bilinear.

lemma *Sprime-subset-separable-lmat-gen*:

(*Sprime* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat set*)
⊆ *separable-lmat-gen*
⟨*proof*⟩

The four spectral / bilinearity helpers needed for the hard direction. Each is a one-line lift of a JNF fact, but the JNF facts (spectral closure and tensor scaleR) live outside the parent heap so they are stated as targeted sorries here for the moment.

lemma *psd-lmat-Tr-real-nonneg*:
fixes *M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*
assumes *M-psd*: *psd-lmat M*
shows *Im* (*Tr-lmat M*) = *0* ∧ *0* ≤ *Re* (*Tr-lmat M*)
⟨*proof*⟩

lemma *psd-lmat-zero-trace-zero*:
fixes *M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*
assumes *M-psd*: *psd-lmat M* **and** *tr-zero*: *Tr-lmat M* = *0*
shows *M* = *0*
⟨*proof*⟩

lemma *unwrap-lm-scaleR-eq*:
fixes *M* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat* **and** *a* :: *real*
shows *unwrap-lm* (*a* ∗$_R$ *M*) = *complex-of-real a* ·$_m$ *unwrap-lm M*
⟨*proof*⟩

lemma *psd-lmat-scaleR-nonneg*:
fixes *M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat* **and** *a* :: *real*
assumes *M-psd*: *psd-lmat M* **and** *a-nn*: *0* ≤ *a*
shows *psd-lmat* (*a* ∗$_R$ *M*)
⟨*proof*⟩

lemma *tensor-lmat-scaleR-left*:
fixes *a* :: *real*
and *σ* :: (*complex*, *'m* :: *finite*, *'m*) *lmat*
and *τ* :: (*complex*, *'p* :: *finite*, *'p*) *lmat*
shows (*tensor-lmat* (*a* ∗$_R$ *σ*) *τ*
:: (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat*)
= *a* ∗$_R$ *tensor-lmat σ τ*
⟨*proof*⟩

lemma *tensor-lmat-scaleR-right*:
fixes *a* :: *real*
and *σ* :: (*complex*, *'m* :: *finite*, *'m*) *lmat*
and *τ* :: (*complex*, *'p* :: *finite*, *'p*) *lmat*
shows (*tensor-lmat σ* (*a* ∗$_R$ *τ*)
:: (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat*)
= *a* ∗$_R$ *tensor-lmat σ τ*
⟨*proof*⟩

lemma *tensor-lmat-zero-left*:
fixes *τ* :: (*complex*, *'p* :: *finite*, *'p*) *lmat*
shows (*tensor-lmat* (*0* :: (*complex*, *'m* :: *finite*, *'m*) *lmat*) *τ*
:: (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat*) = *0*
⟨*proof*⟩

lemma *tensor-lmat-zero-right*:
fixes *σ* :: (*complex*, *'m* :: *finite*, *'m*) *lmat*
shows (*tensor-lmat σ* (*0* :: (*complex*, *'p* :: *finite*, *'p*) *lmat*)
:: (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat*) = *0*

⟨*proof*⟩

lemma *convex-cone-hull-separable-eq-Sprime*:
(*convex-cone hull*
(*separable-lmat-gen*
:: (*complex*, 'm :: *finite* × 'p :: *finite*, 'm × 'p) *lmat set*))
= *convex-cone hull Sprime*
⟨*proof*⟩

theorem *separable-cone-lmat-closed*:
closed (*separable-cone-lmat*
:: (*complex*, 'm :: *finite* × 'p :: *finite*, 'm × 'p) *lmat set*)
⟨*proof*⟩

30.3 The lmat inner product as a real-trace inner product

lemma *inner-lmat-eq-re-tr-dagger*:
fixes *M N* :: (*complex*, 'k :: *finite*, 'k) *lmat*
shows *inner M N* = *Re* (*Tr* ((*unwrap-lm M*)† ∗ *unwrap-lm N*))
⟨*proof*⟩

30.4 Separating hyperplane for non-separable states

Separation. The separable cone C is closed and convex, so a point $\rho \notin C$ is strictly separated from it by a linear functional $x \mapsto \langle a, x\rangle$ and a constant b: $\langle a, \rho\rangle < b < \langle a, x\rangle$ for $x \in C$. Since C is a cone, $b < 0$ and $\langle a, x\rangle \geq 0$ on C.

theorem *separable-cone-lmat-separating-hyperplane*:
fixes *ϱ* :: (*complex*, 'm :: *finite* × 'p :: *finite*, 'm × 'p) *lmat*
assumes *notin*: *ϱ* ∉ *separable-cone-lmat*
shows ∃ *a* :: (*complex*, 'm × 'p, 'm × 'p) *lmat*.
inner a ϱ < *0*
∧ (∀ *σ* ∈ *separable-cone-lmat*. *0* ≤ *inner a σ*)
⟨*proof*⟩

30.5 Entanglement witness in real-trace form

Since $\langle a, x\rangle = \operatorname{Re}\operatorname{Tr}(a^* x)$, the separating functional is a matrix W with $\operatorname{Re}\operatorname{Tr}(W^*\rho) < 0 \leq \operatorname{Re}\operatorname{Tr}(W^*\sigma)$ for every separable σ.

theorem *entanglement-witness-re-tr*:
fixes *ϱ* :: (*complex*, 'm :: *finite* × 'p :: *finite*, 'm × 'p) *lmat*
assumes *notin*: *ϱ* ∉ *separable-cone-lmat*
shows ∃ *W* :: *complex Matrix.mat*.
W ∈ *carrier-mat* (*CARD*('m × 'p)) (*CARD*('m × 'p))
∧ *Re* (*Tr* (*W* ∗ *unwrap-lm ϱ*)) < *0*
∧ (∀ *σ* :: (*complex*, 'm × 'p, 'm × 'p) *lmat*.
σ ∈ *separable-cone-lmat*
⟶ *0* ≤ *Re* (*Tr* (*W* ∗ *unwrap-lm σ*)))
⟨*proof*⟩

30.6 Hermitian entanglement witness

If ρ is Hermitian the witness can be taken Hermitian: replace W by $\frac{1}{2}(W + W^*)$, which leaves $\operatorname{Re}\operatorname{Tr}(W^*\sigma)$ unchanged for Hermitian σ.

theorem *entanglement-witness-hermitian*:
fixes *ϱ* :: (*complex*, 'm :: *finite* × 'p :: *finite*, 'm × 'p) *lmat*
assumes *notin*: *ϱ* ∉ *separable-cone-lmat*

and ϱ-*herm*: *hermitian* (*unwrap-lm* ϱ)
shows $\exists\, W ::$ *complex Matrix.mat*.
$W \in$ *carrier-mat* (*CARD*('*m* × '*p*)) (*CARD*('*m* × '*p*))
$\wedge$ *hermitian W*
$\wedge$ *Re* (*Tr* (*W* ∗ *unwrap-lm* ϱ)) < *0*
$\wedge$ ($\forall \sigma ::$ (*complex*, '*m* × '*p*, '*m* × '*p*) *lmat*.
$\sigma \in$ *separable-cone-lmat*
$\wedge$ *hermitian* (*unwrap-lm* σ)
$\longrightarrow$ *0* ≤ *Re* (*Tr* (*W* ∗ *unwrap-lm* σ)))
⟨*proof*⟩

30.7 Hermitian closure of the separable cone

Every element of *separable-cone-lmat* has hermitian unwrap. Generators *tensor-lmat* σ τ with σ, τ PSD are hermitian, and the hermitian set is closed under + and nonneg real scaling.

lemma *hermitian-zero-mat*:
hermitian (0_m *n n* :: *complex Matrix.mat*)
⟨*proof*⟩

lemma *hermitian-add-mat*:
fixes *P Q* :: *complex Matrix.mat*
assumes *Pcar*: *P* ∈ *carrier-mat n n* **and** *Qcar*: *Q* ∈ *carrier-mat n n*
and *Pherm*: *hermitian P* **and** *Qherm*: *hermitian Q*
shows *hermitian* (*P* + *Q*)
⟨*proof*⟩

Tensor product of hermitians is hermitian (entrywise via *index-tensor-mat*).

lemma *hermitian-tensor-mat*:
fixes *P Q* :: *complex Matrix.mat*
assumes *Pcar*: *P* ∈ *carrier-mat m m* **and** *Qcar*: *Q* ∈ *carrier-mat p p*
and *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p*
and *Pherm*: *hermitian P* **and** *Qherm*: *hermitian Q*
shows *hermitian* (*P* ⨂ *Q*)
⟨*proof*⟩

Generators of *separable-cone-lmat* have hermitian unwrap.

lemma *separable-lmat-gen-hermitian*:
fixes $\sigma ::$ (*complex*, '*m* :: *finite* × '*p* :: *finite*, '*m* × '*p*) *lmat*
assumes $\sigma \in$ *separable-lmat-gen*
shows *hermitian* (*unwrap-lm* σ)
⟨*proof*⟩

The hermitian set in lmat is a convex cone, hence contains the hull of the generators.

lemma *hermitian-lmat-set-convex-cone*:
convex-cone {*M* :: (*complex*, '*k* :: *finite*, '*k*) *lmat*.
hermitian (*unwrap-lm M*)}
⟨*proof*⟩

theorem *separable-cone-lmat-hermitian*:
fixes $\sigma ::$ (*complex*, '*m* :: *finite* × '*p* :: *finite*, '*m* × '*p*) *lmat*
assumes $\sigma \in$ *separable-cone-lmat*
shows *hermitian* (*unwrap-lm* σ)
⟨*proof*⟩

Final form of the entanglement witness theorem: the side condition "σ has hermitian unwrap" is automatic for elements of the separable cone.

theorem *entanglement-witness-hermitian'*:
fixes ϱ :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*
assumes *notin*: $\varrho \notin$ *separable-cone-lmat*
and *ϱ-herm*: *hermitian* (*unwrap-lm* ϱ)
shows $\exists$ *W* :: *complex Matrix.mat*.
W ∈ *carrier-mat* (*CARD*(*'m* × *'p*)) (*CARD*(*'m* × *'p*))
∧ *hermitian W*
∧ *Re* (*Tr* (*W* ∗ *unwrap-lm* ϱ)) < *0*
∧ (∀ σ :: (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat*.
σ ∈ *separable-cone-lmat*
⟶ *0* ≤ *Re* (*Tr* (*W* ∗ *unwrap-lm* σ)))
⟨*proof*⟩

30.8 Main separability characterisation

The Horodecki-style witness characterisation: for hermitian ϱ, membership in the separable cone is equivalent to the nonneg-real-trace property holding against every hermitian witness that is itself nonneg-real-trace on the cone.

theorem *separable-iff-witness*:
fixes ϱ :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*
assumes *ϱ-herm*: *hermitian* (*unwrap-lm* ϱ)
shows $\varrho \in$ *separable-cone-lmat*
⟷
(∀ *W* :: *complex Matrix.mat*.
W ∈ *carrier-mat* (*CARD*(*'m* × *'p*)) (*CARD*(*'m* × *'p*))
∧ *hermitian W*
∧ (∀ σ :: (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat*.
σ ∈ *separable-cone-lmat*
⟶ *0* ≤ *Re* (*Tr* (*W* ∗ *unwrap-lm* σ)))
⟶ *0* ≤ *Re* (*Tr* (*W* ∗ *unwrap-lm* ϱ)))
⟨*proof*⟩

30.9 Density-matrix specialisation

For an actual quantum density matrix ϱ (PSD with unit trace), the conditions *psd-lmat* ϱ and *Tr-lmat* ϱ = *1* imply *hermitian* (*unwrap-lm* ϱ), so the witness theorem and the characterisation specialise cleanly.

theorem *entanglement-witness-for-density*:
fixes ϱ :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*
assumes *ϱ-psd*: *psd-lmat* ϱ
and *ϱ-tr*: *Tr-lmat* ϱ = *1*
and *ϱ-not-sep*: $\varrho \notin$ *separable-cone-lmat*
shows $\exists$ *W* :: *complex Matrix.mat*.
W ∈ *carrier-mat* (*CARD*(*'m* × *'p*)) (*CARD*(*'m* × *'p*))
∧ *hermitian W*
∧ *Re* (*Tr* (*W* ∗ *unwrap-lm* ϱ)) < *0*
∧ (∀ σ :: (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat*.
σ ∈ *separable-cone-lmat*
⟶ *0* ≤ *Re* (*Tr* (*W* ∗ *unwrap-lm* σ)))
⟨*proof*⟩

The witness is also nonneg on every separable density matrix (*Sprime*) since *Sprime* ⊆ *separable-cone-lmat*.

theorem *entanglement-witness-for-density-on-Sprime*:
fixes ϱ :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*
assumes *ϱ-psd*: *psd-lmat* ϱ

and ϱ-*tr*: *Tr-lmat* $\varrho = 1$
and ϱ-*not-sep*: $\varrho \notin$ *separable-cone-lmat*
shows $\exists\, W ::$ *complex Matrix.mat*.
$W \in$ *carrier-mat* (*CARD*('*m* × '*p*)) (*CARD*('*m* × '*p*))
$\wedge$ *hermitian W*
$\wedge$ *Re* (*Tr* (*W* ∗ *unwrap-lm* ϱ)) < *0*
$\wedge$ ($\forall\, \sigma ::$ (*complex*, '*m* × '*p*, '*m* × '*p*) *lmat*.
$\sigma \in$ *Sprime* ⟶ *0* ≤ *Re* (*Tr* (*W* ∗ *unwrap-lm* σ)))
⟨*proof*⟩

The full Horodecki characterisation, specialised to density matrices.

theorem *separable-density-iff-witness*:
fixes $\varrho ::$ (*complex*, '*m* :: *finite* × '*p* :: *finite*, '*m* × '*p*) *lmat*
assumes ϱ-*psd*: *psd-lmat* ϱ
and ϱ-*tr*: *Tr-lmat* $\varrho = 1$
shows $\varrho \in$ *separable-cone-lmat*
⟷
($\forall\, W ::$ *complex Matrix.mat*.
$W \in$ *carrier-mat* (*CARD*('*m* × '*p*)) (*CARD*('*m* × '*p*))
$\wedge$ *hermitian W*
$\wedge$ ($\forall\, \sigma ::$ (*complex*, '*m* × '*p*, '*m* × '*p*) *lmat*.
$\sigma \in$ *separable-cone-lmat*
⟶ *0* ≤ *Re* (*Tr* (*W* ∗ *unwrap-lm* σ)))
⟶ *0* ≤ *Re* (*Tr* (*W* ∗ *unwrap-lm* ϱ)))
⟨*proof*⟩

The trace is identically *1* on the convex hull of *Sprime*.

lemma *Tr-lmat-convex-hull-Sprime-eq-one*:
fixes $\varrho ::$ (*complex*, '*m* :: *finite* × '*p* :: *finite*, '*m* × '*p*) *lmat*
assumes $\varrho \in$ *convex hull Sprime*
shows *Tr-lmat* $\varrho = 1$
⟨*proof*⟩

A density matrix is separable iff it is a convex combination of separable pure-product states. This is the standard textbook definition of separability for mixed states.

theorem *separable-density-iff-in-convex-hull-Sprime*:
fixes $\varrho ::$ (*complex*, '*m* :: *finite* × '*p* :: *finite*, '*m* × '*p*) *lmat*
assumes ϱ-*psd*: *psd-lmat* ϱ
and ϱ-*tr*: *Tr-lmat* $\varrho = 1$
shows $\varrho \in$ *separable-cone-lmat* ⟷ $\varrho \in$ *convex hull Sprime*
⟨*proof*⟩

Topology of the separable density-matrix set.

lemma *compact-convex-hull-Sprime*:
compact (*convex hull*
(*Sprime* :: (*complex*, '*m* :: *finite* × '*p* :: *finite*, '*m* × '*p*) *lmat set*))
⟨*proof*⟩

lemma *closed-convex-hull-Sprime*:
closed (*convex hull*
(*Sprime* :: (*complex*, '*m* :: *finite* × '*p* :: *finite*, '*m* × '*p*) *lmat set*))
⟨*proof*⟩

Strict separation in the inner-product form: by closedness of *convex hull Sprime* we get a strictly negative-inner-product hyperplane. The Re-trace strict-form is then an immediate corollary once the hermitian witness construction is replayed.

lemma *separable-density-separating-hyperplane*:

fixes ϱ :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes ϱ-*psd*: *psd-lmat* ϱ
and ϱ-*tr*: *Tr-lmat* ϱ = *1*
and ϱ-*not-sep*: $\varrho \notin$ *separable-cone-lmat*
shows ∃ *a b*.
inner a ϱ < *b*
∧ (∀ σ :: (*complex*, $'m$ × $'p$, $'m$ × $'p$) *lmat*.
$\sigma \in$ *convex hull Sprime* ⟶ *b* < *inner a* σ)
⟨*proof*⟩

The strict Re-trace form of the entanglement witness.

theorem *entanglement-witness-strict*:
fixes ϱ :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes ϱ-*psd*: *psd-lmat* ϱ
and ϱ-*tr*: *Tr-lmat* ϱ = *1*
and ϱ-*not-sep*: $\varrho \notin$ *separable-cone-lmat*
shows ∃ (*W* :: *complex Matrix.mat*) *c*.
W ∈ *carrier-mat* (*CARD*($'m$ × $'p$)) (*CARD*($'m$ × $'p$))
∧ *hermitian W*
∧ *Re* (*Tr* (*W* ∗ *unwrap-lm* ϱ)) < *c*
∧ (∀ σ :: (*complex*, $'m$ × $'p$, $'m$ × $'p$) *lmat*.
$\sigma \in$ *convex hull Sprime*
⟶ *c* < *Re* (*Tr* (*W* ∗ *unwrap-lm* σ)))
⟨*proof*⟩

30.10 The positive-map criterion (D28-3)

A Hermitian $R \in M_r(M_n)$ is separable iff $\Phi^{(r)}(R) \geq 0$ for every positive $\Phi : M_n \to M_r$.

theorem *separable-iff-positive-maps*:
fixes *R* :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes *Rh*: *hermitian* (*unwrap-lm R*)
shows *R* ∈ *separable-cone-lmat* ⟷
(∀ Φ. *lin-on CARD*($'p$) *CARD*($'m$) (*carrier-mat CARD*($'p$) *CARD*($'p$)) Φ ⟶ *positive-map CARD*($'p$) *CARD*($'m$) Φ
⟶ *psd* (*amplification CARD*($'p$) *CARD*($'m$) *CARD*($'m$) Φ (*unwrap-lm R*)))
⟨*proof*⟩

30.11 The Horodecki–Peres criterion for two qubits (D28-5)

Entries of the partial transpose $\Phi^{(2)}(X)$ of a 4×4 matrix.

lemma *partial-transpose-2-index*:
fixes *M* :: *complex Matrix.mat*
assumes *i*: *i* < *4* **and** *j*: *j* < *4*
shows *amplification 2 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) *M* $$ (*i*,*j*)
= *M* $$ ((*i div 2*) ∗ *2* + *j mod 2*, (*j div 2*) ∗ *2* + *i mod 2*)
⟨*proof*⟩

lemma *product-vec-iff-det-zero*:
fixes *v* :: *complex Matrix.vec*
assumes *vd*: *dim-vec v* = *4*
shows (∃ α β. *dim-vec* α = *2* ∧ *dim-vec* β = *2* ∧ *v* = *tensor-vec* α β)
⟷ *v* $ *0* ∗ *v* $ *3* − *v* $ *1* ∗ *v* $ *2* = *0*
⟨*proof*⟩

The notes' witness $w = (-\overline{v_2}, \overline{v_1})$ gives $\langle w|\Phi^{(2)}(|v\rangle\langle v|)w\rangle = -2|v_0 v_3 - v_1 v_2|^2$ ($= 2|\langle v_2|v_1\rangle|^2 - 2\|v_1\|^2\|v_2\|^2$ by Lagrange's identity).

lemma *partial-transpose-witness-value*:
fixes v :: *complex Matrix.vec*
assumes *vd*: *dim-vec v = 4*
defines $w \equiv$ *Matrix.vec 4* (λk. *if k = 0 then − cnj (v $ 2) else if k = 1 then − cnj (v $ 3)*
else if k = 2 then cnj (v $ 0) else cnj (v $ 1))
shows $\langle w \mid$ *amplification 2 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) (*v* ⊘ *v*) $*_v$ *w*$\rangle$
= *− 2 * cnj (v $ 0 * v $ 3 − v $ 1 * v $ 2) * (v $ 0 * v $ 3 − v $ 1 * v $ 2)*
⟨*proof*⟩

theorem *horodecki-peres-two-qubits*:
fixes v :: *complex Matrix.vec*
assumes *vd*: *dim-vec v = 4*
shows *psd* (*amplification 2 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) (*v* ⊘ *v*))
⟷ (∃ α β. *dim-vec* α *= 2* ∧ *dim-vec* β *= 2* ∧ *v = tensor-vec* α β)
⟨*proof*⟩

corollary *horodecki-peres-entangled*:
fixes v :: *complex Matrix.vec*
assumes *vd*: *dim-vec v = 4*
shows *entangled 2 2 v* ⟷
¬ *psd* (*amplification 2 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) (*v* ⊘ *v*))
⟨*proof*⟩

Part V

Quantum error correction (Days 29–34)

31 Day 29 (7 November 2011): classical error detection and correction

To transmit $v \in \mathbb{Z}_2^n$ through a channel that may flip bits (+1 in $\mathbb{Z}_2$), one encodes $v \mapsto \phi(v) \in \mathbb{Z}_2^{n+k}$, transmits, and decodes. Examples: repetition $v \mapsto (v, v)$ detects one error but corrects none and doubles the cost; the parity check $(a_1, \ldots, a_n) \mapsto (a_1, \ldots, a_n; a_1 + \cdots + a_n)$ detects one error cheaply; the Hamming $[7, 1, 3]$ code detects and corrects one error (below: the code as written detects one error but cannot correct it; the $[7, 4, 3]$ code corrects one).

Definition (notes). The *Hamming weight* $\|w\|$ of $w \in \mathbb{Z}_2^m$ is its number of non-zero entries; the weight of a code $S \subseteq \mathbb{Z}_2^m$ is $\min\{\|v - w\| : v \neq w \in S\}$, the least number of errors turning one code word into another; for a subspace code it is $\min\{\|v\| : 0 \neq v \in S\}$.

Decoding amounts to choosing a left inverse of ϕ (different left inverses correct differently); the best binary codes are cyclic codes, ideals in $\mathbb{Z}_2[x]/\langle x^m - 1\rangle$ (not formalised).

31.1 Binary strings, Hamming weight and distance

definition *hamming-weight* :: *nat list* ⇒ *nat* **where**
hamming-weight v ≡ *length* (*filter* (λb. *b* ≠ *0*) *v*)

definition *hamming-dist* :: *nat list* ⇒ *nat list* ⇒ *nat* **where**
hamming-dist v w ≡ *hamming-weight* (*map2* ($\lambda a\ b$. (*a* + *b*) *mod 2*) *v w*)

lemma *hamming-weight-Nil* [*simp*]: *hamming-weight* [] = *0*
⟨*proof*⟩

lemma *hamming-weight-Cons*:
hamming-weight (*a* # *v*)
= (*if a* ≠ *0 then Suc* (*hamming-weight v*) *else hamming-weight v*)
⟨*proof*⟩

lemma *hamming-dist-Nil* [*simp*]:
hamming-dist [] [] = *0*
⟨*proof*⟩

lemma *hamming-dist-Cons*:
hamming-dist (*a* # *v*) (*b* # *w*)
= (*if* (*a* + *b*) *mod 2* ≠ *0*
then Suc (*hamming-dist v w*)
else hamming-dist v w)
⟨*proof*⟩

The weight counts the ones, and $d(v, w) = \mathrm{wt}(v + w)$ counts the positions where v and w differ: $d(0110, 1100) = \mathrm{wt}(1010) = 2$.

lemma *hamming-dist-symm*:
assumes *length v* = *length w*
shows *hamming-dist v w* = *hamming-dist w v*
⟨*proof*⟩

lemma *hamming-dist-zero*:
assumes *lenw*: *length v* = *length w*
and *bits-v*: ∀ *x* ∈ *set v*. *x* < *2*
and *bits-w*: ∀ *x* ∈ *set w*. *x* < *2*
shows *hamming-dist v w* = *0* ⟷ *v* = *w*
⟨*proof*⟩

lemma *hamming-dist-self* [*simp*]:
hamming-dist v v = *0*
⟨*proof*⟩

31.2 Code distance

A code of distance at least $2t+1$ can correct t errors (recorded as the definition *corrects-errors*; the decoding theorem itself is not formalised).

definition *code-distance* :: *nat list set* ⇒ *nat* **where**
code-distance code ≡
if (∃ *v* ∈ *code*. ∃ *w* ∈ *code*. *v* ≠ *w*)
then Min {*hamming-dist v w* | *v w*. *v* ∈ *code* ∧ *w* ∈ *code* ∧ *v* ≠ *w*}
else 0

definition *corrects-errors* :: *nat list set* ⇒ *nat* ⇒ *bool* **where**
corrects-errors code t ≡ *code-distance code* ≥ *2*∗*t* + *1*

31.3 The parity-check and repetition codes

Parity check: append the sum of the bits; any single error changes the code word. Repetition doubles Hamming distances.

definition *parity-bit* :: *nat list* ⇒ *nat* **where**

parity-bit v ≡ *foldr* (λ*a b*. (*a* + *b*) *mod 2*) *v 0*

definition *parity-check-encode* :: *nat list* ⇒ *nat list* **where**
parity-check-encode v ≡ *v* @ [*parity-bit v*]

lemma *length-parity-check-encode* [*simp*]:
length (*parity-check-encode v*) = *length v* + *1*
⟨*proof*⟩

lemma *hamming-weight-pos-iff*:
0 < *hamming-weight v* ⟷ (∃ *x* ∈ *set v*. *x* ≠ *0*)
⟨*proof*⟩

lemma *hamming-dist-pos-iff*:
assumes *lenw*: *length v* = *length w*
and *bits-v*: ∀ *x* ∈ *set v*. *x* < *2*
and *bits-w*: ∀ *x* ∈ *set w*. *x* < *2*
shows *0* < *hamming-dist v w* ⟷ *v* ≠ *w*
⟨*proof*⟩

lemma *parity-bit-lt-2*: *parity-bit v* < *2*
⟨*proof*⟩

lemma *parity-check-encode-binary*:
assumes ∀ *x* ∈ *set v*. *x* < *2*
shows ∀ *x* ∈ *set* (*parity-check-encode v*). *x* < *2*
⟨*proof*⟩

The parity check $v \mapsto (v, \sum_i v_i)$ produces words of even weight; one flipped bit makes the weight odd, so a single error is detected, though not located.

lemma *parity-check-detects-one-error*:
assumes *lenE*: *length e* = *length v* + *1*
and *bits-v*: ∀ *x* ∈ *set v*. *x* < *2*
and *bits-e*: ∀ *x* ∈ *set e*. *x* < *2*
and *weight*: *hamming-weight e* = *1*
shows *parity-check-encode v*
≠ *map2* (λ*a b*. (*a* + *b*) *mod 2*) (*parity-check-encode v*) *e*
⟨*proof*⟩

definition *repetition-encode* :: *nat list* ⇒ *nat list* **where**
repetition-encode v ≡ *v* @ *v*

Repetition $v \mapsto (v, v)$ doubles every distance, $d((v,v),(w,w)) = 2\,d(v,w)$.

lemma *hamming-dist-repetition-doubles*:
assumes *length v* = *length w*
shows *hamming-dist* (*repetition-encode v*) (*repetition-encode w*)
= *2* ∗ *hamming-dist v w*
⟨*proof*⟩

31.4 More on the repetition code

lemma *hamming-weight-append*:
hamming-weight (*xs* @ *ys*) = *hamming-weight xs* + *hamming-weight ys*
⟨*proof*⟩

lemma *hamming-weight-repetition-encode*:
hamming-weight (*repetition-encode v*) = *2* ∗ *hamming-weight v*
⟨*proof*⟩

31.5 Classical codes over $\mathbb{Z}_2$

Days 29–30: classical codes over $\mathbb{Z}_2$. Words are lists over $\{0,1\}$; all finite checks are decided by *code-simp* (proof-producing evaluation, no oracle).

definition *bin-words* :: *nat* ⇒ *nat list set* **where**
bin-words n = *set* (*List.n-lists n* [*0*, *1*])

definition *xor-word* :: *nat list* ⇒ *nat list* ⇒ *nat list* **where**
xor-word v w = *map2* (λ*a b*. (*a* + *b*) *mod 2*) *v w*

31.5.1 The notes' "Hamming [7,1,3]" code does not correct one error

The notes encode $(a_1,\dots,a_7) \mapsto (a_1,\dots,a_7;\ a_1{+}a_2{+}a_3{+}a_4;\ a_1{+}a_2{+}a_5{+}a_6;\ a_1{+}a_3{+}a_5{+}a_7)$ and claim it detects and corrects one error. It detects one error, but two codewords are at distance 2: the word e_4 is at distance 1 from the codewords of 0 and of e_4, so no decoder corrects every single error.

definition *notes-hamming-encode* :: *nat list* ⇒ *nat list* **where**
notes-hamming-encode a = *a* @ [(*a*!*0* + *a*!*1* + *a*!*2* + *a*!*3*) *mod 2*, (*a*!*0* + *a*!*1* + *a*!*4* + *a*!*5*) *mod 2*, (*a*!*0* + *a*!*2* + *a*!*4* + *a*!*6*) *mod 2*]

definition *notes-hamming-code* :: *nat list set* **where**
notes-hamming-code = *notes-hamming-encode* ‘ *bin-words 7*

definition *flip-at* :: *nat* ⇒ *nat list* ⇒ *nat list* **where**
flip-at i w = *w*[*i* := (*w*!*i* + *1*) *mod 2*]

lemma *notes-hamming-code-iff*:
w ∈ *notes-hamming-code* ⟷ *take 7 w* ∈ *bin-words 7* ∧ *w* = *notes-hamming-encode* (*take 7 w*)
⟨*proof*⟩

lemma *bin-words-iff*: *w* ∈ *bin-words n* ⟷ *length w* = *n* ∧ *set w* ⊆ {*0*, *1*}
⟨*proof*⟩

lemma *bin-nth-lt2*: *set v* ⊆ {*0*,*1*} ⟹ *k* < *length v* ⟹ *v* ! *k* < (*2*::*nat*)
⟨*proof*⟩

lemma *xor-word-length* [*simp*]: *length* (*xor-word v w*) = *min* (*length v*) (*length w*)
⟨*proof*⟩

lemma *xor-word-nth*: *k* < *length v* ⟹ *k* < *length w* ⟹ *xor-word v w* ! *k* = (*v* ! *k* + *w* ! *k*) *mod 2*
⟨*proof*⟩

lemma *xor-word-cancel*:
assumes *length c* = *length v length c* = *length w set v* ⊆ {*0*,*1*} *set w* ⊆ {*0*,*1*}
shows *xor-word c v* = *xor-word c w* ⟷ *v* = *w*
⟨*proof*⟩

lemma *mod2-eq-iff*: (*p*::*nat*) *mod 2* = *q mod 2* ⟷ (*even p* ⟷ *even q*)
⟨*proof*⟩

The encoding is linear over $\mathbb{Z}_2$.

lemma *notes-hamming-encode-xor*:
assumes *length a* = *7 length b* = *7*
shows *notes-hamming-encode* (*xor-word a b*) = *xor-word* (*notes-hamming-encode a*) (*notes-hamming-encode b*)
⟨*proof*⟩

lemma *flip-at-xor*:
assumes *set c* ⊆ {*0*,*1*} *i* < *length c*
shows *flip-at i c* = *xor-word c* ((*replicate* (*length c*) *0*)[*i* := *1*])
⟨*proof*⟩

theorem *notes-hamming-detects-one-error*:
∀ *c* ∈ *notes-hamming-code*. ∀ *i* < *10*. *flip-at i c* ∉ *notes-hamming-code*
⟨*proof*⟩

theorem *notes-hamming-ambiguous-word*:
defines *w* ≡ [*0*,*0*,*0*,*1*,*0*,*0*,*0*,*0*,*0*,*0*]
shows *notes-hamming-encode* (*replicate 7 0*) ∈ *notes-hamming-code*
and *notes-hamming-encode* [*0*,*0*,*0*,*1*,*0*,*0*,*0*] ∈ *notes-hamming-code*
and *notes-hamming-encode* (*replicate 7 0*) ≠ *notes-hamming-encode* [*0*,*0*,*0*,*1*,*0*,*0*,*0*]
and *hamming-dist* (*notes-hamming-encode* (*replicate 7 0*)) *w* = *1*
and *hamming-dist* (*notes-hamming-encode* [*0*,*0*,*0*,*1*,*0*,*0*,*0*]) *w* = *1*
⟨*proof*⟩

theorem *notes-hamming-not-correcting*: ¬ *corrects-errors notes-hamming-code 1*
⟨*proof*⟩

31.5.2 The Hamming [7,4,3] code corrects one error

definition *hamming74-encode* :: *nat list* ⇒ *nat list* **where**
hamming74-encode a = *a* @ [(*a*!*0* + *a*!*1* + *a*!*3*) *mod 2*, (*a*!*0* + *a*!*2* + *a*!*3*) *mod 2*, (*a*!*1* + *a*!*2* + *a*!*3*) *mod 2*]

definition *hamming74-code* :: *nat list set* **where**
hamming74-code = *hamming74-encode* ‘ *bin-words 4*

lemma *ex1-in-set*: (∃!*c*. *c* ∈ *set xs* ∧ *P c*) ⟷ *card* (*set* (*filter P xs*)) = *1*
⟨*proof*⟩

theorem *hamming74-unique-decoding*:
∀ *w* ∈ *bin-words 7*. ∃!*c*. *c* ∈ *hamming74-code* ∧ *hamming-dist c w* ≤ *1*
⟨*proof*⟩

theorem *hamming74-corrects-one-error*: *corrects-errors hamming74-code 1*
⟨*proof*⟩

31.6 Hamming distance and subspace codes (D29-4)

lemma *hamming-dist-xor*: *hamming-dist v w* = *hamming-weight* (*xor-word v w*)
⟨*proof*⟩

The triangle inequality for words of bits of equal length.

lemma *hamming-dist-triangle*:
assumes *length u* = *length v length v* = *length w*
and ∀ *x* ∈ *set u*. *x* < *2* ∀ *x* ∈ *set v*. *x* < *2* ∀ *x* ∈ *set w*. *x* < *2*
shows *hamming-dist u w* ≤ *hamming-dist u v* + *hamming-dist v w*
⟨*proof*⟩

For a subspace code (closed under addition mod 2) the distance is the least weight of a nonzero code word.

definition *zero-word* :: *nat* ⇒ *nat list* **where**
zero-word m = *replicate m 0*

Words add coordinatewise mod 2: $v + v = 0$, $v + 0 = v$, and $v + w = 0$ iff $v = w$. For a code closed under $+$ the distance is therefore the least weight of a non-zero code word,

$$d(S) = \min_{v \neq w \in S} \mathrm{wt}(v + w) = \min_{0 \neq u \in S} \mathrm{wt}(u).$$

lemma *xor-word-self*: *xor-word v v = zero-word (length v)*
⟨*proof*⟩

lemma *xor-word-zero*: ∀ *x* ∈ *set v*. *x* < *2* ⟹ *xor-word v* (*zero-word* (*length v*)) = *v*
⟨*proof*⟩

lemma *xor-word-eq-zero-iff*:
assumes *length v = length w* ∀ *x* ∈ *set v*. *x* < *2* ∀ *x* ∈ *set w*. *x* < *2*
shows *xor-word v w = zero-word* (*length v*) ⟷ *v = w*
⟨*proof*⟩

lemma *hamming-weight-eq-zero-iff*: ∀ *x* ∈ *set v*. *x* < *2* ⟹ *hamming-weight v = 0* ⟷ *v = zero-word* (*length v*)
⟨*proof*⟩

definition *subspace-code* :: *nat* ⇒ *nat list set* ⇒ *bool* **where**
subspace-code m S ⟷ (∀ *v* ∈ *S*. *length v = m* ∧ (∀ *x* ∈ *set v*. *x* < *2*))
∧ (∀ *v* ∈ *S*. ∀ *w* ∈ *S*. *xor-word v w* ∈ *S*)

theorem *subspace-code-distance*:
assumes *S*: *subspace-code m S* **and** *fin*: *finite S* **and** *two*: ∃ *v* ∈ *S*. ∃ *w* ∈ *S*. *v* ≠ *w*
shows *code-distance S = Min* {*hamming-weight v* | *v*. *v* ∈ *S* ∧ *v* ≠ *zero-word m*}
⟨*proof*⟩

A code of distance at least $2t + 1$ corrects t errors: a received word is within distance t of at most one code word.

theorem *corrects-errors-unique-decoding*:
assumes *ce*: *corrects-errors S t* **and** *fin*: *finite S*
and *S*: ∀ *v* ∈ *S*. *length v = m* ∧ (∀ *x* ∈ *set v*. *x* < *2*)
and *r*: *length r = m* ∀ *x* ∈ *set r*. *x* < *2*
and *c*: *c* ∈ *S hamming-dist r c* ≤ *t* **and** *c'*: *c'* ∈ *S hamming-dist r c'* ≤ *t*
shows *c = c'*
⟨*proof*⟩

32 Day 30 (9 November 2011): error probabilities; the three-qubit bit-flip code

If a bit flips with probability p (independently), the repetition code on three bits leaves an error undetected with probability $3p^2(1-p)^4 + 3p^4(1-p)^2 + p^6$, the parity code with probability $\binom{4}{2}p^2(1-p)^2 + p^4$ (the former is smaller for $p < \frac{1}{2}$), and the majority-rule code $0 \mapsto 000$, $1 \mapsto 111$ decodes wrongly with probability $3p^2(1-p) + p^3$ (below; the comparison holds for all $0 < p < 1$).

Quantum error correction: one cannot clone, but basis vectors can be copied and measurements can decode. With $X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ the allowed errors are qubit flips $X \otimes I \otimes I$, $I \otimes X \otimes I$, ...

The *three-qubit bit-flip code* encodes $a|0\rangle + b|1\rangle \mapsto a|000\rangle + b|111\rangle$ (two CNOTs). Decoding: (1) syndrome measurement with $P_0 = |000\rangle\langle 000| + |111\rangle\langle 111|$, $P_1 = |100\rangle\langle 100| + |011\rangle\langle 011|$, $P_2 = |010\rangle\langle 010| + |101\rangle\langle 101|$, $P_3 = |001\rangle\langle 001| + |110\rangle\langle 110|$, $\sum P_i^2 = I$, which leaves a state with at most one flip unchanged and reveals where the flip occurred; (2) recovery $R(Y) =$

$P_0 Y P_0 + (X \otimes I \otimes I) P_1 Y P_1 (X \otimes I \otimes I) + \cdots$, a completely positive trace-preserving map with $R(|\psi_i\rangle\langle\psi_i|) = |\psi\rangle\langle\psi|$ after zero or one error. The code's recovery theorem is *bit-flip-code-recovery* below (syndrome projections, $\sum P_i = I$, R CPTP); X is *Pauli-X* of Day 10.

32.1 Probabilities of undetected and uncorrected errors (D30-1)

Each bit flips independently with probability p; an error pattern e of weight k on n bits has probability $p^k(1-p)^{n-k}$. An error is undetected when it turns the sent codeword into another codeword.

definition *word-prob* :: *real* ⇒ *nat* ⇒ *nat list* ⇒ *real* **where**
word-prob p n e = *p* ^ *hamming-weight e* ∗ (*1* − *p*) ^ (*n* − *hamming-weight e*)

definition *err-prob* :: *real* ⇒ *nat* ⇒ *nat list set* ⇒ *real* **where**
err-prob p n ES = (∑ *e* ∈ *ES*. *word-prob p n e*)

Bits flip independently with probability p, so an error pattern of weight k in n bits has probability $p^k(1-p)^{n-k}$, and a set of patterns the sum of these.

lemma *err-prob-list*: *distinct es* ⟹ *err-prob p n* (*set es*) = (∑ *e*←*es*. *word-prob p n e*)
⟨*proof*⟩

definition *rep-code* :: *nat list set* **where** *rep-code* = *repetition-encode* ‘ *bin-words 3*
definition *par-code* :: *nat list set* **where** *par-code* = *parity-check-encode* ‘ *bin-words 3*

lemma *rep-code-list*: *rep-code* = *set* (*map repetition-encode* (*List.n-lists 3* [*0*, *1*]))
⟨*proof*⟩

lemma *par-code-list*: *par-code* = *set* (*map parity-check-encode* (*List.n-lists 3* [*0*, *1*]))
⟨*proof*⟩

definition *undetected* :: *nat list set* ⇒ *nat* ⇒ *nat list* ⇒ *nat list set* **where**
undetected C n c = {*e* ∈ *bin-words n*. *e* ≠ *replicate n 0* ∧ *xor-word c e* ∈ *C*}

lemma *set-filter-bin*: {*e* ∈ *bin-words n*. *P e*} = *set* (*filter P* (*List.n-lists n* [*0*, *1*]))
⟨*proof*⟩

Repetition on three bits: $3p^2(1-p)^4 + 3p^4(1-p)^2 + p^6$.

theorem *repetition-undetected-prob*:
assumes *c*: *c* ∈ *rep-code*
shows *err-prob p 6* (*undetected rep-code 6 c*) = *3* ∗ *p*^*2* ∗ (*1* − *p*)^*4* + *3* ∗ *p*^*4* ∗ (*1* − *p*)^*2* + *p*^*6*
⟨*proof*⟩

Parity on three bits: $\binom{4}{2}p^2(1-p)^2 + p^4$.

theorem *parity-undetected-prob*:
assumes *c*: *c* ∈ *par-code*
shows *err-prob p 4* (*undetected par-code 4 c*) = *6* ∗ *p*^*2* ∗ (*1* − *p*)^*2* + *p*^*4*
⟨*proof*⟩

The notes compare the two for $p < \frac{1}{2}$; the inequality in fact holds for all $0 < p < 1$.

theorem *repetition-better-than-parity*:
assumes *p0*: *0* < *p* **and** *p1*: *p* < *1*
shows *3* ∗ *p*^*2* ∗ (*1* − *p*)^*4* + *3* ∗ *p*^*4* ∗ (*1* − *p*)^*2* + *p*^*6* < *6* ∗ *p*^*2* ∗ (*1* − *p*)^*2* + (*p*::*real*)^*4*
⟨*proof*⟩

corollary *repetition-better-than-parity-notes*:
0 < *p* ⟹ *p* < *1/2* ⟹ *3* ∗ *p*^*2* ∗ (*1* − *p*)^*4* + *3* ∗ *p*^*4* ∗ (*1* − *p*)^*2* + *p*^*6* < *6* ∗ *p*^*2* ∗ (*1* − *p*)^*2* + (*p*::*real*)^*4*

⟨*proof*⟩

Majority rule $0 \mapsto 000$, $1 \mapsto 111$: decoding fails with probability $3p^2(1-p)+p^3$.

definition *maj-decode* :: *nat list* ⇒ *nat* **where**
maj-decode w = (*if 2* ≤ *sum-list w then 1 else 0*)

theorem *majority-uncorrected-prob*:
assumes *b*: *b* ∈ {*0*, *1*}
shows *err-prob p 3* {*e* ∈ *bin-words 3*. *maj-decode* (*xor-word* (*replicate 3 b*) *e*) ≠ *b*}
= *3* ∗ *p*^*2* ∗ (*1* − *p*) + *p*^*3*
⟨*proof*⟩

32.2 The three-qubit bit-flip code and its recovery map (D30-3, D30-4)

The encoding $a|0\rangle + b|1\rangle \mapsto a|000\rangle + b|111\rangle$ as an 8×2 matrix.

definition *encode-mat* :: *complex Matrix.mat* **where**
encode-mat ≡ *Matrix.mat 8 2* (λ(*i*,*j*).
if i = *0* ∧ *j* = *0 then 1*
else if i = *7* ∧ *j* = *1 then 1*
else 0)

lemma *encode-mat-carrier* [*simp*]: *encode-mat* ∈ *carrier-mat 8 2*
⟨*proof*⟩

The syndrome projections $P_0 = |000\rangle\langle 000| + |111\rangle\langle 111|$, …, as diagonal 0/1 matrices ($|abc\rangle \leftrightarrow e_{4a+2b+c}$), and the single flips $X \otimes I \otimes I$, $I \otimes X \otimes I$, $I \otimes I \otimes X$ as permutation matrices.

definition *syndrome-proj* :: *nat set* ⇒ *complex Matrix.mat* **where**
syndrome-proj K = *Matrix.mat 8 8* (λ(*i*,*j*). *if i* = *j* ∧ *i* ∈ *K then 1 else 0*)

abbreviation P_0 ≡ *syndrome-proj* {*0*, *7*}
abbreviation P_1 ≡ *syndrome-proj* {*4*, *3*}
abbreviation P_2 ≡ *syndrome-proj* {*2*, *5*}
abbreviation P_3 ≡ *syndrome-proj* {*1*, *6*}

definition *flip-bit-mat* :: *nat* ⇒ *complex Matrix.mat* **where**
flip-bit-mat b = *Matrix.mat 8 8* (λ(*i*,*j*). *if j* = (*if i div 2*^*b mod 2* = *0 then i* + *2*^*b else i* − *2*^*b*) *then 1 else 0*)

abbreviation X_1 ≡ *flip-bit-mat 2*
abbreviation X_2 ≡ *flip-bit-mat 1*
abbreviation X_3 ≡ *flip-bit-mat 0*

The code space is span$\{|000\rangle, |111\rangle\}$. A flip of qubit j moves it into the span of the two words with bit j flipped, so the syndrome projections are the projections onto

$$P_0 : \{|000\rangle, |111\rangle\}, \quad P_1 : \{|100\rangle, |011\rangle\}, \quad P_2 : \{|010\rangle, |101\rangle\}, \quad P_3 : \{|001\rangle, |110\rangle\},$$

and X_j flips qubit j (e.g. $X_1 = X \otimes I \otimes I$). Recovery measures $P_0, \ldots, P_3$ and applies X_j after outcome j.

lemma *syndrome-proj-carrier* [*simp*]: *syndrome-proj K* ∈ *carrier-mat 8 8*
⟨*proof*⟩

lemma *flip-bit-mat-carrier* [*simp*]: *flip-bit-mat b* ∈ *carrier-mat 8 8*
⟨*proof*⟩

lemma *syndrome-proj-dims* [*simp*]: *dim-row* (*syndrome-proj K*) = *8 dim-col* (*syndrome-proj K*) = *8*

⟨*proof*⟩

lemma *flip-bit-mat-dims* [*simp*]: *dim-row* (*flip-bit-mat b*) = *8 dim-col* (*flip-bit-mat b*) = *8*
⟨*proof*⟩

lemma *encode-mat-dims* [*simp*]: *dim-row encode-mat = 8 dim-col encode-mat = 2*
⟨*proof*⟩

lemma *flip-proj-carrier* [*simp*]: *flip-bit-mat b ∗ syndrome-proj K ∈ carrier-mat 8 8*
⟨*proof*⟩

lemma *b3-cases*: $b < (3::nat) \Longrightarrow b = 0 \vee b = 1 \vee b = 2$ ⟨*proof*⟩

lemma *mat-mult-entry-8*:
fixes *A B* :: *complex Matrix.mat*
assumes *A ∈ carrier-mat 8 8 B ∈ carrier-mat 8 8 i < 8 j < 8*
shows (*A ∗ B*) $$ (*i*,*j*) = ($\sum k<8$. *A* $$ (*i*,*k*) ∗ *B* $$ (*k*,*j*))
⟨*proof*⟩

lemma *syndrome-proj-index*:
assumes *i < 8 j < 8*
shows *syndrome-proj K* $$ (*i*,*j*) = (*if i = j ∧ i ∈ K then 1 else 0*)
⟨*proof*⟩

lemma *syndrome-proj-two*:
assumes *a*: *a < 8* **and** *b*: *b < 8* **and** *ab*: *a ≠ b*
shows *syndrome-proj* {*a*, *b*} = *unit-vec 8 a ⊘ unit-vec 8 a + unit-vec 8 b ⊘ unit-vec 8 b*
⟨*proof*⟩

The flips are the notes' tensors.

lemma *tensor3-index*:
fixes *A B C* :: *complex Matrix.mat*
assumes *A*: *A ∈ carrier-mat 2 2* **and** *B*: *B ∈ carrier-mat 2 2* **and** *C*: *C ∈ carrier-mat 2 2*
and *i*: *i < 8* **and** *j*: *j < 8*
shows *Tensor.tensor-mat* (*Tensor.tensor-mat A B*) *C* $$ (*i*,*j*)
= *A* $$ (*i div 4*, *j div 4*) ∗ *B* $$ (*i div 2 mod 2*, *j div 2 mod 2*) ∗ *C* $$ (*i mod 2*, *j mod 2*)
⟨*proof*⟩

lemma *flips-as-tensors*:
X_1 = *Tensor.tensor-mat* (*Tensor.tensor-mat Pauli-X* (1_m *2*)) (1_m *2*)
X_2 = *Tensor.tensor-mat* (*Tensor.tensor-mat* (1_m *2*) *Pauli-X*) (1_m *2*)
X_3 = *Tensor.tensor-mat* (*Tensor.tensor-mat* (1_m *2*) (1_m *2*)) *Pauli-X*
⟨*proof*⟩

lemma *flip-bit-mat-mult-vec*:
assumes *b*: *b < 3* **and** *vd*: *dim-vec v = 8* **and** *i*: *i < 8*
shows (*flip-bit-mat b* $*_v$ *v*) $ *i* = *v* $ (*if i div 2^b mod 2 = 0 then i + 2^b else i − 2^b*)
⟨*proof*⟩

lemma *syndrome-proj-mult-vec*:
assumes *vd*: *dim-vec v = 8* **and** *i*: *i < 8*
shows (*syndrome-proj K* $*_v$ *v*) $ *i* = (*if i ∈ K then v* $ *i else 0*)
⟨*proof*⟩

lemma *syndrome-proj-hermitian*: $(syndrome\text{-}proj\ K)^\dagger$ = *syndrome-proj K*
⟨*proof*⟩

lemma *syndrome-proj-idem*: *syndrome-proj K * syndrome-proj K = syndrome-proj K*
⟨*proof*⟩

lemma *syndrome-proj-sum*:
$P_0 + P_1 + P_2 + P_3 = 1_m\ 8$
⟨*proof*⟩

The flip *flip-bit-mat b* permutes the basis by the involution *flip-tgt b*.

definition *flip-tgt* :: *nat ⇒ nat ⇒ nat* **where**
flip-tgt b i = (if i div 2^b mod 2 = 0 then i + 2^b else i − 2^b)

lemma *flip-tgt-lt*:
assumes *b*: *b < 3* **and** *i*: *i < 8* **shows** *flip-tgt b i < 8*
⟨*proof*⟩

lemma *flip-tgt-invol*:
assumes *b*: *b < 3* **and** *i*: *i < 8* **shows** *flip-tgt b (flip-tgt b i) = i*
⟨*proof*⟩

lemma *flip-bit-mat-index*:
assumes *i*: *i < 8* **and** *j*: *j < 8*
shows *flip-bit-mat b $$ (i,j) = (if j = flip-tgt b i then 1 else 0)*
⟨*proof*⟩

lemma *flip-tgt-iff*:
assumes *b*: *b < 3* **and** *i*: *i < 8* **and** *j*: *j < 8*
shows *i = flip-tgt b j ⟷ j = flip-tgt b i*
⟨*proof*⟩

lemma *flip-bit-mat-hermitian*:
assumes *b*: *b < 3*
shows *(flip-bit-mat b)† = flip-bit-mat b*
⟨*proof*⟩

lemma *flip-bit-mat-squared*:
assumes *b*: *b < 3*
shows *flip-bit-mat b * flip-bit-mat b = 1_m 8*
⟨*proof*⟩

Kraus operators of the recovery map $R(Y) = P_0 Y P_0 + \sum_k X_k P_k Y P_k X_k$.

definition *bitflip-recovery-ops* :: *complex Matrix.mat list* **where**
*bitflip-recovery-ops = [P_0, X_1 * P_1, X_2 * P_2, X_3 * P_3]*

definition *bitflip-recovery-map* :: *complex Matrix.mat ⇒ complex Matrix.mat* **where**
bitflip-recovery-map = kraus-map-dim 8 bitflip-recovery-ops

lemma *flip-proj-dag-mult*:
assumes *b*: *b < 3*
shows *((flip-bit-mat b * syndrome-proj K)†) * (flip-bit-mat b * syndrome-proj K) = syndrome-proj K*
⟨*proof*⟩

theorem *bitflip-recovery-ops-kraus*: *kraus-operators 8 8 bitflip-recovery-ops*
⟨*proof*⟩

corollary *bitflip-recovery-map-CPTP*: *bitflip-recovery-map ∈ CPTP 8 8*
⟨*proof*⟩

The encoded state $\psi = a|000\rangle + b|111\rangle$ and its single-flip corruptions.

lemma *encode-mat-mult-vec*:
assumes *xd*: *dim-vec x = 2* **and** *i*: *i < 8*
shows (*encode-mat* $*_v$ *x*) $ *i* = (*if i = 0 then x $ 0 else if i = 7 then x $ 1 else 0*)
⟨*proof*⟩

lemma *encode-mat-dim* [*simp*]: *dim-vec* (*encode-mat* $*_v$ *x*) = *8* ⟨*proof*⟩

lemma *recovery-kraus-term*:
fixes φ :: *complex Matrix.vec*
assumes *φd*: *dim-vec φ = 8*
shows *bitflip-recovery-map* (*φ* ⊘ *φ*) =
(*P*0 $*_v$ *φ*) ⊘ (*P*0 $*_v$ *φ*) + ((*X*1 $*_v$ (*P*1 $*_v$ *φ*)) ⊘ (*X*1 $*_v$ (*P*1 $*_v$ *φ*))
+ ((*X*2 $*_v$ (*P*2 $*_v$ *φ*)) ⊘ (*X*2 $*_v$ (*P*2 $*_v$ *φ*))
+ ((*X*3 $*_v$ (*P*3 $*_v$ *φ*)) ⊘ (*X*3 $*_v$ (*P*3 $*_v$ *φ*)) + *0*$_m$ *8 8*)))
⟨*proof*⟩

theorem *bit-flip-code-recovery*:
fixes *x* :: *complex Matrix.vec*
assumes *xd*: *dim-vec x = 2*
defines *ψ* ≡ *encode-mat* $*_v$ *x*
shows *bitflip-recovery-map* (*ψ* ⊘ *ψ*) = *ψ* ⊘ *ψ*
and *bitflip-recovery-map* ((*X*1 $*_v$ *ψ*) ⊘ (*X*1 $*_v$ *ψ*)) = *ψ* ⊘ *ψ*
and *bitflip-recovery-map* ((*X*2 $*_v$ *ψ*) ⊘ (*X*2 $*_v$ *ψ*)) = *ψ* ⊘ *ψ*
and *bitflip-recovery-map* ((*X*3 $*_v$ *ψ*) ⊘ (*X*3 $*_v$ *ψ*)) = *ψ* ⊘ *ψ*
⟨*proof*⟩

33 Day 31 (11 November 2011): the bit-flip code as a channel; fidelity

Encoding $\tilde{\psi} = a|000\rangle + b|111\rangle$ and flipping each qubit independently with probability p gives the error map $E(Y) = (1-p)^3 Y + p(1-p)^2 \sum_j (X_j) Y (X_j)^* + \cdots + p^3 (X\otimes X\otimes X) Y (X\otimes X\otimes X)^*$, and with the recovery R of Day 30,

$$R \circ E(Y) = \big((1-p)^3 + 3p(1-p)^2\big) Y + \big(3p^2(1-p) + p^3\big)(X\otimes X\otimes X) Y (X\otimes X\otimes X).$$

The code helps iff $(1-p)^3 + 3p(1-p)^2 \geq 1-p$ iff $p \leq \frac{1}{2}$ (below, where the formula for $R \circ E$ holds for Y supported on the code space). Below: the encoding isometry.

33.1 The bit-flip channel

Doing nothing to ψ while a flip occurs with probability p produces the ensemble $\{1-p, \psi\}, \{p, X\psi\}$, i.e. the error map $E(Y) = (1-p)Y + pXYX^*$, which is completely positive and trace preserving.

definition *bit-flip-channel* ::
real ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
bit-flip-channel p ϱ ≡
complex-of-real (*1 − p*) $\cdot_m$ *ϱ*
+ *complex-of-real p* $\cdot_m$ (*Pauli-X* ∗ *ϱ* ∗ (*Pauli-X*†))

The error map is the Kraus map of $\sqrt{1-p}\, I$ and $\sqrt{p}\, X$, so it is completely positive and trace preserving (Day 22); as $XX^* = I$ it is unital too.

definition *bit-flip-kraus* :: *real* ⇒ *complex Matrix.mat list* **where**
bit-flip-kraus p = [*complex-of-real* (*sqrt* (*1 − p*)) $\cdot_m$ *1*$_m$ *2*, *complex-of-real* (*sqrt p*) $\cdot_m$ *Pauli-X*]

lemma *bit-flip-channel-kraus*:
assumes *p*: $0 \le p$ $p \le 1$ **and** ϱ: $\varrho \in$ *carrier-mat 2 2*
shows *bit-flip-channel p* ϱ = *kraus-map-dim 2* (*bit-flip-kraus p*) ϱ
⟨*proof*⟩

lemma *bit-flip-kraus-operators*:
assumes *p*: $0 \le p$ $p \le 1$
shows *kraus-operators 2 2* (*bit-flip-kraus p*)
⟨*proof*⟩

lemma *bit-flip-channel-CPTP*:
assumes *p*: $0 \le p$ $p \le 1$
shows *bit-flip-channel p* ∈ *CPTP 2 2*
⟨*proof*⟩

lemma *bit-flip-channel-UCP*:
assumes *p*: $0 \le p$ $p \le 1$
shows *bit-flip-channel p* ∈ *UCP 2 2*
⟨*proof*⟩

33.2 Fidelity

A state ψ perturbed to $\psi' = a\psi + b\psi^\perp$ is close to ψ when $|a|$ is close to 1, so the *fidelity* of states is $F = |\langle\psi'|\psi\rangle|$; for the density matrices $\rho = |\psi\rangle\langle\psi|$, $\rho' = |\psi'\rangle\langle\psi'|$ one has $\langle\rho|\rho'\rangle = \mathrm{Tr}(\rho\rho') = |\langle\psi'|\psi\rangle|^2$, so $F = \sqrt{\mathrm{Tr}(\rho\rho')}$. Error correction is compared by $F = \sqrt{\langle\psi|R \circ E(|\psi\rangle\langle\psi|)\psi\rangle}$, minimised or averaged over ψ: doing nothing gives $\min F = \sqrt{1-p}$, the bit-flip code gives $\min F = \sqrt{(1-p)^3 + 3p(1-p)^2}$ (below). Formally *fidelity* ψ ϱ is $\sqrt{\Re\langle\psi|\rho\psi\rangle}$.

definition *fidelity* ::
complex Matrix.vec ⇒ *complex Matrix.mat* ⇒ *real* **where**
fidelity ψ ϱ ≡ *sqrt* (*Re* (⟨ψ | ϱ *$_v$ ψ⟩))

lemma *fidelity-def-eq*:
fidelity ψ ϱ = *sqrt* (*Re* (⟨ψ | ϱ *$_v$ ψ⟩))
⟨*proof*⟩

lemma *fidelity-pure-states*:
fixes ψ ψ' :: *complex Matrix.vec*
assumes ψ*-qs*: *quantum-state n* ψ
and ψ'*-qs*: *quantum-state n* ψ'
shows *fidelity* ψ ((*mat-of-cols n* [ψ']) * ((*mat-of-cols n* [ψ'])†))
= *cmod* (⟨ψ | ψ'⟩)
⟨*proof*⟩

lemma *fidelity-self*:
fixes ψ :: *complex Matrix.vec*
assumes ψ*-qs*: *quantum-state n* ψ
shows *fidelity* ψ ((*mat-of-cols n* [ψ]) * ((*mat-of-cols n* [ψ])†)) = *1*
⟨*proof*⟩

lemma *fidelity-orthogonal*:
fixes ψ ψ' :: *complex Matrix.vec*
assumes ψ*-qs*: *quantum-state n* ψ
and ψ'*-qs*: *quantum-state n* ψ'
and *ortho*: ⟨ψ | ψ'⟩ = *0*
shows *fidelity* ψ ((*mat-of-cols n* [ψ']) * ((*mat-of-cols n* [ψ'])†)) = *0*
⟨*proof*⟩

33.3 The three-qubit code against independent bit flips

Day 31: the three-qubit code against independent bit flips. Each qubit flips with probability p, so $E(Y) = (1-p)^3 Y + p(1-p)^2 \sum_i X_i Y X_i + p^2(1-p) \sum_{i<j} X_i X_j Y X_j X_i + p^3 X_1 X_2 X_3 Y X_3 X_2 X_1$. On an encoded state the recovery R leaves $((1-p)^3 + 3p(1-p)^2)|\tilde{\psi}\rangle\langle\tilde{\psi}| + (3p^2(1-p) + p^3)|X_1 X_2 X_3 \tilde{\psi}\rangle\langle X_1 X_2 X_3 \tilde{\psi}|$. The notes state this formula for a general Y; it holds for Y supported on the code space (for $Y = |100\rangle\langle 100|$ and $p = 0$ one gets $R(Y) = |000\rangle\langle 000| \neq Y$).

definition *three-bit-channel* :: *real* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
three-bit-channel p M =
complex-of-real ((*1* − *p*)^*3*) ·$_m$ *M*
+ *complex-of-real* (*p* ∗ (*1* − *p*)^*2*) ·$_m$ (X_1 ∗ *M* ∗ ($X_1^\dagger$))
+ *complex-of-real* (*p* ∗ (*1* − *p*)^*2*) ·$_m$ (X_2 ∗ *M* ∗ ($X_2^\dagger$))
+ *complex-of-real* (*p* ∗ (*1* − *p*)^*2*) ·$_m$ (X_3 ∗ *M* ∗ ($X_3^\dagger$))
+ *complex-of-real* (*p*^*2* ∗ (*1* − *p*)) ·$_m$ ((X_2 ∗ X_3) ∗ *M* ∗ ((X_2 ∗ X_3)†))
+ *complex-of-real* (*p*^*2* ∗ (*1* − *p*)) ·$_m$ ((X_1 ∗ X_3) ∗ *M* ∗ ((X_1 ∗ X_3)†))
+ *complex-of-real* (*p*^*2* ∗ (*1* − *p*)) ·$_m$ ((X_1 ∗ X_2) ∗ *M* ∗ ((X_1 ∗ X_2)†))
+ *complex-of-real* (*p*^*3*) ·$_m$ ((X_1 ∗ X_2 ∗ X_3) ∗ *M* ∗ ((X_1 ∗ X_2 ∗ X_3)†))

definition *swap2* :: *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
swap2 x = *Matrix.vec 2* (λ*i*. *x* $ (*1* − *i*))

lemma *swap2-dim* [*simp*]: *dim-vec* (*swap2 x*) = *2* ⟨*proof*⟩

lemma *flips-on-codeword*:
assumes *x*: *dim-vec x* = *2*
defines ψ ≡ *encode-mat* ∗$_v$ *x* **and** φ ≡ *encode-mat* ∗$_v$ *swap2 x*
shows (X_1 ∗ X_2 ∗ X_3) ∗$_v$ ψ = φ
and (X_2 ∗ X_3) ∗$_v$ ψ = X_1 ∗$_v$ φ
and (X_1 ∗ X_3) ∗$_v$ ψ = X_2 ∗$_v$ φ
and (X_1 ∗ X_2) ∗$_v$ ψ = X_3 ∗$_v$ φ
⟨*proof*⟩

33.3.1 Errors followed by recovery

lemma *bitflip-recovery-ops-carrier'*: ∀ *W* ∈ *set bitflip-recovery-ops*. *W* ∈ *carrier-mat 8 8*
⟨*proof*⟩

lemma *bitflip-recovery-map-add*: *A* ∈ *carrier-mat 8 8* ⟹ *B* ∈ *carrier-mat 8 8* ⟹ *bitflip-recovery-map* (*A* + *B*) = *bitflip-recovery-map A* + *bitflip-recovery-map B*
⟨*proof*⟩

lemma *bitflip-recovery-map-smult*: *A* ∈ *carrier-mat 8 8* ⟹ *bitflip-recovery-map* (*c* ·$_m$ *A*) = *c* ·$_m$ *bitflip-recovery-map A*
⟨*proof*⟩

lemma *comb8*:
fixes *A B* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat n n* **and** *B*: *B* ∈ *carrier-mat n n*
shows *complex-of-real a* ·$_m$ *A* + *complex-of-real b* ·$_m$ *A* + *complex-of-real b* ·$_m$ *A* + *complex-of-real b* ·$_m$ *A*
+ *complex-of-real c* ·$_m$ *B* + *complex-of-real c* ·$_m$ *B* + *complex-of-real c* ·$_m$ *B* + *complex-of-real d* ·$_m$ *B*
= *complex-of-real* (*a* + *3* ∗ *b*) ·$_m$ *A* + *complex-of-real* (*3* ∗ *c* + *d*) ·$_m$ *B*
⟨*proof*⟩

lemma *outer8*: *dim-vec v* = *8* ⟹ *v* ⊘ *v* ∈ *carrier-mat 8 8*
⟨*proof*⟩

D31-2: errors followed by recovery on an encoded state.

theorem *three-bit-code-recovery*:
assumes *x*: *dim-vec x = 2*
defines $\psi \equiv$ *encode-mat* $*_v$ *x* **and** $\varphi \equiv$ *encode-mat* $*_v$ *swap2 x*
shows *bitflip-recovery-map* (*three-bit-channel p* ($\psi \oslash \psi$))
= *complex-of-real* ((*1 − p*)^*3* + *3* ∗ *p* ∗ (*1 − p*)^*2*) $\cdot_m$ ($\psi \oslash \psi$)
+ *complex-of-real* (*3* ∗ *p*^*2* ∗ (*1 − p*) + *p*^*3*) $\cdot_m$ ($\varphi \oslash \varphi$)
⟨*proof*⟩

The code is better than doing nothing iff $(1-p)^3 + 3p(1-p)^2 \geq 1-p$ iff $p \leq \frac{1}{2}$.

theorem *three-bit-code-better-iff*:
assumes *p0*: *0* ≤ *p* **and** *p1*: *p* < *1*
shows (*1 − p*) ≤ (*1 − p*)^*3* + *3* ∗ *p* ∗ (*1 − p*)^*2* ⟷ *p* ≤ (*1/2* :: *real*)
⟨*proof*⟩

33.3.2 Minimum fidelities (D31-4)

lemma *quad-two-outer*:
fixes ψ φ :: *complex Matrix.vec*
assumes ψ*d*: *dim-vec* ψ = *n* **and** φ*d*: *dim-vec* φ = *n*
shows ⟨ψ | (*a* $\cdot_m$ ($\psi \oslash \psi$) + *b* $\cdot_m$ ($\varphi \oslash \varphi$)) $*_v$ ψ⟩
= *a* ∗ (⟨ψ|ψ⟩ ∗ ⟨ψ|ψ⟩) + *b* ∗ (⟨φ|ψ⟩ ∗ ⟨ψ|φ⟩)
⟨*proof*⟩

lemma *encode-inner*:
assumes *u*: *dim-vec u = 2* **and** *v*: *dim-vec v = 2*
shows ⟨*encode-mat* $*_v$ *u* | *encode-mat* $*_v$ *v*⟩ = ⟨*u*|*v*⟩
⟨*proof*⟩

theorem *code-min-fidelity*:
assumes *p0*: *0* ≤ *p* **and** *p1*: *p* ≤ *1*
defines *c1* ≡ (*1 − p*)^*3* + *3* ∗ *p* ∗ (*1 − p*)^*2*
shows ∀ *x*. *dim-vec x = 2* ⟶ ⟨*x*|*x*⟩ = *1* ⟶
sqrt c1 ≤ *fidelity* (*encode-mat* $*_v$ *x*) (*bitflip-recovery-map* (*three-bit-channel p* ((*encode-mat* $*_v$ *x*) ⊘ (*encode-mat* $*_v$ *x*))))
and *fidelity* (*encode-mat* $*_v$ *unit-vec 2 0*)
(*bitflip-recovery-map* (*three-bit-channel p* ((*encode-mat* $*_v$ *unit-vec 2 0*) ⊘ (*encode-mat* $*_v$ *unit-vec 2 0*)))) = *sqrt c1*
⟨*proof*⟩

theorem *nothing-min-fidelity*:
assumes *p0*: *0* ≤ *p* **and** *p1*: *p* ≤ *1*
shows ∀ ψ. *dim-vec* ψ = *2* ⟶ ⟨ψ|ψ⟩ = *1* ⟶ *sqrt* (*1 − p*) ≤ *fidelity* ψ (*bit-flip-channel p* ($\psi \oslash \psi$))
and *fidelity* (*unit-vec 2 0*) (*bit-flip-channel p* (*unit-vec 2 0* ⊘ *unit-vec 2 0*)) = *sqrt* (*1 − p*)
⟨*proof*⟩

34 Day 32 (14 November 2011): the phase-flip code, the Shor code, Pauli magic

34.1 The phase-flip code

With $Z|0\rangle = |0\rangle$, $Z|1\rangle = -|1\rangle$ and $|\pm\rangle = (|0\rangle \pm |1\rangle)/\sqrt{2}$ one has $|+\rangle \perp |-\rangle$, $Z|+\rangle = |-\rangle$, $Z|-\rangle = |+\rangle$: Z acts on $\{|+\rangle, |-\rangle\}$ as the bit flip acts on $\{|0\rangle, |1\rangle\}$. Encoding $|0\rangle \mapsto |++\rangle$, $|1\rangle \mapsto |--\rangle$ gives the three-qubit *phase-flip code*, which detects and corrects phase flips

exactly as the bit-flip code corrects bit flips; with $\beta = Y|+\rangle$ (orthogonal to $|+\rangle$, $Y\beta = |+\rangle$) the same works for Y.

The *Shor code* encodes $|0\rangle \mapsto |0_L\rangle = 2^{-3/2}(|000\rangle + |111\rangle)^{\otimes 3}$ and $|1\rangle \mapsto |1_L\rangle = 2^{-3/2}(|000\rangle - |111\rangle)^{\otimes 3}$ in $\mathbb{C}^{2^9}$.

Theorem (notes) (Shor)**.** There is a completely positive trace-preserving $R : M_{2^9} \to M_{2^9}$ such that for every 2×2 unitary U, every position j, $\tilde{U} = I \otimes \cdots \otimes U \otimes \cdots \otimes I$, and every $\psi = a|0_L\rangle + b|1_L\rangle$, $R(\tilde{U}|\psi\rangle\langle\psi|\tilde{U}^*) = |\psi\rangle\langle\psi|$.

Proposition (notes). If $V \subseteq \mathbb{C}^n$ is a subspace and $U_1, \ldots, U_t$ are unitaries with $U_iV \perp U_jV$ for $i \neq j$, then there is a CPTP R with $R(U_i|\psi\rangle\langle\psi|U_i^*) = |\psi\rangle\langle\psi|$ for all $\psi \in V$ and all i (take $R(X) = \sum_i U_i^* P_i X P_i U_i$ with P_i the projection onto U_iV, $P_0 = I - \sum P_i$, $U_0 = I$).

The $3 \cdot 9 + 1$ *1-Pauli* operators $I \otimes \cdots \otimes U \otimes \cdots \otimes I$ (U a Pauli matrix) map $V = \mathrm{span}\{|0_L\rangle, |1_L\rangle\}$ to mutually orthogonal subspaces (up to the coincidences $Z_1 v = Z_2 v = Z_3 v$ etc.). The proposition is *recovery-single-error* in *Day32-Recovery-Proposition*, together with its linear-combination form *recovery-linear-combination* (Day 33); the Pauli expansion identity is *pauli-expansion-unitary* below. The Shor code and Shor's theorem are *Day33-Shor-Code* (*shor-code-theorem*). As first stated, the orthogonality claim is false ($Z_1 \neq Z_2$ but $Z_1|0_L\rangle = Z_2|0_L\rangle$, *shor-notes-Z1-Z2*); Day 33 corrects it.

definition *ket-plus* :: *complex Matrix.vec* **where**
ket-plus ≡ (*1 / sqrt 2* :: *complex*) ·$_v$ (*unit-vec 2 0 + unit-vec 2 1*)

definition *ket-minus* :: *complex Matrix.vec* **where**
ket-minus ≡ (*1 / sqrt 2* :: *complex*) ·$_v$ (*unit-vec 2 0 − unit-vec 2 1*)

lemma *ket-plus-carrier* [*simp*]: *ket-plus* ∈ *carrier-vec 2*
⟨*proof*⟩

lemma *ket-plus-dim* [*simp*]: *dim-vec ket-plus = 2*
⟨*proof*⟩

lemma *ket-minus-dim* [*simp*]: *dim-vec ket-minus = 2*
⟨*proof*⟩

lemma *ket-plus-index-0*: *ket-plus* $ *0 = 1 / sqrt 2*
⟨*proof*⟩

lemma *ket-plus-index-1*: *ket-plus* $ *1 = 1 / sqrt 2*
⟨*proof*⟩

$|\pm\rangle = \frac{1}{\sqrt{2}}(|0\rangle \pm |1\rangle)$ is an orthonormal basis in which X and Z exchange roles:

$$Z|+\rangle = |-\rangle, \quad Z|-\rangle = |+\rangle, \qquad X|+\rangle = |+\rangle, \quad X|-\rangle = -|-\rangle.$$

A phase flip acts on $|\pm\rangle$ as a bit flip acts on $|0\rangle, |1\rangle$, so the phase-flip code is the bit-flip code in the basis $|\pm\rangle$.

lemma *ket-plus-unit*: ‖*ket-plus*‖ = *1*
⟨*proof*⟩

lemma *ket-minus-unit*: ‖*ket-minus*‖ = *1*
⟨*proof*⟩

lemma *ket-plus-minus-orthog*: ⟨*ket-plus* | *ket-minus*⟩ = *0*
⟨*proof*⟩

lemma *Pauli-Z-ket-plus*: *Pauli-Z* $*_v$ *ket-plus* = *ket-minus*
⟨*proof*⟩

lemma *Pauli-Z-ket-minus*: *Pauli-Z* $*_v$ *ket-minus* = *ket-plus*
⟨*proof*⟩

lemma *Pauli-X-ket-plus*: *Pauli-X* $*_v$ *ket-plus* = *ket-plus*
⟨*proof*⟩

lemma *Pauli-X-ket-minus*: *Pauli-X* $*_v$ *ket-minus* = − *ket-minus*
⟨*proof*⟩

The code words $|+++\rangle$ and $|---\rangle$ of the three-qubit phase-flip code (an earlier version of this file called them *phase-code-zero*/*phase-code-one*; the nine-qubit Shor code words are not defined).

definition *phase-code-zero* :: *complex Matrix.vec* **where**
phase-code-zero ≡
let v = *unit-vec 2 0* + *unit-vec 2 1 in*
(*1* / (*2* ∗ *sqrt 2*) :: *complex*)
$·_v$ *tensor-vec* (*tensor-vec v v*) *v*

definition *phase-code-one* :: *complex Matrix.vec* **where**
phase-code-one ≡
let v = *unit-vec 2 0* − *unit-vec 2 1 in*
(*1* / (*2* ∗ *sqrt 2*) :: *complex*)
$·_v$ *tensor-vec* (*tensor-vec v v*) *v*

34.2 Algebra of projections; towards Days 33 and 34

Day 33 constructs the Shor recovery map from the projections onto the 22 orthogonal subspaces $V, X_1V, \ldots, X_9V, Y_1V, \ldots, Y_9V, Z_1V, Z_4V, Z_7V$ and proves, using $U = a_0I + a_1X + a_2Y + a_3Z$ with $\sum |a_k|^2 = 1$ for unitary U (Pauli magic), that it corrects every single-qubit unitary error. Day 34 gives the general theory:

Proposition (notes). For $V \subseteq \mathbb{C}^n$ with projection P and CPTP maps E (errors) and R (recovery), $R \circ E(|\psi\rangle\langle\psi|) = |\psi\rangle\langle\psi|$ for all $\psi \in V$ iff $R \circ E(PXP) = PXP$ for all X.

Theorem (notes) (1)**.** Let $E(X) = \sum_{i=1}^{r} E_i X E_i^*$ be CPTP. There is a CPTP R with $R \circ E(PXP) = PXP$ for all X iff $PE_i^*E_jP = \alpha_{ij}P$ for some scalars α_{ij}.

Theorem (notes) (2)**.** If R corrects E as in Theorem 1 and $G_i \in \mathrm{span}\{E_j\}$ with $\sum G_i^*G_i = I$, then R also corrects $G(X) = \sum_i G_i X G_i^*$.

Of the Knill–Laflamme results, the equivalence of Theorem 1's two readings (*recovery-states-iff-PXP*), the necessity of $PE_i^*E_jP = \alpha_{ij}P$ (*knill-laflamme-necessity*) and Theorem 2 (*knill-laflamme-closure*) are proved on Day 34, with the converse, the construction of R from the conditions, and the equivalence itself (*knill-laflamme*). The elementary algebra of orthogonal projections used by the proofs follows.

lemma *projection-complement*:
assumes *projection n P*
shows *projection n* (*1*$_m$ *n* − *P*)
⟨*proof*⟩

Two projections are orthogonal if their products in both orders vanish.

definition *orthogonal-projections* ::
nat ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *bool* **where**

orthogonal-projections n P Q ≡
projection n P ∧ projection n Q ∧
P ∗ Q = 0$_m$ n n ∧ Q ∗ P = 0$_m$ n n

lemma *orthogonal-projections-carrier-left*:
orthogonal-projections n P Q ⟹ P ∈ carrier-mat n n
⟨*proof*⟩

lemma *orthogonal-projections-carrier-right*:
orthogonal-projections n P Q ⟹ Q ∈ carrier-mat n n
⟨*proof*⟩

34.3 Pauli magic (D32-6)

Proposition (notes). I, X, Y, Z are orthogonal in M_2 and all have 2-norm $\sqrt{2}$.

proposition *pauli-orthogonal*:
shows ⟨⟨ *1$_m$ 2 | Pauli-X* ⟩⟩ = *0*
and ⟨⟨ *1$_m$ 2 | Pauli-Y* ⟩⟩ = *0*
and ⟨⟨ *1$_m$ 2 | Pauli-Z* ⟩⟩ = *0*
and ⟨⟨ *Pauli-X | Pauli-Y* ⟩⟩ = *0*
and ⟨⟨ *Pauli-X | Pauli-Z* ⟩⟩ = *0*
and ⟨⟨ *Pauli-Y | Pauli-Z* ⟩⟩ = *0*
⟨*proof*⟩

proposition *pauli-norm*:
shows ⟨⟨ *1$_m$ 2 | (1$_m$ 2 :: complex Matrix.mat)* ⟩⟩ = *2*
and ⟨⟨ *Pauli-X | Pauli-X* ⟩⟩ = *2*
and ⟨⟨ *Pauli-Y | Pauli-Y* ⟩⟩ = *2*
and ⟨⟨ *Pauli-Z | Pauli-Z* ⟩⟩ = *2*
⟨*proof*⟩

34.4 The Pauli expansion identity for unitaries (D32-7)

lemma *cmod-parallelogram*:
fixes *a b* :: *complex*
shows (*cmod* (*a* + *b*))^2 + (*cmod* (*a* − *b*))^2 = *2* ∗ (*cmod a*)^2 + *2* ∗ (*cmod b*)^2
⟨*proof*⟩

proposition *pauli-expansion-unitary*:
fixes *U* :: *complex Matrix.mat* **and** *a0 a1 a2 a3* :: *complex*
assumes *Uu*: *unitary U*
and *U-eq*: *U = a0 ·$_m$ 1$_m$ 2 + a1 ·$_m$ Pauli-X + a2 ·$_m$ Pauli-Y + a3 ·$_m$ Pauli-Z*
shows (*cmod a0*)^2 + (*cmod a1*)^2 + (*cmod a2*)^2 + (*cmod a3*)^2 = *1*
⟨*proof*⟩

34.5 The general recovery proposition (D32-4, D33-2)

34.5.1 Matrix sums acting on vectors

lemma *smult-zero-vec* [*simp*]: (*c* :: *complex*) ·$_v$ *0$_v$ n* = *0$_v$ n*
⟨*proof*⟩

lemma *zero-mat-mult-vec*:
assumes *dim-vec w = n* **shows** (*0$_m$ m n* :: *complex Matrix.mat*) ∗$_v$ *w = 0$_v$ m*
⟨*proof*⟩

lemma *mult-mat-vec-zero-vec*:
assumes *Ac*: $A \in$ *carrier-mat m n* **shows** $A *_v (0_v\ n ::$ *complex Matrix.vec*$) = 0_v\ m$
⟨*proof*⟩

lemma *foldr-add-mat-mult-vec-zero*:
fixes $f ::$ *'a* $\Rightarrow$ *complex Matrix.mat*
assumes *fc*: $\forall x \in$ *set xs*. $f\ x \in$ *carrier-mat n n* **and** *wd*: *dim-vec* $w = n$
and *z*: $\bigwedge x.\ x \in$ *set xs* $\Longrightarrow f\ x *_v\ w = 0_v\ n$
shows *foldr* $(\lambda x\ acc.\ f\ x +\ acc)$ *xs* $(0_m\ n\ n) *_v\ w = 0_v\ n$
⟨*proof*⟩

lemma *foldr-add-mat-mult-vec-delta*:
fixes $f ::$ *nat* $\Rightarrow$ *complex Matrix.mat*
assumes *fc*: $\forall x \in$ *set xs*. $f\ x \in$ *carrier-mat n n* **and** *wd*: *dim-vec* $w = n$
and *d*: *distinct xs* **and** *j*: $j \in$ *set xs*
and *del*: $\bigwedge x.\ x \in$ *set xs* $\Longrightarrow f\ x *_v\ w =$ (*if* $x = j$ *then* w *else* $0_v\ n$)
shows *foldr* $(\lambda x\ acc.\ f\ x +\ acc)$ *xs* $(0_m\ n\ n) *_v\ w = w$
⟨*proof*⟩

lemma *vec-sum-delta*:
assumes *fin*: *finite Js* **and** *j*: $j \in$ *Js* **and** *wd*: $\bigwedge i.\ i \in$ *Js* $\Longrightarrow$ *dim-vec* $(w\ i) = n$
shows *vec-sum n Js* $(\lambda i.$ *if* $i = j$ *then* $w\ i$ *else* $0_v\ n) = w\ j$
⟨*proof*⟩

Let $V = \operatorname{ran} P$ for a projection P and let $U_0, \ldots, U_{t-1}$ be unitaries with $U_i V \perp U_j V$ for $i \neq j$, i.e. $PU_i^* U_j P = 0$. With $P_i = U_i P U_i^*$, $Q = I - \sum_i P_i$ and the Kraus operators PU_i^* (the notes write $U_i^* P_i = PU_i^*$) the map $R(X) = QXQ + \sum_i PU_i^* X U_i P$ is CPTP and recovers every $\psi \in V$ from any $\varphi = \sum_i a_i U_i \psi$: $R(|\varphi\rangle\langle\varphi|) = (\sum_i |a_i|^2)|\psi\rangle\langle\psi|$.

definition *image-proj-sum* :: *nat* $\Rightarrow$ *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat list* $\Rightarrow$ *complex Matrix.mat* **where**
image-proj-sum n P Us = *foldr* $(\lambda i\ acc.\ Us\ !\ i * P * ((Us\ !\ i)^\dagger) +\ acc)$ $[0..<$*length Us*$]$ $(0_m\ n\ n)$

definition *recovery-ops* :: *nat* $\Rightarrow$ *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat list* $\Rightarrow$ *complex Matrix.mat list* **where**
recovery-ops n P Us = $(1_m\ n -$ *image-proj-sum n P Us*$)$ # *map* $(\lambda i.\ P * ((Us\ !\ i)^\dagger))$ $[0..<$*length Us*$]$

The hypotheses of the proposition, bundled: P a projection and $PU_i^* U_j P = \delta_{ij} P$. For unitaries U_i the case $i = j$ is automatic (*recovery-setting-unitaries*); the weaker form also covers the rescaled errors of the Knill–Laflamme theorem.

definition *recovery-setting* :: *nat* $\Rightarrow$ *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat list* $\Rightarrow$ *bool* **where**
recovery-setting n P Us $\longleftrightarrow$ *projection n P* $\wedge$ $(\forall U \in$ *set Us*. $U \in$ *carrier-mat n n*$)$
$\wedge$ $(\forall i <$ *length Us*. $\forall j <$ *length Us*. $P * (((Us\ !\ i)^\dagger) * Us\ !\ j) * P =$ (*if* $i = j$ *then* P *else* $0_m\ n\ n$)$)$

lemma *recovery-settingD*:
assumes *recovery-setting n P Us*
shows *projection n P* $\forall U \in$ *set Us*. $U \in$ *carrier-mat n n*
$\forall i <$ *length Us*. $\forall j <$ *length Us*. $i \neq j \longrightarrow P * (((Us\ !\ i)^\dagger) * Us\ !\ j) * P = 0_m\ n\ n$
$\forall i <$ *length Us*. $P * (((Us\ !\ i)^\dagger) * Us\ !\ i) * P = P$
⟨*proof*⟩

lemma *recovery-setting-unitaries*:
assumes *P*: *projection n P* **and** *Us*: $\forall U \in$ *set Us*. $U \in$ *carrier-mat n n* $\wedge$ *unitary U*
and *orth*: $\forall i <$ *length Us*. $\forall j <$ *length Us*. $i \neq j \longrightarrow P * (((Us\ !\ i)^\dagger) * Us\ !\ j) * P = 0_m\ n\ n$
shows *recovery-setting n P Us*
⟨*proof*⟩

lemma

assumes *S*: *recovery-setting n P Us*
shows *rec-Pc*: $P \in$ *carrier-mat n n* **and** *rec-Ph*: $P^\dagger = P$ **and** *rec-PP*: $P * P = P$
⟨*proof*⟩

lemma
assumes *S*: *recovery-setting n P Us*
shows *rec-Uc*: $i <$ *length Us* $\Longrightarrow$ *Us* ! $i \in$ *carrier-mat n n*
and *rec-PUUP*: $i <$ *length Us* $\Longrightarrow P * (((Us\ !\ i)^\dagger) * Us\ !\ i) * P = P$
⟨*proof*⟩

lemma *rec-Pi-c*:
assumes *S*: *recovery-setting n P Us* **and** *i*: $i <$ *length Us*
shows *Us* ! $i * P * ((Us\ !\ i)^\dagger) \in$ *carrier-mat n n*
⟨*proof*⟩

Under $PU_i^* U_j P = \delta_{ij} P$ the images $P_i = U_i P U_i^*$ of the code projection are mutually orthogonal projections,

$$P_i P_j = U_i \, (P U_i^* U_j P) \, U_j^* = \delta_{ij} \, U_i P U_i^*,$$

so $Q = \sum_i P_i$ is a projection. The recovery operators $I - Q$ and PU_i^* then satisfy $(I-Q)^*(I-Q) + \sum_i (PU_i^*)^*(PU_i^*) = (I-Q) + Q = I$.

lemma *rec-Pi-mult*:
assumes *S*: *recovery-setting n P Us* **and** *i*: $i <$ *length Us* **and** *j*: $j <$ *length Us*
shows $(Us\ !\ i * P * ((Us\ !\ i)^\dagger)) * (Us\ !\ j * P * ((Us\ !\ j)^\dagger))$
$= (\textit{if } i = j \textit{ then } Us\ !\ i * P * ((Us\ !\ i)^\dagger) \textit{ else } 0_m\ n\ n)$
⟨*proof*⟩

lemma *image-proj-sum-carrier*:
assumes *S*: *recovery-setting n P Us*
shows *image-proj-sum n P Us* $\in$ *carrier-mat n n*
⟨*proof*⟩

lemma *image-proj-sum-hermitian*:
assumes *S*: *recovery-setting n P Us*
shows $(\textit{image-proj-sum } n\ P\ Us)^\dagger = \textit{image-proj-sum } n\ P\ Us$
⟨*proof*⟩

lemma *image-proj-sum-idem*:
assumes *S*: *recovery-setting n P Us*
shows *image-proj-sum n P Us* $*$ *image-proj-sum n P Us* $=$ *image-proj-sum n P Us*
⟨*proof*⟩

lemma *image-proj-sum-projection*:
assumes *S*: *recovery-setting n P Us*
shows *projection n* (*image-proj-sum n P Us*)
⟨*proof*⟩

lemma *dagger-one-minus*:
assumes *Sc*: $M \in$ *carrier-mat n n* **and** *Sh*: $M^\dagger = M$
shows $(1_m\ n - M)^\dagger = 1_m\ n - M$
⟨*proof*⟩

lemma *recovery-ops-carrier*:
assumes *S*: *recovery-setting n P Us*
shows $\forall\ W \in$ *set* (*recovery-ops n P Us*). $W \in$ *carrier-mat n n*
⟨*proof*⟩

theorem *recovery-ops-kraus-operators*:

assumes *S*: *recovery-setting n P Us*
shows *kraus-operators n n* (*recovery-ops n P Us*)
⟨*proof*⟩

corollary *recovery-map-CPTP′*:
assumes *S*: *recovery-setting n P Us* **and** *npos*: *0 < n*
shows *kraus-map-dim n* (*recovery-ops n P Us*) ∈ *CPTP n n*
⟨*proof*⟩

Action of the Kraus operators on $\varphi = \sum_i a_i U_i \psi$, $\psi \in V$.

lemma *rec-kraus-on-image*:
assumes *S*: *recovery-setting n P Us*
and *ψd*: *dim-vec ψ = n* **and** *ψV*: *P *$_v$ ψ = ψ* **and** *i*: *i < length Us* **and** *j*: *j < length Us*
shows (*P* ∗ ((*Us* ! *i*)†)) *$*_v$ (*Us* ! *j *$_v$ ψ*) = (*if i = j then ψ else 0*$_v$ *n*)
⟨*proof*⟩

lemma *rec-Q-on-image*:
assumes *S*: *recovery-setting n P Us*
and *ψd*: *dim-vec ψ = n* **and** *ψV*: *P *$_v$ ψ = ψ* **and** *j*: *j < length Us*
shows (*1*$_m$ *n* − *image-proj-sum n P Us*) *$*_v$ (*Us* ! *j *$_v$ ψ*) = *0*$_v$ *n*
⟨*proof*⟩

theorem *recovery-linear-combination*:
fixes *a* :: *nat* ⇒ *complex*
assumes *S*: *recovery-setting n P Us* **and** *ψd*: *dim-vec ψ = n* **and** *ψV*: *P *$_v$ ψ = ψ*
defines *φ* ≡ *vec-sum n* {*i*. *i < length Us*} (λ*i*. *a i ·*$_v$ (*Us* ! *i *$_v$ ψ*))
shows *kraus-map-dim n* (*recovery-ops n P Us*) (*φ* ⊘ *φ*)
= *complex-of-real* (∑ *i < length Us*. (*cmod* (*a i*))^2) ·$_m$ (*ψ* ⊘ *ψ*)
⟨*proof*⟩

corollary *recovery-single-error*:
assumes *S*: *recovery-setting n P Us*
and *ψd*: *dim-vec ψ = n* **and** *ψV*: *P *$_v$ ψ = ψ* **and** *j*: *j < length Us*
shows *kraus-map-dim n* (*recovery-ops n P Us*) ((*Us* ! *j *$_v$ ψ*) ⊘ (*Us* ! *j *$_v$ ψ*)) = *ψ* ⊘ *ψ*
⟨*proof*⟩

34.6 The code for Y flips

definition *block-op* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
block-op s U = Tensor.tensor-mat (*Tensor.tensor-mat* (*1*$_m$ (*2*^*s*)) *U*) (*1*$_m$ (*2*^(*2*−*s*)))

lemma *block-op-cases*:
fixes *U* :: *complex Matrix.mat*
assumes *U*: *U* ∈ *carrier-mat 2 2*
shows *block-op 0 U = Tensor.tensor-mat* (*Tensor.tensor-mat U* (*1*$_m$ *2*)) (*1*$_m$ *2*)
and *block-op 1 U = Tensor.tensor-mat* (*Tensor.tensor-mat* (*1*$_m$ *2*) *U*) (*1*$_m$ *2*)
and *block-op 2 U = Tensor.tensor-mat* (*Tensor.tensor-mat* (*1*$_m$ *2*) (*1*$_m$ *2*)) *U*
⟨*proof*⟩

lemma *block-op-carrier*:
assumes *U*: *U* ∈ *carrier-mat 2 2* **and** *s*: *s < 3*
shows *block-op s U* ∈ *carrier-mat 8 8*
⟨*proof*⟩

lemma *gram-entry*:
fixes *A B* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat n k* **and** *B*: *B* ∈ *carrier-mat n k* **and** *a*: *a < k* **and** *b*: *b < k*
shows ((*A*†) ∗ *B*) $$ (*a*,*b*) = ⟨*Matrix.col A a* | *Matrix.col B b*⟩

⟨*proof*⟩

lemma *sandwich-VVd*:
fixes *V W* :: *complex Matrix.mat*
assumes *V*: *V* ∈ *carrier-mat n k* **and** *W*: *W* ∈ *carrier-mat n n*
shows (*V* ∗ (*V*†)) ∗ *W* ∗ (*V* ∗ (*V*†)) = *V* ∗ ((*V*†) ∗ *W* ∗ *V*) ∗ (*V*†)
⟨*proof*⟩

Day 32: the code for Y flips. With $\beta = Y|+\rangle$ one has $|+\rangle \perp \beta$, $Y|+\rangle = \beta$, $Y\beta = |+\rangle$, and the code $|0\rangle \mapsto |++\rangle$, $|1\rangle \mapsto |\beta\beta\rangle$ corrects one Y flip exactly as the bit-flip code corrects one X flip.

definition *ket-beta* :: *complex Matrix.vec* **where** *ket-beta* = i ·$_v$ *ket-minus*

definition *y-basis* :: *bool* ⇒ *complex Matrix.vec* **where** *y-basis s* = (*if s then ket-plus else ket-beta*)

lemma *ket-beta-dim* [*simp*]: *dim-vec ket-beta* = *2* ⟨*proof*⟩
lemma *y-basis-dim* [*simp*]: *dim-vec* (*y-basis s*) = *2* ⟨*proof*⟩

lemma *ket-beta-index*: *ket-beta* $ *0* = i / *sqrt 2 ket-beta* $ *1* = − i / *sqrt 2*
⟨*proof*⟩

lemma *Y-plus*: *Pauli-Y* ∗$_v$ *ket-plus* = *ket-beta*
⟨*proof*⟩

lemma *Y-beta*: *Pauli-Y* ∗$_v$ *ket-beta* = *ket-plus*
⟨*proof*⟩

lemma *Y-ub*: *Pauli-Y* ∗$_v$ *y-basis s* = *y-basis* (¬ *s*)
⟨*proof*⟩

lemma *inner2*: *dim-vec u* = *2* ⟹ *dim-vec v* = *2* ⟹ ⟨*u*|*v*⟩ = *cnj* (*u* $ *0*) ∗ *v* $ *0* + *cnj* (*u* $ *1*) ∗ *v* $ *1*
⟨*proof*⟩

$$\beta = Y|+\rangle = \frac{i}{\sqrt{2}}\begin{pmatrix}1\\-1\end{pmatrix}, \qquad Y\beta = |+\rangle, \qquad \langle+|\beta\rangle = 0,$$

so $\{|+\rangle, \beta\}$ is an orthonormal basis that Y swaps, as X swaps $|0\rangle$ and $|1\rangle$.

lemma *y-basis-inner*: ⟨*y-basis s* | *y-basis t*⟩ = (*if s* = *t then 1 else 0*)
⟨*proof*⟩

theorem *ket-plus-beta-orth*: ⟨*ket-plus* | *ket-beta*⟩ = *0*
⟨*proof*⟩

34.6.1 Codes with orthogonal error images

For a code with orthonormal basis $v_1, \ldots, v_k$ (the columns of V, projection VV^*) and unitary errors whose images of the basis are mutually orthogonal, the recovery proposition gives one CPTP map correcting each error on every code state Vc.

lemma *proj-of-orthonormal*:
assumes *orth*: *orthonormal-list n vs*
defines *V* ≡ *mat-of-cols n vs*
shows *projection n* (*V* ∗ (*V*†))
⟨*proof*⟩

lemma *proj-fixes-code*:

assumes *orth*: *orthonormal-list n vs* **and** *c*: *dim-vec c = length vs*
defines $V \equiv$ *mat-of-cols n vs*
shows $(V * (V^\dagger)) *_v (V *_v c) = V *_v c$
⟨*proof*⟩

theorem *orthogonal-images-recovery*:
assumes *orth*: *orthonormal-list n vs*
and *Us*: $\forall U \in$ *set Us*. $U \in$ *carrier-mat n n* $\wedge$ *unitary U*
and *img*: $\forall i <$ *length Us*. $\forall j <$ *length Us*. $i \neq j \longrightarrow$
$(\forall a <$ *length vs*. $\forall b <$ *length vs*. $\langle Us ! i *_v vs ! a \mid Us ! j *_v vs ! b\rangle = 0)$
defines $V \equiv$ *mat-of-cols n vs*
shows *recovery-setting n* $(V * (V^\dagger))$ *Us*
⟨*proof*⟩

corollary *orthogonal-images-correct*:
assumes *orth*: *orthonormal-list n vs*
and *Us*: $\forall U \in$ *set Us*. $U \in$ *carrier-mat n n* $\wedge$ *unitary U*
and *img*: $\forall i <$ *length Us*. $\forall j <$ *length Us*. $i \neq j \longrightarrow$
$(\forall a <$ *length vs*. $\forall b <$ *length vs*. $\langle Us ! i *_v vs ! a \mid Us ! j *_v vs ! b\rangle = 0)$
shows $\exists$ *Rs*. *kraus-operators n n Rs* $\wedge$
$(\forall (c ::$ *complex Matrix.vec*$)$ *j*. *dim-vec c = length vs* $\longrightarrow$ *j < length Us* $\longrightarrow$
kraus-map-dim n Rs (*Us* ! *j* $*$ ((*mat-of-cols n vs* $*_v$ *c*) $\oslash$ (*mat-of-cols n vs* $*_v$ *c*)) $*$ ((*Us* ! *j*)†))
$=$ (*mat-of-cols n vs* $*_v$ *c*) $\oslash$ (*mat-of-cols n vs* $*_v$ *c*))
⟨*proof*⟩

34.6.2 The Y-flip code

definition *y-word* :: *bool* $\Rightarrow$ *bool* $\Rightarrow$ *bool* $\Rightarrow$ *complex Matrix.vec* **where**
y-word a b c = tensor-vec (*tensor-vec* (*y-basis a*) (*y-basis b*)) (*y-basis c*)

definition *y-errors* :: *complex Matrix.mat list* **where**
y-errors = [1_m 8, *block-op 0 Pauli-Y*, *block-op 1 Pauli-Y*, *block-op 2 Pauli-Y*]

definition *y-codewords* :: *complex Matrix.vec list* **where**
y-codewords = [*y-word True True True*, *y-word False False False*]

lemma *y-word-dim* [*simp*]: *dim-vec* (*y-word a b c*) = 8
⟨*proof*⟩

The code $|0\rangle \mapsto |{+}{+}{+}\rangle$, $|1\rangle \mapsto |\beta\beta\beta\rangle$ is the bit-flip code in the basis $\{|+\rangle, \beta\}$. The eight products of $|+\rangle$ and β are orthonormal and Y on qubit s swaps the factor s, so the images of the code under I, Y_0, Y_1, Y_2 are mutually orthogonal.

lemma *y-word-inner*: $\langle$*y-word a b c* | *y-word a′ b′ c′*$\rangle$ = (*if a = a′* $\wedge$ *b = b′* $\wedge$ *c = c′ then 1 else 0*)
⟨*proof*⟩

lemma *y-actions*:
block-op 0 Pauli-Y $*_v$ *y-word a b c = y-word* ($\neg$ *a*) *b c*
block-op 1 Pauli-Y $*_v$ *y-word a b c = y-word a* ($\neg$ *b*) *c*
block-op 2 Pauli-Y $*_v$ *y-word a b c = y-word a b* ($\neg$ *c*)
⟨*proof*⟩

lemma *y-codewords-orthonormal*: *orthonormal-list 8 y-codewords*
⟨*proof*⟩

lemma *y-errors-unitary*: $\forall U \in$ *set y-errors*. $U \in$ *carrier-mat 8 8* $\wedge$ *unitary U*
⟨*proof*⟩

lemma *y-images-orthogonal*:
$\forall\, i < \textit{length y-errors}.\ \forall\, j < \textit{length y-errors}.\ i \neq j \longrightarrow$
$(\forall\, a < \textit{length y-codewords}.\ \forall\, b < \textit{length y-codewords}.$
$\langle \textit{y-errors} \mathbin{!} i *_v \textit{y-codewords} \mathbin{!} a \mid \textit{y-errors} \mathbin{!} j *_v \textit{y-codewords} \mathbin{!} b\rangle = 0)$
⟨*proof*⟩

D32-2: one CPTP map corrects every single Y flip on every state $a|{+}{+}{+}\rangle + b|\beta\beta\beta\rangle$.

theorem *y-code-corrects*:
$\exists\, \textit{Rs}.\ \textit{kraus-operators 8 8 Rs} \wedge$
$(\forall\, c\ j.\ \textit{dim-vec}\ c = 2 \longrightarrow j < 4 \longrightarrow$
$\textit{kraus-map-dim 8 Rs}\ (\textit{y-errors} \mathbin{!} j * ((\textit{mat-of-cols 8 y-codewords} *_v c) \oslash (\textit{mat-of-cols 8 y-codewords} *_v c)) * ((\textit{y-errors} \mathbin{!} j)^\dagger))$
$= (\textit{mat-of-cols 8 y-codewords} *_v c) \oslash (\textit{mat-of-cols 8 y-codewords} *_v c))$
⟨*proof*⟩

35 Day 33 (16 November 2011): the nine-qubit Shor code

The nine-qubit Shor code of Day 32 and its recovery. The 2^9-dimensional space is never handled entrywise: nine qubits are three blocks of three, every single-qubit operator acts inside one block, and all inner products factor over the blocks (*tensor-inner-prod*).

35.1 Single-qubit operators on nine qubits, and on a block of three

$\tilde{U} = I \otimes \cdots \otimes I \otimes U \otimes I \otimes \cdots \otimes I$ with U in position i ($i = 0, \ldots, 8$).

definition *qubit-op* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
qubit-op i U = *Tensor.tensor-mat* (*Tensor.tensor-mat* (1_m (2^*i*)) *U*) (1_m (2^(8−*i*)))

definition *on-block* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
on-block b M = *Tensor.tensor-mat* (*Tensor.tensor-mat* (1_m (8^*b*)) *M*) (1_m (8^(2−*b*)))

$\tilde{U}_i = I_{2^i} \otimes U \otimes I_{2^{8-i}}$ applies U to qubit i. In three blocks of three qubits, qubit $3b + s$ is qubit s of block b:

$$\tilde{U}_{3b+s} = I_{8^b} \otimes \left(I_{2^s} \otimes U \otimes I_{2^{2-s}}\right) \otimes I_{8^{2-b}},$$

so every computation on $\mathbb{C}^{512}$ reduces to one on $\mathbb{C}^8$.

lemma *qubit-op-carrier*:
assumes *U*: *U* ∈ *carrier-mat 2 2* **and** *i*: *i* < *9*
shows *qubit-op i U* ∈ *carrier-mat 512 512*
⟨*proof*⟩

lemma *qubit-op-blk*:
assumes *b*: *b* < *3* **and** *s*: *s* < *3*
shows *qubit-op* (*3* ∗ *b* + *s*) *U* = *on-block b* (*block-op s U*)
⟨*proof*⟩

lemma *qubit-op-blk-div*:
assumes *i*: *i* < *9*
shows *qubit-op i U* = *on-block* (*i div 3*) (*block-op* (*i mod 3*) *U*)
⟨*proof*⟩

definition *tensor3* :: *complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
tensor3 x y z = *tensor-vec* (*tensor-vec x y*) *z*

lemma *tensor3-dim*: *dim-vec x = 8* ⟹ *dim-vec y = 8* ⟹ *dim-vec z = 8* ⟹ *dim-vec (tensor3 x y z) = 512*
⟨*proof*⟩

lemma *on-block-tv3*:
assumes *M*: *M* ∈ *carrier-mat 8 8* **and** *x*: *dim-vec x = 8* **and** *y*: *dim-vec y = 8* **and** *z*: *dim-vec z = 8*
shows *on-block 0 M* ∗$_v$ *tensor3 x y z = tensor3 (M* ∗$_v$ *x) y z*
and *on-block 1 M* ∗$_v$ *tensor3 x y z = tensor3 x (M* ∗$_v$ *y) z*
and *on-block 2 M* ∗$_v$ *tensor3 x y z = tensor3 x y (M* ∗$_v$ *z)*
⟨*proof*⟩

35.2 The block vectors $|m\rangle \pm |7-m\rangle$

In a block of three qubits, *ghz-vec p 0* is $|000\rangle \pm |111\rangle$; a bit flip moves it to *ghz-vec p m* with $m \in \{4, 2, 1\}$. These eight vectors are mutually orthogonal.

definition *sign-pm* :: *bool* ⇒ *complex* **where** *sign-pm p = (if p then 1 else −1)*

definition *ghz-vec* :: *bool* ⇒ *nat* ⇒ *complex Matrix.vec* **where**
ghz-vec p m = unit-vec 8 m + sign-pm p ·$_v$ *unit-vec 8 (7 − m)*

On a block, with $m = 2^{2-s}$ for qubit s,

$$X_s(|000\rangle \pm |111\rangle) = |m\rangle \pm |7-m\rangle, \qquad Z_s(|000\rangle \pm |111\rangle) = |000\rangle \mp |111\rangle,$$

and Y_s does both, with the factor $-i$. The eight vectors $|m\rangle \pm |7-m\rangle$, $m \in \{0, 1, 2, 4\}$, are orthogonal with norm $\sqrt{2}$.

lemma *ghz-vec-dim* [*simp*]: *dim-vec (ghz-vec p m) = 8*
⟨*proof*⟩

lemma *ghz-vec-index*:
assumes *r*: *r < 8* **and** *m*: *m < 8*
shows *ghz-vec p m* $ *r = (if r = m then 1 else 0) + sign-pm p* ∗ *(if r = 7 − m then 1 else 0)*
⟨*proof*⟩

lemma *mult-gvec0*:
assumes *M*: *M* ∈ *carrier-mat 8 8* **and** *r*: *r < 8*
shows *(M* ∗$_v$ *ghz-vec p 0) $ r = M $$ (r,0) + sign-pm p* ∗ *M $$ (r,7)*
⟨*proof*⟩

lemma *block-op-entry-cases*:
assumes *U*: *U* ∈ *carrier-mat 2 2* **and** *r*: *r < 8* **and** *k*: *k < 8*
shows *block-op 0 U $$ (r,k) = U $$ (r div 4, k div 4)* ∗ 1_m *2 $$ (r div 2 mod 2, k div 2 mod 2)* ∗ 1_m *2 $$ (r mod 2, k mod 2)*
and *block-op 1 U $$ (r,k) =* 1_m *2 $$ (r div 4, k div 4)* ∗ *U $$ (r div 2 mod 2, k div 2 mod 2)* ∗ 1_m *2 $$ (r mod 2, k mod 2)*
and *block-op 2 U $$ (r,k) =* 1_m *2 $$ (r div 4, k div 4)* ∗ 1_m *2 $$ (r div 2 mod 2, k div 2 mod 2)* ∗ *U $$ (r mod 2, k mod 2)*
⟨*proof*⟩

lemma *block-op-X-gvec*:
assumes *s*: *s < 3*
shows *block-op s Pauli-X* ∗$_v$ *ghz-vec p 0 = ghz-vec p (2^(2−s))*
⟨*proof*⟩

lemma *block-op-Y-gvec*:
assumes *s*: *s < 3*
shows *block-op s Pauli-Y* ∗$_v$ *ghz-vec p 0 =* (−i) ·$_v$ *ghz-vec (¬ p) (2^(2−s))*

⟨*proof*⟩

lemma *block-op-Z-gvec*:
assumes *s*: $s < 3$
shows *block-op s Pauli-Z* $*_v$ *ghz-vec p 0* $=$ *ghz-vec* $(\neg\ p)$ *0*
⟨*proof*⟩

lemma *inner-two-units*:
assumes *a*: $a < n$ **and** *b*: $b < n$ **and** *c*: $c < n$ **and** *d*: $d < n$
shows ⟨*unit-vec n a* $+ \alpha \cdot_v$ *unit-vec n b* | *unit-vec n c* $+ \beta \cdot_v$ *unit-vec n d*⟩
$=$ (*if a* $=$ *c then 1 else 0*) $+ \beta *$ (*if a* $=$ *d then 1 else 0*)
$+$ *cnj* $\alpha *$ (*if b* $=$ *c then 1 else 0*) $+$ *cnj* $\alpha * \beta *$ (*if b* $=$ *d then 1 else 0*)
⟨*proof*⟩

lemma *ghz-vec-inner*:
assumes *m*: $m \in \{0,1,2,4\}$ **and** *m'*: $m' \in \{0,1,2,4\}$
shows ⟨*ghz-vec p m* | *ghz-vec p' m'*⟩ $=$ (*if* $m = m' \wedge p = p'$ *then 2 else 0*)
⟨*proof*⟩

35.3 Code words and the images of the errors

definition *ghz-vec-of* :: *bool* $\times$ *nat* $\Rightarrow$ *complex Matrix.vec* **where** *ghz-vec-of x* $=$ *ghz-vec* (*fst x*) (*snd x*)

A label lists the block vectors of a product state, one per block.

definition *label-vec* :: (*bool* $\times$ *nat*) *list* $\Rightarrow$ *complex Matrix.vec* **where**
label-vec l $=$ *tensor3* (*ghz-vec-of* (*l* ! *0*)) (*ghz-vec-of* (*l* ! *1*)) (*ghz-vec-of* (*l* ! *2*))

definition *valid-label* :: (*bool* $\times$ *nat*) *list* $\Rightarrow$ *bool* **where**
valid-label l $\longleftrightarrow$ *length l* $= 3 \wedge (\forall x \in$ *set l. snd x* $\in \{0,1,2,4\})$

definition *shor-scale* :: *complex* **where** *shor-scale* $=$ *complex-of-real* (*1 / sqrt 8*)

definition *shor-cw* :: *bool* $\Rightarrow$ *complex Matrix.vec* **where**
shor-cw p $=$ *shor-scale* $\cdot_v$ *label-vec* (*replicate 3* (*p*, *0*))

The notes' code words $|0_L\rangle, |1_L\rangle$.

lemma *shor-cw-notes*:
shor-cw True $=$ *complex-of-real* (*1 / sqrt* (*2^3*)) $\cdot_v$
tensor-vec (*tensor-vec* (*unit-vec 8 0* $+$ *unit-vec 8 7*) (*unit-vec 8 0* $+$ *unit-vec 8 7*)) (*unit-vec 8 0* $+$ *unit-vec 8 7*)
shor-cw False $=$ *complex-of-real* (*1 / sqrt* (*2^3*)) $\cdot_v$
tensor-vec (*tensor-vec* (*unit-vec 8 0* $-$ *unit-vec 8 7*) (*unit-vec 8 0* $-$ *unit-vec 8 7*)) (*unit-vec 8 0* $-$ *unit-vec 8 7*)
⟨*proof*⟩

lemma *ghz-vec-of-dim* [*simp*]: *dim-vec* (*ghz-vec-of x*) $=$ *8*
⟨*proof*⟩

lemma *label-vec-dim* [*simp*]: *dim-vec* (*label-vec l*) $=$ *512*
⟨*proof*⟩

lemma *shor-cw-dim* [*simp*]: *dim-vec* (*shor-cw p*) $=$ *512*
⟨*proof*⟩

lemma *tensor3-smult*:
assumes *dim-vec x* $=$ *8 dim-vec y* $=$ *8 dim-vec z* $=$ *8*
shows *tensor3* $(k \cdot_v x)$ *y z* $=$ *k* $\cdot_v$ *tensor3 x y z tensor3 x* $(k \cdot_v y)$ *z* $=$ *k* $\cdot_v$ *tensor3 x y z*
tensor3 x y $(k \cdot_v z) = k \cdot_v$ *tensor3 x y z*

⟨*proof*⟩

lemma *qubit-op-tl*:
assumes *i*: $i < 9$ **and** *U*: $U \in$ *carrier-mat 2 2*
and *act*: *block-op* $(i \bmod 3)$ $U *_v$ *ghz-vec p 0* $= \kappa \cdot_v$ *ghz-vec q m*
shows *qubit-op i U* $*_v$ *label-vec* (*replicate 3* $(p, 0)$) $= \kappa \cdot_v$ *label-vec* ((*replicate 3* $(p, 0)$)[*i div 3* := (q, m)])
⟨*proof*⟩

lemma *qubit-op-X-tl*:
$i < 9 \Longrightarrow$ *qubit-op i Pauli-X* $*_v$ *label-vec* (*replicate 3* $(p, 0)$)
$=$ *label-vec* ((*replicate 3* $(p, 0)$)[*i div 3* := $(p, 2\hat{}(2 - i \bmod 3))$])
⟨*proof*⟩

lemma *qubit-op-Y-tl*:
$i < 9 \Longrightarrow$ *qubit-op i Pauli-Y* $*_v$ *label-vec* (*replicate 3* $(p, 0)$)
$= (-\mathrm{i}) \cdot_v$ *label-vec* ((*replicate 3* $(p, 0)$)[*i div 3* := $(\neg\, p, 2\hat{}(2 - i \bmod 3))$])
⟨*proof*⟩

lemma *qubit-op-Z-tl*:
$i < 9 \Longrightarrow$ *qubit-op i Pauli-Z* $*_v$ *label-vec* (*replicate 3* $(p, 0)$)
$=$ *label-vec* ((*replicate 3* $(p, 0)$)[*i div 3* := $(\neg\, p, 0)$])
⟨*proof*⟩

lemma *label-vec-inner*:
assumes *l*: *valid-label l* **and** *l′*: *valid-label l′*
shows $\langle$*label-vec l* $\mid$ *label-vec l′*$\rangle$ = (*if l* = *l′ then 8 else 0*)
⟨*proof*⟩

35.4 The 22 correctable errors

In the notes' numbering $I, X_1, \ldots, X_9, Y_1, \ldots, Y_9, Z_1, Z_4, Z_7$. Here qubits are counted from 0: I, X_i and Y_i ($i < 9$), and Z_0, Z_3, Z_6.

definition *shor-errors* :: *complex Matrix.mat list* **where**
shor-errors = 1_m *512* # *map* (λ*i*. *qubit-op i Pauli-X*) [*0*..<*9*] @ *map* (λ*i*. *qubit-op i Pauli-Y*) [*0*..<*9*]
@ *map* (λ*b*. *qubit-op* (*3* ∗ *b*) *Pauli-Z*) [*0*..<*3*]

definition *shor-labels* :: *bool* ⇒ (*bool* × *nat*) *list list* **where**
shor-labels p = *replicate 3* $(p, 0)$
map (λ*i*. (*replicate 3* $(p, 0)$)[*i div 3* := $(p, 2\hat{}(2 - i \bmod 3))$]) [*0*..<*9*]
@ *map* (λ*i*. (*replicate 3* $(p, 0)$)[*i div 3* := $(\neg\, p, 2\hat{}(2 - i \bmod 3))$]) [*0*..<*9*]
@ *map* (λ*b*. (*replicate 3* $(p, 0)$)[*b* := $(\neg\, p, 0)$]) [*0*..<*3*]

definition *shor-kappa* :: *complex list* **where**
shor-kappa = *1* # *replicate 9 1* @ *replicate 9* $(-\mathrm{i})$ @ *replicate 3 1*

lemma *shor-lengths* [*simp*]: *length shor-errors* = *22 length* (*shor-labels p*) = *22 length shor-kappa* = *22*
⟨*proof*⟩

One Z per block suffices, since Z_i and Z_j agree on the code when i and j share a block. Each error sends a code word to a multiple of a labelled product vector,

$$E_j |c_p\rangle = \kappa_j \, 8^{-1/2} \, |\ell_j(p)\rangle,$$

and distinct errors give distinct labels, hence orthogonal images.

lemma *shor-errors-cases*:
assumes *j*: *j* < *22*

obtains $j = 0$ *shor-errors* ! $j = 1_m$ *512 shor-labels p* ! j = *replicate 3* $(p, 0)$ *shor-kappa* ! $j = 1$
| i **where** $i < 9$ j = *Suc i shor-errors* ! j = *qubit-op i Pauli-X*
shor-labels p ! j = $(replicate\ 3\ (p, 0))[i\ div\ 3 := (p, 2\hat{}(2 - i\ mod\ 3))]$ *shor-kappa* ! $j = 1$
| i **where** $i < 9$ $j = 10 + i$ *shor-errors* ! j = *qubit-op i Pauli-Y*
shor-labels p ! j = $(replicate\ 3\ (p, 0))[i\ div\ 3 := (\neg\ p, 2\hat{}(2 - i\ mod\ 3))]$ *shor-kappa* ! $j = -$i
| b **where** $b < 3$ $j = 19 + b$ *shor-errors* ! j = *qubit-op* $(3 * b)$ *Pauli-Z*
shor-labels p ! j = $(replicate\ 3\ (p, 0))[b := (\neg\ p, 0)]$ *shor-kappa* ! $j = 1$
⟨*proof*⟩

lemma *shor-errors-carrier*: $j < 22 \Longrightarrow$ *shor-errors* ! $j \in$ *carrier-mat 512 512*
⟨*proof*⟩

lemma *shor-errors-action*:
assumes *j*: $j < 22$
shows *shor-errors* ! j $*_v$ *shor-cw p* = (*shor-kappa* ! j $*$ *shor-scale*) $\cdot_v$ *label-vec* (*shor-labels p* ! j)
⟨*proof*⟩

lemma *shor-labels-valid*: $j < 22 \Longrightarrow$ *valid-label* (*shor-labels p* ! j)
⟨*proof*⟩

lemma *shor-labels-distinct*: *distinct* (*shor-labels True* @ *shor-labels False*)
⟨*proof*⟩

lemma *shor-labels-differ*:
assumes *i*: $i < 22$ **and** *j*: $j < 22$ **and** *ij*: $i \neq j$
shows *shor-labels p* ! $i \neq$ *shor-labels p'* ! j
⟨*proof*⟩

lemma *shor-errors-orth*:
assumes *i*: $i < 22$ **and** *j*: $j < 22$ **and** *ij*: $i \neq j$
shows ⟨*shor-errors* ! i $*_v$ *shor-cw p* | *shor-errors* ! j $*_v$ *shor-cw p'*⟩ = 0
⟨*proof*⟩

lemma *shor-scale-sq*: *cnj shor-scale* $*$ *shor-scale* = *1 / 8*
⟨*proof*⟩

lemma *shor-cw-inner*: ⟨*shor-cw p* | *shor-cw p'*⟩ = (*if p* = *p' then 1 else 0*)
⟨*proof*⟩

35.5 The code space and its projection

definition *shor-V* :: *complex Matrix.mat* **where** *shor-V* = *mat-of-cols 512* [*shor-cw True*, *shor-cw False*]

definition *shor-P* :: *complex Matrix.mat* **where** *shor-P* = *shor-V* $*$ (*shor-V*†)

definition *shor-state* :: *complex* ⇒ *complex* ⇒ *complex Matrix.vec* **where**
shor-state α β = α $\cdot_v$ *shor-cw True* + β $\cdot_v$ *shor-cw False*

lemma *shor-state-dim* [*simp*]: *dim-vec* (*shor-state* α β) = *512*
⟨*proof*⟩

lemma *shor-V-carrier*: *shor-V* ∈ *carrier-mat 512 2*
⟨*proof*⟩

lemma *shor-orthonormal*: *orthonormal-list 512* [*shor-cw True*, *shor-cw False*]
⟨*proof*⟩

lemma *shor-VdV*: $(\textit{shor-V}^\dagger) * \textit{shor-V} = 1_m\ 2$
⟨*proof*⟩

lemma *mat-vec-unit-col*:
fixes M :: *complex Matrix.mat*
assumes M: $M \in \textit{carrier-mat}\ n\ k$ **and** a: $a < k$
shows $M *_v \textit{unit-vec}\ k\ a = \textit{Matrix.col}\ M\ a$
⟨*proof*⟩

lemma *shor-V-col*: $a < 2 \Longrightarrow \textit{Matrix.col shor-V}\ a = [\textit{shor-cw True}, \textit{shor-cw False}] ! a$
⟨*proof*⟩

lemma *shor-V-unit*: $\textit{shor-V} *_v \textit{unit-vec}\ 2\ 0 = \textit{shor-cw True}\ \textit{shor-V} *_v \textit{unit-vec}\ 2\ 1 = \textit{shor-cw False}$
⟨*proof*⟩

lemma *shor-P-projection*: *projection 512 shor-P*
⟨*proof*⟩

lemma *shor-P-cw*: $\textit{shor-P} *_v \textit{shor-cw}\ p = \textit{shor-cw}\ p$
⟨*proof*⟩

lemma *shor-P-fixes*: $\textit{shor-P} *_v \textit{shor-state}\ \alpha\ \beta = \textit{shor-state}\ \alpha\ \beta$
⟨*proof*⟩

35.6 Proposition (Day 33): orthogonal images, and $Z_1 v = Z_2 v = Z_3 v$

lemma *qubit-op-unitary*: $\textit{unitary}\ U \Longrightarrow \textit{unitary}\ (\textit{qubit-op}\ i\ U)$
⟨*proof*⟩

lemma *shor-errors-unitary*: $U \in \textit{set shor-errors} \Longrightarrow U \in \textit{carrier-mat 512 512} \wedge \textit{unitary}\ U$
⟨*proof*⟩

theorem *shor-orthogonal-images*:
assumes i: $i < 22$ **and** j: $j < 22$ **and** ij: $i \neq j$
shows $\textit{shor-P} * (((\textit{shor-errors} ! i)^\dagger) * \textit{shor-errors} ! j) * \textit{shor-P} = 0_m\ 512\ 512$
⟨*proof*⟩

lemma *shor-recovery-setting*: *recovery-setting 512 shor-P shor-errors*
⟨*proof*⟩

lemma *qubit-op-Z-cw*:
assumes i: $i < 9$
shows $\textit{qubit-op}\ i\ \textit{Pauli-Z} *_v \textit{shor-cw}\ p = \textit{qubit-op}\ (3 * (i\ \textit{div}\ 3))\ \textit{Pauli-Z} *_v \textit{shor-cw}\ p$
⟨*proof*⟩

theorem *shor-Z-coincide*:
assumes i: $i < 9$ **and** j: $j < 9$ **and** ij: $i\ \textit{div}\ 3 = j\ \textit{div}\ 3$
shows $\textit{qubit-op}\ i\ \textit{Pauli-Z} *_v \textit{shor-state}\ \alpha\ \beta = \textit{qubit-op}\ j\ \textit{Pauli-Z} *_v \textit{shor-state}\ \alpha\ \beta$
⟨*proof*⟩

The Day 32 proposition as first stated (distinct 1-Paulis map V to orthogonal subspaces) is false: $Z_1 \neq Z_2$, yet $Z_1|0_L\rangle = Z_2|0_L\rangle \neq 0$. The notes correct it on Day 33.

lemma *shor-notes-Z1-Z2*:
$\textit{qubit-op 0 Pauli-Z} \neq \textit{qubit-op 1 Pauli-Z}$
$\langle \textit{qubit-op 0 Pauli-Z} *_v \textit{shor-cw True} \mid \textit{qubit-op 1 Pauli-Z} *_v \textit{shor-cw True} \rangle = 1$
⟨*proof*⟩

35.7 Shor's theorem

lemma *pauli-decomp*:
fixes U :: *complex Matrix.mat*
assumes U: $U \in$ *carrier-mat 2 2*
shows U = ((*U $$ (0,0)* + *U $$ (1,1)*) / *2*) ·$_m$ *1*$_m$ *2* + ((*U $$ (0,1)* + *U $$ (1,0)*) / *2*) ·$_m$ *Pauli-X*
+ (−i ∗ (*U $$ (0,1)* − *U $$ (1,0)*) / *2*) ·$_m$ *Pauli-Y* + ((*U $$ (0,0)* − *U $$ (1,1)*) / *2*) ·$_m$ *Pauli-Z*
(**is** U = *?R*)
⟨*proof*⟩

lemma *qubit-op-add*:
assumes A: $A \in$ *carrier-mat 2 2* **and** B: $B \in$ *carrier-mat 2 2*
shows *qubit-op i* (*A* + *B*) = *qubit-op i A* + *qubit-op i B*
⟨*proof*⟩

lemma *qubit-op-smult*:
assumes A: $A \in$ *carrier-mat 2 2*
shows *qubit-op i* (*k* ·$_m$ *A*) = *k* ·$_m$ *qubit-op i A*
⟨*proof*⟩

lemma *qubit-op-one*:
assumes i: $i < 9$
shows *qubit-op i* (*1*$_m$ *2*) = *1*$_m$ *512*
⟨*proof*⟩

lemma *shor-errors-nth*:
assumes i: $i < 9$ **and** b: $b < 3$
shows *shor-errors* ! *0* = *1*$_m$ *512 shor-errors* ! *Suc i* = *qubit-op i Pauli-X*
shor-errors ! (*10* + *i*) = *qubit-op i Pauli-Y shor-errors* ! (*19* + *b*) = *qubit-op* (*3* ∗ *b*) *Pauli-Z*
⟨*proof*⟩

Shor: one CPTP recovery map R corrects every single-qubit unitary error on every qubit, for every state $\psi = a|0_L\rangle + b|1_L\rangle$ of the code space.

theorem *shor-code-theorem*:
∃ *Rs. kraus-operators 512 512 Rs* ∧
(∀ *U i* α β. *U* ∈ *carrier-mat 2 2* ⟶ *unitary U* ⟶ *i* < *9* ⟶
kraus-map-dim 512 Rs (*qubit-op i U* ∗ (*shor-state* α β ⊘ *shor-state* α β) ∗ ((*qubit-op i U*)†))
= *shor-state* α β ⊘ *shor-state* α β)
⟨*proof*⟩

36 Day 34 (18 November 2011): the Knill–Laflamme conditions

36.1 Vectors in the range of a projection; rank-one decompositions inside V

lemma *projection-fixed-of-orth-kernel*:
assumes P: *projection n P* **and** *vd*: *dim-vec v* = *n*
and *orth*: ⋀*x*. *dim-vec x* = *n* ⟹ *P* ∗$_v$ *x* = *0*$_v$ *n* ⟹ ⟨*v* | *x*⟩ = *0*
shows *P* ∗$_v$ *v* = *v*
⟨*proof*⟩

For $X \geq 0$ the matrix PXP is a sum of $|\psi_l\rangle\langle\psi_l|$ with $\psi_l \in V$ (the notes: "non-zero eigenvectors of PXP are all in V").

lemma *psd-PXP-decomp-in-range*:
assumes P: *projection n P* **and** *Xc*: *M* ∈ *carrier-mat n n* **and** *Xpsd*: *psd M*
shows ∃ *vs* :: *complex Matrix.vec list*. (∀ *v* ∈ *set vs*. *dim-vec v* = *n* ∧ *P* ∗$_v$ *v* = *v*) ∧
P ∗ *M* ∗ *P* = *foldr* (λ*v acc*. *v* ⊘ *v* + *acc*) *vs* (*0*$_m$ *n n*)

⟨*proof*⟩

36.2 Linearity facts for the composite map $\mathcal{R} \circ \mathbb{E}$

definition *recover-after* :: *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
recover-after n As Es M = *kraus-map-dim n As* (*kraus-map-dim n Es M*)

lemma *recover-after-carrier*: **assumes** *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat n n* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat n n* **and** *M*: *M* ∈ *carrier-mat n n* **shows** *recover-after n As Es M* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *recover-after-add*: **assumes** *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat n n* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat n n* **and** *M*: *M* ∈ *carrier-mat n n* **and** *N*: *N* ∈ *carrier-mat n n* **shows** *recover-after n As Es* (*M* + *N*) = *recover-after n As Es M* + *recover-after n As Es N*
⟨*proof*⟩

lemma *recover-after-smult*: **assumes** *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat n n* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat n n* **and** *M*: *M* ∈ *carrier-mat n n* **shows** *recover-after n As Es* (*c* ·$_m$ *M*) = *c* ·$_m$ *recover-after n As Es M*
⟨*proof*⟩

lemma *recover-after-minus*:
assumes *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat n n* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat n n*
and *Xc*: *M* ∈ *carrier-mat n n* **and** *Yc*: *N* ∈ *carrier-mat n n*
shows *recover-after n As Es* (*M* − *N*) = *recover-after n As Es M* − *recover-after n As Es N*
⟨*proof*⟩

lemma *recover-after-foldr*:
assumes *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat n n* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat n n* **and** ∀ *v* ∈ *set vs*. *dim-vec v* = *n*
shows *recover-after n As Es* (*foldr* (λ*v acc*. *v* ⊘ *v* + *acc*) *vs* (0$_m$ *n n*)) = *foldr* (λ*v acc*. *recover-after n As Es* (*v* ⊘ *v*) + *acc*) *vs* (0$_m$ *n n*)
⟨*proof*⟩

D34-1: $\mathcal{R} \circ \mathbb{E}$ fixes $|\psi\rangle\langle\psi|$ for all $\psi \in V$ iff it fixes PXP for all X.

proposition *recovery-states-iff-PXP*:
assumes *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat n n* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat n n* **and** *P*: *projection n P*
shows (∀ *ψ*. *dim-vec ψ* = *n* ⟶ *P* ∗$_v$ *ψ* = *ψ* ⟶ *recover-after n As Es* (*ψ* ⊘ *ψ*) = *ψ* ⊘ *ψ*)
⟷ (∀ *X*. *X* ∈ *carrier-mat n n* ⟶ *recover-after n As Es* (*P* ∗ *X* ∗ *P*) = *P* ∗ *X* ∗ *P*)
⟨*proof*⟩

36.3 Knill–Laflamme, Theorem 1, necessity: $PE_i^* E_j P = \alpha_{ij} P$

lemma *lin-indep-complex-mat-single*:
assumes *Pc*: *P* ∈ *carrier-mat d n* **and** *Pnz*: *P* ≠ 0$_m$ *d n*
shows *lin-indep-complex-mat d n* [*P*]
⟨*proof*⟩

The core of the necessity proof: by Choi's Theorem 2 applied to the two Kraus representations $\{P\}$ and $\{A_l E_i P\}$ of $X \mapsto PXP$, every $A_l E_i P$ is a multiple of P.

lemma *kl-kraus-scalars*:
assumes *P*: *projection n P* **and** *Pnz*: *P* ≠ 0$_m$ *n n* **and** *npos*: *0* < *n*
and *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat n n* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat n n*
and *corr*: ∀ *X*. *X* ∈ *carrier-mat n n* ⟶
kraus-map-dim n As (*kraus-map-dim n Es* (*P* ∗ *X* ∗ *P*)) = *P* ∗ *X* ∗ *P*

shows $\exists\,\beta :: nat \Rightarrow nat \Rightarrow complex.\ \forall\, l < length\ As.\ \forall\, i < length\ Es.$
$As\ !\ l * Es\ !\ i * P = \beta\ l\ i \cdot_m P$
⟨*proof*⟩

theorem *knill-laflamme-necessity*:
assumes *P*: *projection n P* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *npos*: $0 < n$
and *As*: *kraus-operators n n As* **and** *Es*: *kraus-operators n n Es*
and *corr*: $\forall\, X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow$
kraus-map-dim n As (*kraus-map-dim n Es* $(P * X * P)) = P * X * P$
shows $\exists\,\alpha :: nat \Rightarrow nat \Rightarrow complex.\ \forall\, i < length\ Es.\ \forall\, j < length\ Es.$
$P * (((Es\ !\ i)^\dagger) * Es\ !\ j) * P = \alpha\ i\ j \cdot_m P$
⟨*proof*⟩

36.4 Kraus lists consisting of multiples of one operator

lemma *kraus-map-dim-scalar-list*:
assumes *Pc*: $P \in carrier\text{-}mat\ n\ n$ **and** *Xc*: $M \in carrier\text{-}mat\ n\ n$
shows *kraus-map-dim n* (*map* $(\lambda\gamma.\ \gamma \cdot_m P)$ *gs*) *M*
$=$ *sum-list* (*map* $(\lambda\gamma.\ cnj\ \gamma * \gamma)$ *gs*) $\cdot_m (P * M * (P^\dagger))$
⟨*proof*⟩

lemma *sum-wdag-w-scalar-list*:
assumes *Pc*: $P \in carrier\text{-}mat\ n\ n$
shows *sum-wdag-w n* (*map* $(\lambda\gamma.\ \gamma \cdot_m P)$ *gs*) $=$ *sum-list* (*map* $(\lambda\gamma.\ cnj\ \gamma * \gamma)$ *gs*) $\cdot_m ((P^\dagger) * P)$
⟨*proof*⟩

lemma *smult-eq-self-imp-one*:
fixes *P* :: *complex Matrix.mat*
assumes *Pc*: $P \in carrier\text{-}mat\ n\ n$ **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *eq*: $s \cdot_m P = P$
shows $s = 1$
⟨*proof*⟩

36.5 Theorem 2

If $\mathcal{R} \circ \mathbb{E}$ fixes PXP and $G_i = \sum_j g_{ij} E_j$ with $\sum_i G_i^* G_i = I$, then $\mathcal{R} \circ \mathbb{G}$ fixes PXP as well (same $\mathcal{R}$).

theorem *knill-laflamme-closure*:
assumes *P*: *projection n P* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *npos*: $0 < n$
and *As*: *kraus-operators n n As*
and *Es-c*: $\forall\, E \in set\ Es.\ E \in carrier\text{-}mat\ n\ n$
and *corr*: $\forall\, X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow$
kraus-map-dim n As (*kraus-map-dim n Es* $(P * X * P)) = P * X * P$
and *Gc*: $G \in carrier\text{-}mat\ t\ (length\ Es)$
and *Gs-tp*: *sum-wdag-w n* (*kraus-mix n n G Es*) $= 1_m\ n$
shows $\forall\, X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow$
kraus-map-dim n As (*kraus-map-dim n* (*kraus-mix n n G Es*) $(P * X * P)) = P * X * P$
⟨*proof*⟩

36.6 Sufficiency of the Knill–Laflamme conditions (D34-2)

D34-2, sufficiency: if $PE_i^* E_j P = \alpha_{ij} P$ then a recovery channel exists. Diagonalise $(\alpha_{ij}) = u\,\mathrm{diag}(d)\,u^*$; the errors $F_k = \sum_i u_{ik} E_i$ give the same channel and satisfy $PF_k^* F_l P = \delta_{kl} d_k P$, so the $F_k/\sqrt{d_k}$ with $d_k > 0$ satisfy the hypotheses of the recovery proposition (*recovery-setting*).

lemma *smult-cancel-nonzero*:
fixes *P* :: *complex Matrix.mat*
assumes *Pc*: $P \in carrier\text{-}mat\ n\ n$ **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *eq*: $c \cdot_m P = d \cdot_m P$
shows $c = d$

⟨*proof*⟩

lemma *outer-prod-smult-vec*:
fixes v :: *complex Matrix.vec*
shows $(s \cdot_v v) \oslash (s \cdot_v v) = (s * cnj\ s) \cdot_m (v \oslash v)$
⟨*proof*⟩

lemma *kraus-map-dim-foldr-add*:
assumes *Ws-c*: $\forall W \in set\ Ws.\ W \in carrier\text{-}mat\ n\ n$ **and** *fc*: $\forall k \in set\ xs.\ f\ k \in carrier\text{-}mat\ n\ n$
shows *kraus-map-dim n Ws* (*foldr* ($\lambda k\ acc.\ f\ k + acc$) *xs* ($0_m\ n\ n$))
$= foldr\ (\lambda k\ acc.\ kraus\text{-}map\text{-}dim\ n\ Ws\ (f\ k) + acc)\ xs\ (0_m\ n\ n)$
⟨*proof*⟩

lemma *kraus-map-dim-outer-as-foldr*:
assumes *Ws-c*: $\forall W \in set\ Ws.\ W \in carrier\text{-}mat\ n\ n$ **and** ψd: *dim-vec* $\psi = n$
shows *kraus-map-dim n Ws* ($\psi \oslash \psi$)
$= foldr\ (\lambda k\ acc.\ (Ws\ !\ k *_v \psi) \oslash (Ws\ !\ k *_v \psi) + acc)\ [0..<length\ Ws]\ (0_m\ n\ n)$
⟨*proof*⟩

lemma *foldr-smult-sandwich*:
fixes V :: *nat* ⇒ *complex Matrix.mat*
assumes *Mc*: $M \in carrier\text{-}mat\ n\ n$ **and** *Nc*: $N \in carrier\text{-}mat\ n\ n$ **and** *Pc*: $P \in carrier\text{-}mat\ n\ n$
and *Vc*: $\forall j < r.\ V\ j \in carrier\text{-}mat\ n\ n$ **and** *h*: $\forall j < r.\ M * V\ j * N = \beta\ j \cdot_m P$
shows $M * foldr\ (\lambda j\ acc.\ b\ j \cdot_m V\ j + acc)\ [0..<r]\ (0_m\ n\ n) * N = (\sum j{<}r.\ b\ j * \beta\ j) \cdot_m P$
⟨*proof*⟩

lemma *foldr-smult-dagger*:
fixes V :: *nat* ⇒ *complex Matrix.mat*
assumes *Vc*: $\forall j < r.\ V\ j \in carrier\text{-}mat\ n\ n$
shows $(foldr\ (\lambda j\ acc.\ b\ j \cdot_m V\ j + acc)\ [0..<r]\ (0_m\ n\ n))^\dagger$
$= foldr\ (\lambda j\ acc.\ cnj\ (b\ j) \cdot_m ((V\ j)^\dagger) + acc)\ [0..<r]\ (0_m\ n\ n)$
⟨*proof*⟩

The bracket $P\,(\sum_i a_i E_i)^*(\sum_j b_j E_j)\,P$ under the Knill–Laflamme conditions.

lemma *kl-mix-bracket*:
assumes *Es-c*: $\forall E \in set\ Es.\ E \in carrier\text{-}mat\ n\ n$ **and** *Pc*: $P \in carrier\text{-}mat\ n\ n$
and *kl*: $\forall i < length\ Es.\ \forall j < length\ Es.\ P * (((Es\ !\ i)^\dagger) * Es\ !\ j) * P = \alpha\ i\ j \cdot_m P$
and *r*: $r = length\ Es$
shows $P * (((foldr\ (\lambda i\ acc.\ a\ i \cdot_m (Es\ !\ i) + acc)\ [0..<r]\ (0_m\ n\ n))^\dagger)$
$* foldr\ (\lambda j\ acc.\ b\ j \cdot_m (Es\ !\ j) + acc)\ [0..<r]\ (0_m\ n\ n)) * P$
$= (\sum j{<}r.\ b\ j * (\sum i{<}r.\ cnj\ (a\ i) * \alpha\ i\ j)) \cdot_m P$
⟨*proof*⟩

lemma *dagger-sandwich*:
assumes *Pc*: $P \in carrier\text{-}mat\ n\ n$ **and** *Ph*: $P^\dagger = P$ **and** *Ac*: $A \in carrier\text{-}mat\ n\ n$ **and** *Bc*: $B \in carrier\text{-}mat\ n\ n$
shows $(P * ((A^\dagger) * B) * P)^\dagger = P * ((B^\dagger) * A) * P$
⟨*proof*⟩

lemma *smult-bracket*:
assumes *Pc*: $P \in carrier\text{-}mat\ n\ n$ **and** *Ac*: $A \in carrier\text{-}mat\ n\ n$ **and** *Bc*: $B \in carrier\text{-}mat\ n\ n$
shows $P * (((x \cdot_m A)^\dagger) * (y \cdot_m B)) * P = (cnj\ x * y) \cdot_m (P * ((A^\dagger) * B) * P)$
⟨*proof*⟩

theorem *knill-laflamme-sufficiency*:
assumes *P*: *projection n P* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *npos*: $0 < n$
and *Es*: *kraus-operators n n Es*
and *kl*: $\forall i < length\ Es.\ \forall j < length\ Es.\ P * (((Es\ !\ i)^\dagger) * Es\ !\ j) * P = \alpha\ i\ j \cdot_m P$

shows $\exists\, As.$ *kraus-operators n n As* $\wedge$
$(\forall\, X.\ X \in$ *carrier-mat n n* $\longrightarrow$ *kraus-map-dim n As* (*kraus-map-dim n Es* $(P * X * P)) = P * X * P$)
⟨*proof*⟩

D34-2 (Theorem 1): the code is correctable iff $PE_i^* E_j P = \alpha_{ij} P$.

theorem *knill-laflamme*:
assumes *P*: *projection n P* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *npos*: $0 < n$
and *Es*: *kraus-operators n n Es*
shows ($\exists\, As.$ *kraus-operators n n As* $\wedge$
$(\forall\, X.\ X \in$ *carrier-mat n n* $\longrightarrow$ *kraus-map-dim n As* (*kraus-map-dim n Es* $(P * X * P)) = P * X * P$))
$\longleftrightarrow$ ($\exists\, \alpha :: nat \Rightarrow nat \Rightarrow complex.\ \forall\, i < length\ Es.\ \forall\, j < length\ Es.$
$P * (((Es\ !\ i)^\dagger) * Es\ !\ j) * P = \alpha\ i\ j \cdot_m P$)
⟨*proof*⟩

Part VI
Beyond the notes

37 Extensions beyond the notes

Nothing from here on is in Paulsen's notes. These are standard quantum-information facts proved with the machinery above: rank-one projections, the bit-flip, phase-flip, bit-phase-flip and unitary-conjugation channels and their membership in UCP and CPTP, composition of channels, Kraus lists, the maximally mixed state, and the Pauli product identities.

37.1 Rank-one projections

lemma *outer-sum-singleton-hermitian*:
fixes ψ :: *complex Matrix.vec*
shows (*outer-sum n* $[\psi]$)† = *outer-sum n* $[\psi]$
⟨*proof*⟩

lemma *outer-sum-singleton-idempotent*:
fixes ψ :: *complex Matrix.vec*
assumes ψ*-qs*: *quantum-state n* ψ
shows *outer-sum n* $[\psi] *$ *outer-sum n* $[\psi]$ = *outer-sum n* $[\psi]$
⟨*proof*⟩

lemma *rank1-projection*:
fixes ψ :: *complex Matrix.vec*
assumes ψ*-qs*: *quantum-state n* ψ
shows *projection n* (*outer-sum n* $[\psi]$)
⟨*proof*⟩

lemma *rank1-projection-acts-identity*:
fixes ψ :: *complex Matrix.vec*
assumes ψ*-qs*: *quantum-state n* ψ
shows *outer-sum n* $[\psi] *_v \psi = \psi$
⟨*proof*⟩

lemma *rank1-projection-trace-one*:
fixes ψ :: *complex Matrix.vec*

assumes *ψ-qs*: *quantum-state n ψ*
shows *Tr* (*outer-sum n* [*ψ*]) = *1*
⟨*proof*⟩

37.2 The bit-flip channel at $p = 0$ and $p = 1$

lemma *bit-flip-channel-zero*:
assumes *ϱ* ∈ *carrier-mat 2 2*
shows *bit-flip-channel 0 ϱ* = *ϱ*
⟨*proof*⟩

lemma *bit-flip-channel-one*:
assumes *ϱ* ∈ *carrier-mat 2 2*
shows *bit-flip-channel 1 ϱ* = *Pauli-X* ∗ *ϱ* ∗ (*Pauli-X*†)
⟨*proof*⟩

37.3 The phase-flip channel

definition *phase-flip-channel* ::
real ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
phase-flip-channel p ϱ ≡
complex-of-real (*1* − *p*) ·$_m$ *ϱ*
+ *complex-of-real p* ·$_m$ (*Pauli-Z* ∗ *ϱ* ∗ (*Pauli-Z*†))

lemma *phase-flip-channel-carrier*:
assumes *ϱ* ∈ *carrier-mat 2 2*
shows *phase-flip-channel p ϱ* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *phase-flip-channel-trace-preserving*:
fixes *p* :: *real*
assumes *ϱ-c*: *ϱ* ∈ *carrier-mat 2 2*
shows *Tr* (*phase-flip-channel p ϱ*) = *Tr ϱ*
⟨*proof*⟩

lemma *phase-flip-channel-psd*:
fixes *p* :: *real*
assumes *p-nn*: *0* ≤ *p*
and *p-le-1*: *p* ≤ *1*
and *ϱ-c*: *ϱ* ∈ *carrier-mat 2 2*
and *ϱ-psd*: *psd ϱ*
shows *psd* (*phase-flip-channel p ϱ*)
⟨*proof*⟩

37.4 Unitary-conjugation channels

For a unitary $U \in M_n$ and $0 \le p \le 1$, $\rho \mapsto (1-p)\rho + pU\rho U^\dagger$ is trace preserving and positive; the bit-flip and phase-flip channels are the instances $U = X$, $U = Z$.

definition *unitary-conj-channel* ::
complex Matrix.mat ⇒ *real* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
unitary-conj-channel U p ϱ ≡
complex-of-real (*1* − *p*) ·$_m$ *ϱ*
+ *complex-of-real p* ·$_m$ (*U* ∗ *ϱ* ∗ (*U*†))

lemma *unitary-conj-channel-carrier*:
assumes *U-c*: *U* ∈ *carrier-mat n n*
and *ϱ-c*: *ϱ* ∈ *carrier-mat n n*
shows *unitary-conj-channel U p ϱ* ∈ *carrier-mat n n*

⟨*proof*⟩

lemma *unitary-conj-channel-trace-preserving*:
fixes *p* :: *real*
assumes *U-c*: *U* ∈ *carrier-mat n n*
and *U-unit*: *unitary U*
and *ϱ-c*: *ϱ* ∈ *carrier-mat n n*
shows *Tr* (*unitary-conj-channel U p ϱ*) = *Tr ϱ*
⟨*proof*⟩

lemma *unitary-conj-channel-psd*:
fixes *p* :: *real*
assumes *p-nn*: 0 ≤ *p*
and *p-le-1*: *p* ≤ 1
and *U-c*: *U* ∈ *carrier-mat n n*
and *ϱ-c*: *ϱ* ∈ *carrier-mat n n*
and *ϱ-psd*: *psd ϱ*
shows *psd* (*unitary-conj-channel U p ϱ*)
⟨*proof*⟩

lemma *bit-flip-eq-unitary-conj*:
bit-flip-channel p ϱ = *unitary-conj-channel Pauli-X p ϱ*
⟨*proof*⟩

lemma *phase-flip-eq-unitary-conj*:
phase-flip-channel p ϱ = *unitary-conj-channel Pauli-Z p ϱ*
⟨*proof*⟩

37.5 The bit-phase-flip channel and the identity channel

definition *bit-phase-flip-channel* ::
real ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
bit-phase-flip-channel p ϱ ≡ *unitary-conj-channel Pauli-Y p ϱ*

lemma *bit-phase-flip-channel-carrier*:
assumes *ϱ* ∈ *carrier-mat 2 2*
shows *bit-phase-flip-channel p ϱ* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *bit-phase-flip-channel-trace-preserving*:
fixes *p* :: *real*
assumes *ϱ-c*: *ϱ* ∈ *carrier-mat 2 2*
shows *Tr* (*bit-phase-flip-channel p ϱ*) = *Tr ϱ*
⟨*proof*⟩

lemma *bit-phase-flip-channel-psd*:
fixes *p* :: *real*
assumes *p-nn*: 0 ≤ *p*
and *p-le-1*: *p* ≤ 1
and *ϱ-c*: *ϱ* ∈ *carrier-mat 2 2*
and *ϱ-psd*: *psd ϱ*
shows *psd* (*bit-phase-flip-channel p ϱ*)
⟨*proof*⟩

definition *identity-channel* ::
complex Matrix.mat ⇒ *complex Matrix.mat* **where**
identity-channel ϱ ≡ *ϱ*

lemma *identity-channel-carrier*:
$\varrho \in$ *carrier-mat n n* $\Longrightarrow$ *identity-channel* $\varrho \in$ *carrier-mat n n*
⟨*proof*⟩

lemma *identity-channel-trace-preserving*:
$\varrho \in$ *carrier-mat n n* $\Longrightarrow$ *Tr* (*identity-channel* ϱ) = *Tr* ϱ
⟨*proof*⟩

lemma *identity-channel-psd*:
$\varrho \in$ *carrier-mat n n* $\Longrightarrow$ *psd* $\varrho \Longrightarrow$ *psd* (*identity-channel* ϱ)
⟨*proof*⟩

lemma *identity-channel-eq-unitary-conj-zero*:
assumes ϱ-*c*: $\varrho \in$ *carrier-mat n n*
shows *identity-channel* ϱ = *unitary-conj-channel* (1_m *n*) *0* ϱ
⟨*proof*⟩

37.6 Composition of channels

lemma *compose-trace-preserving*:
assumes *F-tp*: *trace-preserving n F*
and *G-tp*: *trace-preserving n G*
and *F-range*: $\forall \varrho.\ \varrho \in$ *carrier-mat n n* $\longrightarrow$ *F* $\varrho \in$ *carrier-mat n n*
shows *trace-preserving n* ($\lambda\varrho.$ *G* (*F* ϱ))
⟨*proof*⟩

lemma *identity-channel-compose-left*:
($\lambda\varrho.$ *identity-channel* (*F* ϱ)) = *F*
⟨*proof*⟩

lemma *identity-channel-compose-right*:
($\lambda\varrho.$ *F* (*identity-channel* ϱ)) = *F*
⟨*proof*⟩

lemma *compose-unital-map*:
assumes *F-un*: *unital-map n n F*
and *G-un*: *unital-map n n G*
shows *unital-map n n* ($\lambda\varrho.$ *F* (*G* ϱ))
⟨*proof*⟩

lemma *identity-channel-unital*: *unital-map n n identity-channel*
⟨*proof*⟩

lemma *identity-channel-trace-preserving-full*:
trace-preserving n identity-channel
⟨*proof*⟩

lemma *identity-channel-completely-positive*:
assumes *0* < *n*
shows *completely-positive n n identity-channel*
⟨*proof*⟩

37.7 Conjugation channels are completely positive

definition *conjugation-channel* ::
complex Matrix.mat ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
conjugation-channel U $\varrho \equiv U * \varrho * (U^\dagger)$

lemma *conjugation-channel-eq-kraus*:
assumes *U-c*: $U \in$ *carrier-mat d n*
and *ϱ-c*: $\varrho \in$ *carrier-mat n n*
shows *conjugation-channel U ϱ* = *kraus-map-dim d* [*U*] *ϱ*
⟨*proof*⟩

lemma *conjugation-channel-cp*:
assumes *U-c*: $U \in$ *carrier-mat d n*
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *completely-positive n d* (*kraus-map-dim d* [*U*])
⟨*proof*⟩

lemma *identity-channel-UCP*:
assumes $0 < n$
shows *identity-channel* $\in$ *UCP n n*
⟨*proof*⟩

lemma *identity-channel-CPTP*:
assumes $0 < n$
shows *identity-channel* $\in$ *CPTP n n*
⟨*proof*⟩

lemma *map-conj-cp*:
assumes *U-c*: $U \in$ *carrier-mat d n*
and *U-range*: $\forall \varrho.\ \varrho \in$ *carrier-mat n n*
$\longrightarrow U * \varrho * (U^\dagger) \in$ *carrier-mat d d*
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *completely-positive n d* ($\lambda\varrho.\ U * \varrho * (U^\dagger)$)
⟨*proof*⟩

lemma *map-conj-unital*:
fixes *U* :: *complex Matrix.mat*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
shows $U * (1_m\ n) * (U^\dagger) = 1_m\ n$
⟨*proof*⟩

37.8 Unitary-conjugation channels are UCP and CPTP

lemma *map-conj-in-UCP*:
fixes *U* :: *complex Matrix.mat*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *npos*: $0 < n$
shows ($\lambda\varrho.\ U * \varrho * (U^\dagger)$) $\in$ *UCP n n*
⟨*proof*⟩

lemma *unitary-conj-channel-in-UCP*:
fixes *U* :: *complex Matrix.mat* **and** *p* :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *p-nn*: $0 \leq p$
and *p-le*: $p \leq 1$
and *npos*: $0 < n$
shows *unitary-conj-channel U p* $\in$ *UCP n n*
⟨*proof*⟩

lemma *unitary-conj-channel-in-CPTP*:
fixes U :: *complex Matrix.mat* **and** p :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *p-nn*: $0 \leq p$
and *p-le*: $p \leq 1$
and *npos*: $0 < n$
shows *unitary-conj-channel U p* $\in$ *CPTP n n*
⟨*proof*⟩

37.9 The Pauli channels are UCP and CPTP

lemma *bit-flip-channel-eq*:
bit-flip-channel p = *unitary-conj-channel Pauli-X p*
⟨*proof*⟩

lemma *phase-flip-channel-eq*:
phase-flip-channel p = *unitary-conj-channel Pauli-Z p*
⟨*proof*⟩

lemma *bit-phase-flip-channel-eq*:
bit-phase-flip-channel p = *unitary-conj-channel Pauli-Y p*
⟨*proof*⟩

All three are unitary-conjugation channels $\rho \mapsto (1-p)\rho + pU\rho U^*$ with $U = X, Z, Y$, whose Kraus operators $\sqrt{1-p}\,I$, $\sqrt{p}\,U$ satisfy

$$(1-p)I + p\,U^*U = I = (1-p)I + p\,UU^*,$$

so each is both unital and trace preserving.

lemma *phase-flip-channel-UCP*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
shows *phase-flip-channel p* $\in$ *UCP 2 2*
⟨*proof*⟩

lemma *phase-flip-channel-CPTP*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
shows *phase-flip-channel p* $\in$ *CPTP 2 2*
⟨*proof*⟩

lemma *bit-phase-flip-channel-UCP*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
shows *bit-phase-flip-channel p* $\in$ *UCP 2 2*
⟨*proof*⟩

lemma *bit-phase-flip-channel-CPTP*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
shows *bit-phase-flip-channel p* $\in$ *CPTP 2 2*
⟨*proof*⟩

37.10 Pauli matrices as Kraus operators

lemma *Pauli-X-kraus-operators*:

kraus-operators 2 2 [Pauli-X]
⟨*proof*⟩

lemma *Pauli-Y-kraus-operators*:
kraus-operators 2 2 [Pauli-Y]
⟨*proof*⟩

lemma *Pauli-Z-kraus-operators*:
kraus-operators 2 2 [Pauli-Z]
⟨*proof*⟩

37.11 Kraus maps of Kraus-operator lists are CPTP

lemma *Pauli-X-kraus-map-CPTP*:
kraus-map-dim 2 [Pauli-X] ∈ CPTP 2 2
⟨*proof*⟩

lemma *Pauli-Y-kraus-map-CPTP*:
kraus-map-dim 2 [Pauli-Y] ∈ CPTP 2 2
⟨*proof*⟩

lemma *Pauli-Z-kraus-map-CPTP*:
kraus-map-dim 2 [Pauli-Z] ∈ CPTP 2 2
⟨*proof*⟩

lemma *identity-kraus-operators*:
kraus-operators n n [1$_m$ n]
⟨*proof*⟩

lemma *identity-channel-eq-kraus-map-dim*:
fixes *n* :: *nat*
assumes *ϱ-c*: *ϱ ∈ carrier-mat n n*
shows *identity-channel ϱ = kraus-map-dim n [1$_m$ n] ϱ*
⟨*proof*⟩

37.12 Fixed points of Pauli channels

lemma *Pauli-X-conj-commute*:
fixes *M* :: *complex Matrix.mat*
assumes *M-c*: *M ∈ carrier-mat 2 2*
and *commute*: *Pauli-X ∗ M = M ∗ Pauli-X*
shows *Pauli-X ∗ M ∗ (Pauli-X†) = M*
⟨*proof*⟩

lemma *bit-flip-channel-fixed-point-commute*:
fixes *p* :: *real*
assumes *M-c*: *M ∈ carrier-mat 2 2*
and *commute*: *Pauli-X ∗ M = M ∗ Pauli-X*
shows *bit-flip-channel p M = M*
⟨*proof*⟩

lemma *Pauli-Z-conj-commute*:
fixes *M* :: *complex Matrix.mat*
assumes *M-c*: *M ∈ carrier-mat 2 2*
and *commute*: *Pauli-Z ∗ M = M ∗ Pauli-Z*
shows *Pauli-Z ∗ M ∗ (Pauli-Z†) = M*
⟨*proof*⟩

lemma *phase-flip-channel-fixed-point-commute*:
fixes p :: *real*
assumes *M-c*: $M \in$ *carrier-mat 2 2*
and *commute*: *Pauli-Z* $*$ $M = M *$ *Pauli-Z*
shows *phase-flip-channel p M* $= M$
⟨*proof*⟩

lemma *unitary-conj-channel-identity-op*:
fixes p :: *real*
assumes ϱ*-c*: $\varrho \in$ *carrier-mat n n*
shows *unitary-conj-channel* $(1_m\ n)$ *p* $\varrho = \varrho$
⟨*proof*⟩

lemma *unitary-conj-channel-identity-op-eq*:
fixes p :: *real*
shows $(\lambda\varrho.$ *if* $\varrho \in$ *carrier-mat n n*
then unitary-conj-channel $(1_m\ n)$ *p* ϱ *else identity-channel* $\varrho)$
$=$ *identity-channel*
⟨*proof*⟩

37.13 The maximally mixed state

definition *maximally-mixed-state* :: *nat* $\Rightarrow$ *complex Matrix.mat* **where**
maximally-mixed-state n $\equiv$ *complex-of-real* $(1\ /$ *real n*$)\ \cdot_m\ (1_m\ n)$

lemma *maximally-mixed-state-carrier* [*simp*]:
maximally-mixed-state n $\in$ *carrier-mat n n*
⟨*proof*⟩

lemma *maximally-mixed-state-trace*:
assumes *npos*: $0 < n$
shows *Tr* (*maximally-mixed-state n*) $= 1$
⟨*proof*⟩

lemma *maximally-mixed-state-psd*:
psd (*maximally-mixed-state n*)
⟨*proof*⟩

lemma *maximally-mixed-state-density-matrix*:
assumes *npos*: $0 < n$
shows *density-matrix n* (*maximally-mixed-state n*)
⟨*proof*⟩

37.14 Fixed points of unitary-conjugation channels

lemma *unitary-conj-commute*:
fixes M :: *complex Matrix.mat*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *M-c*: $M \in$ *carrier-mat n n*
and *commute*: $U * M = M * U$
shows $U * M * (U^\dagger) = M$
⟨*proof*⟩

lemma *unitary-conj-channel-fixed-point-commute*:
fixes p :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*

and *M-c*: $M \in$ *carrier-mat n n*
and *commute*: $U * M = M * U$
shows *unitary-conj-channel U p M* $= M$
⟨*proof*⟩

lemma *maximally-mixed-state-commutes*:
fixes U :: *complex Matrix.mat*
assumes *U-c*: $U \in$ *carrier-mat n n*
shows $U *$ *maximally-mixed-state n* $=$ *maximally-mixed-state n* $* U$
⟨*proof*⟩

lemma *unitary-conj-channel-fixes-max-mixed*:
fixes p :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
shows *unitary-conj-channel U p* (*maximally-mixed-state n*)
$=$ *maximally-mixed-state n*
⟨*proof*⟩

lemma *bit-flip-channel-fixes-max-mixed*:
fixes p :: *real*
shows *bit-flip-channel p* (*maximally-mixed-state 2*)
$=$ *maximally-mixed-state 2*
⟨*proof*⟩

lemma *phase-flip-channel-fixes-max-mixed*:
fixes p :: *real*
shows *phase-flip-channel p* (*maximally-mixed-state 2*)
$=$ *maximally-mixed-state 2*
⟨*proof*⟩

lemma *bit-phase-flip-channel-fixes-max-mixed*:
fixes p :: *real*
shows *bit-phase-flip-channel p* (*maximally-mixed-state 2*)
$=$ *maximally-mixed-state 2*
⟨*proof*⟩

37.15 Channels preserve density matrices

lemma *unitary-conj-channel-preserves-density-matrix*:
fixes p :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *p-nn*: $0 \leq p$
and *p-le*: $p \leq 1$
and *dm*: *density-matrix n* ϱ
shows *density-matrix n* (*unitary-conj-channel U p* ϱ)
⟨*proof*⟩

lemma *bit-flip-channel-preserves-density-matrix*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
and *dm*: *density-matrix 2* ϱ
shows *density-matrix 2* (*bit-flip-channel p* ϱ)
⟨*proof*⟩

lemma *phase-flip-channel-preserves-density-matrix*:
fixes p :: *real*

assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
and *dm*: *density-matrix 2* ϱ
shows *density-matrix 2* (*phase-flip-channel p* ϱ)
⟨*proof*⟩

lemma *bit-phase-flip-channel-preserves-density-matrix*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
and *dm*: *density-matrix 2* ϱ
shows *density-matrix 2* (*bit-phase-flip-channel p* ϱ)
⟨*proof*⟩

37.16 Pauli product identities

With the Day 32 convention for Y: $XY = -iZ$, $YZ = -iX$, $ZX = -iY$.

lemma *Pauli-X-times-Pauli-Y*: *Pauli-X* $*$ *Pauli-Y* $= -\mathrm{i} \cdot_m$ *Pauli-Z*
⟨*proof*⟩

lemma *Pauli-Y-times-Pauli-Z*: *Pauli-Y* $*$ *Pauli-Z* $= -\mathrm{i} \cdot_m$ *Pauli-X*
⟨*proof*⟩

lemma *Pauli-Z-times-Pauli-X*: *Pauli-Z* $*$ *Pauli-X* $= -\mathrm{i} \cdot_m$ *Pauli-Y*
⟨*proof*⟩